

Here is a **complete Design Patterns list**, organized from beginner to advanced, with a strong focus on **C# / .NET / ASP.NET Core development**.

1. SOLID Principles

Before learning design patterns deeply, learn:

1. **S — Single Responsibility Principle**
2. **O — Open/Closed Principle**
3. **L — Liskov Substitution Principle**
4. **I — Interface Segregation Principle**
5. **D — Dependency Inversion Principle**

Also learn:

- DRY
- KISS
- YAGNI
- Separation of Concerns
- Composition over Inheritance
- Program to an Interface

2. Creational Design Patterns

Patterns for **creating objects**.

1. **Factory Method**
2. **Singleton**
3. **Abstract Factory**
4. **Builder**
5. **Prototype**
6. **Object Pool**

Learn in this order

Factory → Singleton → Builder → Prototype → Abstract Factory → Object Pool

3. Structural Design Patterns

Patterns for **combining classes and objects**.

1. **Adapter**
2. **Bridge**
3. **Composite**
4. **Decorator**
5. **Facade**
6. **Flyweight**
7. **Proxy**

Important for .NET

- Adapter
 - Decorator
 - Facade
 - Proxy
 - Composite
-

4. Behavioral Design Patterns

Patterns for **communication and behavior between objects**.

1. **Chain of Responsibility**
2. **Command**
3. **Interpreter**
4. **Iterator**
5. **Mediator**
6. **Memento**
7. **Observer**
8. **State**
9. **Strategy**
10. **Template Method**
11. **Visitor**

Very important for .NET

- Strategy
 - Observer
 - Command
 - Chain of Responsibility
 - Mediator
 - State
-

5. Enterprise / Application Patterns

These are not all GoF patterns, but are extremely useful for professional .NET applications.

Architecture & Data Access

1. **Repository Pattern**
2. **Unit of Work**
3. **Service Layer**
4. **Data Mapper**
5. **Active Record**
6. **Specification Pattern**
7. **Query Object**

8. **Identity Map**
9. **Lazy Load**
10. **Domain Model**
11. **Transaction Script**

Dependency & Infrastructure

12. **Dependency Injection**
13. **Dependency Inversion**
14. **Service Locator**
15. **Composition Root**

6. ASP.NET Core Common Patterns

For an ASP.NET Core developer, learn these particularly well:

1. **Dependency Injection**
2. **Repository**
3. **Unit of Work**
4. **Factory**
5. **Strategy**
6. **Decorator**
7. **Adapter**
8. **Facade**
9. **Mediator**
10. **CQRS**
11. **Specification**
12. **Options Pattern**
13. **Middleware Pattern**
14. **Pipeline Pattern**
15. **Result Pattern**

7. Distributed Systems / Microservices Patterns

For advanced .NET backend development:

1. **API Gateway**
2. **Backend for Frontend (BFF)**
3. **CQRS**
4. **Event Sourcing**
5. **Saga**
6. **Outbox Pattern**
7. **Inbox Pattern**
8. **Retry Pattern**
9. **Circuit Breaker**

- 10. Bulkhead
- 11. Timeout
- 12. Rate Limiting
- 13. Cache-Aside
- 14. Sidecar
- 15. Strangler Fig
- 16. Ambassador
- 17. Pub/Sub
- 18. Event-Driven Architecture
- 19. Competing Consumers
- 20. Leader Election

8. Concurrency Patterns

Useful when working with **C#, threading and high-performance applications.**

- 1. Producer–Consumer
- 2. Thread Pool
- 3. Future / Promise
- 4. Active Object
- 5. Monitor Object
- 6. Read-Write Lock
- 7. Double-Checked Locking
- 8. Immutable Object
- 9. Actor Model
- 10. Pipeline

9. Most Important Patterns for a .NET Developer

If your goal is **ASP.NET Core + Web API + SQL + Microservices**, prioritize these:

Priority	Pattern
★★★★★	Dependency Injection
★★★★★	Factory
★★★★★	Strategy
★★★★★	Repository
★★★★★	Unit of Work
★★★★★	Decorator
★★★★★	Middleware / Pipeline
★★★★★	CQRS

Priority	Pattern
★★★★★	Mediator
★★★★★	Circuit Breaker
★★★★	Adapter
★★★★	Facade
★★★★	Specification
★★★★	Observer
★★★★	Command
★★★★	Chain of Responsibility
★★★★	Saga
★★★★	Outbox
★★★★	Cache-Aside
★★★	Singleton
★★★	Builder
★★★	Prototype
★★★	Proxy

1. SOLID Principles

Before learning design patterns deeply, learn:

- 1. **S — Single Responsibility Principle**
- 2. **O — Open/Closed Principle**
- 3. **L — Liskov Substitution Principle**
- 4. **I — Interface Segregation Principle**
- 5. **D — Dependency Inversion Principle**

SOLID is a set of 5 object-oriented design principles that help make C# applications more maintainable, flexible, and scalable.

S - Single Responsibility Principle (SRP)

A class should have only one reason to change.

Bad Example

```
public class Employee
{
    public void CalculateSalary()
```

```
{
    // Salary calculation
}

public void SaveToDatabase()
{
    // Database logic
}
}
```

The class has two responsibilities:

- Salary calculation
- Database operations

Good Example

```
public class Employee
{
    public decimal Salary { get; set; }
}

public class SalaryCalculator
{
    public decimal Calculate(Employee employee)
    {
        return employee.Salary;
    }
}

public class EmployeeRepository
{
    public void Save(Employee employee)
    {
        // Save employee to DB
    }
}
```

O - Open/Closed Principle (OCP)

Software entities should be open for extension but closed for modification.

Bad Example

```
public class DiscountCalculator
{
    public decimal Calculate(string customerType, decimal amount)
    {
```

```
        if (customerType == "Regular")
            return amount * 0.05m;
        else if (customerType == "Premium")
            return amount * 0.10m;

        return 0;
    }
}
```

Every new customer type requires modifying existing code.

Good Example

```
public interface IDiscount
{
    decimal Calculate(decimal amount);
}

public class RegularDiscount : IDiscount
{
    public decimal Calculate(decimal amount)
        => amount * 0.05m;
}

public class PremiumDiscount : IDiscount
{
    public decimal Calculate(decimal amount)
        => amount * 0.10m;
}
```

Usage:

```
IDiscount discount = new PremiumDiscount();
decimal result = discount.Calculate(1000);
```

New discount types can be added without changing existing classes.

L - Liskov Substitution Principle (LSP)

Derived classes should be replaceable with their base classes without breaking behavior.

Bad Example

```
public class Bird
{
    public virtual void Fly()
```

```
    {  
    }  
}  
  
public class Penguin : Bird  
{  
    public override void Fly()  
    {  
        throw new NotImplementedException();  
    }  
}
```

Penguins can't fly, so substitution fails.

Good Example

```
public interface IBird  
{  
}  
  
public interface IFlyable  
{  
    void Fly();  
}  
  
public class Sparrow : IBird, IFlyable  
{  
    public void Fly()  
    {  
        Console.WriteLine("Flying");  
    }  
}  
  
public class Penguin : IBird  
{  
}
```

Now a Penguin can safely replace an **IBird**.

I - Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they don't use.

Bad Example

```
public interface IWorker  
{  
    void Work();  
}
```



```
        void Eat();
    }

    public class Robot : IWorker
    {
        public void Work()
        {
            Console.WriteLine("Working");
        }

        public void Eat()
        {
            throw new NotImplementedException();
        }
    }
}
```

Robot doesn't eat.

Good Example

```
public interface IWorkable
{
    void Work();
}

public interface IEatable
{
    void Eat();
}

public class Human : IWorkable, IEatable
{
    public void Work() { }
    public void Eat() { }
}

public class Robot : IWorkable
{
    public void Work() { }
}
```

D - Dependency Inversion Principle (DIP)

High-level modules should depend on abstractions, not concrete implementations.

Bad Example

```
public class EmailService
{
    public void Send(string message)
    {
        Console.WriteLine("Email Sent");
    }
}

public class Notification
{
    private readonly EmailService _emailService =
        new EmailService();

    public void Notify(string message)
    {
        _emailService.Send(message);
    }
}
```

`Notification` is tightly coupled to `EmailService`.

Good Example

```
public interface IMessageService
{
    void Send(string message);
}

public class EmailService : IMessageService
{
    public void Send(string message)
    {
        Console.WriteLine("Email Sent");
    }
}

public class SmsService : IMessageService
{
    public void Send(string message)
    {
        Console.WriteLine("SMS Sent");
    }
}

public class Notification
{
    private readonly IMessageService _messageService;

    public Notification(IMessageService messageService)
    {
        _messageService = messageService;
    }
}
```

```
    }

    public void Notify(string message)
    {
        _messageService.Send(message);
    }
}
```

Usage:

```
IMessageService service = new EmailService();
Notification notification = new Notification(service);

notification.Notify("Hello");
```

Quick Summary

Principle	Meaning
S	One class = One responsibility
O	Extend behavior without modifying existing code
L	Derived classes should be substitutable for base classes
I	Create small, focused interfaces
D	Depend on abstractions, not implementations

Also learn:

- DRY
- KISS
- YAGNI
- Separation of Concerns
- Composition over Inheritance
- Program to an Interface

2. Creational Design Patterns

Patterns for **creating objects**.

1. Factory Method

Factory Method Design Pattern in C#

The **Factory Method** is a **Creational Design Pattern**.

Its main purpose is:

Define an interface for creating an object, but let subclasses decide which concrete object should be created.

Instead of directly doing:

```
var payment = new CreditCardPayment();
```

you delegate object creation to a factory method.

1. Problem Without Factory Method

Suppose you have different payment types:

```
public class CreditCardPayment
{
    public void Pay()
    {
        Console.WriteLine("Paid using Credit Card");
    }
}

public class UpiPayment
{
    public void Pay()
    {
        Console.WriteLine("Paid using UPI");
    }
}
```

Your code might contain:

```
if (paymentType == "CreditCard")
{
    var payment = new CreditCardPayment();
    payment.Pay();
}
else if (paymentType == "UPI")
{
    var payment = new UpiPayment();
    payment.Pay();
}
```

This creates problems:

- Business code knows concrete classes.
 - Lots of `if/else` or `switch`.
 - Adding a new payment type requires modifying existing code.
 - Violates the **Open/Closed Principle**.
-

2. Create a Common Interface

First create a common abstraction:

```
public interface IPayment
{
    void Pay();
}
```

Implement it:

```
public class CreditCardPayment : IPayment
{
    public void Pay()
    {
        Console.WriteLine("Paid using Credit Card");
    }
}
```

```
public class UpiPayment : IPayment
{
    public void Pay()
    {
        Console.WriteLine("Paid using UPI");
    }
}
```

Now both classes follow the same contract.

3. Factory Method

Create an abstract creator:

```
public abstract class PaymentProcessor
{
    protected abstract IPayment CreatePayment();
}
```

```
    public void ProcessPayment()
    {
        IPayment payment = CreatePayment();

        payment.Pay();
    }
}
```

Here:

```
CreatePayment()
```

is the **Factory Method**.

The base class doesn't know which concrete payment object will be created.

4. Concrete Creators

Create a creator for Credit Card:

```
public class CreditCardProcessor : PaymentProcessor
{
    protected override IPayment CreatePayment()
    {
        return new CreditCardPayment();
    }
}
```

And another for UPI:

```
public class UpiProcessor : PaymentProcessor
{
    protected override IPayment CreatePayment()
    {
        return new UpiPayment();
    }
}
```

5. Usage

```
PaymentProcessor processor = new CreditCardProcessor();  
  
processor.ProcessPayment();
```

Output:

```
Paid using Credit Card
```

For UPI:

```
PaymentProcessor processor = new UpiProcessor();  
  
processor.ProcessPayment();
```

Output:

```
Paid using UPI
```

6. Complete Example

```
using System;  
  
public interface IPayment  
{  
    void Pay();  
}  
  
public class CreditCardPayment : IPayment  
{  
    public void Pay()  
    {  
        Console.WriteLine("Paid using Credit Card");  
    }  
}  
  
public class UpiPayment : IPayment  
{  
    public void Pay()  
    {  
        Console.WriteLine("Paid using UPI");  
    }  
}
```

```
public abstract class PaymentProcessor
{
    protected abstract IPayment CreatePayment();

    public void ProcessPayment()
    {
        IPayment payment = CreatePayment();

        payment.Pay();
    }
}

public class CreditCardProcessor : PaymentProcessor
{
    protected override IPayment CreatePayment()
    {
        return new CreditCardPayment();
    }
}

public class UpiProcessor : PaymentProcessor
{
    protected override IPayment CreatePayment()
    {
        return new UpiPayment();
    }
}

public class Program
{
    public static void Main()
    {
        PaymentProcessor processor =
            new CreditCardProcessor();

        processor.ProcessPayment();

        processor = new UpiProcessor();

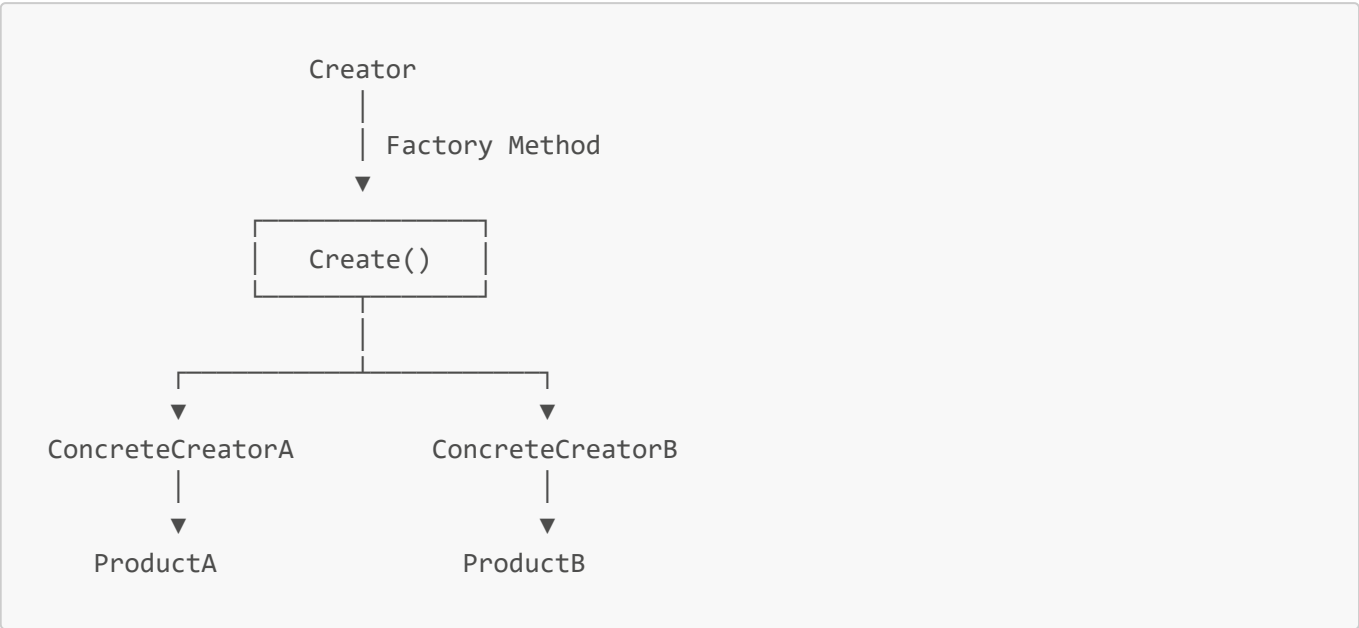
        processor.ProcessPayment();
    }
}
```

Output:

```
Paid using Credit Card
Paid using UPI
```


7. Important Structure

The Factory Method pattern usually looks like this:



In our example:



8. Why Use Factory Method?

Without Factory Method

```
var payment = new CreditCardPayment();
```

The caller knows the concrete implementation.

With Factory Method

```
PaymentProcessor processor =  
    new CreditCardProcessor();  
  
processor.ProcessPayment();
```

`ProcessPayment()` doesn't care about the concrete payment implementation.

It only knows:

```
IPayment
```

This gives you **loose coupling**.

9. Adding a New Payment Type

Suppose you add:

```
public class PayPalPayment : IPayment  
{  
    public void Pay()  
    {  
        Console.WriteLine("Paid using PayPal");  
    }  
}
```

Then create:

```
public class PayPalProcessor : PaymentProcessor  
{  
    protected override IPayment CreatePayment()  
    {  
        return new PayPalPayment();  
    }  
}
```

You don't have to modify:

```
PaymentProcessor  
CreditCardPayment  
UpiPayment
```

You simply add new classes.

This is one of the major advantages of the pattern.

10. Factory Method vs Simple Factory

These are often confused.

Simple Factory

```
public class PaymentFactory
{
    public static IPayment Create(string type)
    {
        if (type == "CreditCard")
            return new CreditCardPayment();

        if (type == "UPI")
            return new UpiPayment();

        throw new Exception("Invalid payment type");
    }
}
```

Usage:

```
var payment = PaymentFactory.Create("UPI");
```

This is commonly called **Simple Factory**, although it isn't formally one of the original GoF patterns.

Factory Method

Instead of a central `if/else` factory:

```
public abstract class PaymentProcessor
{
    protected abstract IPayment CreatePayment();

    public void ProcessPayment()
    {
        var payment = CreatePayment();
        payment.Pay();
    }
}
```

Different subclasses decide what to create:

```
CreditCardProcessor  
    ↓  
CreditCardPayment
```

```
UpiProcessor  
    ↓  
UpiPayment
```

11. Factory Method vs Abstract Factory

This distinction is important for interviews.

Factory Method	Abstract Factory
Creates usually one product	Creates a family of related products
Often uses inheritance	Often uses composition/interfaces
One factory method	Multiple factory methods
Example: Payment	Example: UI components
<code>CreatePayment()</code>	<code>CreateButton(), CreateTextbox()</code>

Example Factory Method:

```
IPayment CreatePayment();
```

Abstract Factory:

```
IButton CreateButton();  
  
ITextbox CreateTextbox();  
  
ICheckbox CreateCheckbox();
```

12. Real ASP.NET Core Example

You can encounter this idea when building systems with multiple implementations.

For example:

```
INotification
├── EmailNotification
├── SmsNotification
└── PushNotification
```

A processor can depend on:

```
public interface INotification
{
    Task SendAsync(string message);
}
```

Different implementations can be created through a factory:

```
public interface INotificationFactory
{
    INotification Create(string type);
}
```

Then:

```
public class NotificationFactory : INotificationFactory
{
    public INotification Create(string type)
    {
        return type switch
        {
            "email" => new EmailNotification(),
            "sms" => new SmsNotification(),
            "push" => new PushNotification(),
            _ => throw new ArgumentException("Invalid notification type")
        };
    }
}
```

In modern ASP.NET Core applications, however, **Dependency Injection + keyed services** or a DI-backed factory is often preferable to manually constructing dependencies with `new`.

13. Interview Definition

A good interview answer:

Factory Method is a creational design pattern that defines a method for creating objects while allowing subclasses or implementations to decide which concrete object is instantiated. It helps reduce coupling between client code and concrete classes and follows the Open/Closed Principle.

Remember this structure

```
Factory Method
  ↓
Creator
  ↓
Factory Method
  ↓
Concrete Creator
  ↓
Concrete Product
```

The key idea is:

"Don't let the main business logic decide which concrete object to instantiate; delegate object creation to a factory method."

2. Singleton

The **Singleton Design Pattern** is a **creational design pattern** that ensures **only one instance of a class exists** and provides a **global point of access** to that instance.

When to Use

Use the Singleton pattern when:

- Only one object of a class should exist.
- The object needs to be shared across the application.
- Examples:
 - Logger
 - Configuration manager
 - Database connection manager (with caution)
 - Cache manager

Basic Singleton Implementation

```
using System;

public sealed class Singleton
```

```
{
    // Static variable to hold the single instance
    private static Singleton instance;

    // Private constructor prevents object creation
    private Singleton()
    {
    }

    // Public method to access the instance
    public static Singleton GetInstance()
    {
        if (instance == null)
        {
            instance = new Singleton();
        }

        return instance;
    }

    public void ShowMessage()
    {
        Console.WriteLine("Singleton instance called.");
    }
}

class Program
{
    static void Main()
    {
        Singleton s1 = Singleton.GetInstance();
        Singleton s2 = Singleton.GetInstance();

        Console.WriteLine(ReferenceEquals(s1, s2)); // True

        s1.ShowMessage();
    }
}
```

Output

```
True
Singleton instance called.
```

Thread-Safe Singleton (Recommended)

The previous implementation is **not thread-safe**. If multiple threads call `GetInstance()` simultaneously, more than one instance could be created.

```
public sealed class Singleton
{
    private static readonly object lockObj = new object();
    private static Singleton instance;

    private Singleton()
    {
    }

    public static Singleton Instance
    {
        get
        {
            lock (lockObj)
            {
                if (instance == null)
                {
                    instance = new Singleton();
                }

                return instance;
            }
        }
    }
}
```

Best Practice: **Lazy<T>** Singleton

The simplest and safest approach in modern C# is to use **Lazy<T>**.

```
using System;

public sealed class Singleton
{
    private static readonly Lazy<Singleton> lazy =
        new Lazy<Singleton>(() => new Singleton());

    private Singleton()
    {
    }

    public static Singleton Instance => lazy.Value;

    public void Display()
    {
        Console.WriteLine("Lazy Singleton");
    }
}
```



```
class Program
{
    static void Main()
    {
        Singleton.Instance.Display();
    }
}
```

Advantages of `Lazy<T>`

- Thread-safe by default.
 - Lazy initialization (created only when first accessed).
 - Simpler code.
 - Better performance than always locking.
-

Real-World Example: Logger

```
public sealed class Logger
{
    private static readonly Lazy<Logger> logger =
        new Lazy<Logger>(() => new Logger());

    private Logger()
    {
    }

    public static Logger Instance => logger.Value;

    public void Log(string message)
    {
        Console.WriteLine($"[{DateTime.Now}] {message}");
    }
}

class Program
{
    static void Main()
    {
        Logger.Instance.Log("Application started.");
        Logger.Instance.Log("User logged in.");
    }
}
```

Advantages

- Ensures a single instance throughout the application.
- Saves memory when only one instance is needed.

- Provides a centralized access point.
- Lazy initialization is possible.

Disadvantages

- Introduces global state, which can make testing harder.
- Can create tight coupling between classes.
- Needs careful implementation to be thread-safe.
- Overusing it can reduce flexibility.

Singleton vs Static Class

Singleton	Static Class
One instance of a class	No instance
Can implement interfaces	Cannot implement interfaces
Can inherit from other classes	Cannot be inherited
Supports lazy initialization	Loaded when first accessed by the runtime
Can maintain instance state	Only static members

Summary

- **Purpose:** Ensure only one instance of a class exists.
- **Best implementation in modern C#:** Use `Lazy<T>`.
- **Common uses:** Logging, configuration, caching, and shared services.
- **Avoid using Singleton** if dependency injection or regular object lifetimes provide a cleaner design, as DI often improves testability and flexibility.

3. Abstract Factory

The **Abstract Factory** design pattern is a **creational design pattern** that provides an interface for creating **families of related or dependent objects** without specifying their concrete classes.

When to Use

Use the Abstract Factory pattern when:

- You need to create related objects together.
- Your application should work with multiple families of products.
- You want to avoid tightly coupling your code to concrete implementations.

Example Scenario

Suppose you're building a cross-platform UI framework.

For Windows:

- Windows Button
- Windows Checkbox

For macOS:

- Mac Button
- Mac Checkbox

You want to create the correct UI components based on the operating system.

Step 1: Create Product Interfaces

```
public interface IButton
{
    void Render();
}

public interface ICheckBox
{
    void Render();
}
```

Step 2: Create Concrete Products

Windows Products

```
public class WindowsButton : IButton
{
    public void Render()
    {
        Console.WriteLine("Windows Button");
    }
}

public class WindowsCheckBox : ICheckBox
{
    public void Render()
    {
        Console.WriteLine("Windows Checkbox");
    }
}
```

macOS Products

```
public class MacButton : IButton
{
```

```
        public void Render()
        {
            Console.WriteLine("Mac Button");
        }
    }

    public class MacCheckBox : ICheckBox
    {
        public void Render()
        {
            Console.WriteLine("Mac Checkbox");
        }
    }
}
```

Step 3: Create the Abstract Factory

```
public interface UIFactory
{
    IButton CreateButton();
    ICheckBox CreateCheckBox();
}
```

Step 4: Create Concrete Factories

```
public class WindowsFactory : UIFactory
{
    public IButton CreateButton()
    {
        return new WindowsButton();
    }

    public ICheckBox CreateCheckBox()
    {
        return new WindowsCheckBox();
    }
}

public class MacFactory : UIFactory
{
    public IButton CreateButton()
    {
        return new MacButton();
    }

    public ICheckBox CreateCheckBox()
    {
        return new MacCheckBox();
    }
}
```

```
}  
}
```

Step 5: Client Code

```
class Application  
{  
    private readonly IButton _button;  
    private readonly ICheckBox _checkBox;  
  
    public Application(IUIFactory factory)  
    {  
        _button = factory.CreateButton();  
        _checkBox = factory.CreateCheckBox();  
    }  
  
    public void RenderUI()  
    {  
        _button.Render();  
        _checkBox.Render();  
    }  
}
```

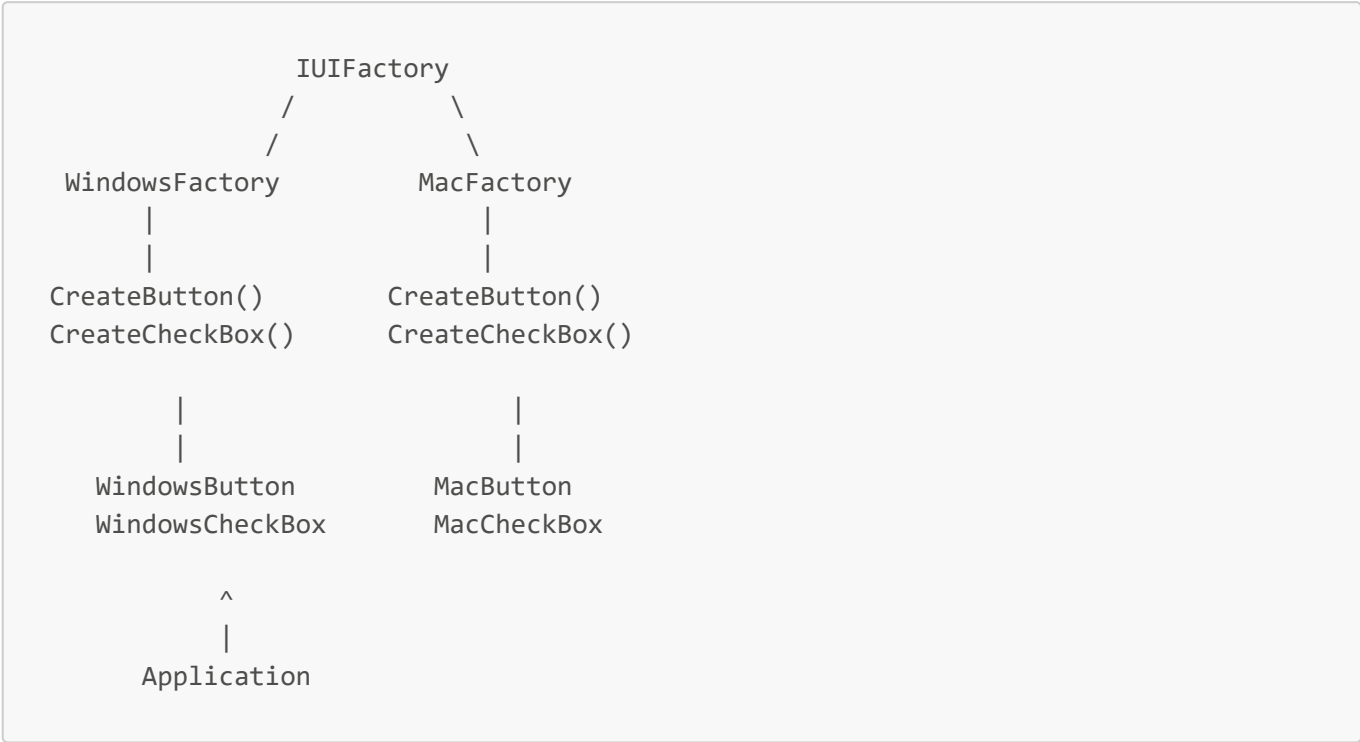
Step 6: Usage

```
class Program  
{  
    static void Main()  
    {  
        IUIFactory factory;  
  
        string os = "Windows";  
  
        if (os == "Windows")  
            factory = new WindowsFactory();  
        else  
            factory = new MacFactory();  
  
        Application app = new Application(factory);  
        app.RenderUI();  
    }  
}
```

Output

Windows Button
Windows Checkbox

Class Diagram



Advantages

- Decouples client code from concrete classes.
- Ensures compatible products are created together.
- Makes switching between product families easy.
- Follows the Open/Closed Principle.

Disadvantages

- Adding a new product type (e.g., **ITextBox**) requires changes to:
 - The abstract factory interface.
 - Every concrete factory.
- Can introduce many classes, increasing complexity.

Factory Method vs. Abstract Factory

Factory Method	Abstract Factory
Creates one product	Creates a family of related products

Factory Method	Abstract Factory
Uses inheritance	Uses composition
Simpler	More flexible but more complex
Example: Create a Car	Example: Create Car, Engine, and Tire together

Real-World Example in ASP.NET Core

The built-in Dependency Injection (DI) container often acts similarly to an Abstract Factory. For example, different implementations of services can be injected based on configuration:

```
public interface INotificationService
{
    void Send(string message);
}

public class EmailNotification : INotificationService
{
    public void Send(string message) =>
        Console.WriteLine($"Email: {message}");
}

public class SmsNotification : INotificationService
{
    public void Send(string message) =>
        Console.WriteLine($"SMS: {message}");
}
```

A factory can return the appropriate notification service based on configuration or runtime conditions, while the client depends only on the interface.

In summary, the Abstract Factory pattern is ideal when you need to create **groups of related objects** (such as UI controls, database providers, or notification components) while keeping your application independent of their concrete implementations.

4. Builder

Builder Design Pattern in C#

The **Builder Design Pattern** is a **creational design pattern** used to construct complex objects **step by step**.

It is especially useful when an object has:

- Many properties
- Optional properties
- Different construction combinations
- A constructor that would otherwise have too many parameters

1. Problem without Builder

Suppose we have a `Computer`:

```
public class Computer
{
    public string CPU { get; set; }
    public string RAM { get; set; }
    public string Storage { get; set; }
    public string GPU { get; set; }
    public bool WiFi { get; set; }
}
```

Without Builder, we might create it like:

```
Computer computer = new Computer
{
    CPU = "Intel i7",
    RAM = "32 GB",
    Storage = "1 TB SSD",
    GPU = "RTX 4070",
    WiFi = true
};
```

For a simple object this is fine.

But imagine a class with **20–30 configuration options**. Constructors can become difficult to maintain:

```
var computer = new Computer(
    "Intel i7",
    "32 GB",
    "1 TB SSD",
    "RTX 4070",
    true,
    true,
    false,
    "Windows 11",
    ...
);
```

It becomes difficult to understand what each parameter means.

2. Builder Pattern Solution

The Builder separates **object construction** from the actual object.

Product

```
public class Computer
{
    public string CPU { get; set; }
    public string RAM { get; set; }
    public string Storage { get; set; }
    public string GPU { get; set; }
    public bool WiFi { get; set; }

    public void Display()
    {
        Console.WriteLine($"CPU      : {CPU}");
        Console.WriteLine($"RAM      : {RAM}");
        Console.WriteLine($"Storage : {Storage}");
        Console.WriteLine($"GPU      : {GPU}");
        Console.WriteLine($"WiFi     : {WiFi}");
    }
}
```

Builder

```
public class ComputerBuilder
{
    private Computer computer = new Computer();

    public ComputerBuilder SetCPU(string cpu)
    {
        computer.CPU = cpu;
        return this;
    }

    public ComputerBuilder SetRAM(string ram)
    {
        computer.RAM = ram;
        return this;
    }

    public ComputerBuilder SetStorage(string storage)
    {
        computer.Storage = storage;
        return this;
    }

    public ComputerBuilder SetGPU(string gpu)
    {
        computer.GPU = gpu;
        return this;
    }
}
```

```
    }

    public ComputerBuilder EnableWiFi()
    {
        computer.WiFi = true;
        return this;
    }

    public Computer Build()
    {
        return computer;
    }
}
```

Usage

```
Computer computer = new ComputerBuilder()
    .SetCPU("Intel i7")
    .SetRAM("32 GB")
    .SetStorage("1 TB SSD")
    .SetGPU("RTX 4070")
    .EnableWiFi()
    .Build();

computer.Display();
```

Output:

```
CPU      : Intel i7
RAM      : 32 GB
Storage  : 1 TB SSD
GPU      : RTX 4070
WiFi     : True
```

3. How it works

The flow is:

```
Client
|
v
ComputerBuilder
|
| SetCPU()
| SetRAM()
```

```
    | SetStorage()  
    | SetGPU()  
    |  
    v  
Build()  
    |  
    v  
Computer
```

The important idea is:

Don't construct the complex object all at once. Construct it step by step.

4. Why `return this`?

Notice:

```
public ComputerBuilder SetCPU(string cpu)  
{  
    computer.CPU = cpu;  
    return this;  
}
```

`return this` allows **method chaining**:

```
builder  
    .SetCPU("Intel i7")  
    .SetRAM("32 GB")  
    .SetStorage("1 TB SSD")  
    .SetGPU("RTX 4070");
```

This is called a **fluent interface**.

5. Builder with Interfaces

A more formal implementation looks like this:

```
public interface IComputerBuilder  
{  
    IComputerBuilder SetCPU(string cpu);  
    IComputerBuilder SetRAM(string ram);  
    IComputerBuilder SetStorage(string storage);  
    IComputerBuilder SetGPU(string gpu);  
}
```

```
    IComputerBuilder EnableWiFi();  
  
    Computer Build();  
}
```

Implementation:

```
public class ComputerBuilder : IComputerBuilder  
{  
    private readonly Computer computer = new Computer();  
  
    public IComputerBuilder SetCPU(string cpu)  
    {  
        computer.CPU = cpu;  
        return this;  
    }  
  
    public IComputerBuilder SetRAM(string ram)  
    {  
        computer.RAM = ram;  
        return this;  
    }  
  
    public IComputerBuilder SetStorage(string storage)  
    {  
        computer.Storage = storage;  
        return this;  
    }  
  
    public IComputerBuilder SetGPU(string gpu)  
    {  
        computer.GPU = gpu;  
        return this;  
    }  
  
    public IComputerBuilder EnableWiFi()  
    {  
        computer.WiFi = true;  
        return this;  
    }  
  
    public Computer Build()  
    {  
        return computer;  
    }  
}
```

Usage:

```
IComputerBuilder builder = new ComputerBuilder();

Computer computer = builder
    .SetCPU("AMD Ryzen 7")
    .SetRAM("16 GB")
    .SetStorage("512 GB SSD")
    .EnableWiFi()
    .Build();
```

6. Builder with Immutable Objects

A better modern approach is to make the final object immutable.

```
public class Computer
{
    public string CPU { get; }
    public string RAM { get; }
    public string Storage { get; }
    public string GPU { get; }
    public bool WiFi { get; }

    public Computer(
        string cpu,
        string ram,
        string storage,
        string gpu,
        bool wifi)
    {
        CPU = cpu;
        RAM = ram;
        Storage = storage;
        GPU = gpu;
        WiFi = wifi;
    }
}
```

Builder:

```
public class ComputerBuilder
{
    private string cpu;
    private string ram;
    private string storage;
    private string gpu;
    private bool wifi;
```

```
public ComputerBuilder SetCPU(string cpu)
{
    this.cpu = cpu;
    return this;
}

public ComputerBuilder SetRAM(string ram)
{
    this.ram = ram;
    return this;
}

public ComputerBuilder SetStorage(string storage)
{
    this.storage = storage;
    return this;
}

public ComputerBuilder SetGPU(string gpu)
{
    this.gpu = gpu;
    return this;
}

public ComputerBuilder EnableWiFi()
{
    wifi = true;
    return this;
}

public Computer Build()
{
    return new Computer(
        cpu,
        ram,
        storage,
        gpu,
        wifi);
}
```

Usage:

```
var computer = new ComputerBuilder()
    .SetCPU("Intel i7")
    .SetRAM("32 GB")
    .SetStorage("1 TB SSD")
    .SetGPU("RTX 4070")
    .EnableWiFi()
    .Build();
```

This is generally a cleaner design because the **Computer** cannot be modified after creation.

7. Builder + Director

The traditional Builder pattern sometimes contains a **Director**.

The Director knows **the construction process**, while the Builder knows **how to construct each part**.

```
public class ComputerDirector
{
    private readonly ComputerBuilder builder;

    public ComputerDirector(ComputerBuilder builder)
    {
        this.builder = builder;
    }

    public Computer BuildGamingComputer()
    {
        return builder
            .SetCPU("Intel i9")
            .SetRAM("64 GB")
            .SetStorage("2 TB SSD")
            .SetGPU("RTX 4090")
            .EnableWiFi()
            .Build();
    }
}
```

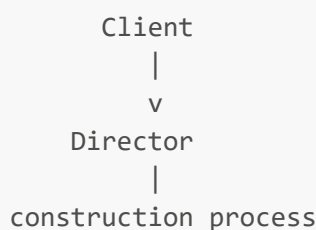
Usage:

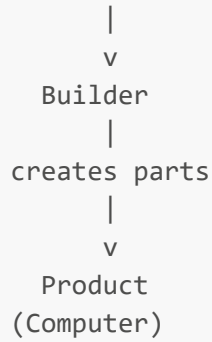
```
var builder = new ComputerBuilder();

var director = new ComputerDirector(builder);

Computer gamingComputer =
    director.BuildGamingComputer();
```

Architecture:





8. Real-world ASP.NET Core Example

Builder is extremely common in .NET.

For example:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

builder.Services.AddSwaggerGen();

builder.Services.AddAuthentication();

builder.Services.AddAuthorization();

var app = builder.Build();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();

app.Run();
```

Here:

```
WebApplication.CreateBuilder(args)
```

creates a builder that lets you configure the application step by step.

Then:

```
builder.Services.AddControllers();
builder.Services.AddSwaggerGen();
```



```
builder.Services.AddAuthentication();
```

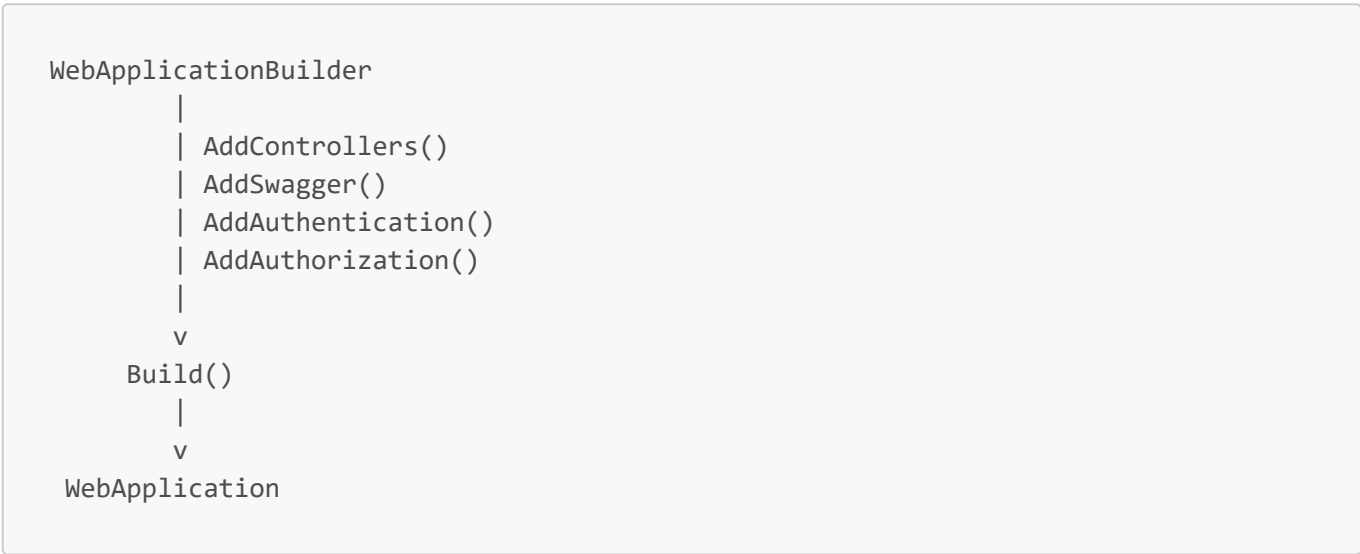
configure it.

Finally:

```
var app = builder.Build();
```

creates the final application.

Conceptually:



9. Builder vs Factory Method

Since you're learning the GoF patterns, this distinction is important.

Builder	Factory Method
Builds a complex object step by step	Creates an object
Focuses on construction process	Focuses on object creation
Usually many configuration options	Usually chooses an implementation
Supports fluent APIs	Usually uses inheritance/polymorphism
Example: Configure a Computer	Example: Create a PdfReport or ExcelReport

Simple way to remember:

```
Factory Method
↓
"What object should I create?"
```

```
Builder
  ↓
"How should I construct this object?"
```

10. Builder vs Abstract Factory

Another important distinction:

```
Factory Method
  → Create one type of object

Abstract Factory
  → Create families of related objects

Builder
  → Construct one complex object step-by-step
```

For example:

```
Factory Method
  → Create WindowsButton

Abstract Factory
  → Create WindowsButton
  → Create WindowsCheckbox
  → Create WindowsTextbox

Builder
  → Build a complex Computer
    CPU
    RAM
    Storage
    GPU
    WiFi
    OS
```

11. When should you use Builder?

Use Builder when:

- An object has **many parameters**
- Many parameters are optional

- Construction involves multiple steps
- You want readable object creation
- You want to avoid huge constructors
- Different configurations of the same object are required
- You want immutable final objects

Avoid Builder for very simple classes such as:

```
var person = new Person("John", 25);
```

A Builder would add unnecessary complexity.

Interview Definition

Builder is a creational design pattern that separates the construction of a complex object from its representation, allowing the same construction process to create different representations.

Core structure

```
Client
|
v
Builder
|
+-- SetPartA()
+-- SetPartB()
+-- SetPartC()
|
+-- Build()
    |
    v
    Product
```

Key idea: Factory decides *what* to create; Builder controls *how* a complex object is constructed.

5. Prototype

Prototype Design Pattern in C#

The **Prototype Design Pattern** is a **Creational Design Pattern** used when you want to create a new object by **copying an existing object**, instead of creating it from scratch.

1. Basic idea

Instead of:

```
new Employee()  
→ set Name  
→ set Department  
→ set Salary
```

you do:

```
Existing Employee  
  ↓  
  Clone()  
  ↓  
New Employee
```

This is useful when object creation is **expensive or complicated**.

2. Simple C# Example

```
public class Employee  
{  
    public string Name { get; set; }  
    public string Department { get; set; }  
    public double Salary { get; set; }  
  
    public Employee Clone()  
    {  
        return new Employee  
        {  
            Name = this.Name,  
            Department = this.Department,  
            Salary = this.Salary  
        };  
    }  
}
```

Usage:

```
Employee employee1 = new Employee  
{  
    Name = "Rahul",  
    Department = "IT",  
    Salary = 60000  
};  
  
Employee employee2 = employee1.Clone();
```

```
employee2.Name = "Amit";

Console.WriteLine(employee1.Name);
Console.WriteLine(employee2.Name);
```

Output:

```
Rahul
Amit
```

The original object remains unchanged.

3. Using `ICloneable`

C# provides the `ICloneable` interface.

```
public class Employee : ICloneable
{
    public string Name { get; set; }
    public string Department { get; set; }
    public double Salary { get; set; }

    public object Clone()
    {
        return MemberwiseClone();
    }
}
```

Usage:

```
Employee employee1 = new Employee
{
    Name = "Rahul",
    Department = "IT",
    Salary = 60000
};

Employee employee2 = (Employee)employee1.Clone();

employee2.Name = "Amit";
```

`MemberwiseClone()` creates a **shallow copy**.

4. Shallow Copy vs Deep Copy

This is very important with Prototype.

Suppose:

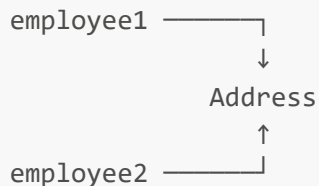
```
public class Address
{
    public string City { get; set; }
}

public class Employee
{
    public string Name { get; set; }
    public Address Address { get; set; }
}
```

If you simply use:

```
Employee employee2 = (Employee)employee1.MemberwiseClone();
```

then both objects can reference the **same Address object**.



This is a **shallow copy**.

Deep copy

With deep copying:

```
employee1 → Address 1
employee2 → Address 2
```

Example:

```
public class Employee
{
```

```
public string Name { get; set; }
public Address Address { get; set; }

public Employee Clone()
{
    return new Employee
    {
        Name = this.Name,
        Address = new Address
        {
            City = this.Address.City
        }
    };
}
```

Now modifying:

```
employee2.Address.City = "Pune";
```

doesn't modify:

```
employee1.Address.City
```

5. Prototype with an Interface

A cleaner implementation is to create a prototype interface.

```
public interface IPrototype<T>
{
    T Clone();
}
```

Then:

```
public class Employee : IPrototype<Employee>
{
    public string Name { get; set; }
    public string Department { get; set; }
    public double Salary { get; set; }

    public Employee Clone()
    {
```

```
        return new Employee
        {
            Name = Name,
            Department = Department,
            Salary = Salary
        };
    }
}
```

Usage:

```
Employee original = new Employee
{
    Name = "Rahul",
    Department = "Finance",
    Salary = 70000
};

Employee copy = original.Clone();

copy.Name = "Amit";
copy.Salary = 80000;
```

6. Real-world Example

Imagine you're building a **banking/fintech application**.

Creating a customer object might involve:

```
Customer
├── Personal Information
├── Address
├── KYC Information
├── Account Information
├── Preferences
└── Risk Configuration
```

Creating this object repeatedly could be expensive.

Instead:

```
Customer Template
├── Clone → Customer 1
└── Clone → Customer 2
```



```
├─ Clone → Customer 3
└─ Clone → Customer 4
```

Then modify only the fields that differ.

7. Prototype vs Factory

Prototype	Factory
Creates by copying an existing object	Creates a new object
Uses <code>Clone()</code>	Uses factory method
Useful for expensive objects	Useful for choosing object types
Focuses on object duplication	Focuses on object creation
Can preserve existing configuration	Usually initializes a new configuration

Factory

```
var employee = EmployeeFactory.CreateEmployee("IT");
```

Prototype

```
var employee = existingEmployee.Clone();
```

8. When Should You Use Prototype?

Use Prototype when:

- Object creation is expensive.
- Object configuration is complicated.
- You need many similar objects.
- You want to avoid repeating initialization.
- You want to preserve an object's current state.
- Creating an object involves many database/API/configuration operations.

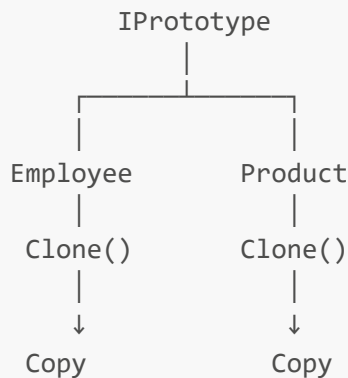
Avoid it when

Objects are simple:

```
var employee = new Employee();
```

There is little benefit from Prototype if creating the object is already cheap and straightforward.

Prototype Pattern Structure



Key point to remember

Prototype = Create a new object by cloning an existing object.

The most important C# concepts to understand with Prototype are `Clone()`, **shallow copy**, **deep copy**, and `MemberwiseClone()`.

6. Object Pool

Object Pool Design Pattern in C#

The **Object Pool** pattern is a **creational design pattern** that reuses expensive-to-create objects instead of creating a new object every time.

Simple idea

Instead of:

```
Create → Use → Destroy
Create → Use → Destroy
Create → Use → Destroy
```

Object Pool does:

```
Create objects once
↓
```

```
    [Pool]
   /  |  \
  Obj1 Obj2 Obj3
   ↓
Take object → Use → Return to pool
```

This can improve performance when object creation is expensive and objects can safely be reused.

1. Basic Example

Suppose creating a database connection, game object, network object, or large buffer is expensive.

```
public class DatabaseConnection
{
    public void Open()
    {
        Console.WriteLine("Connection opened");
    }

    public void Close()
    {
        Console.WriteLine("Connection closed");
    }
}
```

Instead of creating connections repeatedly:

```
var connection = new DatabaseConnection();
connection.Open();

// Use connection

connection.Close();
```

we can maintain a pool.

2. Object Pool Implementation

```
using System.Collections.Concurrent;

public class ObjectPool<T> where T : class
{
    private readonly ConcurrentBag<T> _pool;
    private readonly Func<T> _factory;
```

```

public ObjectPool(Func<T> factory, int initialSize)
{
    _factory = factory;
    _pool = new ConcurrentBag<T>();

    for (int i = 0; i < initialSize; i++)
    {
        _pool.Add(_factory());
    }
}

public T Rent()
{
    if (_pool.TryTake(out T? item))
    {
        return item;
    }

    return _factory();
}

public void Return(T item)
{
    _pool.Add(item);
}
}

```

Using it

```

var pool = new ObjectPool<DatabaseConnection>(
    () => new DatabaseConnection(),
    3
);

var connection = pool.Rent();

connection.Open();

// Perform database operation

connection.Close();

pool.Return(connection);

```

Here:

```

Pool
├─ Connection 1

```

```
├─ Connection 2
└─ Connection 3
```

When we call:

```
pool.Rent();
```

one object is removed from the pool.

When we call:

```
pool.Return(connection);
```

the object goes back into the pool.

3. Why Use Object Pool?

Object Pool is useful when **object creation/destruction is expensive**.

For example:

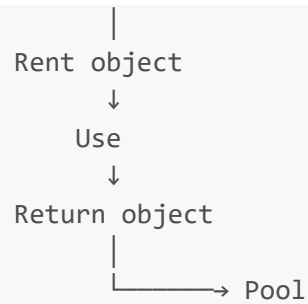
- Database connections
- HTTP/network connections
- Large byte buffers
- Threads
- Game objects
- Socket objects
- Expensive parsers
- Graphics resources
- Reusable memory buffers

Without pooling:

```
Request 1 → Create object → Use → Destroy
Request 2 → Create object → Use → Destroy
Request 3 → Create object → Use → Destroy
```

With pooling:

```
┌─ Object Pool ─┘
```



4. Important Rule: Reset the Object

This is one of the most important parts of Object Pooling.

Suppose:

```
public class UserRequest
{
    public string? UserName { get; set; }
    public string? Token { get; set; }
}
```

User A uses the object:

```
var request = pool.Rent();

request.UserName = "Alice";
request.Token = "ABC123";

pool.Return(request);
```

If we don't reset it, User B could receive:

```
UserName = Alice
Token = ABC123
```

That is dangerous.

Therefore, pooled objects should be reset before reuse.

For example:

```
public class UserRequest
{
```

```
public string? UserName { get; set; }
public string? Token { get; set; }

public void Reset()
{
    UserName = null;
    Token = null;
}
}
```

Then:

```
request.Reset();
pool.Return(request);
```

5. Object Pool with Reset

A better generic implementation can accept a reset action:

```
using System.Collections.Concurrent;

public class ObjectPool<T> where T : class
{
    private readonly ConcurrentBag<T> _pool;
    private readonly Func<T> _factory;
    private readonly Action<T> _reset;

    public ObjectPool(
        Func<T> factory,
        Action<T> reset,
        int initialSize)
    {
        _factory = factory;
        _reset = reset;

        _pool = new ConcurrentBag<T>();

        for (int i = 0; i < initialSize; i++)
        {
            _pool.Add(_factory());
        }
    }

    public T Rent()
    {
        if (_pool.TryTake(out T? item))
        {

```

```
        return item;
    }

    return _factory();
}

public void Return(T item)
{
    _reset(item);
    _pool.Add(item);
}
}
```

Usage:

```
var pool = new ObjectPool<UserRequest>(
    () => new UserRequest(),
    request => request.Reset(),
    10
);
```

Then:

```
var request = pool.Rent();

request.UserName = "John";
request.Token = "XYZ";

pool.Return(request);
```

Before returning, the object is reset.

6. Object Pool in ASP.NET Core

ASP.NET Core already provides an object-pooling mechanism through `Microsoft.Extensions.ObjectPool`.

You can use:

```
using Microsoft.Extensions.ObjectPool;
```

Example:


```
public class MyObject
{
    public string? Data { get; set; }
}
```

Create a pool:

```
var provider = new DefaultObjectPoolProvider();

var pool = provider.Create<MyObject>();
```

Rent:

```
var obj = pool.Get();

obj.Data = "Hello";
```

Return:

```
pool.Return(obj);
```

This is generally preferable to implementing your own pool when the framework's pooling functionality meets your requirements.

7. Object Pool vs Singleton

These patterns are often confused.

Singleton

There is **one shared instance**:

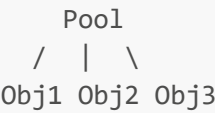
```
Application
  |
  ↓
Object
```

Example:

```
Logger.Instance
```

Object Pool

There are **multiple reusable instances**:



A consumer temporarily obtains one:

```
Rent → Use → Return
```

Singleton	Object Pool
One instance	Multiple instances
Shared permanently	Borrowed temporarily
No return operation	Return required
Useful for shared state/services	Useful for expensive reusable objects

8. Object Pool vs Factory

A **Factory** creates objects.

```
var obj = factory.Create();
```

An **Object Pool** reuses objects.

```
var obj = pool.Rent();

// use

pool.Return(obj);
```

Think of it as:

```
Factory
↓
```

Creates objects

Object Pool



Stores and reuses objects

A pool may internally use a factory to create new objects when it runs out.

9. Advantages

Performance

Avoids repeated expensive object creation.

Reduced GC pressure

Fewer temporary objects can mean less garbage collection pressure.

Resource reuse

Useful for expensive resources.

Better throughput

Particularly useful in high-frequency systems.

10. Disadvantages

Object pooling is **not automatically faster**.

It adds complexity:

- Objects must be reset correctly.
- Objects may contain stale state.
- Pool size must be managed.
- Incorrect sharing can cause concurrency problems.
- Holding objects too long can exhaust the pool.
- Pooling cheap objects can actually make code slower.

For ordinary lightweight C# classes:

```
new Customer();
```

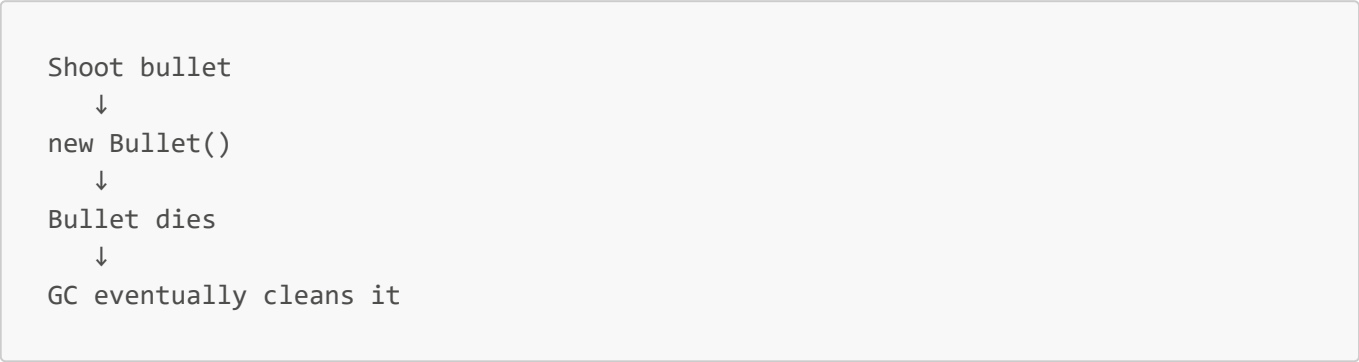
is often perfectly fine.

Don't introduce Object Pool merely because it is a design pattern.

11. Real-World Example

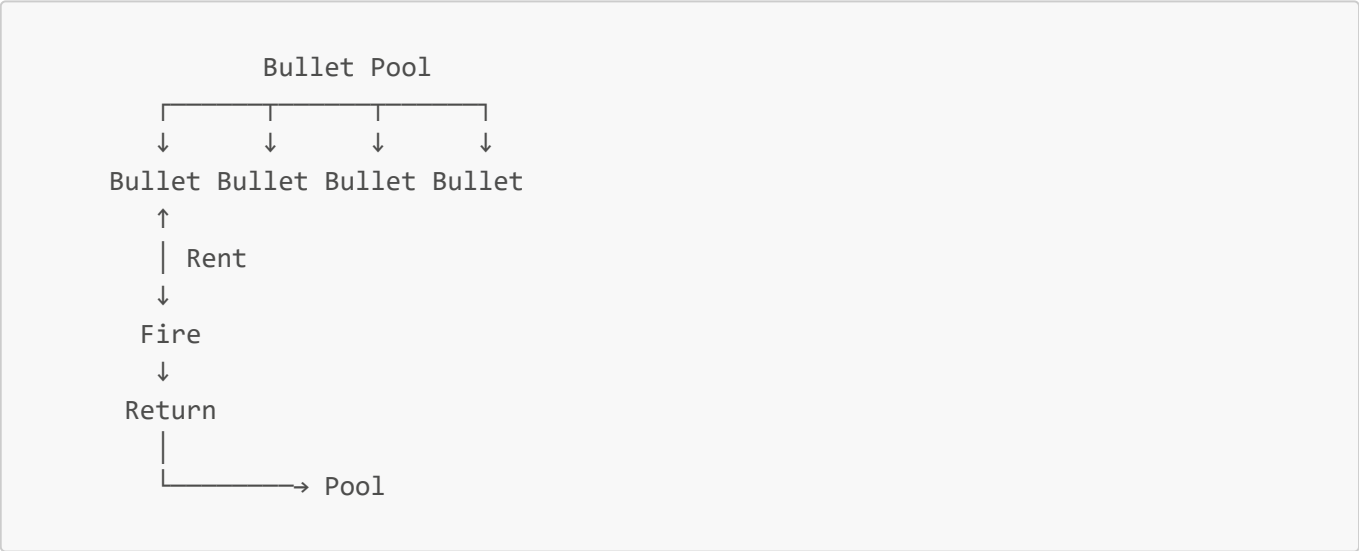
Imagine a game with 10,000 bullets.

Without pooling:



Thousands of allocations can occur.

With pooling:

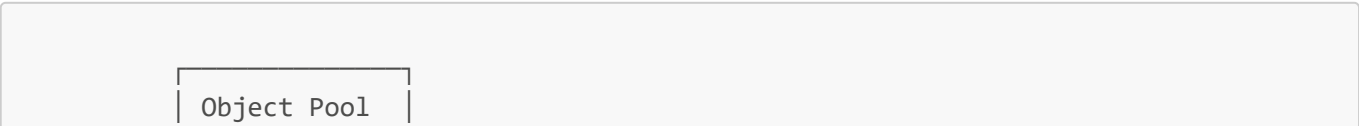


The same bullet objects are reused.

Interview Definition

Object Pool is a creational design pattern that maintains a collection of reusable objects. Instead of repeatedly creating and destroying expensive objects, clients rent an object from the pool, use it, and return it for future reuse.

Remember this lifecycle





Key concept: Rent → Use → Reset → Return → Reuse

Learn in this order

Factory → Singleton → Builder → Prototype → Abstract Factory → Object Pool

Also learn:

- DRY

DRY Design Principle in C#

DRY stands for **Don't Repeat Yourself**.

It is a software design principle that says:

Every piece of knowledge or logic should have a single, authoritative representation in the codebase.

DRY is not exactly a GoF "design pattern"; it is a **design principle** that helps you avoid duplicated logic.

1. Problem: Repeated Code

Suppose you have two methods:

```

public decimal CalculateEmployeeSalary(decimal salary)
{
    decimal tax = salary * 0.10m;
    decimal finalSalary = salary - tax;

    return finalSalary;
}

public decimal CalculateManagerSalary(decimal salary)

```

```
{  
    decimal tax = salary * 0.10m;  
    decimal finalSalary = salary - tax;  
  
    return finalSalary;  
}
```

The tax calculation is duplicated.

If the tax changes from **10% to 12%**, you have to modify multiple places.

2. Apply DRY

Move the common logic into one place:

```
public class SalaryCalculator  
{  
    public decimal CalculateAfterTax(decimal salary)  
    {  
        decimal tax = salary * 0.10m;  
        return salary - tax;  
    }  
}
```

Now:

```
public class EmployeeService  
{  
    private readonly SalaryCalculator _calculator;  
  
    public EmployeeService(SalaryCalculator calculator)  
    {  
        _calculator = calculator;  
    }  
  
    public decimal CalculateEmployeeSalary(decimal salary)  
    {  
        return _calculator.CalculateAfterTax(salary);  
    }  
  
    public decimal CalculateManagerSalary(decimal salary)  
    {  
        return _calculator.CalculateAfterTax(salary);  
    }  
}
```

The tax calculation exists in **one place**.

3. DRY with Methods

✗ Without DRY

```
public void CreateUser(string name)
{
    Console.WriteLine(name.Trim().ToLower());
}

public void UpdateUser(string name)
{
    Console.WriteLine(name.Trim().ToLower());
}

public void DeleteUser(string name)
{
    Console.WriteLine(name.Trim().ToLower());
}
```

The same transformation is repeated.

☑ With DRY

```
private string NormalizeName(string name)
{
    return name.Trim().ToLower();
}
```

Then:

```
public void CreateUser(string name)
{
    Console.WriteLine(NormalizeName(name));
}

public void UpdateUser(string name)
{
    Console.WriteLine(NormalizeName(name));
}

public void DeleteUser(string name)
{
    Console.WriteLine(NormalizeName(name));
}
```

4. DRY with Classes

Suppose you have:

```
public class OrderService
{
    public decimal CalculateTax(decimal amount)
    {
        return amount * 0.18m;
    }
}
```

And another class:

```
public class InvoiceService
{
    public decimal CalculateTax(decimal amount)
    {
        return amount * 0.18m;
    }
}
```

This duplicates business knowledge.

Instead:

```
public class TaxCalculator
{
    public decimal Calculate(decimal amount)
    {
        return amount * 0.18m;
    }
}
```

Use it from both services:

```
public class OrderService
{
    private readonly TaxCalculator _taxCalculator;

    public OrderService(TaxCalculator taxCalculator)
    {
        _taxCalculator = taxCalculator;
    }
}
```



```
    public decimal CalculateTax(decimal amount)
    {
        return _taxCalculator.Calculate(amount);
    }
}
```

```
public class InvoiceService
{
    private readonly TaxCalculator _taxCalculator;

    public InvoiceService(TaxCalculator taxCalculator)
    {
        _taxCalculator = taxCalculator;
    }

    public decimal CalculateTax(decimal amount)
    {
        return _taxCalculator.Calculate(amount);
    }
}
```

5. DRY in ASP.NET Core

A common example is validation.

✗ Repeated validation

```
public IActionResult CreateUser(User user)
{
    if (string.IsNullOrEmpty(user.Name))
        return BadRequest("Name is required");

    if (user.Age < 18)
        return BadRequest("User must be 18+");

    // ...
}
```

Another controller might repeat the same rules.

☑ Centralize validation

For example, with FluentValidation:

```
public class UserValidator : AbstractValidator<User>
{
    public UserValidator()
    {
        RuleFor(x => x.Name)
            .NotEmpty();

        RuleFor(x => x.Age)
            .GreaterThanOrEqualTo(18);
    }
}
```

Now the validation rules have one source of truth.

6. DRY with Extension Methods

Suppose you repeatedly convert strings:

```
name.Trim().ToLower()
```

You can create an extension method:

```
public static class StringExtensions
{
    public static string Normalize(this string value)
    {
        return value.Trim().ToLower();
    }
}
```

Usage:

```
string name = " JOHN ";

string normalized = name.Normalize();
```

7. DRY with Generic Methods

✗ Repeated code

```
public void SaveUser(User user)
{
    // save user
}

public void SaveProduct(Product product)
{
    // save product
}

public void SaveOrder(Order order)
{
    // save order
}
```

If the logic is genuinely identical, generic abstraction may help:

```
public void Save<T>(T entity)
{
    // common save logic
}
```

Usage:

```
Save(user);
Save(product);
Save(order);
```

However, **don't introduce generics just to eliminate a few lines**. The abstraction should represent genuinely common behavior.

8. DRY vs Copy-Paste

Consider:

```
public decimal CalculateOrderTotal(Order order)
{
    return order.Price + order.Tax;
}
```

And:

```
public decimal CalculateInvoiceTotal(Invoice invoice)
{
    return invoice.Price + invoice.Tax;
}
```

These look similar, but they might represent different business concepts.

Don't automatically combine them.

DRY is about eliminating **duplicated knowledge**, not necessarily identical-looking code.

This is important.

9. DRY Does NOT Mean "Never Repeat Code"

A common mistake is trying to eliminate every repeated line.

For example:

```
public void CreateUser()
{
    Console.WriteLine("Creating user...");
}
```

```
public void DeleteUser()
{
    Console.WriteLine("Deleting user...");
}
```

You don't need to create:

```
private void Print(string message)
{
    Console.WriteLine(message);
}
```

just because `Console.WriteLine()` appears twice.

That can make the code harder to understand.

Good DRY

Remove duplicated **business logic / knowledge**.

Bad DRY

Remove every tiny repeated syntax element.

10. DRY vs WET

You will sometimes hear:

WET = Write Everything Twice

Example:

```
var tax1 = amount * 0.18m;
```

and somewhere else:

```
var tax2 = amount * 0.18m;
```

If 18% represents the same business rule, that's a DRY violation.

Instead:

```
public static class TaxRules
{
    public const decimal GST_RATE = 0.18m;
}
```

Then:

```
var tax = amount * TaxRules.GST_RATE;
```

Now changing the rate happens in one place.

11. DRY in a Real .NET Project

A typical structure could be:

```
MyApplication
|
|— Controllers
```

```
├── OrderController.cs
├── Services
│   └── OrderService.cs
├── Validators
│   └── OrderValidator.cs
├── Repositories
│   └── OrderRepository.cs
├── Helpers
│   └── DateTimeHelper.cs
├── Constants
│   └── TaxRules.cs
└── Models
    └── Order.cs
```

The idea is to avoid putting the same business rules in multiple controllers/services.

12. DRY + SOLID

DRY works very well with **SOLID** principles.

For example:

```
DRY
├── Avoid duplicated business rules
├── Centralize common behavior
└── Maintain one source of truth
```

It often leads naturally to:

- **SRP** — put related responsibilities together.
- **OCP** — extend behavior without modifying duplicated code.
- **DIP** — depend on abstractions when sharing services.
- **Strategy Pattern** — replace duplicated conditional algorithms.
- **Template Method** — centralize common algorithm structure.
- **Decorator/Middleware** — centralize cross-cutting behavior.

13. Interview Example

Interviewer: What is DRY?

Good answer:

DRY stands for Don't Repeat Yourself. It is a software design principle that aims to avoid duplication of knowledge and business logic in a codebase. If the same business rule is implemented in multiple places, changing that rule becomes error-prone. I would usually extract the common behavior into a method, class, service, extension method, or another appropriate abstraction. However, I don't blindly eliminate all code duplication because excessive abstraction can make the code harder to understand.

Simple example

```
public class TaxCalculator
{
    public decimal Calculate(decimal amount)
    {
        return amount * 0.18m;
    }
}
```

Instead of implementing:

```
amount * 0.18m
```

in multiple services, they all use `TaxCalculator`.

Key point: DRY means **one source of truth for the same knowledge**, not simply "never write the same line twice."

- KISS

KISS Design Principle in C#

KISS stands for **Keep It Simple, Stupid** (sometimes phrased as *Keep It Simple and Straightforward*).

It is not a formal GoF design pattern. It is a **software design principle** that says:

Prefer the simplest solution that correctly solves the problem.

In C#, KISS means avoiding unnecessary abstractions, complicated logic, patterns, frameworks, and layers when a simpler implementation is sufficient.

1. Bad example — unnecessary complexity

Suppose you simply want to calculate the total price.

```
public interface IPriceCalculationStrategy
{
    decimal Calculate(decimal price, decimal tax);
}

public class PriceCalculationStrategy : IPriceCalculationStrategy
{
    public decimal Calculate(decimal price, decimal tax)
    {
        return price + (price * tax / 100);
    }
}

public class PriceCalculationService
{
    private readonly IPriceCalculationStrategy _strategy;

    public PriceCalculationService(IPriceCalculationStrategy strategy)
    {
        _strategy = strategy;
    }

    public decimal GetTotal(decimal price, decimal tax)
    {
        return _strategy.Calculate(price, tax);
    }
}
```

This works, but for a simple calculation it introduces:

- Interface
- Strategy
- Service
- Dependency injection
- Extra files

That's unnecessary complexity.

2. KISS version

```
public static decimal CalculateTotal(decimal price, decimal tax)
{
    return price + (price * tax / 100);
}
```

Usage:


```
decimal total = CalculateTotal(1000, 18);

Console.WriteLine(total);
```

Output:

```
1180
```

The second solution is easier to understand, test, and maintain.

3. KISS with conditional logic

✗ Overcomplicated

```
public enum CustomerType
{
    Regular,
    Premium
}

public class DiscountCalculator
{
    public decimal CalculateDiscount(
        decimal price,
        CustomerType customerType)
    {
        if (customerType == CustomerType.Regular)
        {
            return price * 0.05m;
        }
        else if (customerType == CustomerType.Premium)
        {
            return price * 0.10m;
        }

        return 0;
    }
}
```

Even this isn't terrible, but if the requirement is extremely simple, we can keep it straightforward.

☑ KISS

```
public decimal CalculateDiscount(decimal price, bool isPremium)
{
    return isPremium
        ? price * 0.10m
        : price * 0.05m;
}
```

4. KISS in ASP.NET Core

Suppose you need an API to get a product.

✗ Over-engineered

```
Controller
  ↓
Mediator
  ↓
Command
  ↓
Handler
  ↓
Service
  ↓
Factory
  ↓
Strategy
  ↓
Repository
  ↓
UnitOfWork
  ↓
Database
```

You *can* build this architecture, but don't automatically introduce every pattern.

For a small CRUD API, this may be unnecessary.

☑ Simple approach

```
[ApiController]
[Route("api/products")]
public class ProductsController : ControllerBase
{
    private readonly AppDbContext _context;

    public ProductsController(AppDbContext context)
```

```

    {
        _context = context;
    }

    [HttpGet("{id}")]
    public async Task<IActionResult> Get(int id)
    {
        var product = await _context.Products.FindAsync(id);

        if (product == null)
            return NotFound();

        return Ok(product);
    }
}

```

This may be completely appropriate for a small application.

5. KISS does NOT mean "never use design patterns"

This is important.

KISS does **not** mean:

"Always write the smallest amount of code."

It means:

Don't introduce complexity unless the complexity provides real value.

For example, if your application has multiple payment providers:

```

PaymentService
  ↓
IPaymentProvider
  ↙   ↘
Stripe Razorpay

```

Using Strategy/Dependency Injection may make sense.

```

public interface IPaymentProvider
{
    Task PayAsync(decimal amount);
}

public class StripePaymentProvider : IPaymentProvider
{
    public Task PayAsync(decimal amount)

```

```
        {
            // Stripe payment
            return Task.CompletedTask;
        }
    }

    public class RazorpayPaymentProvider : IPaymentProvider
    {
        public Task PayAsync(decimal amount)
        {
            // Razorpay payment
            return Task.CompletedTask;
        }
    }
}
```

Here the abstraction has a purpose because you have **multiple implementations**.

6. KISS vs over-engineering

KISS	Over-engineering
Simple solution	Complex solution
Easy to understand	Difficult to understand
Few abstractions	Many abstractions
Easy maintenance	More maintenance
Introduce patterns when needed	Use patterns everywhere
Minimal necessary code	Excessive code
Clear logic	Indirect logic

7. KISS with LINQ

✗ Unnecessarily complicated

```
List<int> result = new List<int>();

foreach (var number in numbers)
{
    if (number % 2 == 0)
    {
        result.Add(number);
    }
}
```

This isn't actually bad code, but if you're comfortable with LINQ:

☒ Simple

```
var result = numbers.Where(x => x % 2 == 0).ToList();
```

However, don't use LINQ merely to make code shorter if the resulting expression becomes difficult to understand.

8. KISS and exception handling

☒ Bad

```
try
{
    // operation
}
catch (Exception ex)
{
    if (ex != null)
    {
        Console.WriteLine(ex.Message);
    }
    else
    {
        Console.WriteLine("Unknown error");
    }
}
```

`catch` already gives you a non-null exception.

☒ KISS

```
try
{
    // operation
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

9. KISS and database queries

✗ Unnecessarily complicated

```
var products = await _context.Products
    .Where(x => x.IsActive == true)
    .Where(x => x.Price > 0)
    .Select(x => x)
    .ToListAsync();
```

☑ KISS

```
var products = await _context.Products
    .Where(x => x.IsActive && x.Price > 0)
    .ToListAsync();
```

10. KISS + YAGNI

KISS works very well with **YAGNI**:

You Aren't Gonna Need It

Don't build functionality just because you *might* need it someday.

For example:

```
public class UserService
{
    public void CreateUser(User user)
    {
        // Create user
    }
}
```

Don't immediately create:

```
IUserService
IUserFactory
IUserBuilder
IUserValidator
IUserRepository
IUserManager
```

```
IUserProvider  
IUserStrategy  
IUserCoordinator
```

unless your requirements actually justify them.

11. KISS + DRY

You recently asked about **DRY**, and these two principles are closely related.

DRY

Don't Repeat Yourself

Avoid duplicated knowledge/code.

KISS

Keep It Simple

Avoid unnecessary complexity.

For example:

```
public decimal CalculateTax(decimal amount)  
{  
    return amount * 0.18m;  
}
```

If you repeat this calculation everywhere, **DRY** tells you to centralize it.

But don't turn it into:

```
TaxCalculationFactory  
    ↓  
TaxCalculationStrategyFactory  
    ↓  
TaxCalculationStrategyResolver  
    ↓  
TaxCalculationProvider  
    ↓  
TaxCalculationService
```

just to avoid two lines of duplicated code.

DRY + KISS together help you find the right level of abstraction.

12. KISS in a real .NET project

A practical approach:

```
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

might be enough for a simple application.

For a more complex system:

```
API
  ↓
Application
  ↓
Domain
  ↓
Infrastructure
```

with CQRS, MediatR, Repository, Unit of Work, etc. can be justified.

The key question is:

Does this abstraction solve a real problem?

If yes → use it.

If no → keep it simple.

Interview definition

If an interviewer asks "**What is KISS?**", you can say:

KISS stands for Keep It Simple, Stupid. It is a software design principle that encourages developers to use the simplest solution that satisfies the requirements. In C#, this means avoiding unnecessary abstractions, design patterns, layers, and complicated logic. We should introduce complexity only when it provides a real benefit such as extensibility, maintainability, or separation of concerns.

One-line example


```
// KISS
return price * quantity;
```

instead of creating multiple classes and interfaces just to perform a simple multiplication.

- YAGNI

YAGNI Principle in C#

YAGNI stands for “**You Aren’t Gonna Need It.**”

It is a software design principle that says:

Don't implement functionality until you actually need it.

YAGNI is **not really a design pattern** like Strategy or Observer. It is a **design/development principle**, closely associated with Agile and Extreme Programming.

1. Without YAGNI

Suppose you need a simple `OrderService`:

```
public class OrderService
{
    public void CreateOrder(Order order)
    {
        // Create order
    }

    // "Maybe we'll need these later..."
    public void ExportToXml(Order order)
    {
    }

    public void ExportToCsv(Order order)
    {
    }

    public void ExportToExcel(Order order)
    {
    }

    public void SendOrderToKafka(Order order)
    {
    }

    public void SendOrderToRabbitMq(Order order)
    {
    }
}
```

The problem is that you're building features **before there is a requirement for them**.

2. Applying YAGNI

If the current requirement is only:

Create an order and save it.

Implement only that:

```
public class OrderService
{
    private readonly IOrderRepository _repository;

    public OrderService(IOrderRepository repository)
    {
        _repository = repository;
    }

    public async Task CreateOrderAsync(Order order)
    {
        await _repository.AddAsync(order);
    }
}
```

Don't add:

- Excel export
- CSV export
- Kafka
- RabbitMQ
- XML export
- caching
- notification system

until those requirements actually appear.

3. Another common example

Over-engineered

```
public interface IUserRepository
{
    User GetById(int id);

    IEnumerable<User> GetAll();
}
```

```
IEnumerable<User> Search(string name);

void ExportToCsv();

void ExportToExcel();

void ExportToXml();

void BackupUsers();

void RestoreUsers();
}
```

If your application currently only needs:

```
User GetById(int id);
```

then the extra methods violate the spirit of YAGNI.

Better

```
public interface IUserRepository
{
    User? GetById(int id);
}
```

Add additional functionality when a real requirement demands it.

4. YAGNI vs DRY

These principles are related but solve different problems.

Principle	Meaning
YAGNI	Don't build functionality you don't currently need
DRY	Don't duplicate knowledge/code
KISS	Keep the solution simple

For example:

```
// YAGNI problem
public void ExportToExcel()
{
}
```

```
// We don't need Excel export yet  
}
```

You're implementing unnecessary functionality.

DRY problem:

```
public void CreateUser()  
{  
    ValidateEmail();  
    ValidatePassword();  
}  
  
public void UpdateUser()  
{  
    ValidateEmail();  
    ValidatePassword();  
}
```

The validation logic might be duplicated.

KISS problem:

```
public string GetUserName(User user)  
{  
    return user?.Name ?? string.Empty;  
}
```

You shouldn't introduce unnecessary abstractions or complexity just to perform a simple operation.

5. YAGNI in ASP.NET Core

Suppose you are creating an API:

```
[ApiController]  
[Route("api/orders")]  
public class OrdersController : ControllerBase  
{  
    [HttpPost]  
    public IActionResult Create(Order order)  
    {  
        // Current requirement  
        return Ok();  
    }  
}
```

Don't immediately create:

```
OrderExportService
OrderNotificationService
OrderKafkaProducer
OrderRabbitMqConsumer
OrderCachingService
OrderAnalyticsService
OrderReportingService
OrderArchivalService
```

unless the application actually requires them.

Start with the simplest architecture that satisfies the current requirement.

When a requirement appears, introduce the appropriate abstraction.

6. YAGNI does NOT mean "never design for change"

This is important.

YAGNI doesn't mean:

"Write bad code because we only need it today."

Instead:

Implement today's requirements cleanly, but don't implement speculative future requirements.

For example, this is reasonable:

```
public interface IPaymentService
{
    Task ProcessPaymentAsync(decimal amount);
}
```

But this may be premature if you only have one payment provider:

```
public interface IPaymentService
{
    Task ProcessPaymentAsync(decimal amount);
}

public interface IPaymentProviderFactory
{
    IPaymentProvider Create(PaymentProviderType type);
}
```

```
public interface IPaymentProvider
{
    Task ProcessAsync(decimal amount);
}

public interface IPaymentStrategy
{
    // ...
}

public interface IPaymentAdapter
{
    // ...
}
```

If there is no actual requirement for multiple providers, you're potentially creating unnecessary complexity.

7. Real-world example

Imagine you're developing a fintech payment API.

Requirement

POST /api/payments

Currently:

```
Payment API
  ↓
PaymentService
  ↓
PaymentRepository
  ↓
Database
```

That's enough.

Don't automatically add:

```
Payment API
  ↓
BFF
  ↓
API Gateway
  ↓
Mediator
```

↓
CQRS
↓
Event Bus
↓
Saga
↓
Outbox
↓
Kafka
↓
Redis
↓
Microservices

Those patterns can be valuable, but **they should solve actual problems.**

This is particularly important in .NET applications because it's easy to create a large architecture containing Repository, Unit of Work, CQRS, Mediator, Specification, Factory, Strategy, etc., even when the application doesn't need them.

8. YAGNI with C# classes

Don't do this prematurely

```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; } = "";

    // Maybe needed someday
    public string TwitterHandle { get; set; } = "";

    // Maybe needed someday
    public string LinkedInProfile { get; set; } = "";

    // Maybe needed someday
    public string InstagramHandle { get; set; } = "";

    // Maybe needed someday
    public string BlockchainWallet { get; set; } = "";
}
```

If the current requirement is only:

```
public class Customer
{
    public int Id { get; set; }
```

```
    public string Name { get; set; } = "";  
}
```

keep it that way.

9. Benefits of YAGNI

Less code

Less code means fewer things to maintain.

Less complexity

Your architecture remains easier to understand.

Faster development

You spend time implementing actual requirements.

Fewer bugs

Unused functionality can still contain bugs.

Easier refactoring

You can introduce an abstraction when you understand the actual requirement.

10. YAGNI in one sentence

YAGNI = Don't write code for hypothetical future requirements; implement the simplest clean solution that satisfies the requirements you have now.

A good mindset for C# development is:

```
Requirement  
↓  
Simple implementation  
↓  
Real problem appears  
↓  
Refactor  
↓  
Introduce pattern/abstraction when justified
```

rather than:

```
Maybe we'll need it
↓
Add abstraction
↓
Add pattern
↓
Add framework
↓
Add complexity
↓
Never use it
```

Interview answer:

"YAGNI stands for You Aren't Gonna Need It. It is a software development principle that says we should not implement functionality based on assumptions about future requirements. In C#, I apply it by keeping classes, interfaces, abstractions, and architectural patterns focused on current requirements and introducing additional complexity only when there is a real need."

- Separation of Concerns

Separation of Concerns (SoC) in C#

Separation of Concerns (SoC) is a software design principle where you divide an application into **separate parts, with each part responsible for one specific concern**.

It is not a GoF design pattern like Factory or Strategy. It is a **fundamental design principle** used in many architectures and patterns.

Simple idea

Instead of putting everything into one class:

```
Controller
├── Validation
├── Business Logic
├── Database Access
├── Email
├── Logging
└── Payment
```

separate the responsibilities:

```
Controller
↓
Service
↓
Repository
```

↓
Database

Each layer has a clear responsibility.

1. Bad Example — No Separation of Concerns

Imagine an API that creates an order.

```
public class OrderController
{
    public string CreateOrder(Order order)
    {
        // Validate
        if (order.Amount <= 0)
            return "Invalid amount";

        // Business logic
        decimal discount = 0;

        if (order.Amount > 1000)
            discount = order.Amount * 0.10m;

        decimal finalAmount = order.Amount - discount;

        // Database
        using var connection = new SqlConnection("connection-string");

        connection.Open();

        var command = new SqlCommand(
            "INSERT INTO Orders (Amount) VALUES (@Amount)",
            connection);

        command.Parameters.AddWithValue("@Amount", finalAmount);

        command.ExecuteNonQuery();

        // Send email
        Console.WriteLine("Order confirmation email sent");

        // Logging
        Console.WriteLine("Order created");

        return "Success";
    }
}
```

This class is doing **too many things**:

- HTTP handling
- Validation
- Business logic
- Database access
- Email
- Logging

This violates Separation of Concerns.

2. Better Design

We can separate the responsibilities.

```
OrderController
  ↓
OrderService
  ↓
OrderRepository
  ↓
Database
```

And separately:

```
OrderService
├── EmailService
└── Logger
```

Controller

The controller should primarily deal with HTTP/API concerns.

```
[ApiController]
[Route("api/orders")]
public class OrderController : ControllerBase
{
    private readonly IOrderService _orderService;

    public OrderController(IOrderService orderService)
    {
        _orderService = orderService;
    }
}
```

```
[HttpPost]
public IActionResult Create(Order order)
{
    var result = _orderService.CreateOrder(order);

    return Ok(result);
}
```

The controller doesn't know:

- SQL
- discount calculation
- email implementation
- database connection

3. Service Layer

The service contains the **business logic**.

```
public interface IOrderService
{
    string CreateOrder(Order order);
}
```

Implementation:

```
public class OrderService : IOrderService
{
    private readonly IOrderRepository _repository;
    private readonly IEmailService _emailService;

    public OrderService(
        IOrderRepository repository,
        IEmailService emailService)
    {
        _repository = repository;
        _emailService = emailService;
    }

    public string CreateOrder(Order order)
    {
        if (order.Amount <= 0)
            return "Invalid amount";

        decimal discount = 0;
```

```
        if (order.Amount > 1000)
            discount = order.Amount * 0.10m;

        var finalAmount = order.Amount - discount;

        _repository.Save(finalAmount);

        _emailService.Send(
            "customer@example.com",
            "Order created");

        return "Success";
    }
}
```

Now the service focuses on **order business logic**.

4. Repository

The repository handles database access.

```
public interface IOrderRepository
{
    void Save(decimal amount);
}
```

Implementation:

```
public class OrderRepository : IOrderRepository
{
    public void Save(decimal amount)
    {
        using var connection =
            new SqlConnection("connection-string");

        connection.Open();

        var command = new SqlCommand(
            "INSERT INTO Orders (Amount) VALUES (@Amount)",
            connection);

        command.Parameters.AddWithValue("@Amount", amount);

        command.ExecuteNonQuery();
    }
}
```

The repository's concern is:

How do I persist orders?

It doesn't care about HTTP or discounts.

5. Email Service

Email has its own responsibility.

```
public interface IEmailService
{
    void Send(string email, string message);
}
```

```
public class EmailService : IEmailService
{
    public void Send(string email, string message)
    {
        Console.WriteLine(
            $"Sending email to {email}: {message}");
    }
}
```

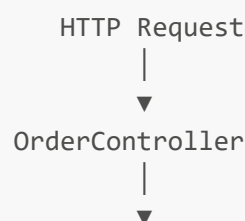
Now email logic is isolated.

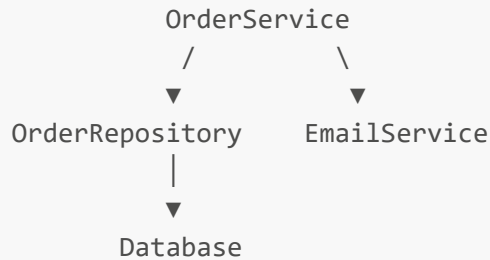
6. Dependency Injection

ASP.NET Core can connect these components.

```
builder.Services.AddScoped<IOrderService, OrderService>();
builder.Services.AddScoped<IOrderRepository, OrderRepository>();
builder.Services.AddScoped<IEmailService, EmailService>();
```

The architecture becomes:





Each component has a different concern.

7. Why Separation of Concerns Is Useful

1. Easier testing

You can test `OrderService` without a real database.

```
var repository = new Mock<IOrderRepository>();
var email = new Mock<IEmailService>();

var service = new OrderService(
    repository.Object,
    email.Object);
```

2. Easier maintenance

Suppose you change SQL Server to PostgreSQL.

You mainly change the repository implementation.

Before:

```
OrderService
  ↓
SqlOrderRepository
  ↓
SQL Server
```

Later:

```
OrderService
  ↓
PostgresOrderRepository
  ↓
PostgreSQL
```

The business logic can remain unchanged.

3. Less coupling

Bad:

```
OrderService
  ↓
SqlConnection
```

Better:

```
OrderService
  ↓
IOrderRepository
  ↓
SqlOrderRepository
```

The service depends on an abstraction rather than a specific database implementation.

4. Easier team development

Different developers can work on different concerns:

```
Developer A → Controller/API
Developer B → Business Logic
Developer C → Database
Developer D → Integration/Email
```

8. Separation of Concerns vs Single Responsibility Principle

They are closely related but not identical.

Separation of Concerns

Says:

Separate different kinds of responsibilities from each other.

Single Responsibility Principle

Says:

A class should have one reason to change.

For example:

```
OrderController
```

should deal with HTTP/API concerns.

```
OrderService
```

should deal with order business rules.

```
OrderRepository
```

should deal with persistence.

SoC is broader, while SRP is commonly applied at the **class/module level**.

9. Where You See SoC in ASP.NET Core

A typical ASP.NET Core application might use:

```
MyApp
├── Controllers
│   └── OrderController.cs
├── Services
│   └── OrderService.cs
├── Repositories
│   └── OrderRepository.cs
├── Models
│   └── Order.cs
├── DTOs
│   └── CreateOrderRequest.cs
└── Validators
```

```
├── OrderValidator.cs
├── Middleware
│   └── ExceptionMiddleware.cs
├── Infrastructure
│   └── Database.cs
```

This is Separation of Concerns in practice.

10. Important Interview Point

If an interviewer asks:

What is Separation of Concerns?

A good answer is:

Separation of Concerns is a software design principle where an application is divided into independent components, with each component handling a specific responsibility. This reduces coupling, improves maintainability and testability, and makes the system easier to change. In ASP.NET Core, controllers can handle HTTP concerns, services handle business logic, repositories handle data access, and middleware handles cross-cutting concerns.

One-line memory trick

One concern → One responsibility → Less coupling → Easier maintenance

Common patterns that help implement SoC: Dependency Injection, Repository, Service Layer, Middleware, MVC, CQRS, and Clean/Onion Architecture.

- Composition over Inheritance

Composition Over Inheritance in C#

Composition over inheritance is a design principle that says:

Prefer building classes by **combining smaller objects** rather than creating deep inheritance hierarchies.

Instead of saying:

"Car is a Vehicle"

and putting lots of behavior into the inheritance hierarchy, we often say:

"Car has a Engine, BrakeSystem, Transmission, etc."

1. The problem with inheritance

Suppose we have:

```
public class Animal
{
    public void Eat()
    {
        Console.WriteLine("Eating");
    }

    public void Walk()
    {
        Console.WriteLine("Walking");
    }
}

public class Dog : Animal
{
    public void Bark()
    {
        Console.WriteLine("Barking");
    }
}

public class Bird : Animal
{
    public void Fly()
    {
        Console.WriteLine("Flying");
    }
}
```

This looks fine initially.

But what happens when we add:

```
Dog
Bird
Fish
Cat
Penguin
Ostrich
```

Not every animal can fly, walk, or behave in the same way.

Inheritance can lead to:

- Large base classes

- Deep inheritance hierarchies
- Tight coupling
- Difficult testing
- Unwanted behavior inherited by subclasses
- Changes in the base class affecting many subclasses

2. Composition approach

Instead of putting behaviors into the inheritance hierarchy, extract behaviors into separate classes/interfaces.

For example:

```
public interface IFlyBehavior
{
    void Fly();
}

public interface IWalkBehavior
{
    void Walk();
}

public class CanFly : IFlyBehavior
{
    public void Fly()
    {
        Console.WriteLine("Flying");
    }
}

public class CannotFly : IFlyBehavior
{
    public void Fly()
    {
        Console.WriteLine("Cannot fly");
    }
}

public class CanWalk : IWalkBehavior
{
    public void Walk()
    {
        Console.WriteLine("Walking");
    }
}

public class CannotWalk : IWalkBehavior
{
    public void Walk()
    {

```

```
        Console.WriteLine("Cannot walk");
    }
}
```

Now the `Animal` class **composes** these behaviors:

```
public class Animal
{
    private readonly IFlyBehavior _flyBehavior;
    private readonly IWalkBehavior _walkBehavior;

    public Animal(
        IFlyBehavior flyBehavior,
        IWalkBehavior walkBehavior)
    {
        _flyBehavior = flyBehavior;
        _walkBehavior = walkBehavior;
    }

    public void Fly()
    {
        _flyBehavior.Fly();
    }

    public void Walk()
    {
        _walkBehavior.Walk();
    }
}
```

Create different animals by combining behaviors:

```
var eagle = new Animal(
    new CanFly(),
    new CanWalk());

var penguin = new Animal(
    new CannotFly(),
    new CanWalk());

eagle.Fly();
eagle.Walk();

penguin.Fly();
penguin.Walk();
```

The important point is that **behavior is composed rather than inherited**.

3. Real-world example: Payment system

This principle is especially useful in backend applications.

Suppose you have:

```
PaymentService
├── CreditCardPayment
├── UpiPayment
├── PayPalPayment
└── BankTransferPayment
```

You might initially use inheritance:

```
public abstract class Payment
{
    public abstract void Pay();
}

public class CreditCardPayment : Payment
{
    public override void Pay()
    {
        Console.WriteLine("Credit card payment");
    }
}

public class UpiPayment : Payment
{
    public override void Pay()
    {
        Console.WriteLine("UPI payment");
    }
}
```

This works, but payment behavior can become more complicated.

For example:

```
Payment
├── PaymentMethod
├── PaymentValidation
├── FraudDetection
├── Notification
├── Retry
└── Logging
```

Instead, compose these capabilities.

```
public interface IPaymentMethod
{
    void Pay(decimal amount);
}

public interface IFraudChecker
{
    bool IsFraud(decimal amount);
}

public interface INotifier
{
    void Notify(string message);
}
```

Implement them:

```
public class UpiPayment : IPaymentMethod
{
    public void Pay(decimal amount)
    {
        Console.WriteLine($"Paid ₹{amount} using UPI");
    }
}

public class CreditCardPayment : IPaymentMethod
{
    public void Pay(decimal amount)
    {
        Console.WriteLine($"Paid ₹{amount} using Credit Card");
    }
}

public class FraudChecker : IFraudChecker
{
    public bool IsFraud(decimal amount)
    {
        return amount > 100000;
    }
}

public class EmailNotifier : INotifier
{
    public void Notify(string message)
    {
        Console.WriteLine($"Email: {message}");
    }
}
```

Now compose them:

```
public class PaymentService
{
    private readonly IPaymentMethod _paymentMethod;
    private readonly IFraudChecker _fraudChecker;
    private readonly INotifier _notifier;

    public PaymentService(
        IPaymentMethod paymentMethod,
        IFraudChecker fraudChecker,
        INotifier notifier)
    {
        _paymentMethod = paymentMethod;
        _fraudChecker = fraudChecker;
        _notifier = notifier;
    }

    public void Process(decimal amount)
    {
        if (_fraudChecker.IsFraud(amount))
        {
            Console.WriteLine("Fraud detected");
            return;
        }

        _paymentMethod.Pay(amount);

        _notifier.Notify(
            $"Payment of ₹{amount} successful");
    }
}
```

Usage:

```
var paymentService = new PaymentService(
    new UpiPayment(),
    new FraudChecker(),
    new EmailNotifier());

paymentService.Process(5000);
```

Here `PaymentService` doesn't inherit from `UPIPayment`, `CreditCardPayment`, etc.

It **has**:


```
PaymentService
|
+-- IPaymentMethod
|
+-- IFraudChecker
|
+-- INotifier
```

That's composition.

4. Composition + Dependency Injection

In ASP.NET Core, composition is commonly combined with **Dependency Injection**.

```
builder.Services.AddScoped<IPaymentMethod, UpiPayment>();
builder.Services.AddScoped<IFraudChecker, FraudChecker>();
builder.Services.AddScoped<INotifier, EmailNotifier>();

builder.Services.AddScoped<PaymentService>();
```

Then:

```
public class PaymentService
{
    private readonly IPaymentMethod _paymentMethod;
    private readonly IFraudChecker _fraudChecker;
    private readonly INotifier _notifier;

    public PaymentService(
        IPaymentMethod paymentMethod,
        IFraudChecker fraudChecker,
        INotifier notifier)
    {
        _paymentMethod = paymentMethod;
        _fraudChecker = fraudChecker;
        _notifier = notifier;
    }
}
```

ASP.NET Core's DI container performs the composition for you.

5. Composition vs Inheritance

Inheritance	Composition
is-a relationship	has-a relationship
Tight coupling	Looser coupling
Behavior inherited	Behavior injected/composed
Often harder to change	Easier to change
Can create deep hierarchies	Usually flatter structure
Compile-time relationship	Can be configured at runtime
Good for genuine subtype relationships	Good for interchangeable behaviors

Inheritance

```
public class Dog : Animal
{
}
```

Means:

Dog IS-A Animal

Composition

```
public class Car
{
    private readonly IEngine _engine;
}
```

Means:

Car HAS-A Engine

6. Another simple example

Bad inheritance design

```
public class Bird
{
    public virtual void Fly()
    {
        Console.WriteLine("Flying");
    }
}

public class Penguin : Bird
{
    public override void Fly()
    {
        throw new NotSupportedException();
    }
}
```

A penguin shouldn't inherit a capability it cannot perform.

Composition

```
public interface IMovement
{
    void Move();
}

public class FlyMovement : IMovement
{
    public void Move()
    {
        Console.WriteLine("Flying");
    }
}

public class SwimMovement : IMovement
{
    public void Move()
    {
        Console.WriteLine("Swimming");
    }
}
```

Now:

```
public class Penguin
{
    private readonly IMovement _movement;

    public Penguin(IMovement movement)
```

```

    {
        _movement = movement;
    }

    public void Move()
    {
        _movement.Move();
    }
}

```

```

var penguin = new Penguin(new SwimMovement());

penguin.Move();

```

No unnecessary inheritance.

7. When should you use composition?

Composition is particularly useful when:

Behavior changes frequently

```

Payment
├── UPI
├── Credit Card
└── PayPal

```

and you want to replace implementations easily.

Objects need combinations of behaviors

For example:

```

Robot
├── CanWalk
├── CanFly
├── CanShoot
└── CanScan

```

Different robots can combine these capabilities.

You want easier unit testing

You can inject mocks:

```
var payment = new Mock<IPaymentMethod>();  
var fraud = new Mock<IFraudChecker>();  
var notifier = new Mock<INotifier>();
```

Then test `PaymentService` independently.

8. Important interview point

Composition over inheritance does NOT mean "never use inheritance."

Inheritance is appropriate when there is a genuine and stable **is-a** relationship.

For example:

```
public abstract class Animal  
{  
    public string Name { get; set; }  
}  
  
public class Dog : Animal  
{  
}
```

This can be perfectly reasonable.

The principle means:

Don't use inheritance merely to reuse code or behavior when composition provides a more flexible design.

Easy way to remember

```
Inheritance  
  ↓  
"IS-A"  
  
Dog IS-A Animal  
Car IS-A Vehicle  
  
Composition  
  ↓  
"HAS-A"
```

```
Car HAS-A Engine
Order HAS-A PaymentProcessor
PaymentService HAS-A FraudChecker
```

Typical C# implementation

```
public class Service
{
    private readonly IRepository _repository;
    private readonly ILogger _logger;
    private readonly IValidator _validator;

    public Service(
        IRepository repository,
        ILogger logger,
        IValidator validator)
    {
        _repository = repository;
        _logger = logger;
        _validator = validator;
    }
}
```

This is **composition + dependency injection** and is one of the most common forms of composition you'll encounter in real-world ASP.NET Core applications.

Rule of thumb:

Use inheritance for true, stable "is-a" relationships; use composition when you want to assemble or replace behaviors.

- Program to an Interface

Program to an Interface in C#

Program to an Interface is a design principle where your code depends on an **interface (abstraction)** rather than a concrete implementation.

It is closely related to the **Dependency Inversion Principle (DIP)** and is heavily used with **Dependency Injection**.

1. Without programming to an interface

Here the `OrderService` directly depends on `EmailNotification`.

```
public class EmailNotification
{
    public void Send(string message)
```

```
    {
        Console.WriteLine($"Email sent: {message}");
    }
}

public class OrderService
{
    private readonly EmailNotification _notification;

    public OrderService()
    {
        _notification = new EmailNotification();
    }

    public void PlaceOrder()
    {
        // Place order logic

        _notification.Send("Order placed successfully");
    }
}
```

Problem

`OrderService` is tightly coupled to `EmailNotification`.

If tomorrow you want to send an SMS:

```
EmailNotification
    ↑
OrderService
```

You have to modify `OrderService`.

2. Program to an Interface

Create an abstraction:

```
public interface INotification
{
    void Send(string message);
}
```

Implement it using email:

```
public class EmailNotification : INotification
{
    public void Send(string message)
    {
        Console.WriteLine($"Email sent: {message}");
    }
}
```

Implement another notification:

```
public class SmsNotification : INotification
{
    public void Send(string message)
    {
        Console.WriteLine($"SMS sent: {message}");
    }
}
```

Now **OrderService** depends on the interface:

```
public class OrderService
{
    private readonly INotification _notification;

    public OrderService(INotification notification)
    {
        _notification = notification;
    }

    public void PlaceOrder()
    {
        // Place order

        _notification.Send("Order placed successfully");
    }
}
```

Usage:

```
INotification notification = new EmailNotification();

OrderService orderService = new OrderService(notification);

orderService.PlaceOrder();
```


Output:

```
Email sent: Order placed successfully
```

We can easily switch to SMS:

```
INotification notification = new SmsNotification();

OrderService orderService = new OrderService(notification);

orderService.PlaceOrder();
```

Output:

```
SMS sent: Order placed successfully
```

The important part is that **OrderService** **doesn't know or care which notification implementation it receives.**

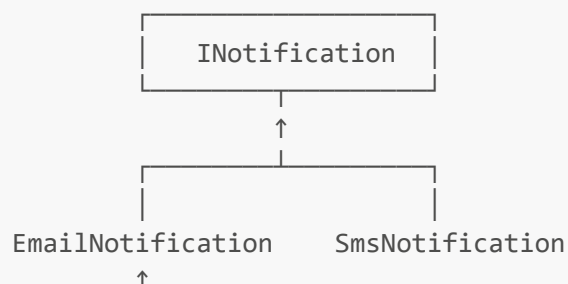
3. Visual representation

Without interface

```
OrderService
  |
  ↓
EmailNotification
```

Strong coupling.

With interface




OrderService

`OrderService` depends on `INotification`, not on a specific notification class.

4. Real ASP.NET Core example

This principle is very common in ASP.NET Core.

Interface:

```
public interface IPaymentService
{
    Task<bool> ProcessPaymentAsync(decimal amount);
}
```

Implementation:

```
public class RazorpayPaymentService : IPaymentService
{
    public async Task<bool> ProcessPaymentAsync(decimal amount)
    {
        Console.WriteLine($"Processing Razorpay payment: {amount}");

        return true;
    }
}
```

Controller:

```
[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly IPaymentService _paymentService;

    public OrdersController(IPaymentService paymentService)
    {
        _paymentService = paymentService;
    }

    [HttpPost]
    public async Task<IActionResult> CreateOrder(decimal amount)
    {
        bool result = await _paymentService.ProcessPaymentAsync(amount);
    }
}
```

```
        if (!result)
            return BadRequest("Payment failed");

        return Ok("Order created");
    }
}
```

Register the implementation:

```
builder.Services.AddScoped<IPaymentService, RazorpayPaymentService>();
```

Now ASP.NET Core automatically injects `RazorpayPaymentService` whenever something asks for `IPaymentService`.

5. Why use it?

Loose coupling

Instead of:

```
OrderService → EmailNotification
```

you have:

```
OrderService → INotification ← EmailNotification
```

Easy testing

You can create a fake implementation:

```
public class FakeNotification : INotification
{
    public void Send(string message)
    {
        Console.WriteLine("Fake notification");
    }
}
```

Test:

```
var notification = new FakeNotification();

var service = new OrderService(notification);

service.PlaceOrder();
```

You don't need to send a real email during testing.

Easy replacement

Today:

```
IPaymentService
    ↓
RazorpayPaymentService
```

Tomorrow:

```
IPaymentService
    ↓
StripePaymentService
```

`OrderService` doesn't need to change.

Key idea to remember

Program to an interface, not an implementation.

Prefer:

```
private readonly IPaymentService _paymentService;
```

over:

```
private readonly RazorpayPaymentService _paymentService;
```

And prefer:

```
public OrderService(IPaymentService paymentService)
```

over:

```
public OrderService(RazorpayPaymentService paymentService)
```

Relationship with other principles

```
Program to an Interface
  ↓
Dependency Inversion
  ↓
Dependency Injection
  ↓
Loose Coupling
  ↓
Better Testing + Maintainability
```

Interview definition: Programming to an interface means designing classes to depend on abstractions rather than concrete implementations, allowing implementations to be replaced without changing the dependent class.

3. Structural Design Patterns

Patterns for **combining classes and objects**.

1. Adapter

Adapter Design Pattern in C#

The **Adapter Design Pattern** is a **structural design pattern** that allows two incompatible interfaces to work together.

Think of it like a **mobile charger adapter**:

Your wall socket provides one type of connection, but your phone expects another. The adapter sits between them and converts one interface into the other.

In software:

Adapter = converts the interface of an existing class into an interface that your application expects.

1. Problem Without Adapter

Suppose your application expects:

```
public interface IPaymentProcessor
{
    void Pay(decimal amount);
}
```

Your application uses it like this:

```
public class PaymentService
{
    private readonly IPaymentProcessor _processor;

    public PaymentService(IPaymentProcessor processor)
    {
        _processor = processor;
    }

    public void MakePayment(decimal amount)
    {
        _processor.Pay(amount);
    }
}
```

Now imagine you have an old third-party payment library:

```
public class LegacyPaymentSystem
{
    public void MakePayment(double amount)
    {
        Console.WriteLine($"Payment processed: {amount}");
    }
}
```

The problem is:

Application expects:

IPaymentProcessor

|
| Pay(decimal)
↓

LegacyPaymentSystem

|
| MakePayment(double)
↓

✗ Interfaces don't match

You cannot directly use `LegacyPaymentSystem` where `IPaymentProcessor` is expected.

2. Adapter Solution

Create an adapter:

```
public class PaymentAdapter : IPaymentProcessor
{
    private readonly LegacyPaymentSystem _legacyPayment;

    public PaymentAdapter(LegacyPaymentSystem legacyPayment)
    {
        _legacyPayment = legacyPayment;
    }

    public void Pay(decimal amount)
    {
        _legacyPayment.MakePayment((double)amount);
    }
}
```

Now everything works:

```
var legacyPayment = new LegacyPaymentSystem();

IPaymentProcessor processor =
    new PaymentAdapter(legacyPayment);

var paymentService = new PaymentService(processor);

paymentService.MakePayment(1000);
```

Output:

```
Payment processed: 1000
```

3. Structure

The basic structure is:



Components

Component	Purpose
Target	Interface expected by your application
Adapter	Converts Target calls into Adaptee calls
Adaptee	Existing incompatible class
Client	Uses the Target interface

4. Complete Example

```
// Target
public interface IPaymentProcessor
{
    void Pay(decimal amount);
}
```

Existing third-party/legacy class:

```
// Adaptee
public class RazorPayLegacy
{
    public void ProcessPayment(double amount)
    {
        Console.WriteLine($"RazorPay processed ₹{amount}");
    }
}
```


Adapter:

```
// Adapter
public class RazorPayAdapter : IPaymentProcessor
{
    private readonly RazorPayLegacy _razorPay;

    public RazorPayAdapter(RazorPayLegacy razorPay)
    {
        _razorPay = razorPay;
    }

    public void Pay(decimal amount)
    {
        _razorPay.ProcessPayment((double)amount);
    }
}
```

Client:

```
public class OrderService
{
    private readonly IPaymentProcessor _paymentProcessor;

    public OrderService(IPaymentProcessor paymentProcessor)
    {
        _paymentProcessor = paymentProcessor;
    }

    public void PlaceOrder(decimal amount)
    {
        Console.WriteLine("Order created");

        _paymentProcessor.Pay(amount);

        Console.WriteLine("Order completed");
    }
}
```

Usage:

```
var razorPay = new RazorPayLegacy();

var adapter = new RazorPayAdapter(razorPay);

var orderService = new OrderService(adapter);
```

```
orderService.PlaceOrder(5000);
```

Output:

```
Order created  
RazorPay processed ₹5000  
Order completed
```

5. Why Do We Need Adapter?

Suppose your application has:

```
IPaymentProcessor
```

but you receive different payment libraries:

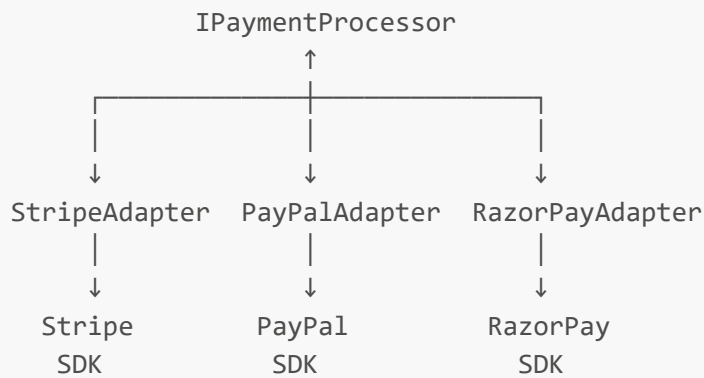
```
Stripe  
PayPal  
RazorPay  
OldPaymentSystem  
BankPaymentSDK
```

Each may have completely different APIs:

```
Stripe  
    Charge()  
  
PayPal  
    ExecutePayment()  
  
RazorPay  
    ProcessPayment()  
  
LegacyBank  
    TransferMoney()
```

You don't want your business logic to know about all these APIs.

Instead:



Your application only knows:

IPaymentProcessor

6. Multiple Adapters

You can create different adapters for different systems.

Stripe

```

public class StripeApi
{
    public void ChargeCustomer(double amount)
    {
        Console.WriteLine($"Stripe charged {amount}");
    }
}
  
```

Adapter:

```

public class StripeAdapter : IPaymentProcessor
{
    private readonly StripeApi _stripe;

    public StripeAdapter(StripeApi stripe)
    {
        _stripe = stripe;
    }

    public void Pay(decimal amount)
    {
        _stripe.ChargeCustomer((double)amount);
    }
}
  
```

```
}
}
```

PayPal

```
public class PayPalApi
{
    public void ExecutePayment(decimal amount)
    {
        Console.WriteLine($"PayPal payment: {amount}");
    }
}
```

Adapter:

```
public class PayPalAdapter : IPaymentProcessor
{
    private readonly PayPalApi _paypal;

    public PayPalAdapter(PayPalApi paypal)
    {
        _paypal = paypal;
    }

    public void Pay(decimal amount)
    {
        _paypal.ExecutePayment(amount);
    }
}
```

Now the business logic doesn't change:

```
IPaymentProcessor processor =
    new StripeAdapter(new StripeApi());

var service = new OrderService(processor);

service.PlaceOrder(1000);
```

Or:

```
IPaymentProcessor processor =
    new PayPalAdapter(new PayPalApi());

var service = new OrderService(processor);
```

```
service.PlaceOrder(1000);
```

7. Adapter with Dependency Injection in ASP.NET Core

This is where the pattern becomes particularly useful in real .NET applications.

Suppose:

```
public interface IPaymentProcessor
{
    Task PayAsync(decimal amount);
}
```

Third-party API:

```
public class LegacyPaymentApi
{
    public Task ProcessAsync(double amount)
    {
        Console.WriteLine($"Processing {amount}");
        return Task.CompletedTask;
    }
}
```

Adapter:

```
public class PaymentAdapter : IPaymentProcessor
{
    private readonly LegacyPaymentApi _api;

    public PaymentAdapter(LegacyPaymentApi api)
    {
        _api = api;
    }

    public async Task PayAsync(decimal amount)
    {
        await _api.ProcessAsync((double)amount);
    }
}
```

Register:

```
builder.Services.AddScoped<LegacyPaymentApi>();

builder.Services.AddScoped<IPaymentProcessor, PaymentAdapter>();
```

Controller:

```
[ApiController]
[Route("api/orders")]
public class OrderController : ControllerBase
{
    private readonly IPaymentProcessor _paymentProcessor;

    public OrderController(IPaymentProcessor paymentProcessor)
    {
        _paymentProcessor = paymentProcessor;
    }

    [HttpPost]
    public async Task<IActionResult> CreateOrder(decimal amount)
    {
        await _paymentProcessor.PayAsync(amount);

        return Ok("Payment successful");
    }
}
```

The controller doesn't know that the actual implementation is a legacy payment API.

8. Object Adapter vs Class Adapter

There are two common approaches.

Object Adapter

This is the most common approach in C#.

```
Adapter
|
| contains
↓
Adaptee
```

Example:

```
public class PaymentAdapter : IPaymentProcessor
{
    private readonly LegacyPaymentSystem _legacy;

    public PaymentAdapter(LegacyPaymentSystem legacy)
    {
        _legacy = legacy;
    }

    public void Pay(decimal amount)
    {
        _legacy.MakePayment((double)amount);
    }
}
```

This uses **composition**.

Class Adapter

A class adapter uses inheritance.

For example:

```
public class PaymentAdapter
    : LegacyPaymentSystem, IPaymentProcessor
{
    public void Pay(decimal amount)
    {
        MakePayment((double)amount);
    }
}
```

Usage:

```
IPaymentProcessor payment =
    new PaymentAdapter();

payment.Pay(500);
```

However, object adapters are generally preferred in C# because **composition is more flexible than inheritance**.

9. Adapter vs Decorator

These patterns are commonly confused.

Adapter

Changes the **interface**.

```
A interface
  ↓
Adapter
  ↓
B interface
```

Example:

```
MakePayment()
  ↓
Adapter
  ↓
ProcessPayment()
```

Decorator

Keeps the same interface but adds behavior.

```
IPaymentProcessor
  ↓
LoggingDecorator
  ↓
IPaymentProcessor
```

For example:

```
public class LoggingPaymentDecorator : IPaymentProcessor
{
    private readonly IPaymentProcessor _processor;

    public LoggingPaymentDecorator(IPaymentProcessor processor)
    {
        _processor = processor;
    }

    public void Pay(decimal amount)
    {
        Console.WriteLine("Payment started");

        _processor.Pay(amount);
    }
}
```



```
        Console.WriteLine("Payment completed");  
    }  
}
```

So:

Adapter changes interface.

Decorator changes/adds behavior.

10. Adapter vs Facade

Another common interview question.

Adapter

Makes **incompatible interfaces compatible**.

```
Client  
  ↓  
Adapter  
  ↓  
Existing system
```

Facade

Provides a **simplified interface to a complicated system**.

```
      Facade  
      ↓  
  ┌───┴───┐  
  ↓   ↓   ↓  
Service1 Service2 Service3
```

For example:

```
public class OrderFacade  
{  
    public void PlaceOrder()  
    {  
        ValidateOrder();  
        ProcessPayment();  
        UpdateInventory();  
        SendEmail();  
    }  
}
```

```
}  
}
```

The facade simplifies multiple operations.

11. Real-World .NET Examples

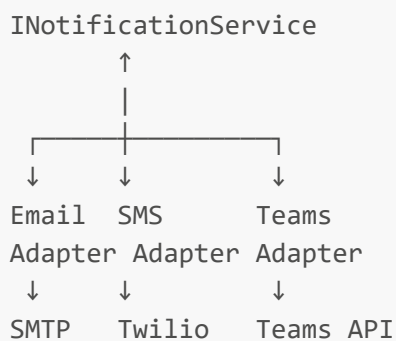
You will encounter adapter-like designs when integrating:

- Payment gateways
- External REST APIs
- Legacy systems
- Third-party SDKs
- Cloud services
- Logging libraries
- Message brokers
- Storage providers
- Authentication providers
- Database providers
- Old enterprise systems

For example, your application can define:

```
public interface INotificationService  
{  
    Task SendAsync(string message);  
}
```

Then adapters can wrap different systems:



Your business logic remains independent of the external APIs.

12. Key Advantages

1. Reuse existing code

You don't need to rewrite the old class.

2. Supports third-party libraries

You can adapt external SDKs to your application's interfaces.

3. Reduces coupling

Your business logic doesn't directly depend on external APIs.

4. Easier testing

You can mock your own interface:

```
var mock = new Mock<IPaymentProcessor>();
```

5. Follows Open/Closed Principle

You can add another adapter without changing existing business logic.

13. Disadvantages

Adapter can introduce:

- Additional classes
- Additional abstraction
- More code
- Potentially complicated mappings
- Conversion overhead in some cases

Don't create an adapter just because you can. Use it when there is a real **interface incompatibility** or an architectural boundary worth isolating.

14. Interview Definition

A good interview answer:

Adapter is a structural design pattern that allows incompatible interfaces to work together. It wraps an existing class and translates the interface expected by the client into the interface provided by the existing class. In C#, it is commonly implemented using composition.

Remember this formula:

```
Client
  ↓
Target Interface
  ↓
Adapter
  ↓
Existing/Legacy Class
```

Main purpose:

Convert one interface into another interface that the client expects.

2. Bridge

Bridge Design Pattern in C#

The **Bridge Design Pattern** is a **structural design pattern** that separates an abstraction from its implementation so that both can evolve independently.

Instead of creating many subclasses for every combination, Bridge uses **composition** instead of inheritance.

Problem Without Bridge

Suppose you have:

- Shapes:
 - Circle
 - Rectangle
- Colors:
 - Red
 - Blue

Using inheritance:

```
Shape
├── RedCircle
├── BlueCircle
├── RedRectangle
└── BlueRectangle
```

If more colors are added:

```
GreenCircle
GreenRectangle
BlackCircle
BlackRectangle
```

The number of classes grows rapidly.

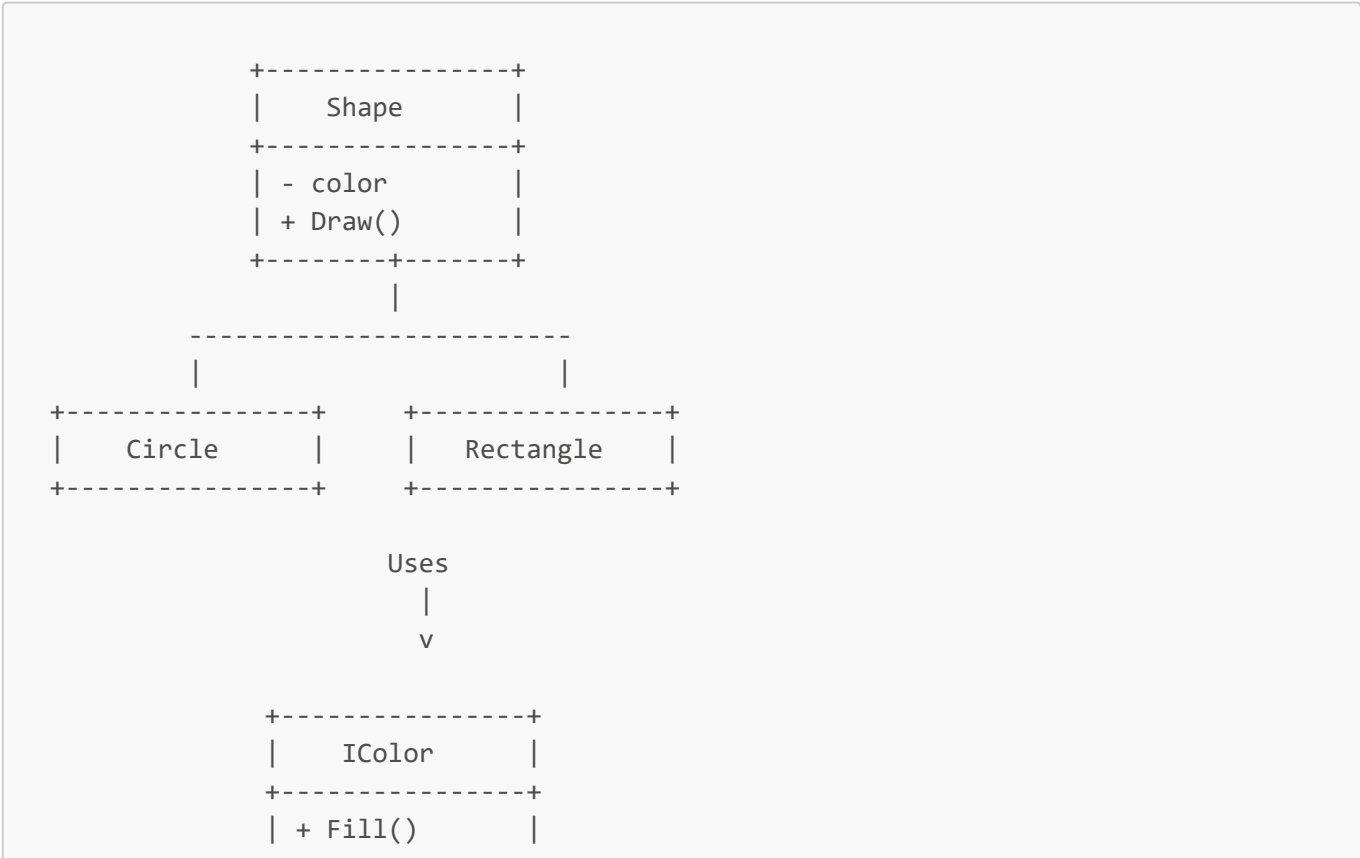
Bridge solves this by separating:

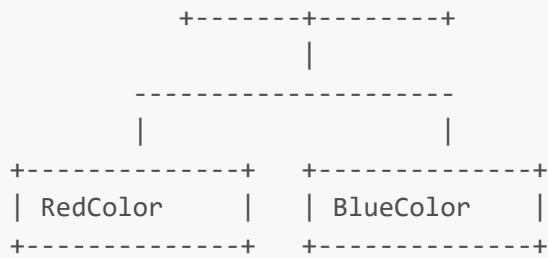
```
Shape -----> Color
```

Structure

```
Abstraction
|
RefinedAbstraction
|
+----> Implementor
        |
        ConcreteImplementor
```

UML Diagram





C# Example

Step 1: Implementor Interface

```

public interface IColor
{
    string Fill();
}
  
```

Step 2: Concrete Implementors

```

public class RedColor : IColor
{
    public string Fill()
    {
        return "Red Color";
    }
}

public class BlueColor : IColor
{
    public string Fill()
    {
        return "Blue Color";
    }
}
  
```

Step 3: Abstraction

```

public abstract class Shape
{
    protected IColor color;

    public Shape(IColor color)
    {
  
```

```
        this.color = color;
    }

    public abstract void Draw();
}
```

Step 4: Refined Abstractions

```
public class Circle : Shape
{
    public Circle(IColor color) : base(color)
    {
    }

    public override void Draw()
    {
        Console.WriteLine($"Drawing Circle with {color.Fill()}");
    }
}

public class Rectangle : Shape
{
    public Rectangle(IColor color) : base(color)
    {
    }

    public override void Draw()
    {
        Console.WriteLine($"Drawing Rectangle with {color.Fill()}");
    }
}
```

Step 5: Client Code

```
class Program
{
    static void Main()
    {
        Shape redCircle = new Circle(new RedColor());
        redCircle.Draw();

        Shape blueCircle = new Circle(new BlueColor());
        blueCircle.Draw();

        Shape redRectangle = new Rectangle(new RedColor());
        redRectangle.Draw();
    }
}
```

```
}
}
```

Output

```
Drawing Circle with Red Color
Drawing Circle with Blue Color
Drawing Rectangle with Red Color
```

Real-World Example

Remote Control and Devices

```
Remote Control (Abstraction)
|
+----> TV
+----> Radio
```

Example:

```
BasicRemote + TV
AdvancedRemote + TV
BasicRemote + Radio
AdvancedRemote + Radio
```

Code Example

```
public interface IDevice
{
    void On();
    void Off();
}

public class TV : IDevice
{
    public void On() => Console.WriteLine("TV ON");
    public void Off() => Console.WriteLine("TV OFF");
}

public class Radio : IDevice
{
    public void On() => Console.WriteLine("Radio ON");
}
```



```
        public void Off() => Console.WriteLine("Radio OFF");
    }

    public class RemoteControl
    {
        protected IDevice device;

        public RemoteControl(IDevice device)
        {
            this.device = device;
        }

        public void Toggle()
        {
            device.On();
        }
    }
}
```

Usage:

```
RemoteControl remote = new RemoteControl(new TV());
remote.Toggle();
```

When to Use Bridge Pattern

Use Bridge when:

- ✔ You want to avoid many subclasses. ✔ Abstraction and implementation should change independently.
- ✔ You prefer composition over inheritance. ✔ Multiple dimensions of change exist.

Advantages

- Reduces class explosion
- Follows Open/Closed Principle
- Better maintainability
- Independent extension
- Uses composition

Disadvantages

- More classes
- Slightly complex structure

Bridge vs Adapter

Feature	Bridge	Adapter
Purpose	Separate abstraction and implementation	Convert incompatible interfaces
Timing	Designed upfront	Added later
Relationship	Composition	Wrapper
Goal	Flexibility	Compatibility

Common .NET Examples

- Database providers ([SqlConnection](#), [OracleConnection](#))
- Logging frameworks
- UI framework + rendering engine
- Payment gateway integrations
- Notification systems (Email, SMS, Push)

Simple Formula

Inheritance:
Circle + Red
Circle + Blue
Rectangle + Red
Rectangle + Blue

Bridge:
Shape → Color

Bridge Pattern = Composition + Independent Variations.

3. Composite

Composite Design Pattern in C#

The **Composite Design Pattern** is a **structural design pattern** that lets you treat:

- **Individual objects (Leaf)**
- **Groups of objects (Composite)**

in the **same way**.

Simple idea

Think about a **file system**:

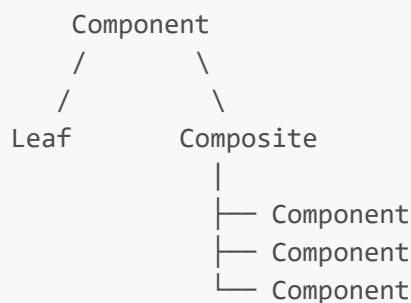
```

Root Folder
├── file1.txt      ← Leaf
├── file2.txt      ← Leaf
└── Documents     ← Composite
    ├── resume.pdf ← Leaf
    └── notes.txt  ← Leaf
  
```

You can perform an operation like `Display()` on both a file and a folder without the client needing to know whether it is dealing with a single object or a group.

1. Structure

The Composite pattern normally contains three parts:



Component

Common interface for both individual and group objects.

Leaf

Represents an individual object.

Composite

Contains other `Component` objects.

2. Basic C# Example

Let's create a company organization structure.

```

using System;
using System.Collections.Generic;

public interface IEmployee
  
```

```
{  
    void ShowDetails();  
}
```

`IEmployee` is our **Component**.

Leaf

```
public class Developer : IEmployee  
{  
    private readonly string _name;  
  
    public Developer(string name)  
    {  
        _name = name;  
    }  
  
    public void ShowDetails()  
    {  
        Console.WriteLine($"Developer: {_name}");  
    }  
}
```

Another leaf:

```
public class Designer : IEmployee  
{  
    private readonly string _name;  
  
    public Designer(string name)  
    {  
        _name = name;  
    }  
  
    public void ShowDetails()  
    {  
        Console.WriteLine($"Designer: {_name}");  
    }  
}
```

3. Composite

Now create a `Department`.

```
public class Department : IEmployee
{
    private readonly string _name;
    private readonly List<IEmployee> _employees = new();

    public Department(string name)
    {
        _name = name;
    }

    public void Add(IEmployee employee)
    {
        _employees.Add(employee);
    }

    public void Remove(IEmployee employee)
    {
        _employees.Remove(employee);
    }

    public void ShowDetails()
    {
        Console.WriteLine($"Department: {_name}");

        foreach (var employee in _employees)
        {
            employee.ShowDetails();
        }
    }
}
```

`Department` is the **Composite** because it contains `IEmployee` objects.

4. Using the Composite

```
var backendTeam = new Department("Backend Team");

backendTeam.Add(new Developer("John"));
backendTeam.Add(new Developer("David"));

var designTeam = new Department("Design Team");

designTeam.Add(new Designer("Alice"));
designTeam.Add(new Designer("Sarah"));

var company = new Department("Company");

company.Add(backendTeam);
```

```
company.Add(designTeam);  
  
company.ShowDetails();
```

Output:

```
Department: Company  
Department: Backend Team  
Developer: John  
Developer: David  
Department: Design Team  
Designer: Alice  
Designer: Sarah
```

Notice something important:

```
company.Add(backendTeam);  
company.Add(designTeam);
```

company doesn't care whether it receives:

```
Developer  
Designer  
Department
```

because all of them implement:

```
IEmployee
```

That's the main benefit of Composite.

5. More Realistic Example: File System

Composite is especially useful for hierarchical structures.

```
FileSystem  
|  
├─ File: a.txt  
├─ File: b.txt  
|  
└─ Folder: Documents
```

```
|  
├─ File: resume.pdf  
└─ File: notes.txt
```

We can model it like this:

```
public interface IFileSystemItem  
{  
    void Display();  
}
```

Leaf

```
public class FileItem : IFileSystemItem  
{  
    private readonly string _name;  
  
    public FileItem(string name)  
    {  
        _name = name;  
    }  
  
    public void Display()  
    {  
        Console.WriteLine($"File: {_name}");  
    }  
}
```

Composite

```
public class Folder : IFileSystemItem  
{  
    private readonly string _name;  
    private readonly List<IFileSystemItem> _items = new();  
  
    public Folder(string name)  
    {  
        _name = name;  
    }  
  
    public void Add(IFileSystemItem item)  
    {  
        _items.Add(item);  
    }  
  
    public void Remove(IFileSystemItem item)  
    {
```

```
        _items.Remove(item);
    }

    public void Display()
    {
        Console.WriteLine($"Folder: {_name}");

        foreach (var item in _items)
        {
            item.Display();
        }
    }
}
```

Usage:

```
var root = new Folder("Root");

root.Add(new FileItem("file1.txt"));
root.Add(new FileItem("file2.txt"));

var documents = new Folder("Documents");

documents.Add(new FileItem("resume.pdf"));
documents.Add(new FileItem("notes.txt"));

root.Add(documents);

root.Display();
```

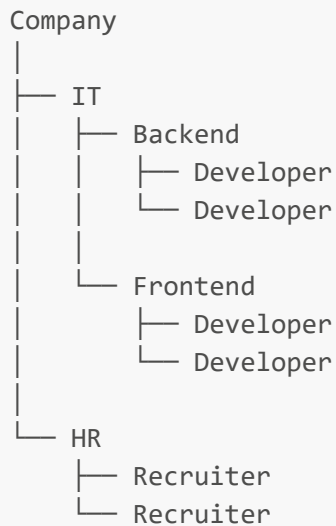
Output:

```
Folder: Root
File: file1.txt
File: file2.txt
Folder: Documents
File: resume.pdf
File: notes.txt
```

6. Composite Pattern with Recursion

The real power of Composite comes from **recursive structures**.

For example:



Every **Department** can contain another **Department**.

```

var company = new Department("Company");

var it = new Department("IT");

var backend = new Department("Backend");
backend.Add(new Developer("John"));
backend.Add(new Developer("Mike"));

var frontend = new Department("Frontend");
frontend.Add(new Developer("David"));
frontend.Add(new Developer("Alex"));

it.Add(backend);
it.Add(frontend);

var hr = new Department("HR");

hr.Add(new Designer("Sarah"));

company.Add(it);
company.Add(hr);

company.ShowDetails();
  
```

The recursive call:

```

foreach (var employee in _employees)
{
    employee.ShowDetails();
}
  
```

works regardless of whether the item is a:

```
Developer
Designer
Department
```

7. Where Composite Is Used

You should consider Composite when your application has a **tree structure**.

Common examples:

File systems

```
Folder
├── File
├── File
└── Folder
    ├── File
    └── File
```

UI components

```
Window
├── Panel
│   ├── Button
│   └── TextBox
└── Panel
    └── Button
```

Organization

```
Company
├── Department
│   ├── Employee
│   └── Employee
└── Department
    └── Employee
```

Menu systems

```

Menu
├── MenuItem
├── MenuItem
└── SubMenu
    ├── MenuItem
    └── MenuItem
  
```

Graphics

```

Scene
├── Circle
├── Rectangle
└── Group
    ├── Circle
    └── Rectangle
  
```

8. Composite vs Normal Collection

Without Composite:

```

if (item is Department department)
{
    department.ShowDetails();
}
else if (item is Developer developer)
{
    developer.ShowDetails();
}
  
```

The client needs to understand every type.

With Composite:

```

item.ShowDetails();
  
```

The client doesn't care whether `item` is a leaf or composite.

That's the key idea:

Treat individual objects and compositions of objects uniformly.

9. Advantages

1. Simplifies client code

Client works with:

```
IFileSystemItem
```

instead of checking concrete types.

2. Supports recursive structures

A composite can contain another composite.

```
Composite
  ↓
Composite
  ↓
Composite
  ↓
Leaf
```

3. Easy to extend

You can add:

```
class ImageFile : IFileSystemItem
```

without changing existing client code.

4. Follows Open/Closed Principle

New leaf/composite types can be added without modifying the existing structure.

10. Disadvantages

Composite isn't always appropriate.

It can make the design more complicated because you introduce:

```
Component
Leaf
```

Composite

Also, if certain operations only make sense for composites:

```
Add()  
Remove()
```

you need to decide whether they belong in the common interface.

For example:

```
public interface IComponent  
{  
    void Operation();  
    void Add(IComponent component);  
}
```

A **Leaf** doesn't naturally support **Add()**.

Therefore, a common design is to keep child-management operations only in **Composite**:

```
public interface IComponent  
{  
    void Operation();  
}  
  
public class Leaf : IComponent  
{  
    public void Operation()  
    {  
    }  
}  
  
public class Composite : IComponent  
{  
    private readonly List<IComponent> _children = new();  
  
    public void Add(IComponent component)  
    {  
        _children.Add(component);  
    }  
  
    public void Operation()  
    {  
        foreach (var child in _children)  
        {  
            child.Operation();  
        }  
    }  
}
```

```
}  
  }  
}
```

This is generally cleaner.

11. Interview Definition

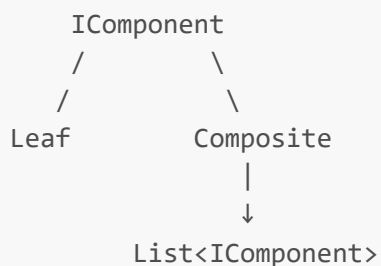
If an interviewer asks:

"What is the Composite Design Pattern?"

You can answer:

Composite is a structural design pattern that allows individual objects and groups of objects to be treated uniformly through a common interface. It is commonly used for tree structures such as file systems, UI hierarchies, organizational structures, and menus.

Remember this structure



The most important concept is:

Composite = Tree structure + Common interface + Recursive composition.

4. Decorator

Decorator Design Pattern in C#

The **Decorator Design Pattern** is a **structural design pattern** that lets you **add new behavior to an existing object dynamically without modifying its original class**.

Simple idea

Wrap an object with another object that adds extra functionality.

For example:

```
Coffee
  ↓
MilkDecorator
  ↓
SugarDecorator
```

The original `Coffee` doesn't need to know anything about milk or sugar.

1. Problem without Decorator

Suppose we have a coffee system:

```
public class Coffee
{
    public double GetCost()
    {
        return 50;
    }

    public string GetDescription()
    {
        return "Coffee";
    }
}
```

Now we want:

- Coffee
- Coffee + Milk
- Coffee + Sugar
- Coffee + Milk + Sugar
- Coffee + Cream

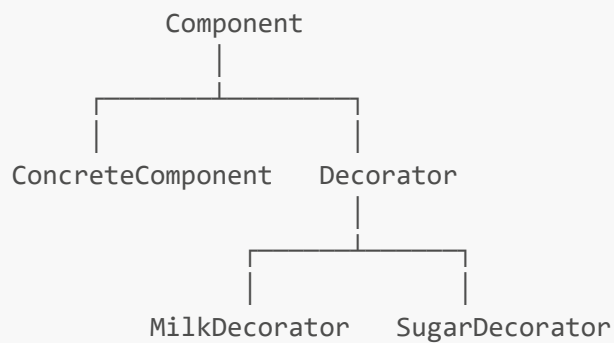
One approach would be to create many classes:

```
Coffee
MilkCoffee
SugarCoffee
MilkSugarCoffee
CreamCoffee
MilkCreamCoffee
...
```

This becomes difficult to maintain.

Decorator solves this problem.

2. Decorator Structure



The important part is that the decorator **implements the same interface** as the object it wraps.

3. C# Example

Step 1: Create Component

```
public interface ICoffee
{
    string GetDescription();
    double GetCost();
}
```

Step 2: Concrete Component

```
public class SimpleCoffee : ICoffee
{
    public string GetDescription()
    {
        return "Simple Coffee";
    }

    public double GetCost()
    {
        return 50;
    }
}
```

Step 3: Create Base Decorator


```
public abstract class CoffeeDecorator : ICoffee
{
    protected ICoffee coffee;

    protected CoffeeDecorator(ICoffee coffee)
    {
        this.coffee = coffee;
    }

    public abstract string GetDescription();
    public abstract double GetCost();
}
```

The decorator contains another `ICoffee`:

```
protected ICoffee coffee;
```

This is **composition**.

Step 4: Milk Decorator

```
public class MilkDecorator : CoffeeDecorator
{
    public MilkDecorator(ICoffee coffee)
        : base(coffee)
    {
    }

    public override string GetDescription()
    {
        return coffee.GetDescription() + ", Milk";
    }

    public override double GetCost()
    {
        return coffee.GetCost() + 10;
    }
}
```

Step 5: Sugar Decorator

```
public class SugarDecorator : CoffeeDecorator
{
    public SugarDecorator(ICoffee coffee)
```

```
        : base(coffee)
    {
    }

    public override string GetDescription()
    {
        return coffee.GetDescription() + ", Sugar";
    }

    public override double GetCost()
    {
        return coffee.GetCost() + 5;
    }
}
```

4. Using the Decorators

```
ICoffee coffee = new SimpleCoffee();

Console.WriteLine(coffee.GetDescription());
Console.WriteLine(coffee.GetCost());
```

Output:

```
Simple Coffee
50
```

Now add milk:

```
coffee = new MilkDecorator(coffee);

Console.WriteLine(coffee.GetDescription());
Console.WriteLine(coffee.GetCost());
```

Output:

```
Simple Coffee, Milk
60
```

Now add sugar:

```
coffee = new SugarDecorator(coffee);  
  
Console.WriteLine(coffee.GetDescription());  
Console.WriteLine(coffee.GetCost());
```

Output:

```
Simple Coffee, Milk, Sugar  
65
```

The object chain is:

```
SugarDecorator  
  |  
  ▼  
MilkDecorator  
  |  
  ▼  
SimpleCoffee
```

5. How it works internally

When you call:

```
coffee.GetCost();
```

the call travels through the decorators.

```
SugarDecorator.GetCost()  
  |  
  └─ coffee.GetCost()  
  |  
  ▼  
MilkDecorator.GetCost()  
  |  
  └─ coffee.GetCost()  
  |  
  ▼  
SimpleCoffee.GetCost()  
  |  
  ▼  
50
```

Then:

```
SimpleCoffee = 50
Milk         = +10
Sugar        = +5
-----
Total        = 65
```

6. Real-world example: Logging

Decorator is extremely useful when you want to add things such as:

- Logging
- Caching
- Authentication
- Authorization
- Validation
- Retry
- Metrics
- Transaction handling

For example:

```
public interface IPaymentService
{
    void Pay(double amount);
}
```

Actual implementation:

```
public class PaymentService : IPaymentService
{
    public void Pay(double amount)
    {
        Console.WriteLine($"Payment processed: {amount}");
    }
}
```

Logging decorator:

```
public class LoggingPaymentDecorator : IPaymentService
{
    private readonly IPaymentService paymentService;

    public LoggingPaymentDecorator(IPaymentService paymentService)
    {
        this.paymentService = paymentService;
    }

    public void Pay(double amount)
    {
        Console.WriteLine($"Payment started: {amount}");

        paymentService.Pay(amount);

        Console.WriteLine("Payment completed");
    }
}
```

Usage:

```
IPaymentService payment = new PaymentService();

payment = new LoggingPaymentDecorator(payment);

payment.Pay(1000);
```

Output:

```
Payment started: 1000
Payment processed: 1000
Payment completed
```

The original `PaymentService` was **not modified**.

7. Multiple Decorators

You can combine decorators.

```
IPaymentService payment = new PaymentService();

payment = new LoggingPaymentDecorator(payment);
payment = new ValidationPaymentDecorator(payment);
payment = new RetryPaymentDecorator(payment);
```

Conceptually:

```
RetryDecorator
  ↓
ValidationDecorator
  ↓
LoggingDecorator
  ↓
PaymentService
```

This is one of the biggest advantages of Decorator.

8. Decorator vs Inheritance

Inheritance

```
PaymentService
├── LoggingPaymentService
├── RetryPaymentService
└── CachedPaymentService
```

You create subclasses.

Decorator

```
Logging
  ↓
Retry
  ↓
Cache
  ↓
PaymentService
```

You compose behavior dynamically.

Decorator therefore favors:

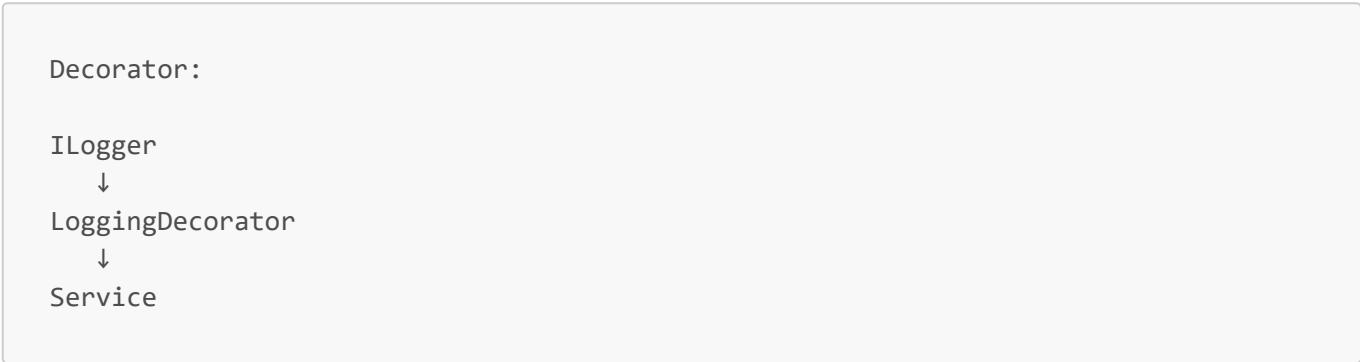
Composition over inheritance

9. Decorator vs Adapter

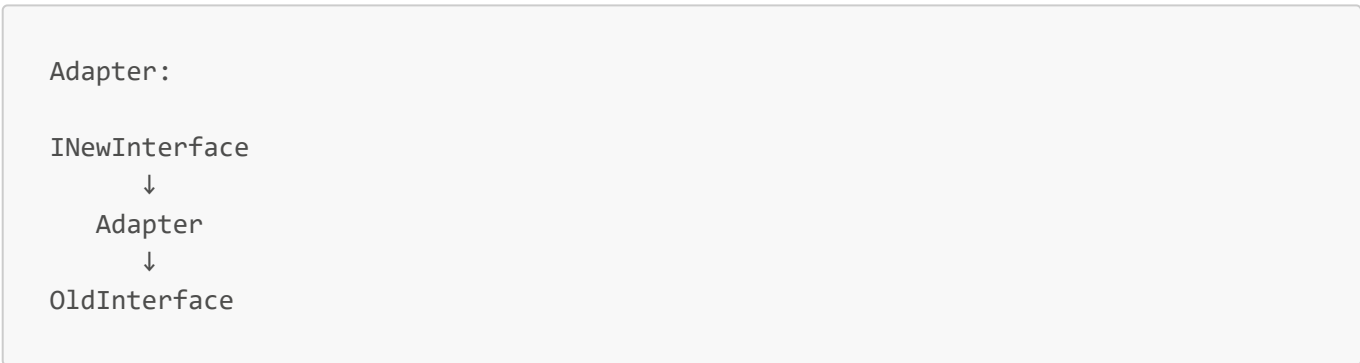
These two patterns are commonly confused.

Decorator	Adapter
Adds behavior	Changes interface
Keeps same interface	Provides another interface
Wraps object	Wraps incompatible object
Purpose = enhancement	Purpose = compatibility

Example:



vs.



10. Decorator vs Proxy

They look very similar because both wrap objects.

Decorator	Proxy
Adds responsibilities/behavior	Controls access
Usually focuses on enhancement	Usually focuses on access/control
Logging, compression, formatting	Security, lazy loading, remote access

The distinction can sometimes be subtle in real applications.

11. Advantages

☒ Open/Closed Principle

You can add behavior without modifying existing classes.

☒ Runtime flexibility

You can decide which decorators to use at runtime.

```
service = new LoggingDecorator(service);  
service = new CacheDecorator(service);
```

☒ Avoids subclass explosion

You don't need:

```
LoggingService  
LoggingCacheService  
LoggingRetryService  
LoggingCacheRetryService  
...
```

☒ Behaviors can be combined

```
Validation  
  ↓  
Logging  
  ↓  
Caching  
  ↓  
Service
```

12. Disadvantages

✗ Many small classes

A large application can end up with many decorators.

✗ Debugging can be harder

You may have:


```
Decorator A
  ↓
Decorator B
  ↓
Decorator C
  ↓
Service
```

Understanding the execution path can become difficult.

✗ Order matters

These can behave differently:

```
Logging → Cache → Service
```

and:

```
Cache → Logging → Service
```

13. Where you will see Decorator in .NET

A very important example is **ASP.NET Core middleware**.

Conceptually:

```
Request
  ↓
Exception Middleware
  ↓
Authentication Middleware
  ↓
Authorization Middleware
  ↓
Logging Middleware
  ↓
Controller
```

Each middleware wraps/continues the request pipeline and adds behavior around the next component.

For example:

```
app.UseAuthentication();  
app.UseAuthorization();  
app.UseMiddleware<LoggingMiddleware>();
```

This has the same **wrapping/composition idea** as the Decorator pattern, although ASP.NET Core middleware is specifically a pipeline mechanism rather than simply a textbook Decorator implementation.

14. Interview Definition

If an interviewer asks:

What is the Decorator Design Pattern?

You can answer:

Decorator is a structural design pattern that dynamically adds responsibilities or behavior to an object by wrapping it with another object that implements the same interface. It follows composition over inheritance and allows behavior to be combined without modifying the original class.

Remember this

Decorator = Same interface + Wrapper + Extra behavior

And the core structure is:

```
Client  
  ↓  
Decorator  
  ↓  
Decorator  
  ↓  
Concrete Object
```

5. Facade

Facade Design Pattern in C#

The **Facade Design Pattern** is a **structural design pattern** that provides a **simple interface to a complex subsystem**.

Simple idea

Instead of making the client interact with many classes:

```
Client
↓
OrderService
↓
PaymentService
InventoryService
EmailService
ShippingService
```

we create a **Facade**:

```
Client
↓
OrderFacade
↓
PaymentService
InventoryService
EmailService
ShippingService
```

The client only needs to know about **OrderFacade**.

1. Problem Without Facade

Suppose we have an e-commerce application.

```
public class InventoryService
{
    public bool CheckStock(int productId)
    {
        Console.WriteLine("Checking inventory...");
        return true;
    }
}
```

```
public class PaymentService
{
    public bool MakePayment(decimal amount)
    {
        Console.WriteLine($"Processing payment: {amount}");
        return true;
    }
}
```

```
public class ShippingService
{
    public void CreateShipment(int productId)
    {
        Console.WriteLine("Creating shipment...");
    }
}
```

```
public class EmailService
{
    public void SendConfirmation(string email)
    {
        Console.WriteLine($"Confirmation email sent to {email}");
    }
}
```

Now the client has to know about **all these classes**:

```
var inventory = new InventoryService();
var payment = new PaymentService();
var shipping = new ShippingService();
var email = new EmailService();

if (inventory.CheckStock(101))
{
    if (payment.MakePayment(5000))
    {
        shipping.CreateShipment(101);
        email.SendConfirmation("user@example.com");
    }
}
```

This makes the client tightly coupled to the subsystem.

2. Create a Facade

We create one class that hides the complexity.

```
public class OrderFacade
{
    private readonly InventoryService _inventory;
    private readonly PaymentService _payment;
```

```
private readonly ShippingService _shipping;
private readonly EmailService _email;

public OrderFacade()
{
    _inventory = new InventoryService();
    _payment = new PaymentService();
    _shipping = new ShippingService();
    _email = new EmailService();
}

public bool PlaceOrder(
    int productId,
    decimal amount,
    string email)
{
    // 1. Check inventory
    if (!_inventory.CheckStock(productId))
    {
        return false;
    }

    // 2. Process payment
    if (!_payment.MakePayment(amount))
    {
        return false;
    }

    // 3. Create shipment
    _shipping.CreateShipment(productId);

    // 4. Send confirmation
    _email.SendConfirmation(email);

    return true;
}
}
```

Now the client becomes very simple:

```
var orderFacade = new OrderFacade();

bool result = orderFacade.PlaceOrder(
    101,
    5000,
    "user@example.com"
);

Console.WriteLine(result ? "Order successful" : "Order failed");
```

The client doesn't need to know:

- how inventory is checked
- how payment works
- how shipping works
- how email is sent
- the order of operations

That's the main purpose of **Facade**.

3. Real-world analogy

Think about a **hotel receptionist**.

Without a receptionist, you might need to contact:

```
Customer
  ↓
Room Service
Housekeeping
Restaurant
Payment
Laundry
Reservation
```

With a receptionist:

```
Customer
  ↓
Receptionist
  ↓
-----
Room Service
Housekeeping
Restaurant
Payment
Laundry
Reservation
```

The receptionist is acting as a **Facade**.

You simply say:

```
"I want room 205 cleaned."
```

The receptionist handles the internal complexity.

4. ASP.NET Core Example

Facade is very useful in ASP.NET Core applications.

Imagine an API endpoint needs to:

```
Validate Order
  ↓
Check Inventory
  ↓
Process Payment
  ↓
Create Shipment
  ↓
Send Email
```

Instead of putting everything in the Controller:

```
[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    [HttpPost]
    public IActionResult CreateOrder(OrderRequest request)
    {
        // Validate
        // Inventory
        // Payment
        // Shipping
        // Email

        return Ok();
    }
}
```

Create an **OrderFacade**.

```
public interface IOrderFacade
{
    Task<bool> PlaceOrderAsync(OrderRequest request);
}
```

```
public class OrderFacade : IOrderFacade
{
    private readonly IInventoryService _inventory;
```

```

private readonly IPaymentService _payment;
private readonly IShippingService _shipping;
private readonly IEmailService _email;

public OrderFacade(
    IInventoryService inventory,
    IPaymentService payment,
    IShippingService shipping,
    IEmailService email)
{
    _inventory = inventory;
    _payment = payment;
    _shipping = shipping;
    _email = email;
}

public async Task<bool> PlaceOrderAsync(OrderRequest request)
{
    var available =
        await _inventory.IsAvailableAsync(request.ProductId);

    if (!available)
        return false;

    var paymentResult =
        await _payment.ProcessAsync(request.Amount);

    if (!paymentResult)
        return false;

    await _shipping.CreateShipmentAsync(request.ProductId);

    await _email.SendConfirmationAsync(request.Email);

    return true;
}
}

```

The Controller is now clean:

```

[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly IOrderFacade _orderFacade;

    public OrdersController(IOrderFacade orderFacade)
    {
        _orderFacade = orderFacade;
    }

    [HttpPost]

```



```
public async Task<IActionResult> CreateOrder(OrderRequest request)
{
    var result = await _orderFacade.PlaceOrderAsync(request);

    if (!result)
        return BadRequest("Order failed");

    return Ok("Order successful");
}
```

Register it with Dependency Injection:

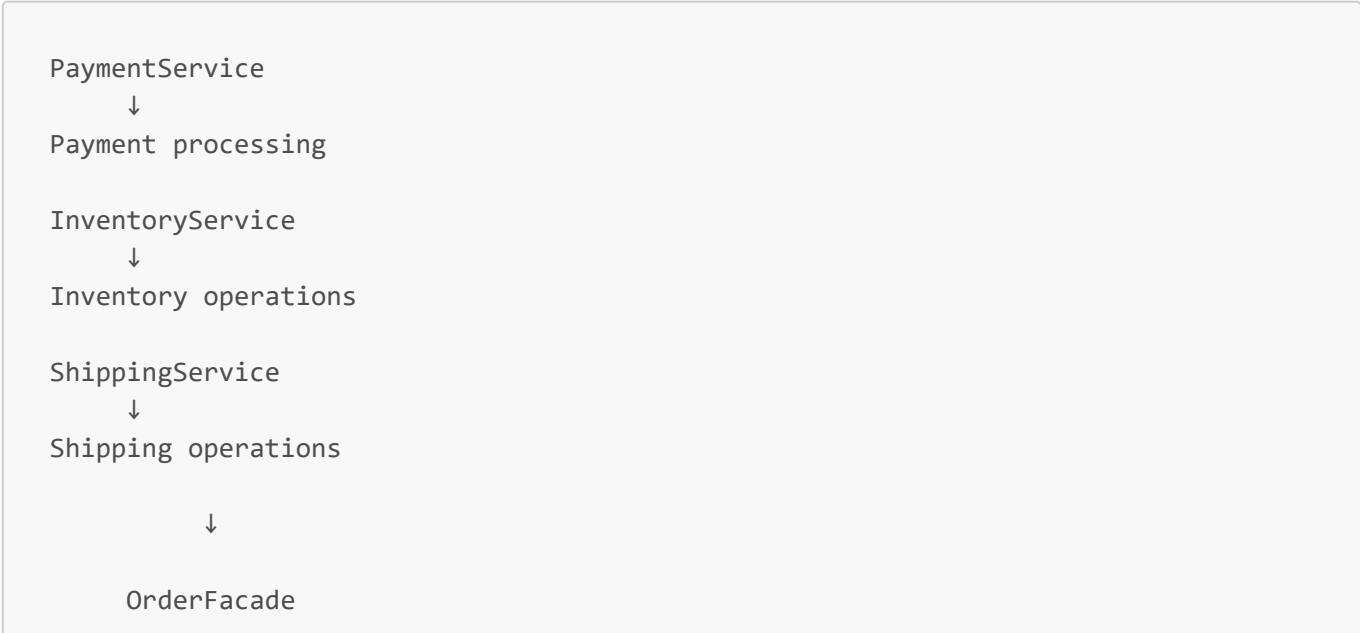
```
builder.Services.AddScoped<IOrderFacade, OrderFacade>();
```

5. Facade vs Service

They can look similar, but their purposes differ.

Service	Facade
Performs a specific responsibility	Coordinates multiple services
Usually focused on one domain operation	Provides a simplified entry point
PaymentService → payment	OrderFacade → payment + inventory + shipping
Contains business/technical functionality	Often orchestrates existing functionality
Can be used directly by many components	Hides subsystem complexity

For example:



↓
Coordinates all three

6. Facade vs Adapter

These patterns are often confused.

Adapter

Changes one interface into another interface.

```
Client
  ↓
Expected Interface
  ↓
Adapter
  ↓
Existing incompatible class
```

Example:

```
ILogger
  ↓
Adapter
  ↓
ThirdPartyLogger
```

Facade

Simplifies a complex subsystem.

```
Client
  ↓
Facade
  ↓
Service A
Service B
Service C
```

Remember

Adapter = make interfaces compatible.

Facade = make a complex system simple.

7. Advantages

1. Reduces complexity

Client sees only one interface.

2. Reduces coupling

The client doesn't depend directly on many subsystem classes.

3. Easier maintenance

Internal implementation can change without affecting clients.

4. Better API design

A facade can expose high-level operations such as:

```
PlaceOrder()  
CancelOrder()  
RefundOrder()
```

instead of requiring the client to perform many low-level operations.

5. Good for large applications

Especially useful in:

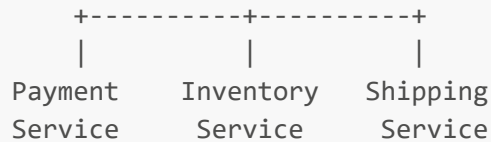
- ASP.NET Core
- Microservices
- Payment systems
- E-commerce
- Banking/fintech
- Legacy-system integration

8. Important point

A Facade **doesn't necessarily replace the underlying classes.**

They still exist:

```
OrderFacade  
|
```



The Facade simply provides a **simplified entry point** to them.

Interview definition

Facade is a structural design pattern that provides a simplified, unified interface to a set of interfaces in a complex subsystem, hiding the subsystem's complexity from the client.

Easy way to remember all four structural patterns you've been studying

```

Adapter    → Convert interface
Bridge     → Separate abstraction from implementation
Composite  → Treat individual and groups uniformly
Decorator  → Add behavior dynamically
Facade     → Simplify a complex subsystem
  
```

Facade = "One simple door to a complicated building."

6. Flyweight

Flyweight Design Pattern in C#

The **Flyweight Design Pattern** is a **structural design pattern** used to **reduce memory consumption** by sharing common objects instead of creating many identical objects.

1. The basic idea

Suppose you are creating a text editor containing:

```

Hello World
Hello C#
Hello .NET
Hello World
Hello C#
  
```

If every character/object stores its own font, color, size, etc., memory usage can become large.

Instead, we separate data into:

- **Intrinsic state** → shared data
- **Extrinsic state** → data specific to each usage

For example:

```
Character
├── Character = 'A'      ← extrinsic
├── Position = (10, 20)  ← extrinsic
└── Font = Arial 12     ← intrinsic/shared
```

2. Real-world example

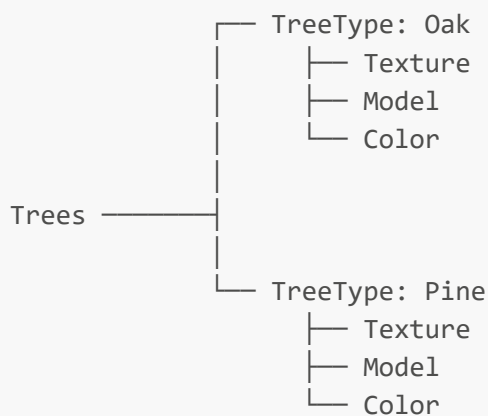
Imagine a game with **1 million trees**.

Without Flyweight:

```
Tree 1 → texture + color + model + position
Tree 2 → texture + color + model + position
Tree 3 → texture + color + model + position
...
```

You might unnecessarily store the same texture/model repeatedly.

With Flyweight:



Each tree only stores its unique position.

3. C# Example

Flyweight

```
public class TreeType
{
    private readonly string name;
    private readonly string color;
    private readonly string texture;

    public TreeType(string name, string color, string texture)
    {
        this.name = name;
        this.color = color;
        this.texture = texture;
    }

    public void Draw(int x, int y)
    {
        Console.WriteLine(
            $"Drawing {name} tree at ({x}, {y}) " +
            $"Color: {color}, Texture: {texture}");
    }
}
```

The `TreeType` contains **shared/intrinsic data**.

4. Flyweight Factory

The factory ensures that we don't create duplicate `TreeType` objects.

```
public class TreeFactory
{
    private readonly Dictionary<string, TreeType> treeTypes = new();

    public TreeType GetTreeType(
        string name,
        string color,
        string texture)
    {
        string key = $"{name}_{color}_{texture}";

        if (!treeTypes.ContainsKey(key))
        {
            treeTypes[key] = new TreeType(
                name,
                color,
                texture);
        }

        return treeTypes[key];
    }
}
```

The dictionary acts as a **cache**.

5. Context / Client Object

Each individual tree contains only its unique data.

```
public class Tree
{
    private readonly int x;
    private readonly int y;
    private readonly TreeType treeType;

    public Tree(int x, int y, TreeType treeType)
    {
        this.x = x;
        this.y = y;
        this.treeType = treeType;
    }

    public void Draw()
    {
        treeType.Draw(x, y);
    }
}
```

Here:

```
x
y
```

are **extrinsic state**.

And:

```
name
color
texture
```

are **intrinsic state**.

6. Using the pattern

```
TreeFactory factory = new TreeFactory();

TreeType oak = factory.GetTreeType(
    "Oak",
    "Green",
    "OakTexture");

TreeType pine = factory.GetTreeType(
    "Pine",
    "Dark Green",
    "PineTexture");

Tree tree1 = new Tree(10, 20, oak);
Tree tree2 = new Tree(30, 50, oak);
Tree tree3 = new Tree(100, 200, oak);
Tree tree4 = new Tree(500, 300, pine);

tree1.Draw();
tree2.Draw();
tree3.Draw();
tree4.Draw();
```

Notice:

```
tree1
tree2
tree3
```

all use the **same TreeType instance**.

You can verify it:

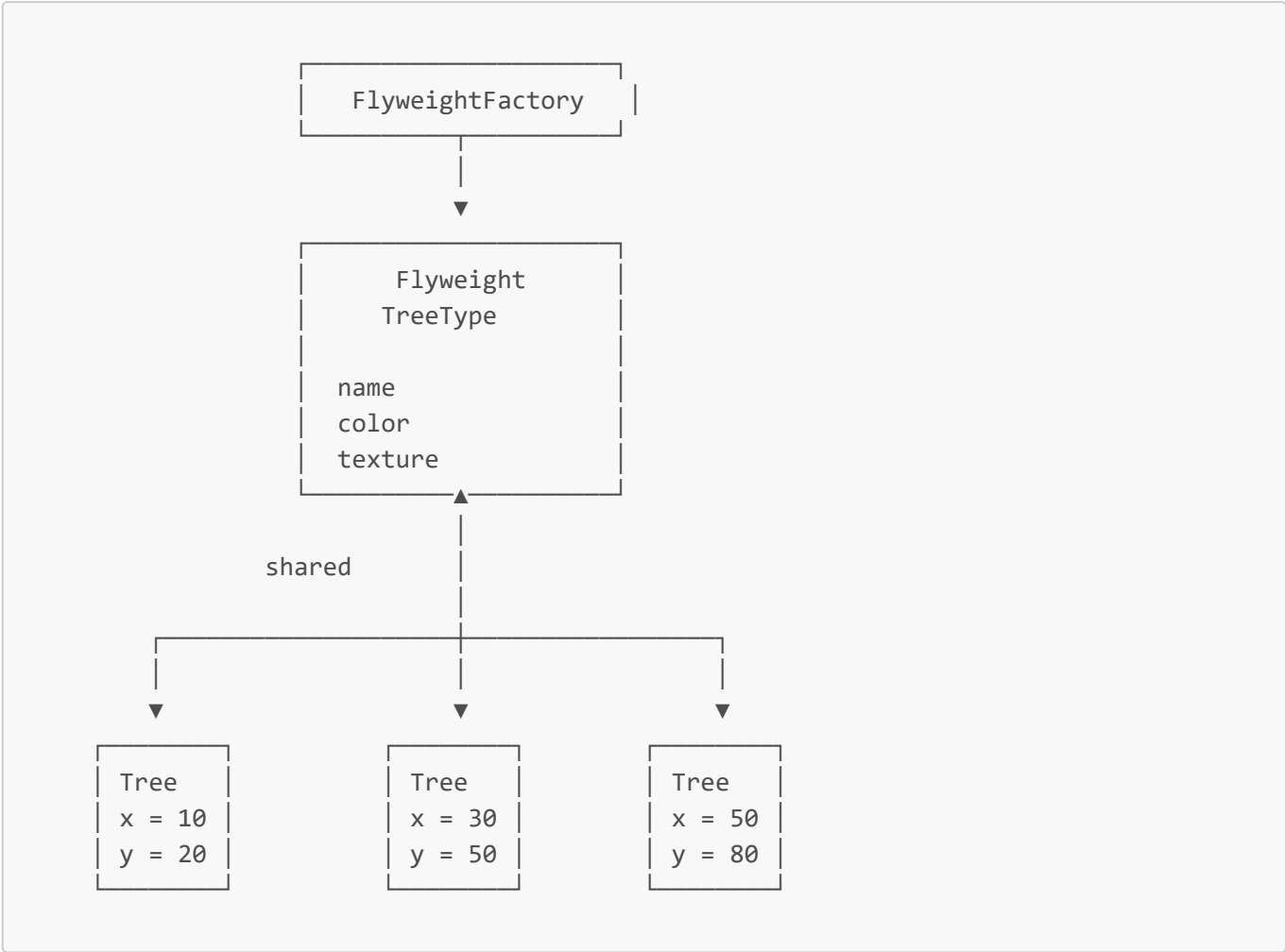
```
Console.WriteLine(
    ReferenceEquals(oak, factory.GetTreeType(
        "Oak",
        "Green",
        "OakTexture")));
```

Output:

```
True
```

7. Structure

The overall architecture looks like this:



8. Intrinsic vs Extrinsic State

This is the **most important concept** in Flyweight.

State	Meaning	Example
Intrinsic	Can be shared	Tree type, texture, color
Extrinsic	Unique to each object	Position, ID, current state

For our example:

TreeType

├─ name

├─ color

└─ texture

→ intrinsic

→ intrinsic

→ intrinsic

Tree

├─ x

├─ y

└─ treeType

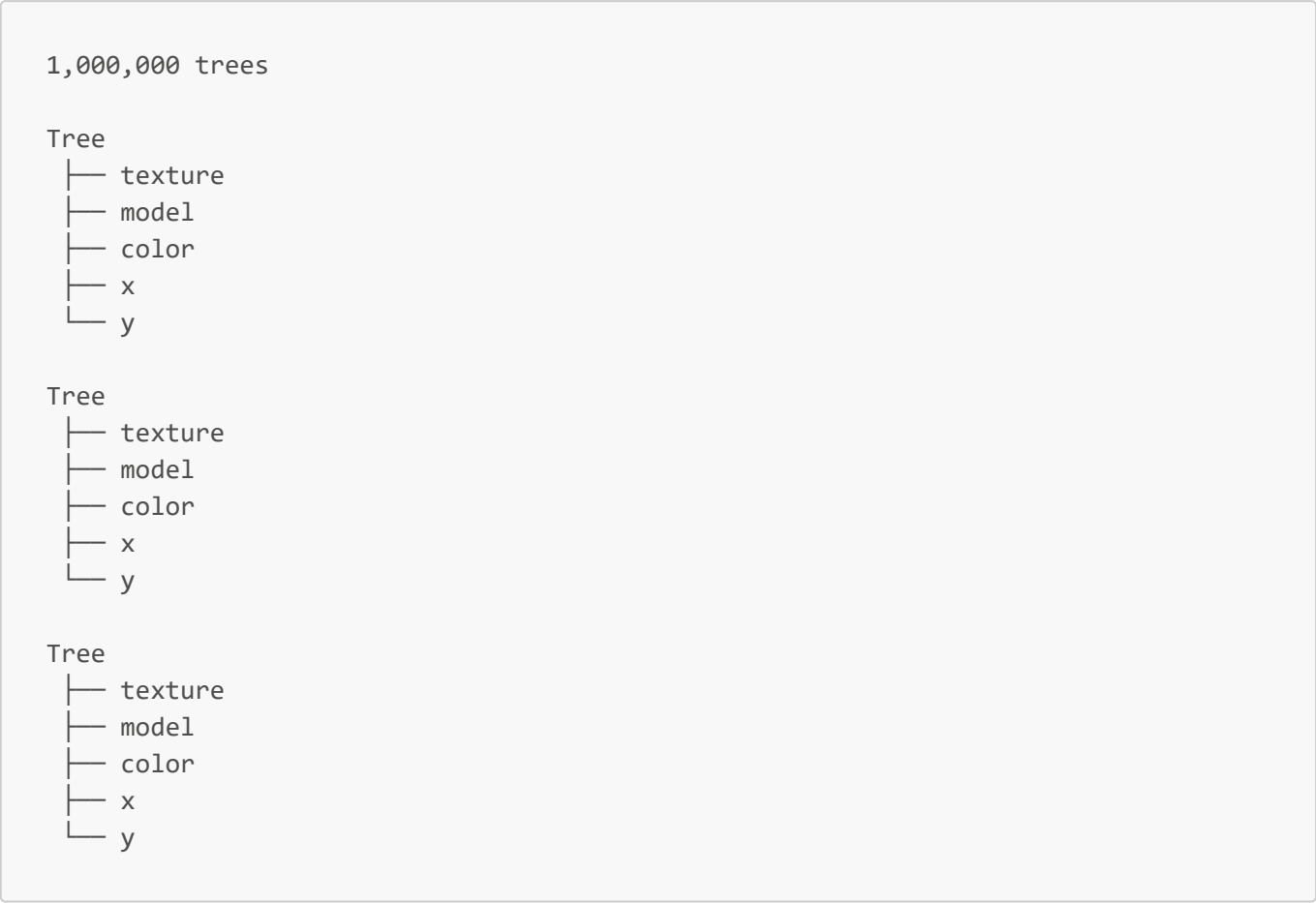
→ extrinsic

→ extrinsic

→ reference to shared object

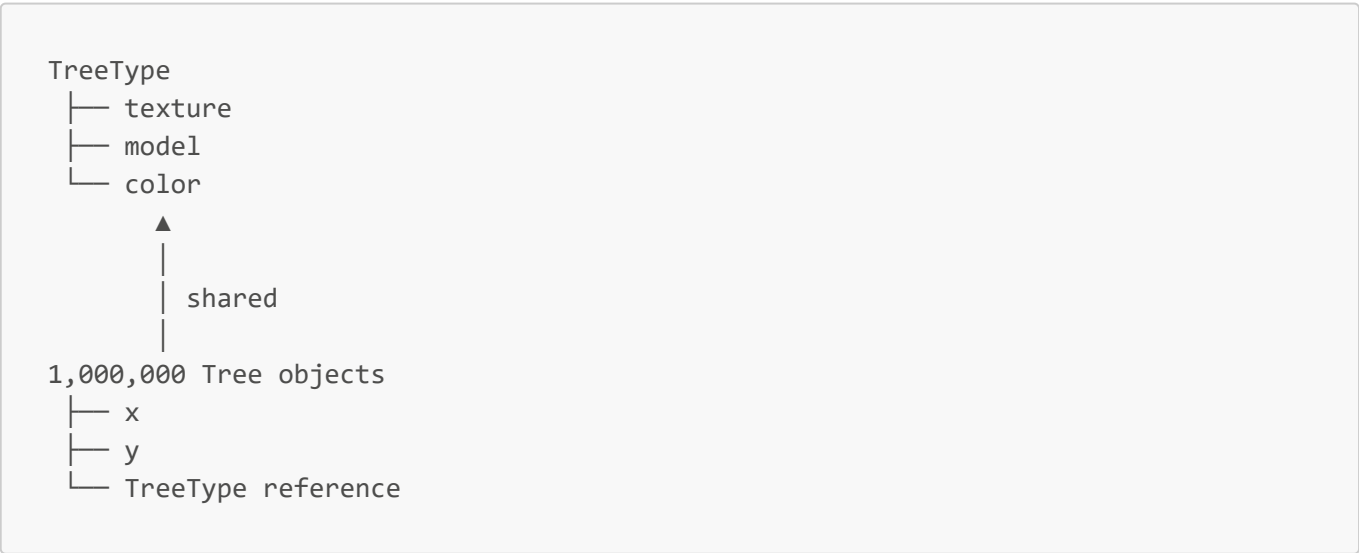
9. Without Flyweight vs With Flyweight

Without Flyweight



Potentially huge memory usage.

With Flyweight



Much less duplicated data.

10. When should you use Flyweight?

Use Flyweight when:

- You have **huge numbers of objects**.
- Many objects contain the **same data**.
- Object creation causes significant memory usage.
- Shared data is mostly **immutable**.
- You can separate common data from unique data.

Typical examples:

```
Game engines
Text editors
Document processing
GUI systems
Caching systems
Large datasets
Object rendering
String/interning-like scenarios
```

11. Flyweight vs Singleton

These patterns are often confused.

Singleton

Guarantees essentially **one shared instance** of a particular object.



Flyweight

Allows **multiple shared instances**, usually one per unique intrinsic state.

```
Factory
├── Oak TreeType
└── Pine TreeType
```

```
├─ Maple TreeType
└─ Birch TreeType
```

So:

Singleton = one instance

Flyweight = reuse instances where possible

12. Flyweight vs Object Pool

Another important distinction:

Object Pool

```
Create expensive object
      ↓
Put into pool
      ↓
Reuse object
      ↓
Return to pool
```

Used when objects are expensive to create and can be reused.

Examples:

```
Database connections
Threads
Sockets
Game bullets
Game particles
```

Flyweight

```
Common data
      ↓
Share it among many objects
```

The primary goal is **memory optimization through sharing**.

13. Interview definition

If an interviewer asks:

What is Flyweight Design Pattern?

A good answer is:

Flyweight is a structural design pattern that reduces memory consumption by sharing common intrinsic state between multiple objects while keeping object-specific extrinsic state outside the shared object. A Flyweight Factory is commonly used to cache and reuse existing Flyweight instances.

Key terms to remember

```
Flyweight
  ↓
Structural Pattern
  ↓
Memory Optimization
  ↓
Share Common Data
  ↓
Intrinsic State
  +
Extrinsic State
  ↓
Flyweight Factory / Cache
```

The main idea: don't create 1,000,000 copies of the same data—**create it once and share it.**

7. Proxy

Proxy Design Pattern in C#

The **Proxy Design Pattern** is a **structural design pattern** that provides a substitute or representative object for another object.

The proxy controls access to the **real object**.

Simple idea

```
Client
  ↓
Proxy
  ↓
Real Object
```

Instead of the client directly calling the real object:

```
Client → RealService
```

we introduce a proxy:

```
Client → Proxy → RealService
```

The Proxy can perform additional work before or after forwarding the request.

1. When to use Proxy

Common use cases:

- **Lazy loading** — create an expensive object only when needed
 - **Access control** — check permissions
 - **Caching** — return cached results
 - **Logging** — record method calls
 - **Remote access** — represent an object located on another machine
 - **Validation** — validate requests before calling the real object
 - **Rate limiting** — restrict how frequently an operation can be performed
-

2. Basic C# Example

Suppose we have an expensive image-loading service.

Common interface

```
public interface IImage
{
    void Display();
}
```

Real object

```
public class RealImage : IImage
{
    private readonly string _fileName;

    public RealImage(string fileName)
    {
        _fileName = fileName;
    }
}
```

```
        LoadFromDisk();
    }

    private void LoadFromDisk()
    {
        Console.WriteLine($"Loading {_fileName} from disk...");
    }

    public void Display()
    {
        Console.WriteLine($"Displaying {_fileName}");
    }
}
```

Creating `RealImage` immediately loads the image.

3. Proxy

```
public class ImageProxy : IImage
{
    private readonly string _fileName;
    private RealImage? _realImage;

    public ImageProxy(string fileName)
    {
        _fileName = fileName;
    }

    public void Display()
    {
        if (_realImage == null)
        {
            _realImage = new RealImage(_fileName);
        }

        _realImage.Display();
    }
}
```

The proxy delays creation of `RealImage` until `Display()` is actually called.

4. Client

```
class Program
{
```

```
static void Main()
{
    IImage image = new ImageProxy("photo.jpg");

    Console.WriteLine("Proxy created.");

    image.Display();

    Console.WriteLine("Display again.");

    image.Display();
}
```

Output:

```
Proxy created.

Loading photo.jpg from disk...
Displaying photo.jpg

Display again.

Displaying photo.jpg
```

Notice:

```
IImage image = new ImageProxy("photo.jpg");
```

does **not** create `RealImage`.

It is created only here:

```
if (_realImage == null)
{
    _realImage = new RealImage(_fileName);
}
```

This is called **Virtual Proxy / Lazy Proxy**.

5. Proxy for Access Control

Proxy isn't only for lazy loading.

Suppose only administrators can delete users.

Interface

```
public interface IUserService
{
    void DeleteUser(int userId);
}
```

Real service

```
public class UserService : IUserService
{
    public void DeleteUser(int userId)
    {
        Console.WriteLine($"User {userId} deleted.");
    }
}
```

Proxy

```
public class UserServiceProxy : IUserService
{
    private readonly UserService _userService;
    private readonly string _role;

    public UserServiceProxy(string role)
    {
        _role = role;
        _userService = new UserService();
    }

    public void DeleteUser(int userId)
    {
        if (_role != "Admin")
        {
            Console.WriteLine("Access denied.");
            return;
        }

        _userService.DeleteUser(userId);
    }
}
```

Client

```
IUserService service =  
    new UserServiceProxy("User");  
  
service.DeleteUser(10);
```

Output:

```
Access denied.
```

For an administrator:

```
IUserService service =  
    new UserServiceProxy("Admin");  
  
service.DeleteUser(10);
```

Output:

```
User 10 deleted.
```

Here:

```
Client  
  ↓  
UserServiceProxy  
  ↓  
UserService
```

The proxy controls access to the real service.

6. Proxy with Logging

Another common implementation is a logging proxy.

```
public interface IPaymentService  
{  
    void Pay(decimal amount);  
}
```

Real service:

```
public class PaymentService : IPaymentService
{
    public void Pay(decimal amount)
    {
        Console.WriteLine($"Payment processed: {amount}");
    }
}
```

Proxy:

```
public class PaymentServiceProxy : IPaymentService
{
    private readonly IPaymentService _realService;

    public PaymentServiceProxy(IPaymentService realService)
    {
        _realService = realService;
    }

    public void Pay(decimal amount)
    {
        Console.WriteLine($"Starting payment: {amount}");

        _realService.Pay(amount);

        Console.WriteLine("Payment completed.");
    }
}
```

Usage:

```
IPaymentService realService = new PaymentService();

IPaymentService proxy =
    new PaymentServiceProxy(realService);

proxy.Pay(5000);
```

Output:

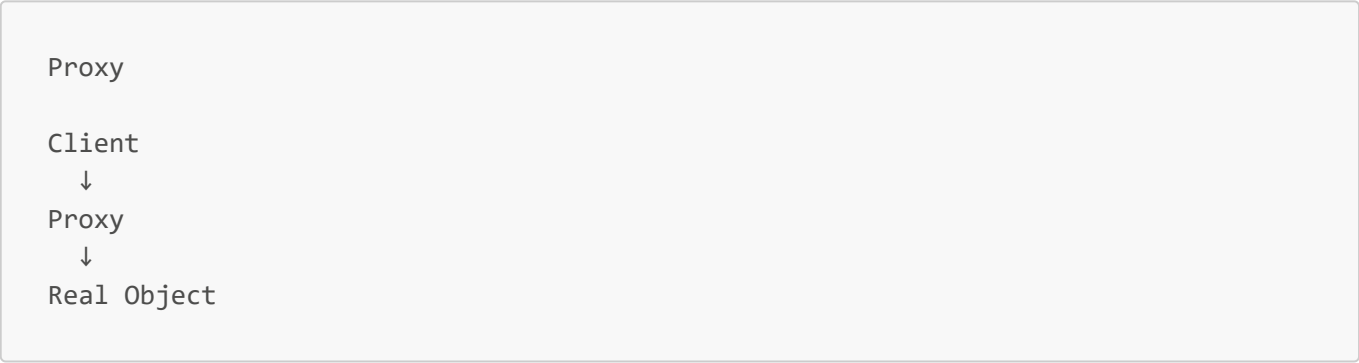
```
Starting payment: 5000
Payment processed: 5000
Payment completed.
```

7. Proxy vs Decorator

These two patterns look very similar.

Proxy	Decorator
Controls access to an object	Adds responsibilities/features
Often manages lifecycle/access	Usually enhances behavior
Can delay object creation	Usually wraps an already-created object
Common for security, remote access, caching	Common for logging, compression, formatting
Focus: access/control	Focus: behavior enhancement

For example:



The proxy asks:

"Should the client be allowed to access this object?"

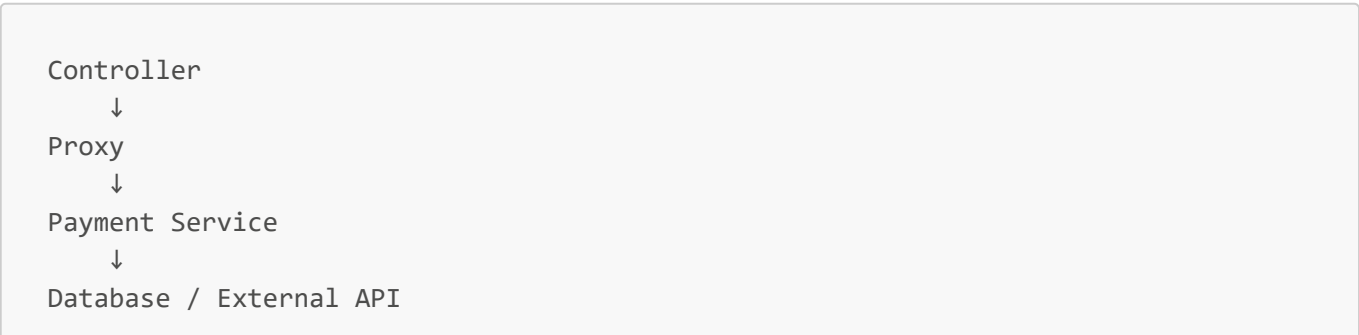
Decorator asks:

"What additional behavior can I add to this object?"

8. Proxy in ASP.NET Core

Proxy concepts are extremely useful in backend development.

For example:



The proxy could perform:

```
Authentication
  ↓
Authorization
  ↓
Rate Limiting
  ↓
Caching
  ↓
Logging
  ↓
Real Service
```

For example:

```
public interface IOrderService
{
    Task<Order> GetOrderAsync(int id);
}
```

Real service:

```
public class OrderService : IOrderService
{
    public async Task<Order> GetOrderAsync(int id)
    {
        // Database call
        return await Task.FromResult(
            new Order { Id = id }
        );
    }
}
```

Proxy:

```
public class CachedOrderService : IOrderService
{
    private readonly IOrderService _inner;

    public CachedOrderService(IOrderService inner)
    {
        _inner = inner;
    }
}
```

```
public async Task<Order> GetOrderAsync(int id)
{
    Console.WriteLine("Checking cache...");

    // If cache exists:
    // return cached order;

    var order = await _inner.GetOrderAsync(id);

    // Store in cache

    return order;
}
```

This gives:

```
Controller
  ↓
CachedOrderService
  ↓
OrderService
  ↓
Database
```

9. Types of Proxy

You should know these names for interviews:

1. Virtual Proxy

Delays creation of expensive objects.

Proxy → creates RealObject only when needed

2. Protection Proxy

Controls access.

Proxy → checks permissions → RealObject

3. Remote Proxy

Represents an object located remotely.

Client → Proxy → Network → Remote Object

4. Caching Proxy

Stores previous results.

```
Client
  ↓
Cache Proxy
  ↓
Cache hit? → Return cached data
  ↓
Cache miss
  ↓
Real Service
```

5. Logging Proxy

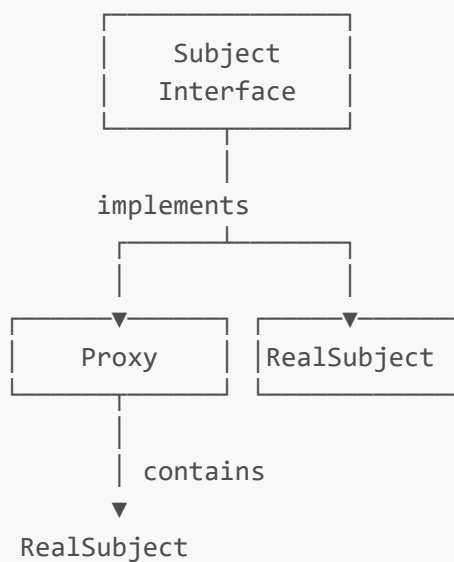
Adds logging around the real object.

6. Smart Proxy

Performs additional management such as reference counting, locking, validation, etc.

10. Proxy Structure

The classic structure is:



C# structure:

```
public interface IService
{
    void Execute();
}

public class RealService : IService
{
    public void Execute()
    {
        Console.WriteLine("Real service executing...");
    }
}

public class ServiceProxy : IService
{
    private readonly RealService _realService;

    public ServiceProxy()
    {
        _realService = new RealService();
    }

    public void Execute()
    {
        // Additional control

        _realService.Execute();

        // Additional behavior
    }
}
```

11. Real-world analogy

Think about a **bank ATM**.

You don't directly access the bank's database.

```
You
↓
ATM
↓
Bank System
↓
Database
```


The ATM acts like a proxy.

It can:

- authenticate you
- validate your PIN
- check balance
- validate withdrawal amount
- apply transaction rules
- communicate with the real banking system

So conceptually:

```
Client → Proxy → Real Object
```

Interview definition

Proxy is a structural design pattern that provides a surrogate or placeholder for another object and controls access to the real object.

The most important thing to remember is:

Proxy = controlled access to another object.

Important for .NET

- Adapter
- Decorator
- Facade
- Proxy
- Composite

4. Behavioral Design Patterns

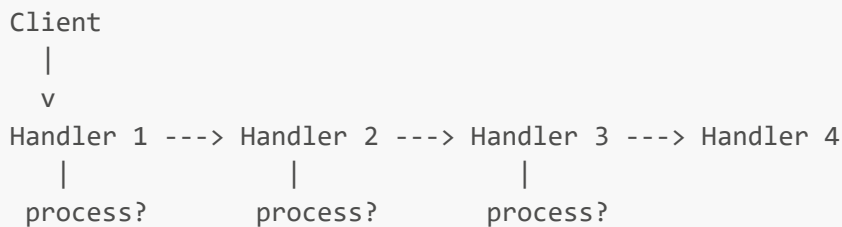
Patterns for **communication and behavior between objects**.

1. Chain of Responsibility

Chain of Responsibility Design Pattern in C#

The **Chain of Responsibility (CoR)** is a **behavioral design pattern** where a request is passed through a chain of handlers. Each handler decides whether it can process the request or passes it to the next handler.

Simple idea



A common example is **expense approval**:

Employee → Manager → Director → CEO

If the manager can approve the expense, it stops there. Otherwise, it moves to the director.

1. Basic Structure

There are usually three parts:

Handler

Defines the interface for processing requests.

Concrete Handler

Actually processes the request or forwards it.

Client

Creates the chain and sends the request.

2. C# Example — Expense Approval

```

public class Expense
{
    public decimal Amount { get; set; }
}
  
```

Handler

```

public abstract class ExpenseHandler
{
    protected ExpenseHandler? NextHandler;
}
  
```

```
    public void SetNext(ExpenseHandler handler)
    {
        NextHandler = handler;
    }

    public abstract void Approve(Expense expense);
}
```

Manager

```
public class Manager : ExpenseHandler
{
    public override void Approve(Expense expense)
    {
        if (expense.Amount <= 10_000)
        {
            Console.WriteLine("Manager approved the expense.");
        }
        else if (NextHandler != null)
        {
            NextHandler.Approve(expense);
        }
    }
}
```

Director

```
public class Director : ExpenseHandler
{
    public override void Approve(Expense expense)
    {
        if (expense.Amount <= 50_000)
        {
            Console.WriteLine("Director approved the expense.");
        }
        else if (NextHandler != null)
        {
            NextHandler.Approve(expense);
        }
    }
}
```

CEO

```
public class CEO : ExpenseHandler
{

```

```
public override void Approve(Expense expense)
{
    if (expense.Amount <= 1_00_000)
    {
        Console.WriteLine("CEO approved the expense.");
    }
    else
    {
        Console.WriteLine("Expense rejected.");
    }
}
}
```

Client

```
var manager = new Manager();
var director = new Director();
var ceo = new CEO();

manager.SetNext(director);
director.SetNext(ceo);

var expense = new Expense
{
    Amount = 30_000
};

manager.Approve(expense);
```

Output

Director approved the expense.

Because:

```
30,000
↓
Manager
↓
Cannot approve
↓
Director
↓
Can approve
↓
Approved
```

3. Better Modern C# Implementation

You can make the handler reusable with an interface.

```
public interface IHandler<T>
{
    IHandler<T>? Next { get; set; }

    void Handle(T request);
}
```

Example:

```
public class AuthenticationHandler : IHandler<string>
{
    public IHandler<string>? Next { get; set; }

    public void Handle(string request)
    {
        Console.WriteLine("Authentication checking...");

        if (request == "valid")
        {
            Next?.Handle(request);
        }
        else
        {
            Console.WriteLine("Authentication failed.");
        }
    }
}
```

Another handler:

```
public class AuthorizationHandler : IHandler<string>
{
    public IHandler<string>? Next { get; set; }

    public void Handle(string request)
    {
        Console.WriteLine("Authorization checking...");

        if (request == "valid")
        {
            Next?.Handle(request);
        }
    }
}
```

```
    }  
  }  
}
```

Create the chain:

```
var authentication = new AuthenticationHandler();  
var authorization = new AuthorizationHandler();  
  
authentication.Next = authorization;  
  
authentication.Handle("valid");
```

4. ASP.NET Core Example

This pattern is particularly useful for **request processing pipelines**.

For example:

```
HTTP Request  
  ↓  
Logging  
  ↓  
Authentication  
  ↓  
Authorization  
  ↓  
Validation  
  ↓  
Business Logic  
  ↓  
Response
```

ASP.NET Core middleware works very similarly to the **Chain of Responsibility** pattern.

Example:

```
app.UseMiddleware<LoggingMiddleware>();  
  
app.UseMiddleware<AuthenticationMiddleware>();  
  
app.UseMiddleware<AuthorizationMiddleware>();
```

Each middleware can:

1. Process something before the next handler.
2. Call the next handler.
3. Process something after the next handler.
4. Stop the chain.

Conceptually:



5. Real-World Examples

You can use Chain of Responsibility for:

- **ASP.NET Core Middleware**
- Authentication pipelines
- Authorization pipelines
- Exception handling
- Logging
- Validation
- Approval workflows
- Payment processing
- Fraud detection
- Rate limiting
- Support ticket escalation
- Command processing
- Request filtering

For example, in a fintech application:



```
↓  
Fraud Check  
↓  
Rate Limit Check  
↓  
Payment Processing
```

Each handler can reject the request or pass it forward.

6. Chain of Responsibility vs if-else

Without the pattern:

```
if (amount <= 10000)  
{  
    ManagerApprove();  
}  
else if (amount <= 50000)  
{  
    DirectorApprove();  
}  
else if (amount <= 100000)  
{  
    CEOApprove();  
}  
else  
{  
    Reject();  
}
```

This becomes difficult to maintain when you add more approval levels.

With Chain of Responsibility:

```
Manager  
↓  
Director  
↓  
CEO
```

You can add another handler without heavily modifying existing handlers.

7. Advantages

Loose coupling

The sender doesn't need to know which handler will process the request.

Easy to extend

You can add:

```
SeniorManager
```

without changing the existing handlers.

Flexible ordering

You can change:

```
A → B → C
```

to:

```
A → C → B
```

Single Responsibility

Each handler focuses on one type of processing.

8. Disadvantages

Request may not be handled

If every handler passes the request forward:

```
A → B → C → null
```

nobody may process it.

Debugging can be harder

You need to trace the entire chain.

Too many handlers

A very long chain can make the system unnecessarily complicated.

9. Key Difference from Other Behavioral Patterns

Pattern	Main Purpose
Chain of Responsibility	Pass request through handlers
Command	Encapsulate a request as an object
Mediator	Centralize communication between objects
Observer	Notify multiple interested objects
Strategy	Select one algorithm
State	Change behavior based on state

Remember it like this:

Chain of Responsibility = "I can't handle it, let the next guy try."

For **C#/.NET interviews**, the most important practical example to understand is **ASP.NET Core Middleware**, because its request pipeline closely follows the same chain-of-handlers concept.

2. Command

Command Design Pattern in C#

The **Command Design Pattern** is a **behavioral design pattern** that encapsulates a request as an object.

In simple words:

Instead of directly calling a method, we create an object representing the operation and execute that object.

This is useful when you want to support **undo/redo, queuing, logging, transactions, scheduling, or decoupling the caller from the actual operation.**

1. Problem without Command Pattern

Suppose we have a remote control:

```
public class Light
{
    public void TurnOn()
    {
        Console.WriteLine("Light ON");
    }
}
```

```
    public void TurnOff()
    {
        Console.WriteLine("Light OFF");
    }
}
```

The remote directly depends on `Light`:

```
public class RemoteControl
{
    public void PressOnButton()
    {
        Light light = new Light();
        light.TurnOn();
    }
}
```

This creates tight coupling.

What if tomorrow the button needs to control:

- Light
- Fan
- TV
- AC
- Door

The remote would need to know about all these classes.

2. Command Pattern Solution

The Command pattern introduces an abstraction:

```
Client
|
v
Command
|
v
Receiver
```

The important participants are:

Command

Defines the operation.

Concrete Command

Implements the command and knows which receiver to call.

Receiver

Actually performs the operation.

Invoker

Triggers the command.

Client

Creates and configures the objects.

3. Complete C# Example

Step 1 — Receiver

The receiver contains the actual business operation.

```
public class Light
{
    public void TurnOn()
    {
        Console.WriteLine("Light is ON");
    }

    public void TurnOff()
    {
        Console.WriteLine("Light is OFF");
    }
}
```

Step 2 — Command Interface

```
public interface ICommand
{
    void Execute();
    void Undo();
}
```

We have included `Undo()` because Command is commonly used for undo/redo functionality.

Step 3 — Concrete Command

```
public class LightOnCommand : ICommand
{
    private readonly Light _light;

    public LightOnCommand(Light light)
    {
        _light = light;
    }

    public void Execute()
    {
        _light.TurnOn();
    }

    public void Undo()
    {
        _light.TurnOff();
    }
}
```

Another command:

```
public class LightOffCommand : ICommand
{
    private readonly Light _light;

    public LightOffCommand(Light light)
    {
        _light = light;
    }

    public void Execute()
    {
        _light.TurnOff();
    }

    public void Undo()
    {
        _light.TurnOn();
    }
}
```

Step 4 — Invoker

The invoker doesn't know how the operation is performed.

```
public class RemoteControl
{
    private ICommand? _command;

    public void SetCommand(ICommand command)
    {
        _command = command;
    }

    public void PressButton()
    {
        _command?.Execute();
    }

    public void PressUndo()
    {
        _command?.Undo();
    }
}
```

Step 5 — Client

```
class Program
{
    static void Main()
    {
        Light light = new Light();

        ICommand lightOn = new LightOnCommand(light);
        ICommand lightOff = new LightOffCommand(light);

        RemoteControl remote = new RemoteControl();

        remote.SetCommand(lightOn);

        remote.PressButton();

        remote.PressUndo();

        remote.SetCommand(lightOff);

        remote.PressButton();

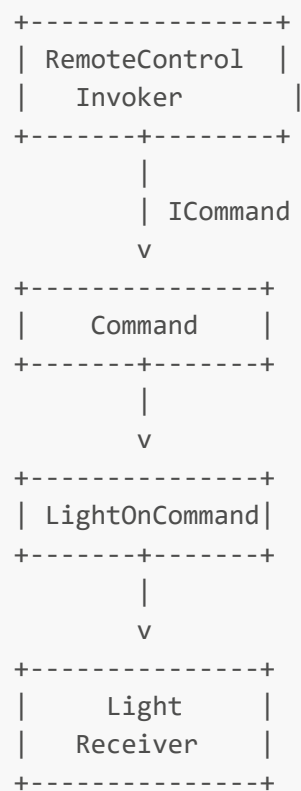
        remote.PressUndo();
    }
}
```

Output:

```
Light is ON
Light is OFF
Light is OFF
Light is ON
```

4. How It Works

The important relationship is:



The `RemoteControl` doesn't need to know:

```
light.TurnOn();
```

Instead, it simply does:

```
command.Execute();
```

That is the main idea of the pattern.

5. Real-World Example: Order Processing

Command becomes more useful in business applications.

Imagine an order system:

```
public class OrderService
{
    public void CreateOrder()
    {
        Console.WriteLine("Order created");
    }

    public void CancelOrder()
    {
        Console.WriteLine("Order cancelled");
    }
}
```

Command:

```
public interface ICommand
{
    void Execute();
}
```

Create command:

```
public class CreateOrderCommand : ICommand
{
    private readonly OrderService _orderService;

    public CreateOrderCommand(OrderService orderService)
    {
        _orderService = orderService;
    }

    public void Execute()
    {
        _orderService.CreateOrder();
    }
}
```

Invoker:


```
public class CommandProcessor
{
    public void Execute(ICommand command)
    {
        command.Execute();
    }
}
```

Usage:

```
var orderService = new OrderService();

var command = new CreateOrderCommand(orderService);

var processor = new CommandProcessor();

processor.Execute(command);
```

Output:

```
Order created
```

6. Command Pattern with Multiple Commands

This is where the pattern becomes powerful.

```
User
|
v
CommandProcessor
|
+----- CreateOrderCommand
|
+----- CancelOrderCommand
|
+----- UpdateOrderCommand
|
+----- SendEmailCommand
|
+----- GenerateInvoiceCommand
```

The processor doesn't need to understand each operation.

```
public void Execute(ICommand command)
{
    command.Execute();
}
```

It only understands:

```
ICommand
```

7. Command Queue

One major advantage is that commands can be stored.

```
Queue<ICommand> commands = new();

commands.Enqueue(new CreateOrderCommand(orderService));
commands.Enqueue(new SendEmailCommand(emailService));
commands.Enqueue(new GenerateInvoiceCommand(invoiceService));
```

Then:

```
while (commands.Count > 0)
{
    ICommand command = commands.Dequeue();

    command.Execute();
}
```

This allows you to build systems involving:

- background jobs
- task queues
- message processing
- scheduled operations
- retry mechanisms

8. Undo/Redo

Command is particularly useful for undo/redo.

For example:

```
Stack<ICommand> history = new();

ICommand command = new LightOnCommand(light);

command.Execute();

history.Push(command);
```

Undo:

```
ICommand lastCommand = history.Pop();

lastCommand.Undo();
```

Conceptually:

```
Execute
  |
  v
Command
  |
  v
History Stack
  |
  v
Undo
```

For multiple undo operations:

```
Stack<ICommand> history = new();

history.Push(command1);
history.Push(command2);
history.Push(command3);
```

Then:

```
history.Pop().Undo();
history.Pop().Undo();
history.Pop().Undo();
```

9. Command vs Strategy

These two patterns can look very similar.

Command	Strategy
Encapsulates a request/action	Encapsulates an algorithm
Usually represents an operation	Usually represents how something is done
Can support undo/redo	Usually doesn't
Can be queued	Usually not
Can be logged	Usually not the primary purpose
<code>Execute()</code> is common	Algorithm-specific method is common

Example:

```
Command
"Delete this order"
```

versus:

```
Strategy
"Calculate shipping using FedEx"
```

10. Command vs Mediator

Command represents an action:

```
CreateOrderCommand
```

Mediator coordinates communication between objects:

```
Controller
|
v
Mediator
|
+--> OrderService
+--> PaymentService
+--> NotificationService
```

In modern ASP.NET Core applications, these patterns can also appear together.

11. Command in ASP.NET Core

A very common architecture is:

```
HTTP Request
  |
  v
Controller
  |
  v
Command
  |
  v
Handler
  |
  v
Service
  |
  v
Database
```

For example:

```
public record CreateUserCommand(
    string Name,
    string Email);
```

Handler:

```
public class CreateUserCommandHandler
{
    public async Task Handle(CreateUserCommand command)
    {
        Console.WriteLine(
            $"Creating user: {command.Name}");
    }
}
```

Controller:

```
[ApiController]
[Route("api/users")]
public class UsersController : ControllerBase
{
    private readonly CreateUserCommandHandler _handler;

    public UsersController(
        CreateUserCommandHandler handler)
    {
        _handler = handler;
    }

    [HttpPost]
    public async Task<IActionResult> Create(
        CreateUserCommand command)
    {
        await _handler.Handle(command);

        return Ok();
    }
}
```

This style is particularly common in **CQRS/MediatR-style architectures**.

12. When Should You Use Command?

Use Command when you need one or more of these:

- **Undo/Redo**
- **Command history**
- **Queue operations**
- **Logging operations**
- **Retry operations**
- **Schedule operations**
- **Decouple caller and receiver**
- **Represent business operations as objects**
- **Implement CQRS**
- **Build job/message processing systems**

13. When NOT to Use It

Don't create commands for every trivial method.

For example, this:

```
user.SetName("John");
```

doesn't necessarily need:

```
SetUserNameCommand
```

If there is no need for:

- queuing
- undo
- logging
- decoupling
- command handling
- transaction/workflow processing

then a normal method may be simpler.

14. Interview Definition

A good interview answer is:

Command is a behavioral design pattern that encapsulates a request as an object, allowing the request to be parameterized, queued, logged, undone, or executed later without the invoker needing to know how the operation is performed.

Remember the structure:

```
Client  
↓  
Invoker  
↓  
Command  
↓  
Receiver
```

The **most important idea** is:

Turn an operation into an object.

3. Interpreter

Interpreter Design Pattern in C#

The **Interpreter Design Pattern** is a **behavioral design pattern** used when you have a language or grammar that needs to be interpreted and executed.

In simple terms:

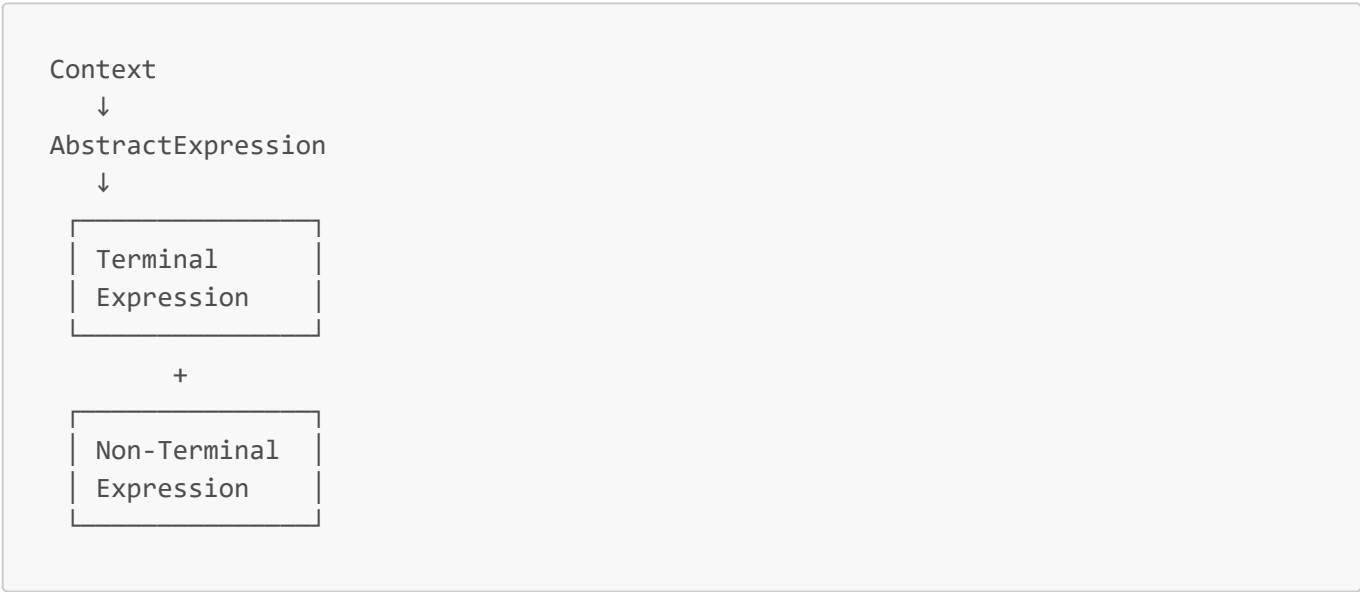
Interpreter converts a sentence/expression written in a particular grammar into an action or result.

It is useful when you need to evaluate expressions such as:

```
5 + 10
10 > 5
AND / OR conditions
price > 100 AND category = 'Electronics'
```

1. Main Components

The Interpreter pattern generally has:



Components

Component	Purpose
Expression	Defines <code>Interpret()</code>
TerminalExpression	Represents a basic value/token
NonTerminalExpression	Combines other expressions
Context	Contains information required during interpretation
Client	Builds the expression tree

2. Simple Example

Let's build an interpreter for:

```
5 + 10
```

Step 1 — Abstract Expression

```
public interface IExpression
{
    int Interpret();
}
```

Every expression must implement:

```
Interpret()
```

Step 2 — Terminal Expression

A number is a terminal expression because it doesn't contain another expression.

```
public class NumberExpression : IExpression
{
    private readonly int _number;

    public NumberExpression(int number)
    {
        _number = number;
    }

    public int Interpret()
    {
        return _number;
    }
}
```

For example:

```
var five = new NumberExpression(5);
```

Calling:

```
five.Interpret();
```

returns:

```
5
```

3. Non-Terminal Expression

Now create an addition expression.

```
public class AddExpression : IExpression
{
    private readonly IExpression _left;
    private readonly IExpression _right;

    public AddExpression(IExpression left, IExpression right)
    {
        _left = left;
        _right = right;
    }

    public int Interpret()
    {
        return _left.Interpret() + _right.Interpret();
    }
}
```

This expression contains two other expressions.

4. Using the Interpreter

```
class Program
{
    static void Main()
    {
        IExpression five = new NumberExpression(5);
        IExpression ten = new NumberExpression(10);

        IExpression addition =
```

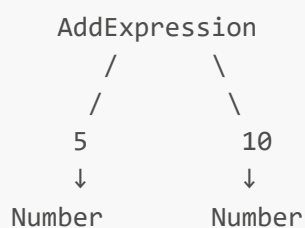
```
        new AddExpression(five, ten);

        Console.WriteLine(addition.Interpret());
    }
}
```

Output:

15

The structure is:



When:

```
addition.Interpret();
```

is called:

```
AddExpression
  ↓
left.Interpret() → 5
right.Interpret() → 10
  ↓
5 + 10
  ↓
15
```

5. Adding Subtraction

We can easily add another expression.

```
public class SubtractExpression : IExpression
{
```

```

    private readonly IExpression _left;
    private readonly IExpression _right;

    public SubtractExpression(
        IExpression left,
        IExpression right)
    {
        _left = left;
        _right = right;
    }

    public int Interpret()
    {
        return _left.Interpret() - _right.Interpret();
    }
}

```

Now:

```

IExpression expression =
    new SubtractExpression(
        new NumberExpression(20),
        new NumberExpression(5));

Console.WriteLine(expression.Interpret());

```

Output:

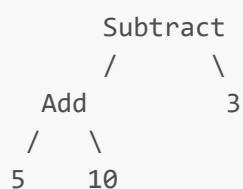
15

6. More Complex Expression

Suppose we want:

5 + 10 - 3

We can construct:



C#:

```
IExpression expression =  
    new SubtractExpression(  
        new AddExpression(  
            new NumberExpression(5),  
            new NumberExpression(10)  
        ),  
        new NumberExpression(3)  
    );  
  
Console.WriteLine(expression.Interpret());
```

Output:

12

This is the key idea of the Interpreter pattern:

Expressions form a tree, and each node knows how to interpret itself.

7. Adding Multiplication

```
public class MultiplyExpression : IExpression  
{  
    private readonly IExpression _left;  
    private readonly IExpression _right;  
  
    public MultiplyExpression(  
        IExpression left,  
        IExpression right)  
    {  
        _left = left;  
        _right = right;  
    }  
  
    public int Interpret()  
    {  
        return _left.Interpret() * _right.Interpret();  
    }  
}
```

Now we can represent:

```
5 + 10 * 2
```

For example:

```
IExpression expression =  
    new AddExpression(  
        new NumberExpression(5),  
        new MultiplyExpression(  
            new NumberExpression(10),  
            new NumberExpression(2)  
        )  
    );  
  
Console.WriteLine(expression.Interpret());
```

Output:

```
25
```

Tree:

```
      Add  
     /  \  
    5   Multiply  
     /   \  
    10    2
```

8. Complete Example

```
public interface IExpression  
{  
    int Interpret();  
}  
  
public class NumberExpression : IExpression  
{  
    private readonly int _number;  
  
    public NumberExpression(int number)  
    {  
        _number = number;  
    }  
}
```

```
    }

    public int Interpret()
    {
        return _number;
    }
}

public class AddExpression : IExpression
{
    private readonly IExpression _left;
    private readonly IExpression _right;

    public AddExpression(
        IExpression left,
        IExpression right)
    {
        _left = left;
        _right = right;
    }

    public int Interpret()
    {
        return _left.Interpret() + _right.Interpret();
    }
}

public class MultiplyExpression : IExpression
{
    private readonly IExpression _left;
    private readonly IExpression _right;

    public MultiplyExpression(
        IExpression left,
        IExpression right)
    {
        _left = left;
        _right = right;
    }

    public int Interpret()
    {
        return _left.Interpret() * _right.Interpret();
    }
}

class Program
{
    static void Main()
    {
        // 5 + 10 * 2

        IExpression expression =
            new AddExpression(
```

```
        new NumberExpression(5),
        new MultiplyExpression(
            new NumberExpression(10),
            new NumberExpression(2)
        )
    );

    int result = expression.Interpret();

    Console.WriteLine(result);
}
}
```

Output:

25

9. Real-World Example

Interpreter becomes more interesting when we interpret **business rules**.

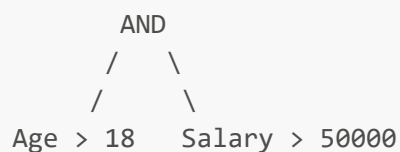
For example:

Age > 18 AND Salary > 50000

We can create expressions such as:

GreaterThanExpression
AndExpression
OrExpression

Conceptually:



This can be used for:

- Rule engines
- Filtering systems

- Search queries
- Permission systems
- Mathematical expressions
- SQL-like expressions
- Configuration languages
- Domain-specific languages (DSLs)
- Validation rules

10. Example: Rule Interpreter

Suppose our context is:

```
public class UserContext
{
    public int Age { get; set; }
    public decimal Salary { get; set; }
}
```

Expression interface:

```
public interface IExpression
{
    bool Interpret(UserContext context);
}
```

Age expression:

```
public class AgeGreaterThanExpression : IExpression
{
    private readonly int _age;

    public AgeGreaterThanExpression(int age)
    {
        _age = age;
    }

    public bool Interpret(UserContext context)
    {
        return context.Age > _age;
    }
}
```

Salary expression:

```
public class SalaryGreaterThanExpression : IExpression
{
    private readonly decimal _salary;

    public SalaryGreaterThanExpression(decimal salary)
    {
        _salary = salary;
    }

    public bool Interpret(UserContext context)
    {
        return context.Salary > _salary;
    }
}
```

AND expression:

```
public class AndExpression : IExpression
{
    private readonly IExpression _left;
    private readonly IExpression _right;

    public AndExpression(
        IExpression left,
        IExpression right)
    {
        _left = left;
        _right = right;
    }

    public bool Interpret(UserContext context)
    {
        return _left.Interpret(context)
            && _right.Interpret(context);
    }
}
```

Usage:

```
var rule =
    new AndExpression(
        new AgeGreaterThanExpression(18),
        new SalaryGreaterThanExpression(50000)
    );

var user = new UserContext
{
    Age = 25,
    Salary = 70000
}
```

```
};

bool result = rule.Interpret(user);

Console.WriteLine(result);
```

Output:

True

The expression tree is:

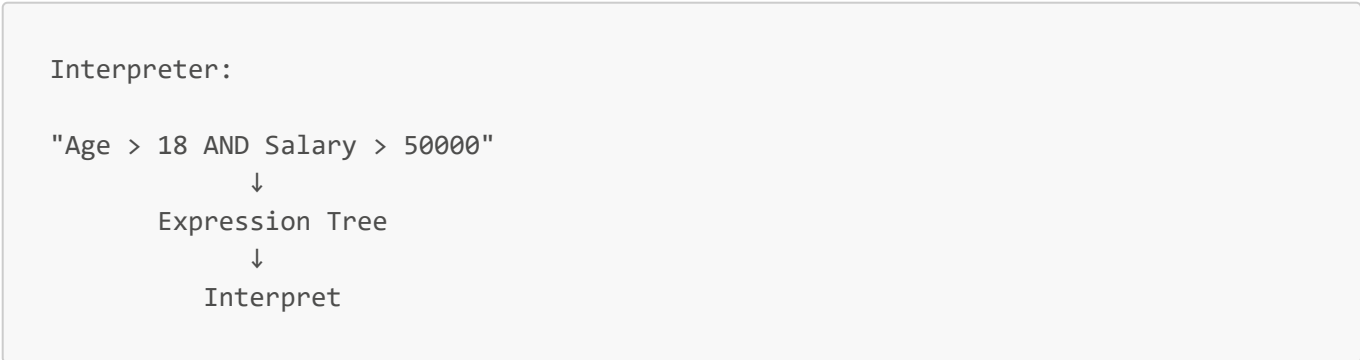


11. Interpreter vs Strategy

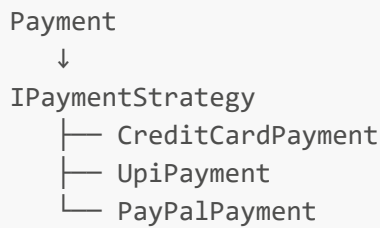
These two patterns can look similar but solve different problems.

Interpreter	Strategy
Interprets a language/grammar	Selects an algorithm
Usually builds expression tree	Usually chooses one implementation
<code>Interpret()</code> is central	Algorithm method is central
Good for rules/DSLs	Good for interchangeable algorithms
Often recursive	Usually not recursive

Example:



Whereas Strategy:



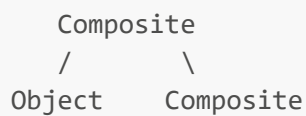
12. Interpreter vs Composite

They are closely related.

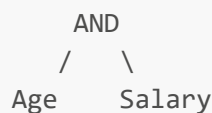
Composite focuses on treating individual objects and groups uniformly.

Interpreter uses a similar tree structure specifically to represent and evaluate a grammar.

Composite:



Interpreter:



In fact, an Interpreter implementation often uses **Composite-like tree structures**.

13. Advantages

☒ Easy to extend grammar

You can add:

```

AddExpression
SubtractExpression
MultiplyExpression
DivideExpression
  
```

```
AndExpression  
OrExpression
```

without changing existing expressions.

☒ Expression tree

Complex expressions can be represented naturally.

☒ Separation of concerns

Each expression knows how to interpret itself.

14. Disadvantages

The biggest problem is **class explosion**.

A complex grammar might require:

```
NumberExpression  
AddExpression  
SubtractExpression  
MultiplyExpression  
DivideExpression  
AndExpression  
OrExpression  
GreaterThanExpression  
LessThanExpression  
...
```

For complex programming languages, manually implementing Interpreter becomes impractical.

That's why real compilers/interpreters usually use more sophisticated techniques involving:

```
Lexer  
↓  
Parser  
↓  
AST  
↓  
Semantic Analysis  
↓  
Interpreter / Compiler
```

15. Where You Might Actually Use It in .NET

For a .NET backend developer, the most practical applications are:

```
Rule Engine
  ↓
Expression
  ↓
Condition
  ↓
AND / OR
  ↓
Action
```

For example, a fintech system could have:

```
IF
    transaction.Amount > 100000
    AND
    transaction.Country != "IN"
THEN
    FlagTransaction()
```

The Interpreter pattern can represent and evaluate the condition:

```
      AND
     /  \
    /    \
Amount > 100000  Country != IN
```

This is one of the most useful ways to understand the pattern beyond the textbook calculator example.

Interview definition

Interpreter is a behavioral design pattern that defines a representation for a language's grammar and provides an interpreter to evaluate sentences in that language.

The key thing to remember is:

Grammar → Expression Tree → Interpret() → Result

4. Iterator

Iterator Design Pattern in C#

The **Iterator Design Pattern** is a **behavioral design pattern** that provides a standard way to traverse elements of a collection **without exposing the internal structure of that collection**.

In simple terms:

Iterator = traverse a collection one element at a time without knowing how the collection is implemented.

C# already provides excellent support for this pattern through:

- `IEnumerable<T>`
- `IEnumerator<T>`
- `yield return`
- `foreach`

1. Problem Without Iterator

Suppose we have a custom collection:

```
class EmployeeCollection
{
    private Employee[] employees =
    {
        new Employee { Id = 1, Name = "John" },
        new Employee { Id = 2, Name = "Alice" },
        new Employee { Id = 3, Name = "Bob" }
    };

    public Employee[] GetEmployees()
    {
        return employees;
    }
}
```

The caller needs to know that the collection internally uses an array:

```
Employee[] employees = collection.GetEmployees();

for (int i = 0; i < employees.Length; i++)
{
    Console.WriteLine(employees[i].Name);
}
```

This creates unnecessary dependency on the collection's internal structure.

What if tomorrow we change:

```
Employee[]
```

to:

```
List<Employee>
```

or:

```
LinkedList<Employee>
```

The traversal logic may need to change.

2. Iterator Pattern

The Iterator pattern separates:

```
Collection  
  ↓  
Iterator  
  ↓  
Traverse elements
```

The caller only knows:

```
"Give me the next employee."
```

It doesn't care whether employees are stored in an array, list, tree, database, etc.

3. C# Built-in Iterator Support

The simplest implementation is:

```
using System.Collections.Generic;  
  
class EmployeeCollection : IEnumerable<Employee>  
{  
    private List<Employee> employees = new()
```



```
{
    new Employee(1, "John"),
    new Employee(2, "Alice"),
    new Employee(3, "Bob")
};

public IEnumerator<Employee> GetEnumerator()
{
    return employees.GetEnumerator();
}

IEnumerator IEnumerable.GetEnumerator()
{
    return GetEnumerator();
}
}
```

Employee:

```
class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }

    public Employee(int id, string name)
    {
        Id = id;
        Name = name;
    }
}
```

Now the client can simply use:

```
EmployeeCollection collection = new EmployeeCollection();

foreach (Employee employee in collection)
{
    Console.WriteLine(employee.Name);
}
```

Output:

```
John
Alice
Bob
```

The client doesn't need to know that the employees are stored in a `List<Employee>`.

4. Understanding `IEnumerable`

The important interface is:

```
IEnumerable<T>
```

It provides:

```
IEnumerator<T> GetEnumerator();
```

The relationship is:

```
IEnumerator<T>
  |
  | GetEnumerator()
  ↓
IEnumerator<T>
  |
  | MoveNext()
  | Current
  | Reset()
  ↓
Elements
```

5. `IEnumerator<T>`

An enumerator keeps track of the current position.

For example:

```
Employee collection

[John] [Alice] [Bob]
  ↑
current position
```

Calling:

```
MoveNext()
```

moves forward:

```
[John] [Alice] [Bob]
      ↑
```

Then:

```
Current
```

returns:

```
Alice
```

6. Manually Using an Enumerator

You can use an enumerator directly:

```
var collection = new EmployeeCollection();

IEnumerator<Employee> iterator = collection.GetEnumerator();

while (iterator.MoveNext())
{
    Employee employee = iterator.Current;

    Console.WriteLine(employee.Name);
}
```

Output:

```
John
Alice
Bob
```

This is essentially what `foreach` does internally.

Conceptually:

```
foreach (var employee in collection)
{
    Console.WriteLine(employee.Name);
}
```

is similar to:

```
var iterator = collection.GetEnumerator();

while (iterator.MoveNext())
{
    var employee = iterator.Current;

    Console.WriteLine(employee.Name);
}
```

7. Using `yield return`

C# makes implementing the Iterator pattern much easier with:

```
yield return
```

For example:

```
class EmployeeCollection
{
    private List<Employee> employees = new()
    {
        new Employee(1, "John"),
        new Employee(2, "Alice"),
        new Employee(3, "Bob")
    };

    public IEnumerable<Employee> GetEmployees()
    {
        foreach (var employee in employees)
        {
            yield return employee;
        }
    }
}
```

Usage:

```
var collection = new EmployeeCollection();

foreach (var employee in collection.GetEmployees())
{
    Console.WriteLine(employee.Name);
}
```

This is usually the preferred approach in modern C# when you need custom iteration.

8. Why `yield return` Is Powerful

Consider:

```
public IEnumerable<int> GetNumbers()
{
    yield return 1;
    yield return 2;
    yield return 3;
}
```

Usage:

```
foreach (var number in GetNumbers())
{
    Console.WriteLine(number);
}
```

The values don't have to be created as a complete collection first.

This is useful for large or generated sequences.

For example:

```
public IEnumerable<int> GetNumbers()
{
    for (int i = 1; i <= 1_000_000; i++)
    {
        yield return i;
    }
}
```

You can consume them one at a time:

```
foreach (var number in GetNumbers())
{
    Console.WriteLine(number);
}
```

9. Real Custom Iterator Example

Imagine a company has employees and we only want employees from the IT department.

```
class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Department { get; set; }
}
```

Collection:

```
class EmployeeCollection
{
    private List<Employee> employees = new()
    {
        new Employee
        {
            Id = 1,
            Name = "John",
            Department = "IT"
        },

        new Employee
        {
            Id = 2,
            Name = "Alice",
            Department = "HR"
        },

        new Employee
        {
            Id = 3,
            Name = "Bob",
            Department = "IT"
        }
    };
}
```

```
public IEnumerable<Employee> GetITEmployees()
{
    foreach (var employee in employees)
    {
        if (employee.Department == "IT")
        {
            yield return employee;
        }
    }
}
```

Client:

```
var collection = new EmployeeCollection();

foreach (var employee in collection.GetITEmployees())
{
    Console.WriteLine(employee.Name);
}
```

Output:

```
John
Bob
```

The collection controls **how iteration happens**, while the client only consumes the results.

10. Iterator with Custom Traversal

Iterator doesn't have to mean simply "go from first to last."

You can define your own traversal.

For example, reverse iteration:

```
class NumberCollection
{
    private int[] numbers = { 10, 20, 30, 40, 50 };

    public IEnumerable<int> ReverseIterator()
    {
        for (int i = numbers.Length - 1; i >= 0; i--)
        {
            yield return numbers[i];
        }
    }
}
```

```
        }  
    }  
}
```

Usage:

```
var collection = new NumberCollection();  
  
foreach (var number in collection.ReverseIterator())  
{  
    Console.WriteLine(number);  
}
```

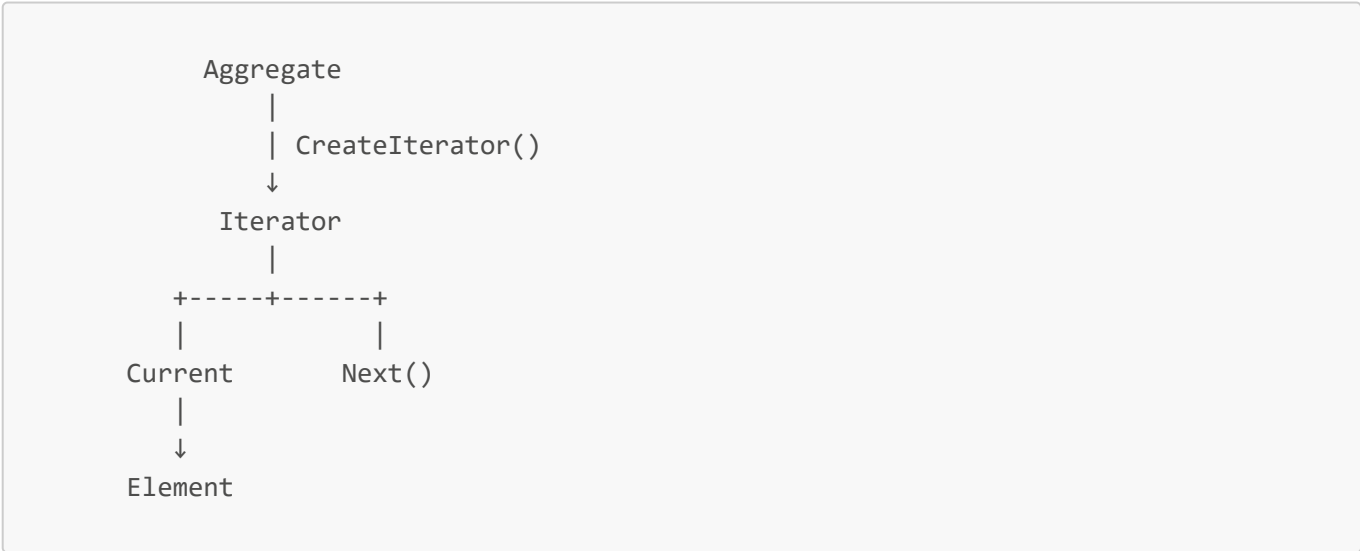
Output:

```
50  
40  
30  
20  
10
```

The important point is that the **iteration algorithm is separated from the client.**

11. Iterator Pattern Structure

The classic GoF Iterator pattern contains:



In C#:

Pattern Concept	C#
-----------------	----

Pattern Concept	C#
Aggregate	<code>IEnumerable<T></code>
Concrete Aggregate	<code>List<T></code> , custom collection
Iterator	<code>IEnumerator<T></code>
Concrete Iterator	Generated/collection enumerator
Traversal	<code>foreach</code> / <code>yield return</code>

12. Iterator vs `foreach`

You normally write:

```
foreach (var item in collection)
{
    Console.WriteLine(item);
}
```

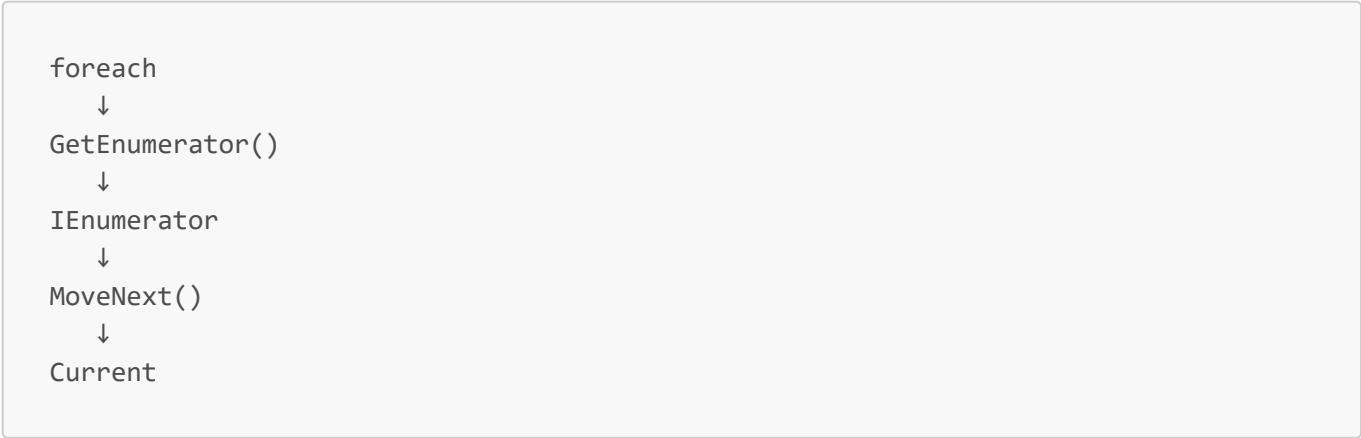
But internally, the concept is:

```
var enumerator = collection.GetEnumerator();

while (enumerator.MoveNext())
{
    var item = enumerator.Current;

    Console.WriteLine(item);
}
```

So:



13. Iterator vs IEnumerable

They are related but different.

`IEnumerable<T>`

Means:

"This object can provide an iterator."

```
IEnumerable<Employee>
```

`IEnumerator<T>`

Means:

"This object is the iterator that keeps track of traversal."

```
IEnumerator<Employee>
```

Think of it like:

```
IEnumerable
  ↓
Factory for iterator

IEnumerator
  ↓
Actual iterator
```

14. Advantages

1. Encapsulation

The internal collection structure is hidden.

2. Simplifies client code

Client simply uses:

```
foreach
```

3. Supports different collection types

The underlying data could be:

```
Array  
List  
LinkedList  
Tree  
Database  
Generated data
```

4. Lazy evaluation

With:

```
yield return
```

data can be generated when requested.

5. Memory efficient

You don't necessarily need to create a complete result collection.

15. Real-World Examples in .NET

You use the Iterator pattern constantly in C# without explicitly thinking about it.

For example:

```
List<int> numbers = new() { 1, 2, 3 };  
  
foreach (var number in numbers)  
{  
    Console.WriteLine(number);  
}
```

Also:

```
Dictionary<int, string>
```

```
HashSet<int>
```

```
Queue<int>
```

```
Stack<int>
```

and LINQ:

```
var result = employees
    .Where(x => x.Department == "IT")
    .OrderBy(x => x.Name);
```

LINQ heavily relies on `IEnumerable<T>` and enumeration.

16. When Should You Use Iterator?

Use it when:

- You need to traverse a custom collection.
- You want to hide collection implementation details.
- You need custom traversal logic.
- You want lazy data generation.
- You want to expose a sequence without exposing the underlying collection.
- You are building reusable collection abstractions.

In normal C# development, you usually **don't implement `IEnumerator<T>` manually**. Prefer:

```
IEnumerable<T>
```

and:

```
yield return
```

when appropriate.

Interview Definition

The Iterator Design Pattern provides a way to sequentially access elements of a collection without exposing its underlying representation. In C#, it is commonly implemented using

IEnumerable<T>, IEnumerator<T>, foreach, and yield return.

Key things to remember

```

Iterator
  ↓
Behavioral Design Pattern

IEnumerable<T>
  ↓
Provides iterator

IEnumerator<T>
  ↓
Traverses collection

MoveNext()
  ↓
Moves to next element

Current
  ↓
Gets current element

yield return
  ↓
Easy custom iterator implementation

foreach
  ↓
Common client-side usage
  
```

5. Mediator

Mediator Design Pattern in C#

The **Mediator Design Pattern** is a **behavioral design pattern** that reduces direct communication between multiple objects.

Instead of objects communicating directly with each other, they communicate through a **Mediator**.

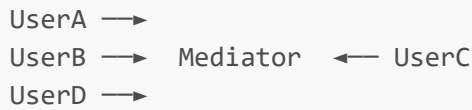
Simple idea

Without Mediator:





With Mediator:



The objects don't need to know about each other.

1. Problem without Mediator

Suppose we have a chat application.

```

public class User
{
    public void SendMessage(User user, string message)
    {
        // Send message directly
    }
}
  
```

Now imagine:

```

User1 → User2
User1 → User3
User2 → User3
User3 → User4
...
  
```

Every **User** needs references to other users.

This creates **tight coupling**.

2. Mediator Solution

Create a mediator that manages communication.

```

public interface IChatMediator
{
  
```

```
void SendMessage(string message, User sender);  
}
```

The users communicate through the mediator.

```
public abstract class User  
{  
    protected IChatMediator Mediator;  
    public string Name { get; }  
  
    protected User(IChatMediator mediator, string name)  
    {  
        Mediator = mediator;  
        Name = name;  
    }  
  
    public abstract void Send(string message);  
  
    public abstract void Receive(string message);  
}
```

Concrete user:

```
public class ChatUser : User  
{  
    public ChatUser(IChatMediator mediator, string name)  
        : base(mediator, name)  
    {  
    }  
  
    public override void Send(string message)  
    {  
        Console.WriteLine($"{Name} sends: {message}");  
  
        Mediator.SendMessage(message, this);  
    }  
  
    public override void Receive(string message)  
    {  
        Console.WriteLine($"{Name} received: {message}");  
    }  
}
```

Mediator implementation:

```
public class ChatMediator : IChatMediator  
{
```

```
private readonly List<User> _users = new();

public void Register(User user)
{
    _users.Add(user);
}

public void SendMessage(string message, User sender)
{
    foreach (var user in _users)
    {
        if (user != sender)
        {
            user.Receive(message);
        }
    }
}
```

Usage:

```
var mediator = new ChatMediator();

var john = new ChatUser(mediator, "John");
var david = new ChatUser(mediator, "David");
var alice = new ChatUser(mediator, "Alice");

mediator.Register(john);
mediator.Register(david);
mediator.Register(alice);

john.Send("Hello everyone!");
```

Output:

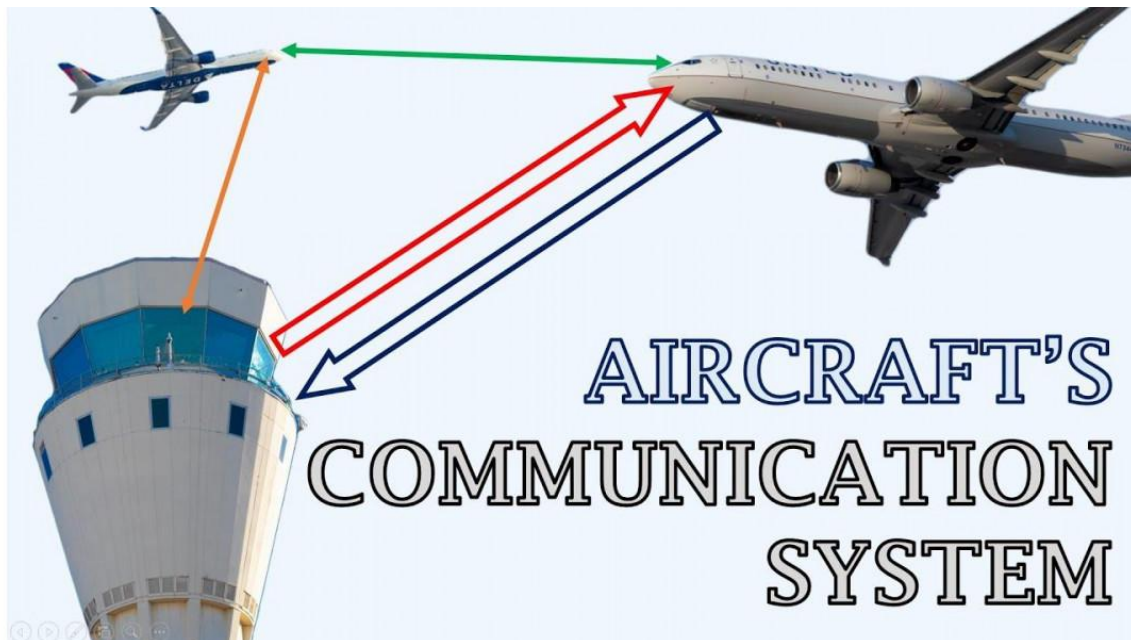
```
John sends: Hello everyone!
David received: Hello everyone!
Alice received: Hello everyone!
```

Notice that **John doesn't know about David or Alice.**

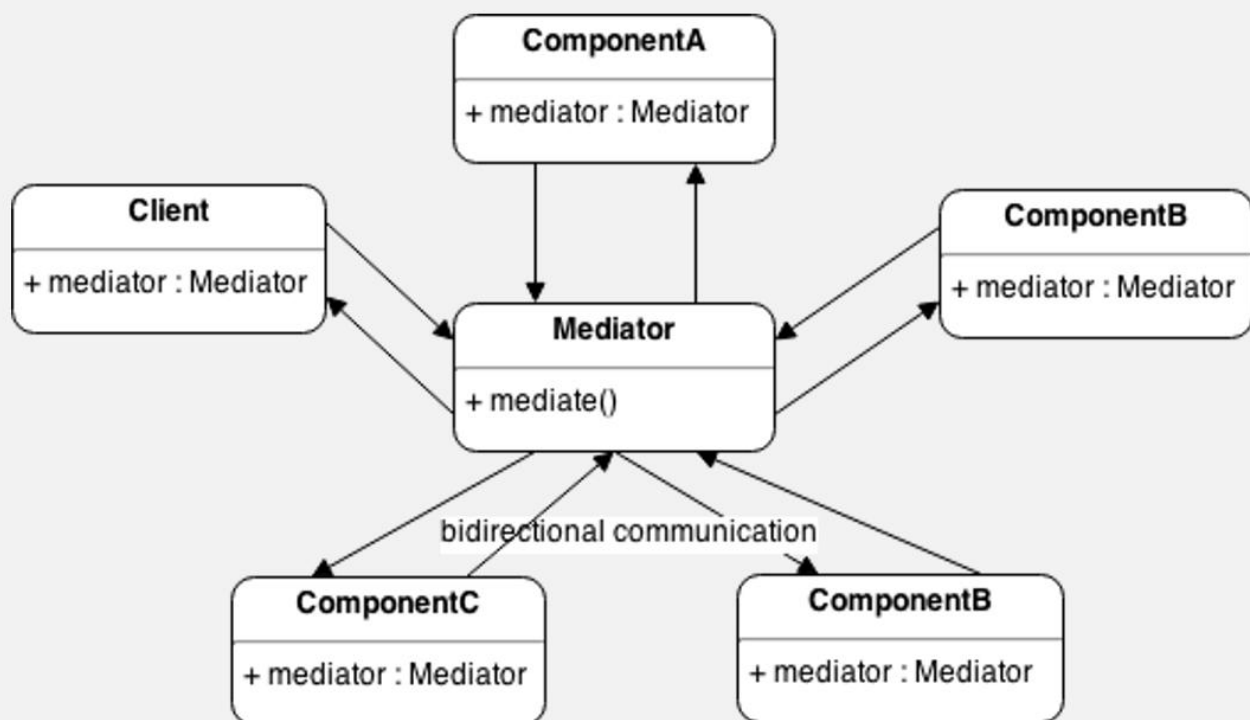
He only knows about the mediator.

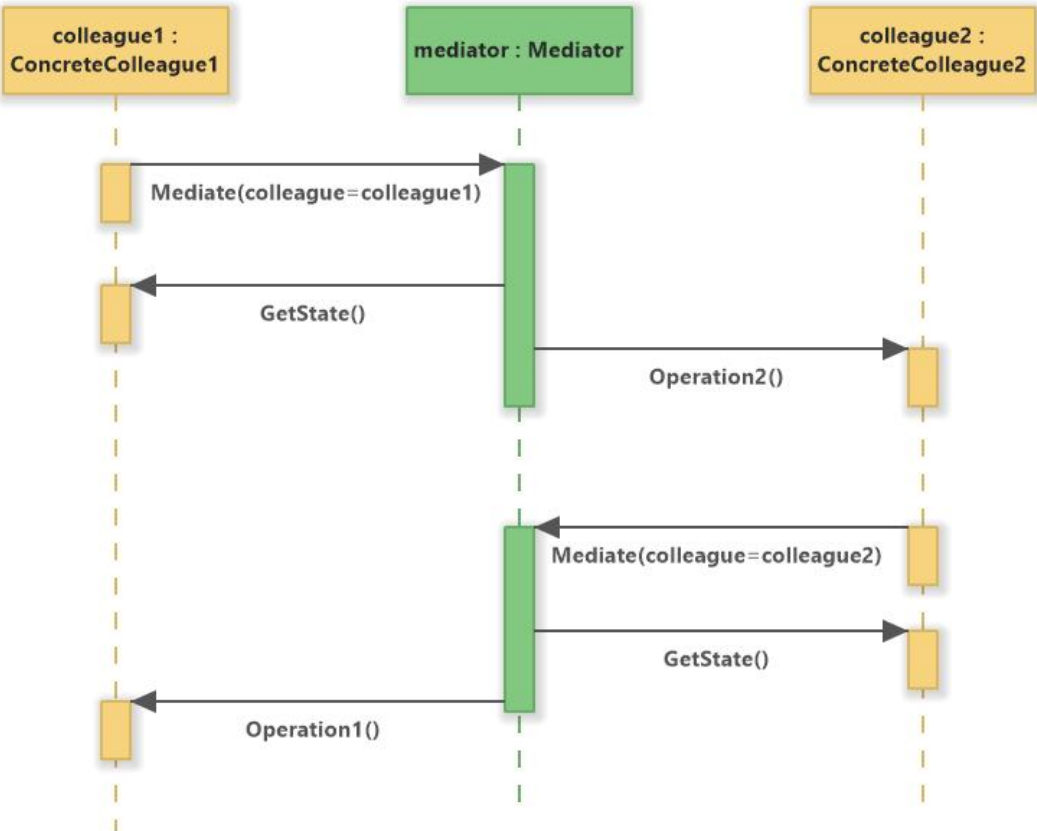
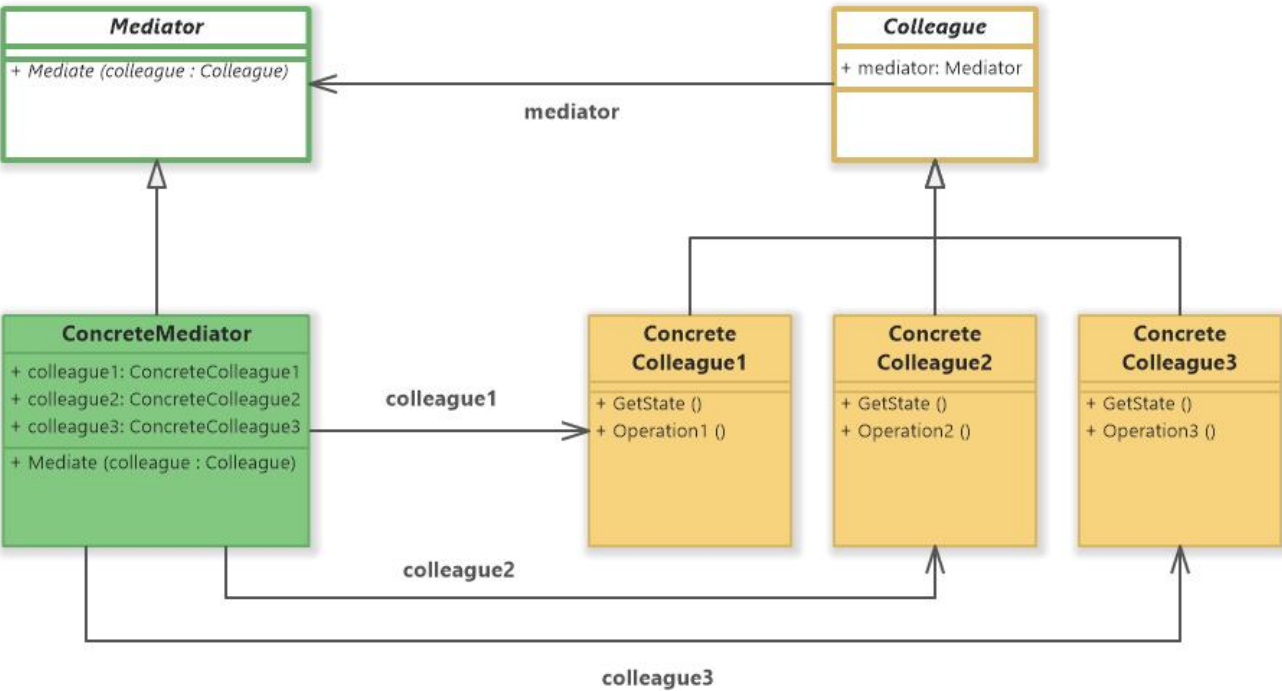
3. Real-world example

A good analogy is an **air traffic control tower**.



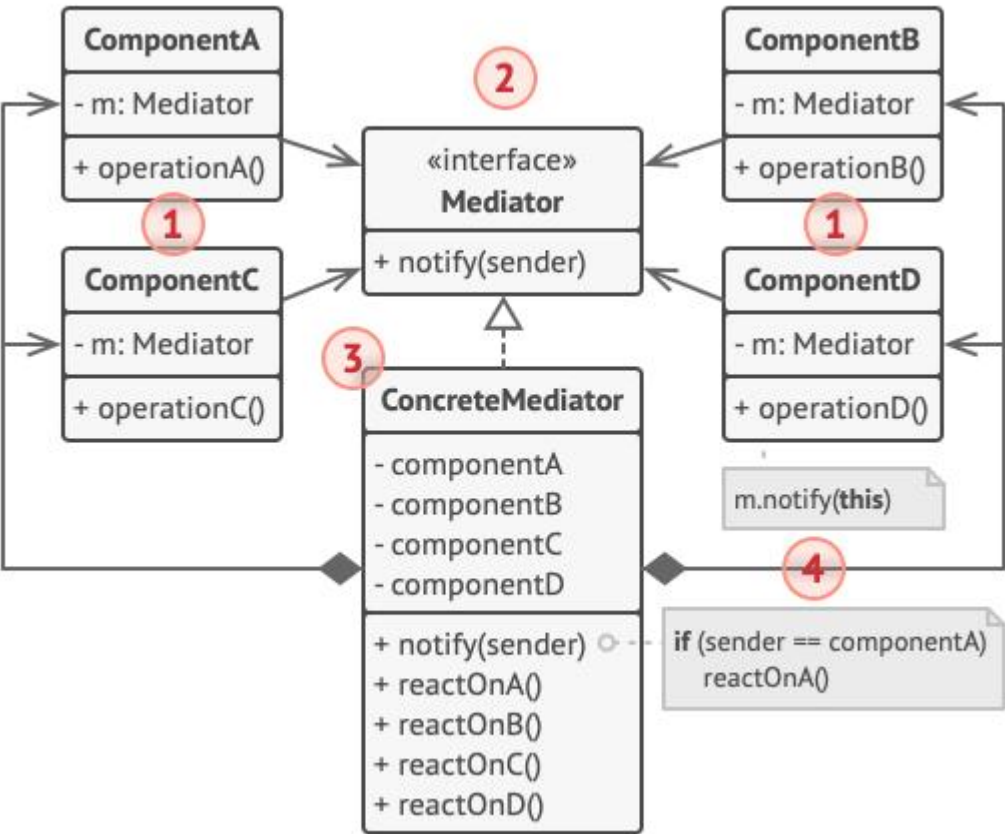
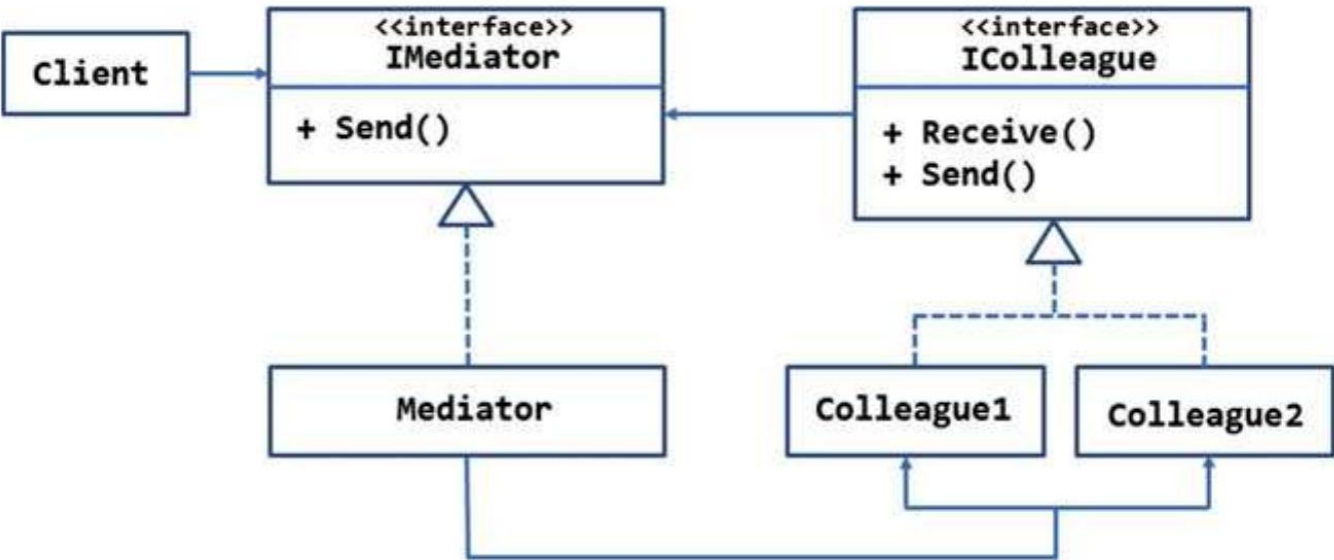
Mediator pattern – Class diagram





design pattern

Mediator

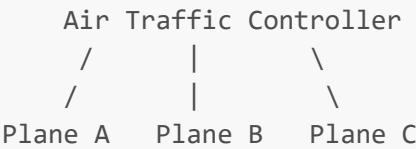


Without a mediator:

Plane A ↔ Plane B
Plane A ↔ Plane C
Plane B ↔ Plane C
Plane C ↔ Plane D

This becomes complicated.

With an air traffic controller:

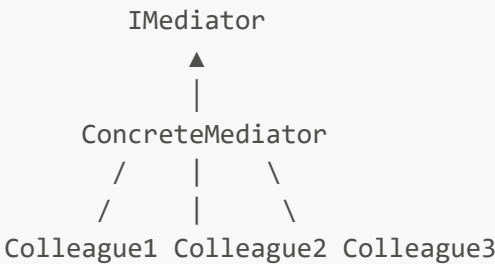


Planes communicate through the controller.

The **air traffic controller is the Mediator**.

4. Mediator Structure

The typical structure is:



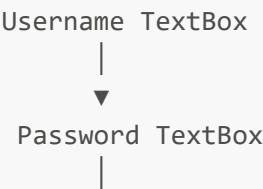
Components

Component	Responsibility
IMediator	Defines communication
ConcreteMediator	Coordinates objects
Colleague	Object that communicates through mediator
ConcreteColleague	Actual implementation

5. Another C# Example — Login Form

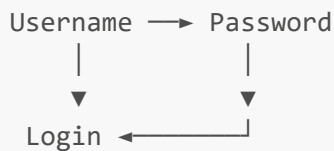
Mediator is very useful for UI components.

Imagine a login form:



▼
Login Button

Instead of components directly communicating with each other:



Use a mediator:



Example:

```
public interface ILoginMediator
{
    void Notify(object sender, string eventName);
}
```

Components:

```
public class UsernameBox
{
    private readonly ILoginMediator _mediator;

    public UsernameBox(ILoginMediator mediator)
    {
        _mediator = mediator;
    }

    public void EnterUsername(string username)
    {
        Console.WriteLine($"Username entered: {username}");

        _mediator.Notify(this, "UsernameChanged");
    }
}
```

```
public class PasswordBox
{
    private readonly ILoginMediator _mediator;

    public PasswordBox(ILoginMediator mediator)
    {
        _mediator = mediator;
    }

    public void EnterPassword(string password)
    {
        Console.WriteLine("Password entered");

        _mediator.Notify(this, "PasswordChanged");
    }
}
```

Mediator:

```
public class LoginMediator : ILoginMediator
{
    private readonly UsernameBox _username;
    private readonly PasswordBox _password;

    public LoginMediator(
        UsernameBox username,
        PasswordBox password)
    {
        _username = username;
        _password = password;
    }

    public void Notify(object sender, string eventName)
    {
        if (eventName == "UsernameChanged")
        {
            Console.WriteLine("Mediator: Username changed");
        }

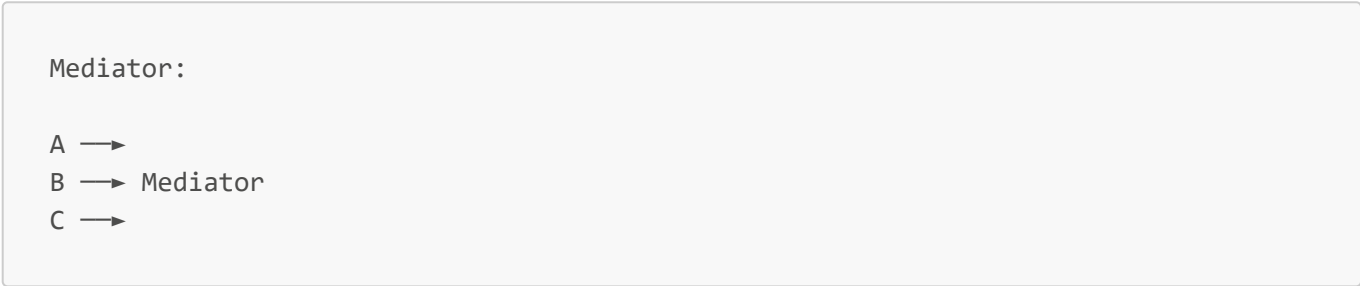
        if (eventName == "PasswordChanged")
        {
            Console.WriteLine("Mediator: Password changed");
        }
    }
}
```

6. Mediator vs Observer

These patterns can look similar.

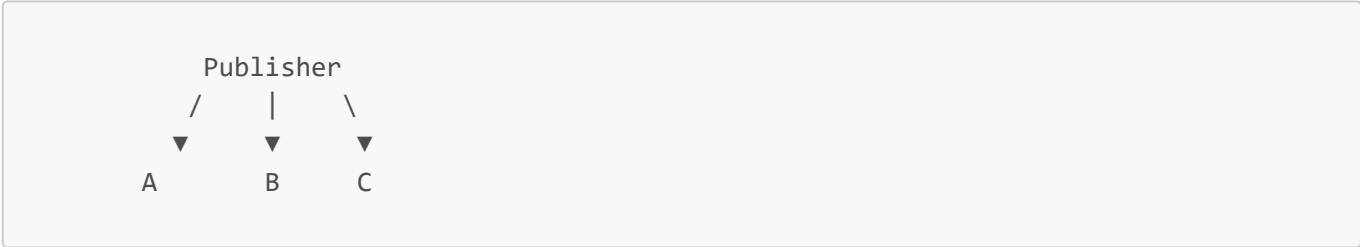
Mediator	Observer
Central object coordinates communication	Publisher broadcasts notifications
Controls interaction between objects	Mainly handles event notification
Usually contains business interaction logic	Usually contains subscription logic
Reduces many-to-many communication	Implements one-to-many notification
Common in workflows/UI coordination	Common in events

For example:



The mediator decides **what should happen**.

Observer:



The publisher says:

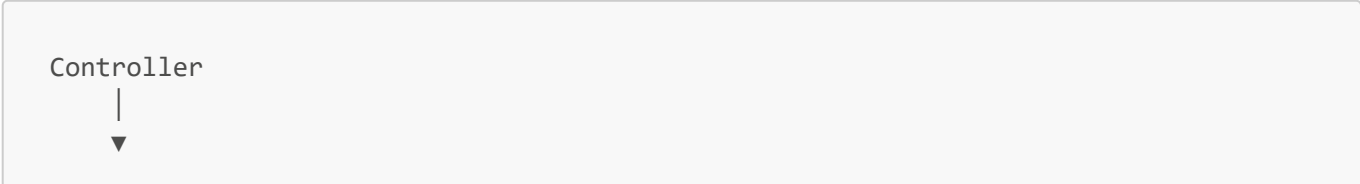
"Something happened."

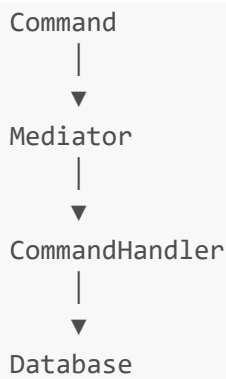
Observers react.

7. Mediator in ASP.NET Core

You will frequently encounter the **Mediator concept** in ASP.NET Core applications through libraries such as **MediatR**.

For example:





A simplified command:

```
public record CreateUserCommand(  
    string Name,  
    string Email);
```

Handler:

```
public class CreateUserHandler  
{  
    public async Task Handle(CreateUserCommand command)  
    {  
        // Business logic  
        Console.WriteLine($"Creating {command.Name}");  
    }  
}
```

Controller:

```
public class UsersController : ControllerBase  
{  
    private readonly IMediator _mediator;  
  
    public UsersController(IMediator mediator)  
    {  
        _mediator = mediator;  
    }  
  
    [HttpPost]  
    public async Task<IActionResult> Create(CreateUserCommand command)  
    {  
        await _mediator.Send(command);  
  
        return Ok();  
    }  
}
```


Conceptually:

```
HTTP Request
  ↓
Controller
  ↓
Mediator
  ↓
Handler
  ↓
Service
  ↓
Repository
  ↓
Database
```

This is one reason the pattern is popular in **CQRS architectures**.

8. Advantages

1. Reduces coupling

Objects don't need references to many other objects.

2. Centralizes communication

Communication rules are located in one place.

3. Easier maintenance

Changing interaction logic can often be done inside the mediator.

4. Good for complex workflows

Useful when many objects interact with each other.

5. Supports SRP

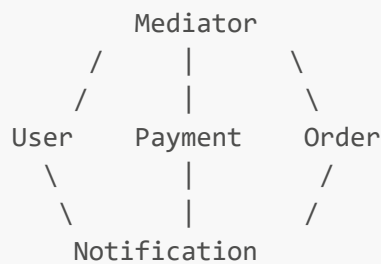
Individual components focus on their own responsibilities while the mediator coordinates them.

9. Disadvantages

The biggest problem is:

The mediator itself can become too complicated.

For example:



If everything is placed into one mediator, it can become a **God Object**.

Therefore, don't use Mediator just because multiple classes exist.

10. When should you use Mediator?

Use it when:

- Many objects communicate with each other.
- Objects have many dependencies on each other.
- Communication logic is becoming complicated.
- You want to centralize workflow coordination.
- You are implementing UI interactions.
- You are implementing CQRS.
- You want controllers to delegate requests to handlers.

Avoid it when:

- Only two objects communicate.
- Communication is extremely simple.
- The mediator would contain almost all application logic.

Interview Definition

Mediator is a behavioral design pattern that encapsulates how a set of objects interact with each other, reducing direct dependencies between them by making them communicate through a central mediator.

Remember it like this:

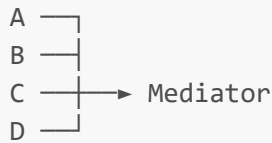
Without Mediator:

```

A ↔ B
↑   ↓
C ↔ D
  
```

Many dependencies ✕

With Mediator:



Centralized communication ✓

Key idea: "Don't let objects talk directly to each other; let the mediator coordinate them."

6. Memento

Memento Design Pattern in C#

The **Memento Design Pattern** is a **behavioral design pattern** used to **save and restore the previous state of an object without exposing its internal details**.

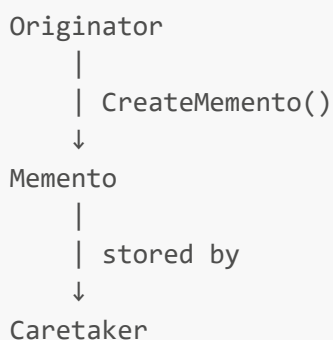
A common example is:

Undo/Redo functionality in a text editor.

You type something → save the state → make changes → if you don't like them, restore the previous state.

1. Main Components

The Memento pattern usually has three components:



1. Originator

The object whose state needs to be saved.

Example:

```
class TextEditor
{
    public string Text { get; set; }

    public EditorMemento Save()
    {
        return new EditorMemento(Text);
    }

    public void Restore(EditorMemento memento)
    {
        Text = memento.Text;
    }
}
```

2. Memento

Stores a snapshot of the Originator's state.

```
class EditorMemento
{
    public string Text { get; }

    public EditorMemento(string text)
    {
        Text = text;
    }
}
```

3. Caretaker

Keeps the saved states but doesn't modify them.

```
class History
{
    private readonly Stack<EditorMemento> _history = new();

    public void Save(EditorMemento memento)
    {
        _history.Push(memento);
    }

    public EditorMemento Undo()
    {
        return _history.Pop();
    }
}
```

2. Complete Example

Let's create a simple **text editor with Undo**.

```
using System;
using System.Collections.Generic;

class TextEditor
{
    public string Text { get; set; }

    public EditorMemento Save()
    {
        return new EditorMemento(Text);
    }

    public void Restore(EditorMemento memento)
    {
        Text = memento.Text;
    }
}

class EditorMemento
{
    public string Text { get; }

    public EditorMemento(string text)
    {
        Text = text;
    }
}

class History
{
    private readonly Stack<EditorMemento> _states = new();

    public void Save(EditorMemento memento)
    {
        _states.Push(memento);
    }

    public EditorMemento Undo()
    {
        if (_states.Count == 0)
            throw new InvalidOperationException("Nothing to undo.");

        return _states.Pop();
    }
}
```

```
class Program
{
    static void Main()
    {
        var editor = new TextEditor();
        var history = new History();

        editor.Text = "Hello";

        // Save state
        history.Save(editor.Save());

        editor.Text = "Hello World";

        // Save state
        history.Save(editor.Save());

        editor.Text = "Hello World!";

        Console.WriteLine(editor.Text);

        // Undo
        editor.Restore(history.Undo());

        Console.WriteLine(editor.Text);

        // Undo again
        editor.Restore(history.Undo());

        Console.WriteLine(editor.Text);
    }
}
```

Output:

```
Hello World!
Hello World
Hello
```

3. How It Works

Suppose the editor goes through these states:

```
State 1
Text = "Hello"

↓ Save
```

```
Memento 1
"Hello"

      ↓

State 2
Text = "Hello World"

      ↓ Save

Memento 2
"Hello World"

      ↓

State 3
Text = "Hello World!"
```

The history contains:

```
Stack
-----
"Hello World"
"Hello"
-----
```

When `Undo()` is called:

```
editor.Restore(history.Undo());
```

the latest state is retrieved:

```
"Hello World"
```

and restored.

4. Why use Memento?

Without Memento, you might expose all of an object's internal state:

```
editor.GetInternalState();
editor.SetInternalState(...);
```

This creates **tight coupling** between the outside code and the object's implementation.

Memento lets the object control how its state is saved:

```
public EditorMemento Save()  
{  
    return new EditorMemento(Text);  
}
```

The Caretaker only knows:

```
Save state  
Store state  
Restore state
```

It doesn't need to know how the state is represented internally.

5. Real-World Examples

Memento is useful for:

Text editors

```
Type  
↓  
Save  
↓  
Type more  
↓  
Save  
↓  
Undo
```

Game save points

```
Player  
├── Health  
├── Level  
├── Position  
└── Inventory
```


Save all of these into a Memento:

```
GameMemento Save()
```

Then restore:

```
player.Restore(savePoint);
```

Undo/Redo

```
Action 1
  ↓
Action 2
  ↓
Action 3
  ↓
Undo
  ↓
Action 2
```

Form editing

Save the state of a form before changes:

```
Name
Email
Address
Phone
```

Then restore the previous form state.

6. Memento with Multiple Properties

A more realistic example:

```
class Player
{
    public int Health { get; private set; }
    public int Level { get; private set; }
    public int X { get; private set; }
    public int Y { get; private set; }
```

```
public PlayerMemento Save()
{
    return new PlayerMemento(
        Health,
        Level,
        X,
        Y);
}

public void Restore(PlayerMemento memento)
{
    Health = memento.Health;
    Level = memento.Level;
    X = memento.X;
    Y = memento.Y;
}
}
```

Memento:

```
class PlayerMemento
{
    public int Health { get; }
    public int Level { get; }
    public int X { get; }
    public int Y { get; }

    public PlayerMemento(
        int health,
        int level,
        int x,
        int y)
    {
        Health = health;
        Level = level;
        X = x;
        Y = y;
    }
}
```

Now you can create checkpoints:

```
var checkpoint = player.Save();
```

and later:

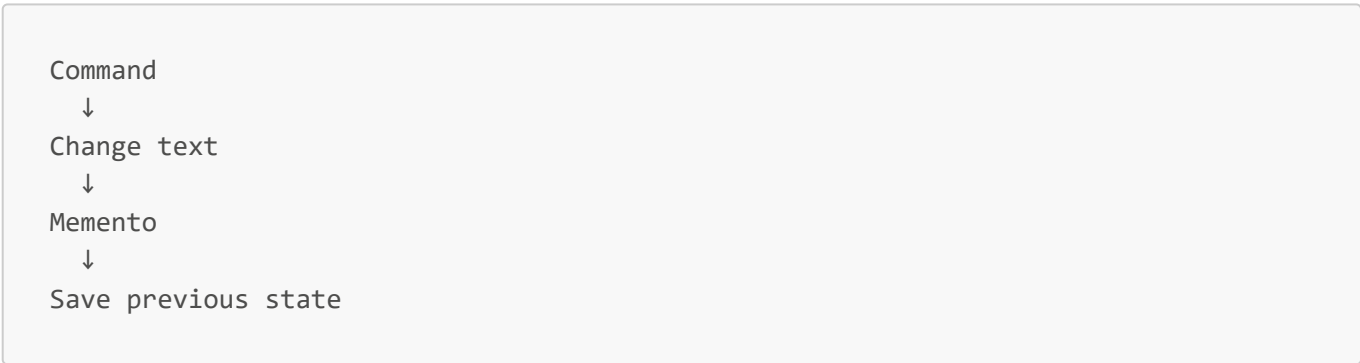
```
player.Restore(checkpoint);
```

7. Memento vs Command

These two patterns are often used together for **Undo**.

Memento	Command
Stores object state	Stores an operation
State-based undo	Action-based undo
Easy to implement	More flexible
Can consume more memory	Usually stores less state
Useful for snapshots	Useful for reversible operations

For example:



A sophisticated editor might use both.

8. Advantages

Encapsulation

Internal state doesn't need to be exposed.

Easy Undo

```
editor.Restore(previousState);
```

Simple implementation

Especially useful for relatively small objects.

Separation of responsibility

```
Originator → creates/restores state
Memento   → stores state
Caretaker → manages history
```

9. Disadvantages

Memory usage

If the object is large:

```
10 MB state
×
100 snapshots
=
1 GB
```

So storing complete snapshots can become expensive.

Deep-copy problems

If the object contains mutable objects:

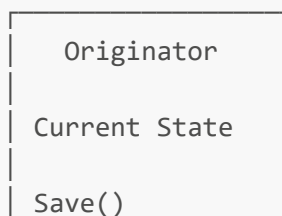
```
List<Item>
Dictionary<string, object>
```

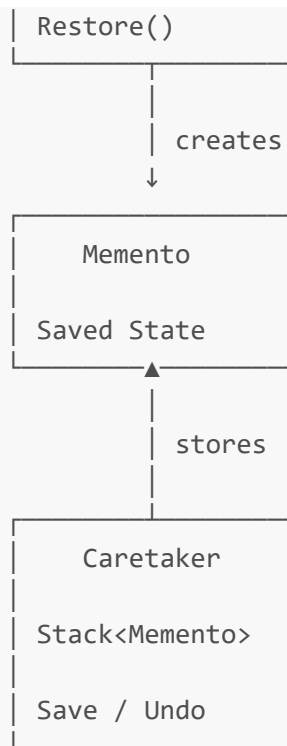
you may need a proper **deep copy** rather than simply copying references.

Large histories

Undo systems can potentially keep many Mementos in memory.

10. Memento Pattern Structure





Interview Definition

Memento is a behavioral design pattern that captures and stores an object's internal state so that the object can be restored to that state later, without violating encapsulation.

Remember it as:

```

Memento = Snapshot

Originator = Object being saved

Memento = Saved snapshot

Caretaker = History manager
  
```

For C# applications, the most common practical use is **Undo/Redo, checkpoints, transaction-like rollback, and restoring previous UI/editor/game state.**

7. Observer

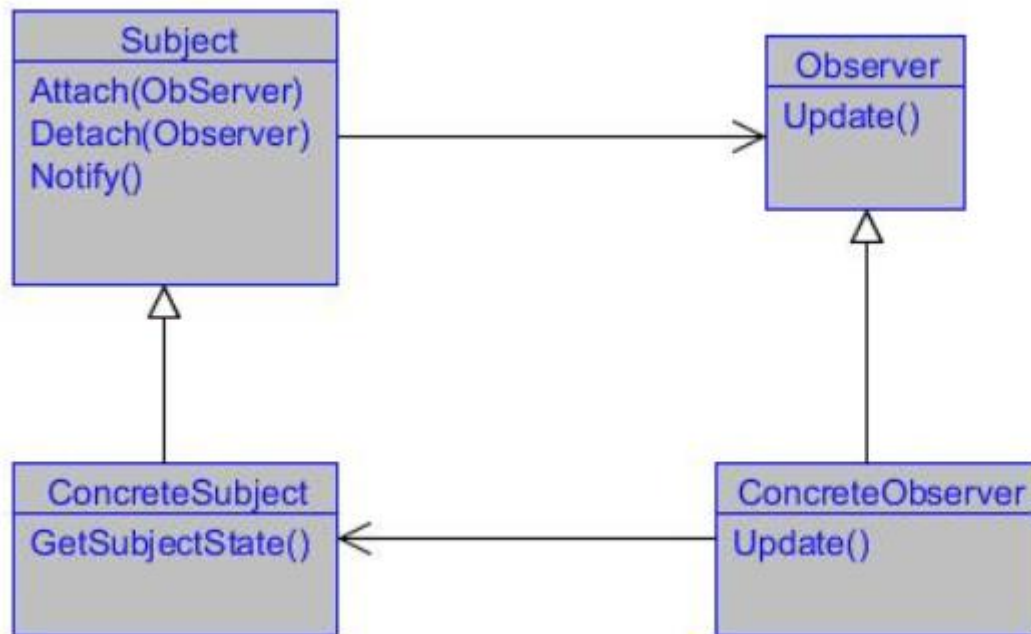
Observer Design Pattern in C#

The **Observer Design Pattern** is a **behavioral design pattern** where one object (the **Subject**) maintains a list of dependent objects (**Observers**) and automatically notifies them when its state changes.

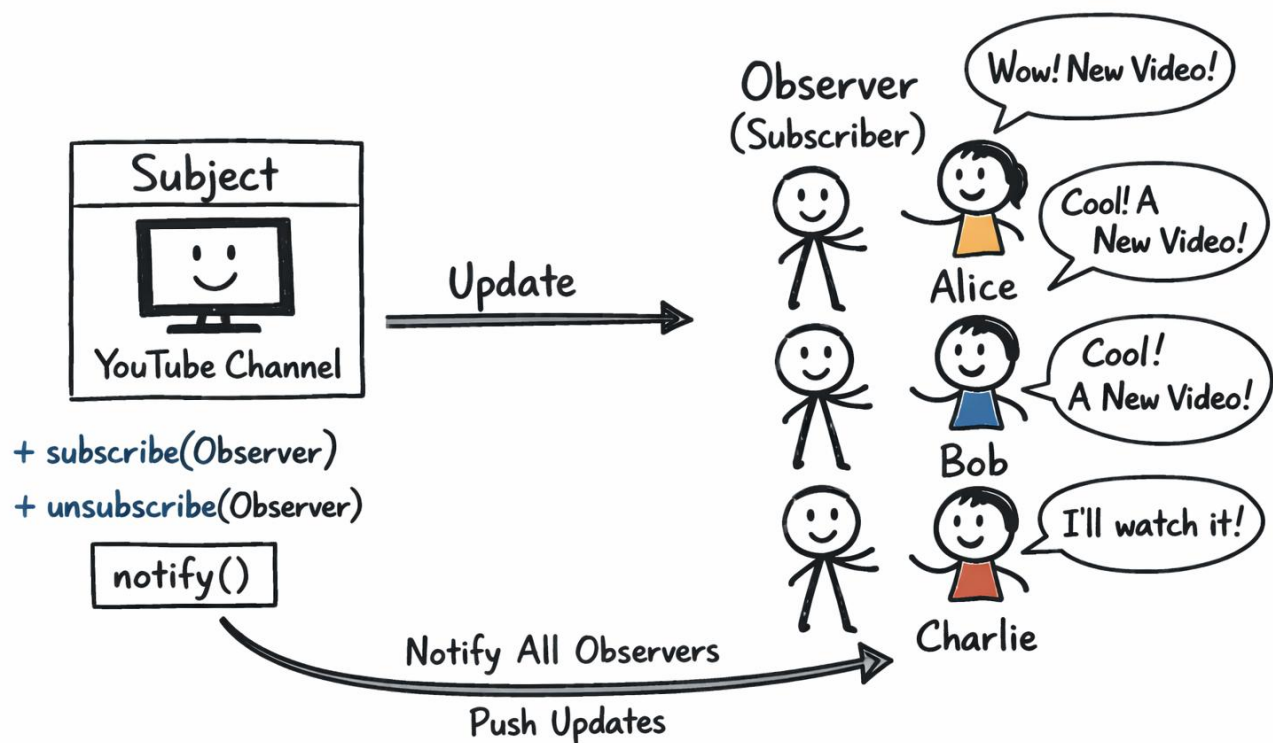
Real-world example

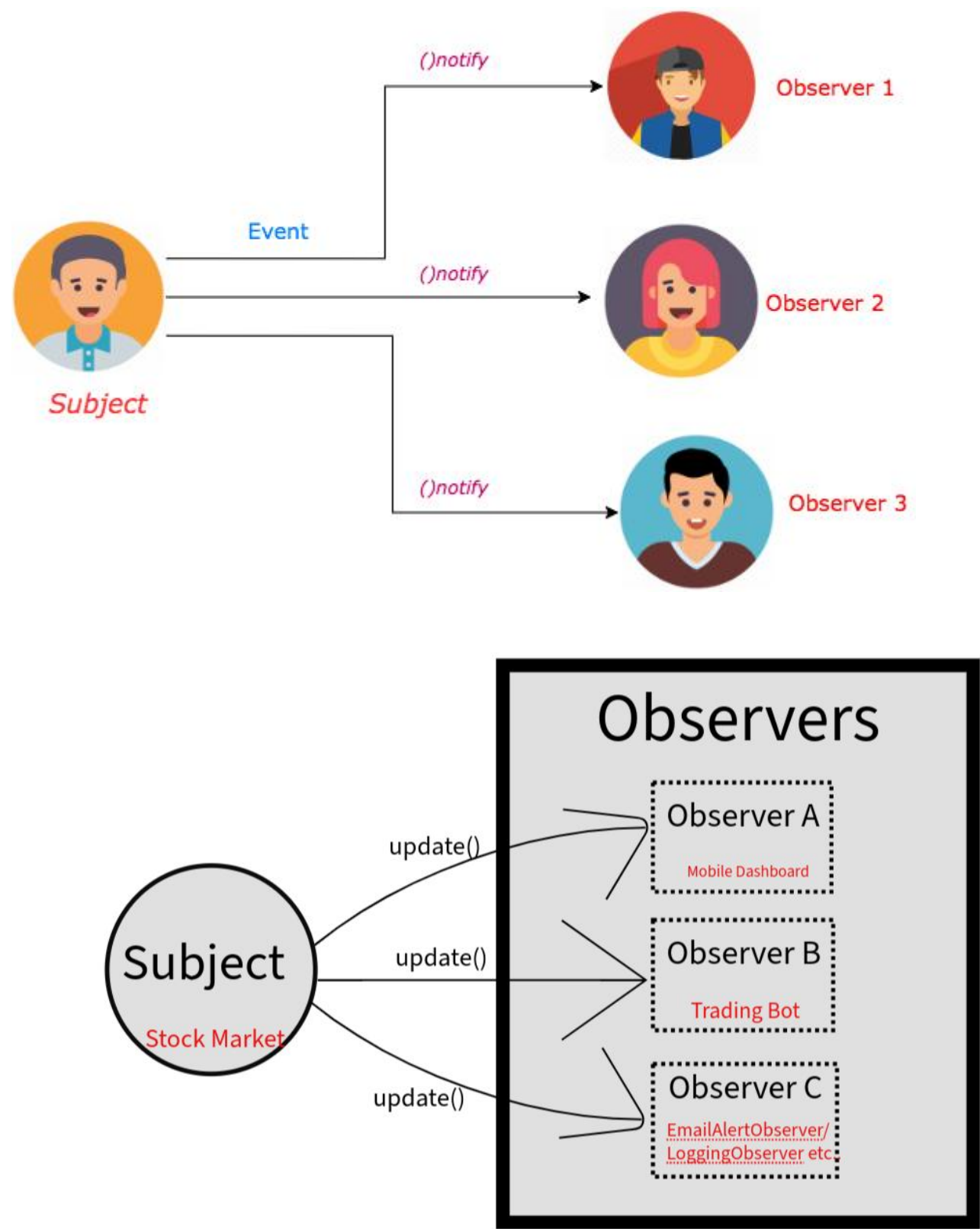
Think of a **YouTube channel**:

- YouTube Channel → **Subject**
- Subscribers → **Observers**
- New video uploaded → **State change**
- All subscribers receive notification → **Notify observers**

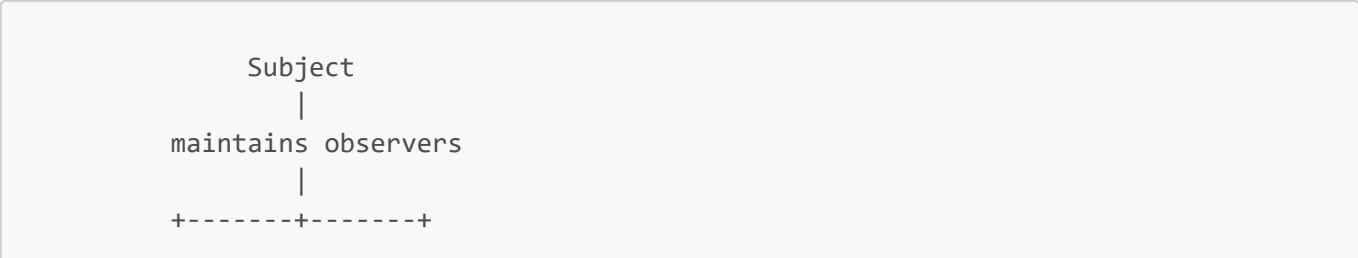


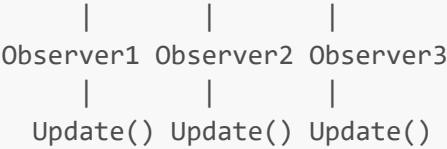
Observer Design Pattern





1. Basic Structure





The main participants are:

Component	Responsibility
Subject	Maintains observers
Observer	Receives notifications
ConcreteSubject	Contains actual state
ConcreteObserver	Performs action when notified

2. Simple C# Example

Suppose we have a **Stock Market** application.

When the stock price changes, multiple investors should be notified.

Observer interface

```
public interface IObserver
{
    void Update(decimal price);
}
```

Subject interface

```
public interface ISubject
{
    void Subscribe(IObserver observer);
    void Unsubscribe(IObserver observer);
    void Notify();
}
```

Concrete Subject

```
public class Stock : ISubject
{
    private readonly List<IObserver> _observers = new();
}
```



```
private decimal _price;

public decimal Price
{
    get => _price;

    set
    {
        _price = value;
        Notify();
    }
}

public void Subscribe(IObserver observer)
{
    _observers.Add(observer);
}

public void Unsubscribe(IObserver observer)
{
    _observers.Remove(observer);
}

public void Notify()
{
    foreach (var observer in _observers)
    {
        observer.Update(_price);
    }
}
}
```

Concrete Observer

```
public class Investor : IObserver
{
    private readonly string _name;

    public Investor(string name)
    {
        _name = name;
    }

    public void Update(decimal price)
    {
        Console.WriteLine(
            $"{_name} received notification. New price: {price}");
    }
}
```

Usage

```
Stock stock = new Stock();

Investor investor1 = new Investor("Rahul");
Investor investor2 = new Investor("Amit");
Investor investor3 = new Investor("Priya");

stock.Subscribe(investor1);
stock.Subscribe(investor2);
stock.Subscribe(investor3);

stock.Price = 100;
```

Output:

```
Rahul received notification. New price: 100
Amit received notification. New price: 100
Priya received notification. New price: 100
```

Now:

```
stock.Price = 150;
```

Output:

```
Rahul received notification. New price: 150
Amit received notification. New price: 150
Priya received notification. New price: 150
```

3. Unsubscribe

One important feature is that an observer can stop receiving notifications.

```
stock.Unsubscribe(investor2);

stock.Price = 200;
```

Now only:

```
Rahul received notification. New price: 200
Priya received notification. New price: 200
```

Amit no longer receives notifications.

4. Observer Pattern Using C# Events

In real C# applications, you will frequently use **events**, which are essentially a language-supported way of implementing the observer idea.

For example:

```
public class Stock
{
    private decimal _price;

    public event EventHandler<StockPriceChangedEventArgs>? PriceChanged;

    public decimal Price
    {
        get => _price;

        set
        {
            if (_price == value)
                return;

            decimal oldPrice = _price;
            _price = value;

            PriceChanged?.Invoke(
                this,
                new StockPriceChangedEventArgs(oldPrice, value));
        }
    }
}
```

Event arguments:

```
public class StockPriceChangedEventArgs : EventArgs
{
    public decimal OldPrice { get; }
    public decimal NewPrice { get; }

    public StockPriceChangedEventArgs(
        decimal oldPrice,
```

```
        decimal newPrice)
    {
        OldPrice = oldPrice;
        NewPrice = newPrice;
    }
}
```

Observer/subscriber:

```
public class Investor
{
    private readonly string _name;

    public Investor(string name)
    {
        _name = name;
    }

    public void OnPriceChanged(
        object? sender,
        StockPriceChangedEventArgs e)
    {
        Console.WriteLine(
            $"{_name}: Stock changed from " +
            $"{e.OldPrice} to {e.NewPrice}");
    }
}
```

Usage:

```
var stock = new Stock();

var rahul = new Investor("Rahul");
var amit = new Investor("Amit");

stock.PriceChanged += rahul.OnPriceChanged;
stock.PriceChanged += amit.OnPriceChanged;

stock.Price = 100;
stock.Price = 150;
```

Unsubscribe:

```
stock.PriceChanged -= amit.OnPriceChanged;
```

This is very common C# code.

5. ASP.NET Core Example

Observer-like behavior is useful in backend systems.

Imagine:

```
Order Created
|
+----> Send Email
|
+----> Send Notification
|
+----> Update Analytics
|
+----> Update Inventory
```

Instead of putting everything into `CreateOrder()`:

```
public async Task CreateOrder(Order order)
{
    await SaveOrder(order);

    await SendEmail(order);
    await SendNotification(order);
    await UpdateAnalytics(order);
    await UpdateInventory(order);
}
```

you can publish an event:

```
public record OrderCreatedEvent(int OrderId);
```

Then different handlers can respond to it:

```
OrderCreatedEvent
|
+---- EmailHandler
|
+---- NotificationHandler
|
+---- AnalyticsHandler
|
+---- InventoryHandler
```

This creates **loose coupling** between the order creation process and the systems interested in the event.

6. Observer vs Event

They are closely related but not exactly the same.

Observer Pattern	C# Event
Design pattern	Language feature
Uses Subject/Observer abstractions	Uses event and delegates
Can be implemented manually	Built into C#
More general	Very convenient in C#
Common in design-pattern discussions	Common in real C# applications

For normal C# applications, **events are often the simplest implementation of the Observer pattern.**

7. Advantages

Loose coupling

The subject doesn't need to know what observers actually do.

```
Stock
|
+-- Investor
+-- EmailService
+-- MobileNotification
```

The stock only knows:

```
observer.Update(price);
```

Easy to add observers

You can add:

```
MobileApp
EmailService
TradingDashboard
LoggingService
```

without changing the subject's core logic.

Dynamic subscription

Observers can subscribe/unsubscribe at runtime.

8. Disadvantages

Too many notifications

If there are thousands of observers:

```
Subject
|
+-- Observer 1
+-- Observer 2
+-- Observer 3
...
+-- Observer 10000
```

a single state change may trigger a lot of work.

Memory leaks with events

If an object subscribes to a long-lived publisher and never unsubscribes, it can remain referenced longer than intended.

```
publisher.Event += subscriber.Handle;
```

Sometimes you need:

```
publisher.Event -= subscriber.Handle;
```

Notification ordering

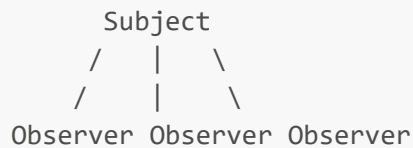
If multiple observers are notified, you may not want your business logic to depend on the order in which they execute.

9. Observer vs Mediator

Since you're learning the behavioral patterns, this distinction is important.

Observer

One object notifies **many interested objects**.



Example:

```
Stock Price Changed
      ↓
Investors notified
```

Mediator

Multiple objects communicate **through a central mediator**.



Example:

```
User A → Chat Mediator → User B
```

Easy way to remember:

Observer = "Something changed; tell everyone interested."

Mediator = "Don't talk directly; talk through me."

10. When Should You Use Observer?

Use Observer when:

- One object's state changes.
- Multiple objects are interested in that change.

- You don't want the subject tightly coupled to those objects.
- Subscribers may change dynamically.
- You need event-driven behavior.

Typical examples:

```
Stock price updates
UI events
Order events
Notifications
Logging
Real-time dashboards
Cache invalidation
Domain events
Message/event systems
```

Interview definition

Observer is a behavioral design pattern that defines a one-to-many dependency between objects so that when the state of one object changes, all its registered observers are automatically notified and updated.

In **C#**, the pattern is commonly implemented using **interfaces, delegates, and events**.

8. State

State Design Pattern in C#

The **State Design Pattern** is a **behavioral design pattern** that allows an object to **change its behavior when its internal state changes**.

Instead of writing a large **if/else** or **switch** statement for every possible state, we move each state's behavior into a separate class.

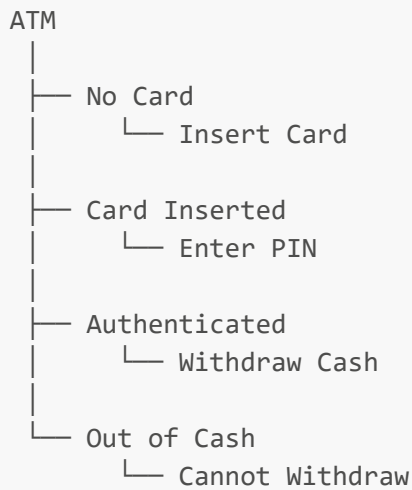
Simple real-world example

Think about an **ATM**:

- **NoCardState**
- **CardInsertedState**
- **AuthenticatedState**
- **CashWithdrawalState**

The ATM behaves differently depending on its current state.

For example:



1. Problem without State Pattern

Suppose we implement an order system:

```
public class Order
{
    public string Status { get; set; }

    public void Process()
    {
        if (Status == "New")
        {
            Console.WriteLine("Processing new order");
        }
        else if (Status == "Paid")
        {
            Console.WriteLine("Order is already paid");
        }
        else if (Status == "Shipped")
        {
            Console.WriteLine("Order has been shipped");
        }
        else if (Status == "Cancelled")
        {
            Console.WriteLine("Order is cancelled");
        }
    }
}
```

As the number of states grows:

New
Paid

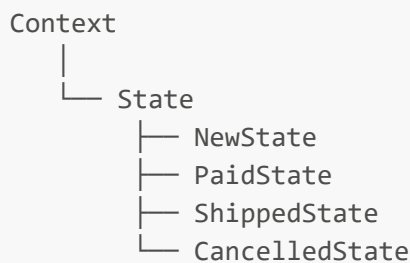
```
Processing
Shipped
Delivered
Cancelled
Refunded
Returned
```

the class becomes full of conditional statements.

This violates the **Open/Closed Principle** and becomes difficult to maintain.

2. State Pattern Solution

We create:



The **Context** maintains the current state.

3. State Interface

```
public interface IOrderState
{
    void Process(OrderContext order);
    void Cancel(OrderContext order);
}
```

Every state must implement these operations.

4. Context

The **OrderContext** maintains the current state.

```
public class OrderContext
{
    private IOrderState _state;
```

```
public OrderContext()
{
    _state = new NewState();
}

public void SetState(IOrderState state)
{
    _state = state;
}

public void Process()
{
    _state.Process(this);
}

public void Cancel()
{
    _state.Cancel(this);
}
}
```

Notice that `OrderContext` doesn't know the detailed behavior of every state.

5. New State

```
public class NewState : IOrderState
{
    public void Process(OrderContext order)
    {
        Console.WriteLine("Processing new order...");

        order.SetState(new PaidState());
    }

    public void Cancel(OrderContext order)
    {
        Console.WriteLine("Order cancelled.");

        order.SetState(new CancelledState());
    }
}
```

When processing a new order, we move it to `PaidState`.

6. Paid State

```
public class PaidState : IOrderState
{
    public void Process(OrderContext order)
    {
        Console.WriteLine("Order is paid.");

        order.SetState(new ShippedState());
    }

    public void Cancel(OrderContext order)
    {
        Console.WriteLine("Refunding payment...");

        order.SetState(new CancelledState());
    }
}
```

7. Shipped State

```
public class ShippedState : IOrderState
{
    public void Process(OrderContext order)
    {
        Console.WriteLine("Order has been shipped.");
    }

    public void Cancel(OrderContext order)
    {
        Console.WriteLine("Cannot cancel shipped order.");
    }
}
```

8. Cancelled State

```
public class CancelledState : IOrderState
{
    public void Process(OrderContext order)
    {
        Console.WriteLine("Cannot process a cancelled order.");
    }

    public void Cancel(OrderContext order)
    {
    }
}
```

```
        Console.WriteLine("Order is already cancelled.");  
    }  
}
```

9. Using the Pattern

```
class Program  
{  
    static void Main()  
    {  
        var order = new OrderContext();  
  
        order.Process();  
        order.Process();  
        order.Process();  
        order.Cancel();  
    }  
}
```

Output:

```
Processing new order...  
Order is paid.  
Order has been shipped.  
Cannot cancel shipped order.
```

The important point is that:

```
order.Process();
```

does different things depending on the current state.

10. How it works

Initially:

```
OrderContext  
  |  
  v  
NewState
```

Calling:

```
order.Process();
```

causes:

```
NewState.Process()  
  |  
  v  
PaidState
```

Next:

```
order.Process();
```

causes:

```
PaidState.Process()  
  |  
  v  
ShippedState
```

So the state transitions are:

```
New  
↓  
Paid  
↓  
Shipped
```

11. Main Components

Component	Responsibility
IOrderState	Defines state behavior
NewState	Behavior when order is new
PaidState	Behavior when order is paid

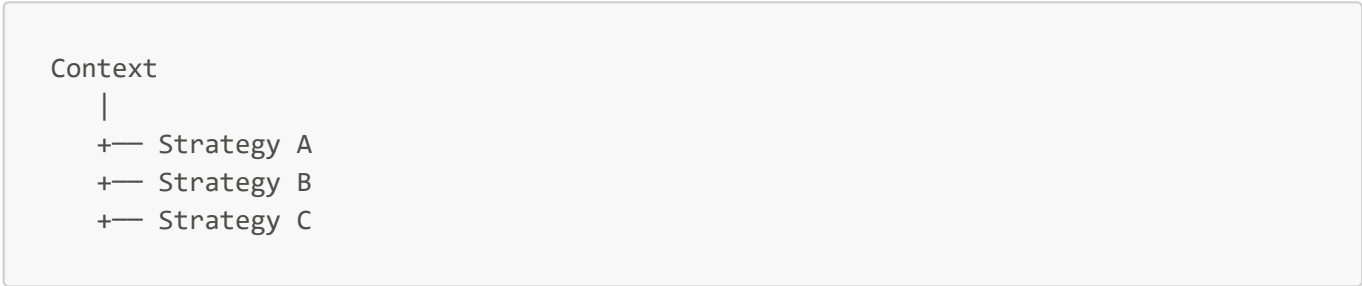
Component	Responsibility
ShippedState	Behavior when order is shipped
CancelledState	Behavior when order is cancelled
OrderContext	Maintains current state

12. State Pattern vs Strategy Pattern

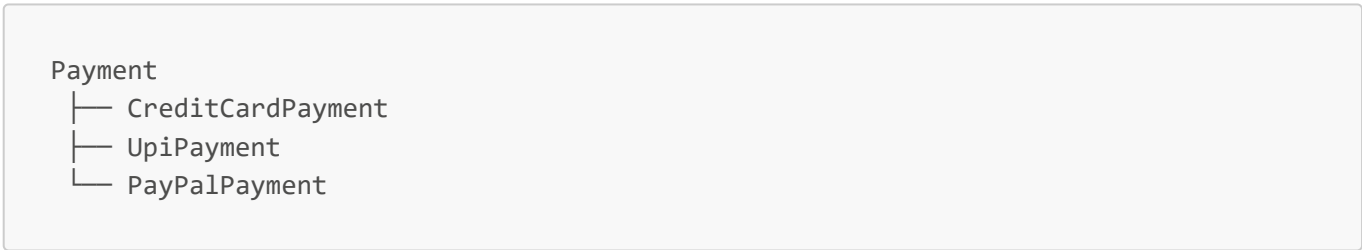
These two patterns look very similar because both use composition and interfaces.

Strategy

The client usually chooses the algorithm:



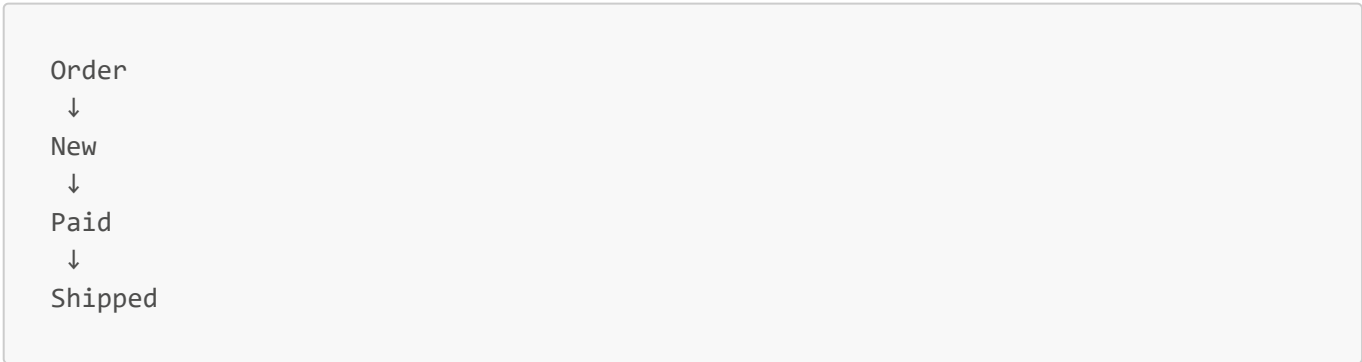
Example:



The client chooses the strategy.

State

The object's behavior changes because its **state changes**:



The state objects often trigger transitions to other states.

Easy way to remember

Strategy = choose an algorithm.

State = change behavior based on current state.

13. When should you use State Pattern?

Use it when:

- An object has many states.
- Behavior changes depending on the state.
- You have large **if/else** or **switch** statements.
- State transitions are becoming complicated.
- You want each state to have its own class.
- You expect new states to be added.

For example:

```
Order System
Payment System
ATM
Vending Machine
Traffic Light
Media Player
Document Workflow
Game Character
Ticket System
```

14. State Pattern in ASP.NET Core

A realistic backend example could be a **payment transaction**:

```
Created
  ↓
Processing
  ↓
Success
```

or:

```
Created
  ↓
```

```
Processing
  ↓
Failed
  ↓
Retrying
  ↓
Processing
```

You could have:

```
IPaymentState
├── CreatedState
├── ProcessingState
├── SuccessState
├── FailedState
└── RetryingState
```

This is particularly useful when business rules become complex.

15. Advantages

Without State Pattern

```
if (status == "New")
{
}
else if (status == "Paid")
{
}
else if (status == "Shipped")
{
}
else if (status == "Cancelled")
{
}
```

With State Pattern

```
_state.Process(this);
```

Each state contains its own behavior.

Benefits:

- Cleaner code
 - Less conditional logic
 - Follows **Single Responsibility Principle**
 - Easier to test
 - Easier to add new states
 - State-specific logic is isolated
-

16. Disadvantages

State Pattern can be overkill when there are only two or three simple states.

It also introduces more classes:

```
OrderContext
IOrderState
NewState
PaidState
ShippedState
CancelledState
```

So don't use it simply because a class has a `Status` property.

Use it when **state-dependent behavior is sufficiently complex**.

Interview Definition

The State Design Pattern is a behavioral design pattern that allows an object to alter its behavior when its internal state changes. It encapsulates state-specific behavior into separate classes and allows the context to switch between those states.

Key interview question

If an interviewer asks:

"Why would you use State Pattern instead of a switch statement?"

A strong answer is:

When state-dependent behavior becomes large or changes frequently, a switch statement creates a highly complex class. State Pattern moves each state's behavior into its own class, making the system easier to maintain, test, and extend without modifying a large conditional block.

9. Strategy

Strategy Design Pattern in C#

The **Strategy Design Pattern** is a **behavioral design pattern** that lets you define multiple algorithms/behaviors, put each one in a separate class, and **select the required behavior at runtime**.

Simple idea

Instead of doing this:

```
if (paymentType == "CreditCard")
{
    // Credit card payment
}
else if (paymentType == "UPI")
{
    // UPI payment
}
else if (paymentType == "PayPal")
{
    // PayPal payment
}
```

we create separate strategies:

```
PaymentService
|
+---- CreditCardStrategy
+---- UpiStrategy
+---- PayPalStrategy
```

The **PaymentService** doesn't need to know the details of each payment method.

1. Real-world example

Suppose an application supports:

- Credit Card
- UPI
- PayPal

All of them perform payment, but the implementation is different.

We can define a common interface:

```
public interface IPaymentStrategy
{
    void Pay(decimal amount);
}
```

Credit Card Strategy

```
public class CreditCardPayment : IPaymentStrategy
{
    public void Pay(decimal amount)
    {
        Console.WriteLine($"Paid ₹{amount} using Credit Card.");
    }
}
```

UPI Strategy

```
public class UpiPayment : IPaymentStrategy
{
    public void Pay(decimal amount)
    {
        Console.WriteLine($"Paid ₹{amount} using UPI.");
    }
}
```

PayPal Strategy

```
public class PayPalPayment : IPaymentStrategy
{
    public void Pay(decimal amount)
    {
        Console.WriteLine($"Paid ₹{amount} using PayPal.");
    }
}
```

2. Context

The **Context** is the class that uses a strategy.

```
public class PaymentService
{
    private IPaymentStrategy _paymentStrategy;

    public PaymentService(IPaymentStrategy paymentStrategy)
    {
        _paymentStrategy = paymentStrategy;
    }
}
```

```
    public void MakePayment(decimal amount)
    {
        _paymentStrategy.Pay(amount);
    }
}
```

Now we can choose the strategy at runtime.

```
class Program
{
    static void Main()
    {
        IPaymentStrategy strategy = new UpiPayment();

        PaymentService paymentService =
            new PaymentService(strategy);

        paymentService.MakePayment(5000);
    }
}
```

Output:

```
Paid ₹5000 using UPI.
```

We can change the strategy:

```
IPaymentStrategy strategy = new CreditCardPayment();

PaymentService paymentService =
    new PaymentService(strategy);

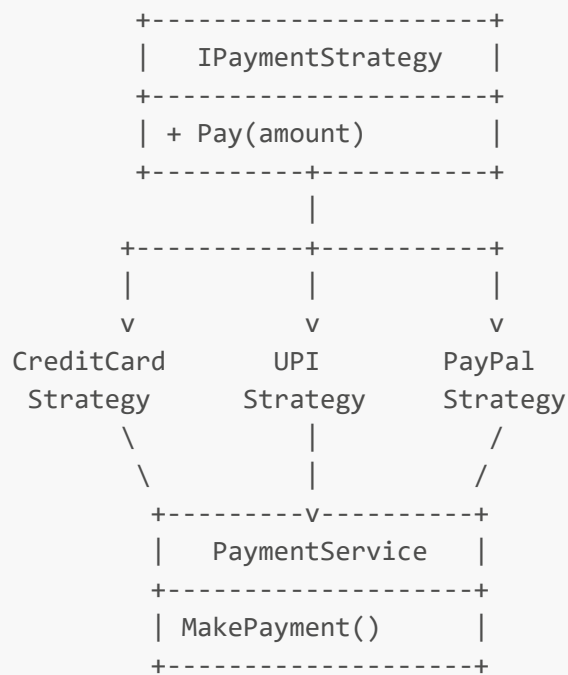
paymentService.MakePayment(5000);
```

Output:

```
Paid ₹5000 using Credit Card.
```

No changes are required inside `PaymentService`.

3. Structure



There are three important components:

Strategy

Defines the common interface.

```

public interface IPaymentStrategy
{
    void Pay(decimal amount);
}
  
```

Concrete Strategies

Implement different algorithms.

```

CreditCardPayment
UpiPayment
PayPalPayment
  
```

Context

Uses a strategy.

```

PaymentService
  
```

4. Strategy with Dependency Injection

In ASP.NET Core, Strategy Pattern works very well with **Dependency Injection**.

Register strategies:

```
builder.Services.AddTransient<IPaymentStrategy, UpiPayment>();
```

But if you have multiple strategies, a better approach is to register all of them:

```
builder.Services.AddTransient<CreditCardPayment>();  
builder.Services.AddTransient<UpiPayment>();  
builder.Services.AddTransient<PayPalPayment>();
```

Then select one based on the request.

For example:

```
public interface IPaymentStrategy  
{  
    string PaymentType { get; }  
  
    void Pay(decimal amount);  
}
```

Implementations:

```
public class UpiPayment : IPaymentStrategy  
{  
    public string PaymentType => "UPI";  
  
    public void Pay(decimal amount)  
    {  
        Console.WriteLine($"UPI payment: ₹{amount}");  
    }  
}
```

```
public class CreditCardPayment : IPaymentStrategy  
{  
    public string PaymentType => "CreditCard";  
  
    public void Pay(decimal amount)  
    {
```



```
        Console.WriteLine($"Credit Card payment: ₹{amount}");  
    }  
}
```

Now a factory/resolver can select the strategy.

5. Another common example — Discount calculation

Suppose an e-commerce application has different discount rules:

```
Normal Customer → 0% discount  
Premium Customer → 10%  
VIP Customer → 20%
```

Without Strategy Pattern:

```
public decimal CalculateDiscount(  
    string customerType,  
    decimal amount)  
{  
    if (customerType == "Premium")  
    {  
        return amount * 0.10m;  
    }  
  
    if (customerType == "VIP")  
    {  
        return amount * 0.20m;  
    }  
  
    return 0;  
}
```

As the number of rules grows, this becomes difficult to maintain.

Using Strategy Pattern:

```
public interface IDiscountStrategy  
{  
    decimal CalculateDiscount(decimal amount);  
}
```

Normal

```
public class NormalDiscount : IDiscountStrategy
{
    public decimal CalculateDiscount(decimal amount)
    {
        return 0;
    }
}
```

Premium

```
public class PremiumDiscount : IDiscountStrategy
{
    public decimal CalculateDiscount(decimal amount)
    {
        return amount * 0.10m;
    }
}
```

VIP

```
public class VipDiscount : IDiscountStrategy
{
    public decimal CalculateDiscount(decimal amount)
    {
        return amount * 0.20m;
    }
}
```

Context:

```
public class DiscountService
{
    private readonly IDiscountStrategy _strategy;

    public DiscountService(IDiscountStrategy strategy)
    {
        _strategy = strategy;
    }

    public decimal GetDiscount(decimal amount)
    {
        return _strategy.CalculateDiscount(amount);
    }
}
```

Usage:

```
var strategy = new VipDiscount();

var discountService = new DiscountService(strategy);

decimal discount = discountService.GetDiscount(10000);

Console.WriteLine(discount);
```

Output:

```
2000
```

6. Strategy vs Factory

These two patterns are often used together.

Factory

Creates/selects an object.

```
Factory
|
+----> UpiPayment
+----> CreditCardPayment
+----> PayPalPayment
```

Strategy

Defines interchangeable behavior.

```
PaymentService
|
+----> IPaymentStrategy
      |
      +---- UPI
      +---- Credit Card
      +---- PayPal
```

A real application can use both:

```
API Request
  |
  v
Payment Strategy Factory
  |
  +---- UPI Strategy
  +---- Card Strategy
  +---- PayPal Strategy
      |
      v
    Payment Service
```

7. When should you use Strategy?

Use Strategy when:

- You have multiple algorithms for the same operation.
- You have large **if/else** or **switch** statements.
- The behavior needs to change at runtime.
- You want to follow **Open/Closed Principle**.
- You want each algorithm independently testable.
- Different customers/configurations require different behavior.

Examples:

```
Payment methods
Discount calculation
Tax calculation
Shipping calculation
Compression algorithms
Authentication methods
Notification methods
Sorting algorithms
File export formats
Fraud detection rules
```

8. Advantages

☒ Removes complex if/else

Instead of:

```
if (...)  
else if (...)  
else if (...)
```

you have separate classes.

☒ Follows Open/Closed Principle

You can add:

```
ApplePayPayment
```

without modifying existing payment strategies.

☒ Easy testing

You can test:

```
UpiPayment  
CreditCardPayment  
PayPalPayment
```

independently.

☒ Runtime flexibility

You can change:

```
IPaymentStrategy strategy
```

without changing the context.

☒ Good for ASP.NET Core

It works naturally with:

```
Dependency Injection  
SOLID  
Unit Testing  
Clean Architecture  
CQRS  
Microservices
```

9. Strategy vs State

Since you are studying the GoF patterns, this distinction is important.

Strategy	State
Changes an algorithm	Changes behavior based on state
Usually selected by client/configuration	State often changes internally
Focuses on interchangeable algorithms	Focuses on state transitions
Example: payment method	Example: order state
UPI, CreditCard	Pending, Shipped, Delivered

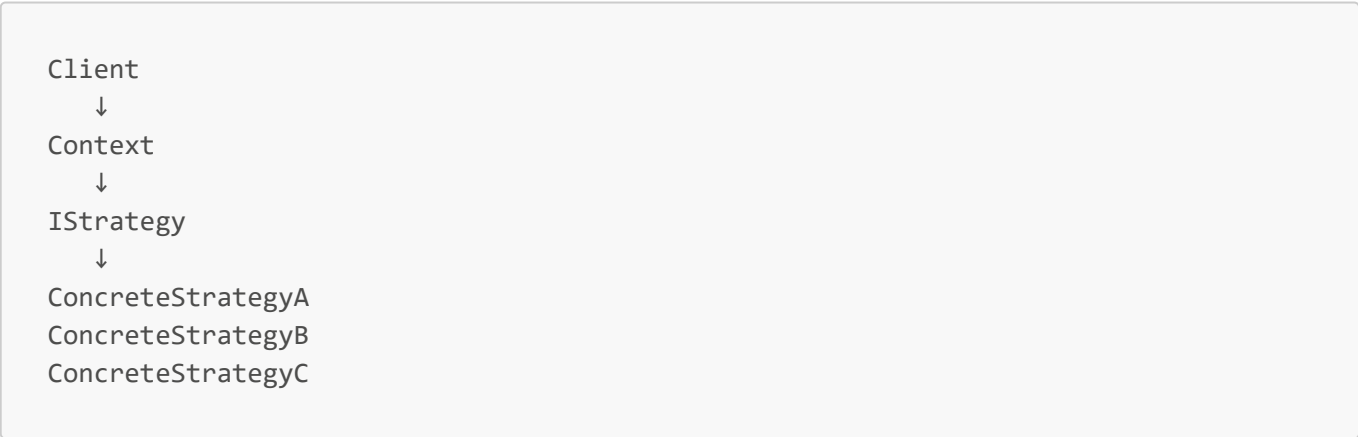
Easy way to remember:

- Strategy = "How should I do something?"
- State = "What state am I currently in?"

Interview definition

Strategy Pattern is a behavioral design pattern that defines a family of interchangeable algorithms, encapsulates each algorithm in a separate class, and allows the algorithm to be selected or changed at runtime.

For C#/.NET interviews, remember this structure:



Key principle: Program to an interface, not a concrete implementation.

10. Template Method

Template Method Design Pattern in C#

The **Template Method** is a **behavioral design pattern** that defines the **skeleton of an algorithm in a base class**, while allowing subclasses to customize specific steps.

Simple idea

Parent class decides the order of steps; child classes decide how some steps are implemented.

1. Real-world example

Suppose we have different types of reports:

- PDF Report
- Excel Report
- CSV Report

All reports follow the same process:

```
Generate Report
  ↓
Fetch Data
  ↓
Format Data
  ↓
Export Data
  ↓
Send Notification
```

The overall process is the same, but individual steps can differ.

This is a good use case for **Template Method**.

2. Basic C# example

```
public abstract class ReportGenerator
{
    // Template Method
    public void GenerateReport()
    {
        FetchData();
        FormatData();
        ExportData();
        SendNotification();
    }

    protected abstract void FetchData();

    protected abstract void FormatData();

    protected abstract void ExportData();

    protected virtual void SendNotification()
```

```
    {  
        Console.WriteLine("Sending notification...");  
    }  
}
```

Here, `GenerateReport()` is the **Template Method**.

It controls the sequence:

```
FetchData()  
  ↓  
FormatData()  
  ↓  
ExportData()  
  ↓  
SendNotification()
```

The subclasses cannot change this sequence.

3. Concrete classes

PDF Report

```
public class PdfReport : ReportGenerator  
{  
    protected override void FetchData()  
    {  
        Console.WriteLine("Fetching data for PDF...");  
    }  
  
    protected override void FormatData()  
    {  
        Console.WriteLine("Formatting data as PDF...");  
    }  
  
    protected override void ExportData()  
    {  
        Console.WriteLine("Exporting PDF...");  
    }  
}
```

Excel Report

```
public class ExcelReport : ReportGenerator  
{
```



```
protected override void FetchData()
{
    Console.WriteLine("Fetching data for Excel...");
}

protected override void FormatData()
{
    Console.WriteLine("Formatting data as Excel...");
}

protected override void ExportData()
{
    Console.WriteLine("Exporting Excel...");
}
}
```

4. Using the classes

```
class Program
{
    static void Main()
    {
        ReportGenerator pdf = new PdfReport();

        pdf.GenerateReport();

        Console.WriteLine();

        ReportGenerator excel = new ExcelReport();

        excel.GenerateReport();
    }
}
```

Output:

```
Fetching data for PDF...
Formatting data as PDF...
Exporting PDF...
Sending notification...

Fetching data for Excel...
Formatting data as Excel...
Exporting Excel...
Sending notification...
```

Notice that we call:

```
GenerateReport();
```

for both objects.

But each subclass performs the individual steps differently.

5. Why `GenerateReport()` is important

This is the key part:

```
public void GenerateReport()
{
    FetchData();
    FormatData();
    ExportData();
    SendNotification();
}
```

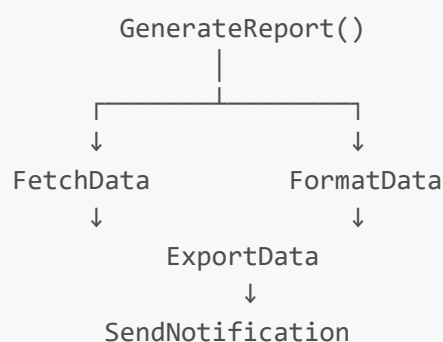
It defines the **algorithm structure**.

The child class cannot do:

```
ExportData
FetchData
FormatData
```

because the parent controls the sequence.

The algorithm is:



6. Abstract vs Virtual methods

Template Method commonly uses both.

Abstract method

```
protected abstract void FetchData();
```

The child **must implement** it.

```
protected override void FetchData()
{
    Console.WriteLine("Fetching PDF data...");
}
```

Virtual method

```
protected virtual void SendNotification()
{
    Console.WriteLine("Sending notification...");
}
```

The child **may override** it.

For example:

```
protected override void SendNotification()
{
    Console.WriteLine("Sending PDF notification by email...");
}
```

If the child doesn't override it, the base implementation is used.

7. Template Method with optional hooks

A **hook** is a method that subclasses can optionally override.

Example:

```
public abstract class DataProcessor
{
    public void Process()
    {
        ReadData();
        ProcessData();

        if (ShouldSave())
        {
            SaveData();
        }
    }

    protected abstract void ReadData();

    protected abstract void ProcessData();

    protected virtual bool ShouldSave()
    {
        return true;
    }

    protected virtual void SaveData()
    {
        Console.WriteLine("Saving data...");
    }
}
```

Subclass:

```
public class TestDataProcessor : DataProcessor
{
    protected override void ReadData()
    {
        Console.WriteLine("Reading test data...");
    }

    protected override void ProcessData()
    {
        Console.WriteLine("Processing test data...");
    }

    protected override bool ShouldSave()
    {
        return false;
    }
}
```

Now:

```
DataProcessor processor = new TestDataProcessor();  
  
processor.Process();
```

Output:

```
Reading test data...  
Processing test data...
```

`SaveData()` isn't executed because the subclass changed the hook:

```
ShouldSave()
```

8. Practical ASP.NET Core example

Imagine different payment providers:

```
Payment Processing  
  ↓  
Validate Payment  
  ↓  
Authenticate  
  ↓  
Process Payment  
  ↓  
Save Transaction  
  ↓  
Send Notification
```

You could create:

```
public abstract class PaymentProcessor  
{  
    public void ProcessPayment()  
    {  
        ValidatePayment();  
        Authenticate();  
        Process();  
        SaveTransaction();  
        SendNotification();  
    }  
}
```

```
protected abstract void ValidatePayment();

protected abstract void Authenticate();

protected abstract void Process();

protected virtual void SaveTransaction()
{
    Console.WriteLine("Saving transaction...");
}

protected virtual void SendNotification()
{
    Console.WriteLine("Sending notification...");
}
}
```

Then:

```
public class CardPaymentProcessor : PaymentProcessor
{
    protected override void ValidatePayment()
    {
        Console.WriteLine("Validating card...");
    }

    protected override void Authenticate()
    {
        Console.WriteLine("Authenticating card...");
    }

    protected override void Process()
    {
        Console.WriteLine("Processing card payment...");
    }
}
```

And:

```
public class UpiPaymentProcessor : PaymentProcessor
{
    protected override void ValidatePayment()
    {
        Console.WriteLine("Validating UPI...");
    }

    protected override void Authenticate()
    {
        Console.WriteLine("Authenticating UPI...");
    }
}
```

```
    }

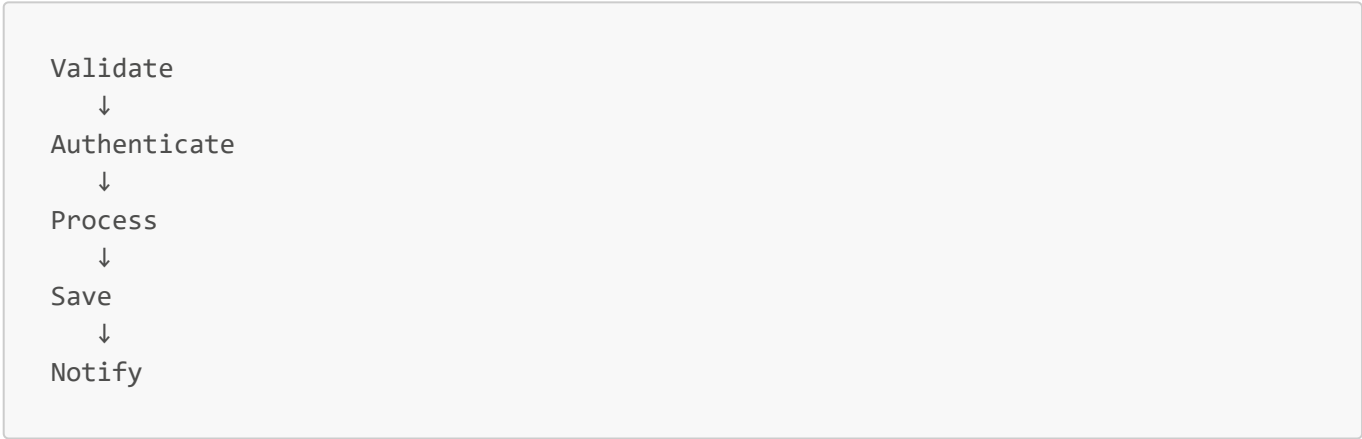
    protected override void Process()
    {
        Console.WriteLine("Processing UPI payment...");
    }
}
```

Usage:

```
PaymentProcessor processor = new UpiPaymentProcessor();

processor.ProcessPayment();
```

The parent guarantees:



while each payment type provides its own implementation.

9. Template Method vs Strategy

Since you're studying design patterns, this distinction is important.

Template Method	Strategy
Usually uses inheritance	Usually uses composition
Algorithm structure is in base class	Algorithm is encapsulated in separate classes
Subclasses customize steps	Strategy objects can be swapped
Compile-time structure	More runtime flexibility
Uses abstract/virtual methods	Uses interfaces/classes

Template Method

```
abstract class Payment
{
    public void Pay()
    {
        Validate();
        Process();
        Save();
    }

    protected abstract void Validate();
    protected abstract void Process();

    protected virtual void Save()
    {
    }
}
```

Strategy

```
public interface IPaymentStrategy
{
    void Pay();
}

public class CardPayment : IPaymentStrategy
{
    public void Pay()
    {
        Console.WriteLine("Card payment");
    }
}

public class UpiPayment : IPaymentStrategy
{
    public void Pay()
    {
        Console.WriteLine("UPI payment");
    }
}
```

Strategy focuses on:

Which algorithm should I use?

Template Method focuses on:

What steps must the algorithm follow, and which steps can subclasses customize?

10. When should you use Template Method?

Use it when:

- Multiple classes follow the **same algorithm**
- The **order of operations must remain fixed**
- Some steps vary between implementations
- You want to avoid duplicating common workflow code
- You want to enforce a common process

Typical examples:

```
Report generation
Payment processing
Data import/export
File processing
Authentication workflows
Order processing
ETL pipelines
Document generation
```

11. Advantages

Code reuse

Common logic stays in the base class.

Consistent algorithm

The subclass can't accidentally change the workflow.

Easy customization

Only variable steps need to be implemented.

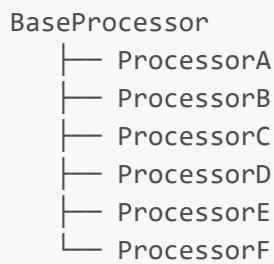
Follows DRY

You avoid repeating the same algorithm in multiple classes.

12. Disadvantages

The biggest limitation is **inheritance**.

If you have many variations, you can end up with:



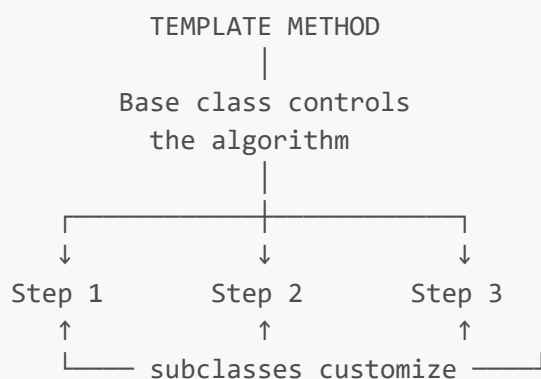
This can create a large inheritance hierarchy.

In such situations, **Strategy** + **composition** may be a better choice.

Interview definition

Template Method is a behavioral design pattern that defines the skeleton of an algorithm in a base class while allowing subclasses to override specific steps without changing the overall algorithm structure.

Remember it like this:



Core idea: "Fixed workflow + customizable steps."

11. Visitor

Visitor Design Pattern in C#

The **Visitor Design Pattern** is a **behavioral design pattern** that lets you add new operations to a group of related classes **without modifying those classes**.

It is especially useful when:

- You have a stable set of object types.
- You frequently need to add new operations.
- You want to keep operations separate from the domain classes.

1. The Problem

Suppose we have different types of employees:

```
public class Developer
{
    public string Name { get; set; }
}

public class Manager
{
    public string Name { get; set; }
}
```

Now we want to perform different operations:

- Calculate salary
- Generate reports
- Calculate bonus
- Export data

Without Visitor, we might keep adding methods to **Developer** and **Manager**.

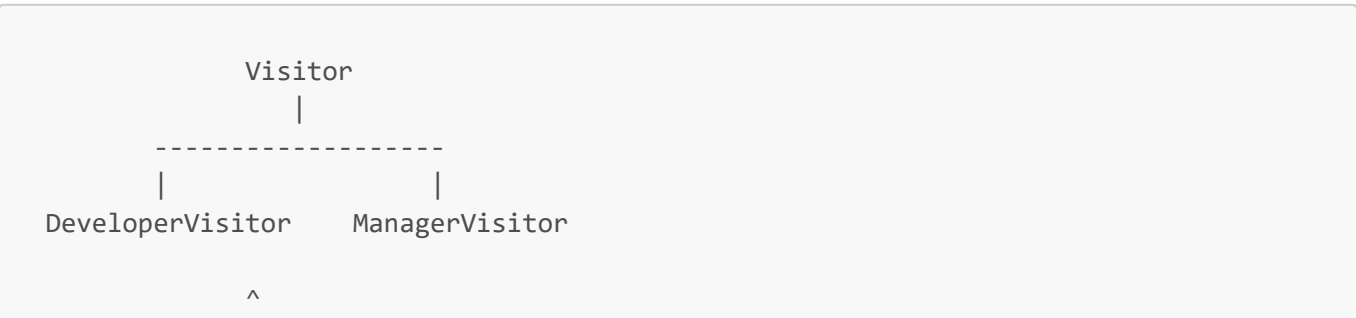
```
public class Developer
{
    public void CalculateSalary() { }
    public void GenerateReport() { }
    public void CalculateBonus() { }
}
```

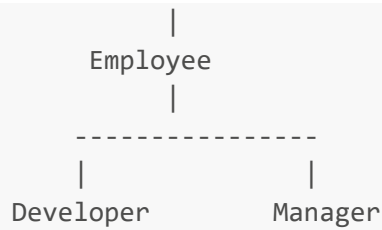
This can make the classes grow with unrelated responsibilities.

Visitor separates these operations from the classes.

2. Basic Structure

The Visitor pattern usually contains:





There are two important concepts:

Element

The object being visited.

Developer
Manager

Visitor

The object that performs an operation.

SalaryVisitor
BonusVisitor
ReportVisitor

3. C# Example

Step 1 — Visitor Interface

```
public interface IEmployeeVisitor
{
    void Visit(Developer developer);
    void Visit(Manager manager);
}
```

The visitor knows how to handle each concrete employee type.

Step 2 — Element Interface

```
public interface IEmployee
{
```

```
void Accept(IEmployeeVisitor visitor);  
}
```

Step 3 — Developer

```
public class Developer : IEmployee  
{  
    public string Name { get; set; }  
  
    public Developer(string name)  
    {  
        Name = name;  
    }  
  
    public void Accept(IEmployeeVisitor visitor)  
    {  
        visitor.Visit(this);  
    }  
}
```

Step 4 — Manager

```
public class Manager : IEmployee  
{  
    public string Name { get; set; }  
  
    public Manager(string name)  
    {  
        Name = name;  
    }  
  
    public void Accept(IEmployeeVisitor visitor)  
    {  
        visitor.Visit(this);  
    }  
}
```

Notice:

```
visitor.Visit(this);
```

This is the important part of the Visitor pattern.

4. Create a Visitor

Let's create a salary calculator.

```
public class SalaryVisitor : IEmployeeVisitor
{
    public void Visit(Developer developer)
    {
        Console.WriteLine(
            $"Calculating salary for Developer: {developer.Name}");
    }

    public void Visit(Manager manager)
    {
        Console.WriteLine(
            $"Calculating salary for Manager: {manager.Name}");
    }
}
```

5. Another Visitor

We can create a bonus calculator without modifying `Developer` or `Manager`.

```
public class BonusVisitor : IEmployeeVisitor
{
    public void Visit(Developer developer)
    {
        Console.WriteLine(
            $"Calculating developer bonus for {developer.Name}");
    }

    public void Visit(Manager manager)
    {
        Console.WriteLine(
            $"Calculating manager bonus for {manager.Name}");
    }
}
```

This is the main advantage.

We added a completely new operation without changing the employee classes.

6. Using the Visitor

```
List<IEmployee> employees = new()
{
    new Developer("John"),
    new Manager("David"),
    new Developer("Mike")
};

IEmployeeVisitor salaryVisitor = new SalaryVisitor();

foreach (var employee in employees)
{
    employee.Accept(salaryVisitor);
}
```

Output:

```
Calculating salary for Developer: John
Calculating salary for Manager: David
Calculating salary for Developer: Mike
```

Now use another visitor:

```
IEmployeeVisitor bonusVisitor = new BonusVisitor();

foreach (var employee in employees)
{
    employee.Accept(bonusVisitor);
}
```

Output:

```
Calculating developer bonus for John
Calculating manager bonus for David
Calculating developer bonus for Mike
```

7. Why `Accept()` Is Important

This line:

```
employee.Accept(visitor);
```

eventually calls:

```
visitor.Visit(this);
```

Suppose the object is:

```
Developer
```

Then:

```
visitor.Visit(this);
```

becomes:

```
visitor.Visit(Developer);
```

If the object is:

```
Manager
```

it becomes:

```
visitor.Visit(Manager);
```

This is called **Double Dispatch**.

8. Double Dispatch

Visitor is strongly associated with **double dispatch**.

Normally, method selection in C# depends primarily on the runtime type of the object receiving the method call.

Visitor effectively uses both:

```
Type of Element  
+
```



```
Type of Visitor
      ↓
Specific Visit() method
```

For example:

```
Developer.Accept(SalaryVisitor)
```

results in:

```
SalaryVisitor.Visit(Developer)
```

Whereas:

```
Manager.Accept(SalaryVisitor)
```

results in:

```
SalaryVisitor.Visit(Manager)
```

9. Complete Example

```
using System;
using System.Collections.Generic;

public interface IEmployee
{
    void Accept(IEmployeeVisitor visitor);
}

public interface IEmployeeVisitor
{
    void Visit(Developer developer);
    void Visit(Manager manager);
}

public class Developer : IEmployee
{
    public string Name { get; }
```

```
    public Developer(string name)
    {
        Name = name;
    }

    public void Accept(IEmployeeVisitor visitor)
    {
        visitor.Visit(this);
    }
}

public class Manager : IEmployee
{
    public string Name { get; }

    public Manager(string name)
    {
        Name = name;
    }

    public void Accept(IEmployeeVisitor visitor)
    {
        visitor.Visit(this);
    }
}

public class SalaryVisitor : IEmployeeVisitor
{
    public void Visit(Developer developer)
    {
        Console.WriteLine(
            $"Developer {developer.Name}: Salary = ₹80,000");
    }

    public void Visit(Manager manager)
    {
        Console.WriteLine(
            $"Manager {manager.Name}: Salary = ₹1,20,000");
    }
}

public class BonusVisitor : IEmployeeVisitor
{
    public void Visit(Developer developer)
    {
        Console.WriteLine(
            $"Developer {developer.Name}: Bonus = ₹10,000");
    }

    public void Visit(Manager manager)
    {
        Console.WriteLine(
            $"Manager {manager.Name}: Bonus = ₹25,000");
    }
}
```

```

}

public class Program
{
    public static void Main()
    {
        List<IEmployee> employees = new()
        {
            new Developer("John"),
            new Manager("David"),
            new Developer("Mike")
        };

        var salaryVisitor = new SalaryVisitor();

        foreach (var employee in employees)
        {
            employee.Accept(salaryVisitor);
        }

        Console.WriteLine();

        var bonusVisitor = new BonusVisitor();

        foreach (var employee in employees)
        {
            employee.Accept(bonusVisitor);
        }
    }
}

```

10. Real-World Example

A very common example is an **expression tree**.

Imagine:

```

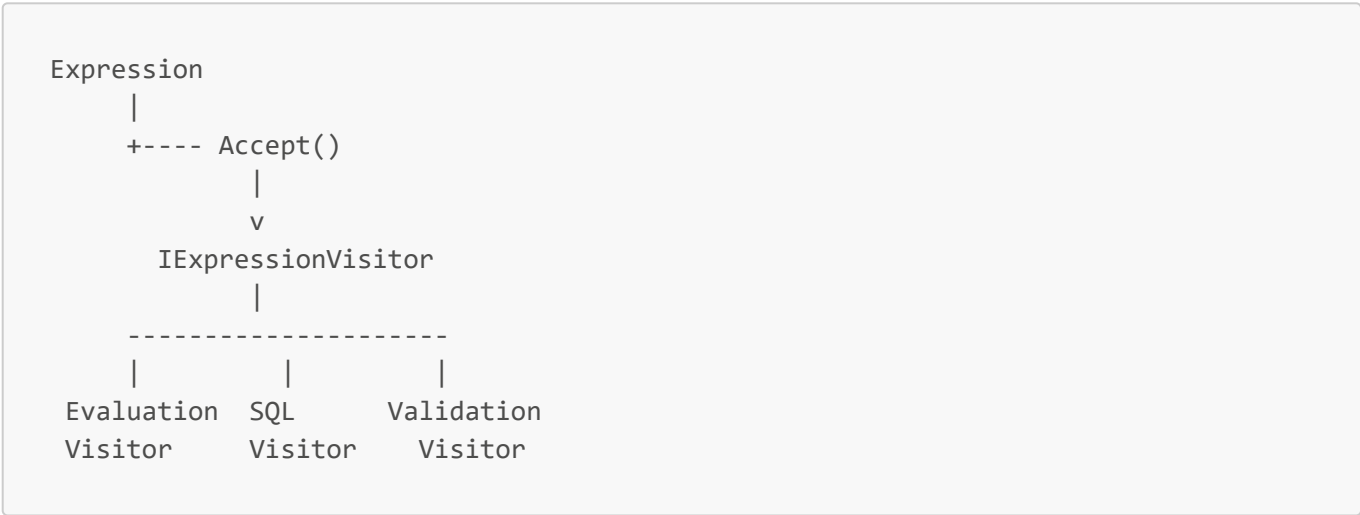
Expression
|
+-- NumberExpression
|
+-- AddExpression
|
+-- MultiplyExpression

```

You might want to perform many operations:

```
Expression
|
+-- Evaluation
+-- SQL Generation
+-- Validation
+-- Logging
+-- Pretty Printing
+-- Serialization
```

Instead of putting all these operations inside every expression class, use visitors:



This is one reason Visitor appears in **compilers, parsers, ASTs, and expression processing**.

11. Visitor vs Strategy

These two patterns can look similar, but they solve different problems.

Visitor	Strategy
Adds operations to an object structure	Changes an algorithm
Works with multiple object types	Usually works with one abstraction
Uses Visit() methods	Usually uses a common strategy interface
Commonly uses double dispatch	Usually uses normal polymorphism
Useful for stable object structures	Useful when algorithms vary

Strategy

```
payment.Process(new CreditCardStrategy());
```

The strategy represents **how something is done**.

Visitor

```
employee.Accept(new SalaryVisitor());
```

The visitor represents **an operation performed on different object types**.

12. Advantages

☒ Add new operations easily

You can create:

```
TaxVisitor  
BonusVisitor  
ReportVisitor  
SalaryVisitor
```

without changing the employee classes.

☒ Separates responsibilities

Employee classes represent employee data/behavior.

Visitors represent external operations.

☒ Useful for complex object structures

Especially:

- ASTs
- Compilers
- File systems
- Document structures
- UI component trees
- Expression trees

13. Disadvantages

The biggest disadvantage is that **adding a new element type is difficult**.

Suppose we add:

```
public class Intern : IEmployee
```

Then `IEmployeeVisitor` must change:

```
public interface IEmployeeVisitor
{
    void Visit(Developer developer);
    void Visit(Manager manager);
    void Visit(Intern intern);
}
```

And **every visitor** must implement:

```
void Visit(Intern intern);
```

So Visitor favors:

```
Few/stable element types
+
Many/new operations
```

rather than:

```
Many/new element types
+
Few operations
```

14. When Should You Use Visitor?

Use Visitor when:

```
Object structure changes rarely
      +
Operations change frequently
      ↓
    Visitor
```

For example:

```
AST nodes
↓
Evaluation
SQL generation
Validation
Optimization
Logging
Visualization
```

Visitor is particularly powerful when the **class hierarchy is stable but you continuously need new operations**.

Interview Definition

Visitor is a behavioral design pattern that allows us to add new operations to a group of related objects without modifying their classes. It separates the operation from the object structure and commonly uses double dispatch through `Accept()` and `Visit()` methods.

Remember it like this:

```
Element
↓
Accept(Visitor)
↓
Visitor.Visit(Element)
↓
Perform operation
```

Key idea: "Add new operations without changing the existing object classes."

Very important for .NET

- Strategy
- Observer
- Command
- Chain of Responsibility
- Mediator
- State

5. Enterprise / Application Patterns

These are not all GoF patterns, but are extremely useful for professional .NET applications.

Architecture & Data Access

1. Repository Pattern

Repository Pattern in C#

The **Repository Pattern** is a design pattern used to separate **data-access logic** from your **business logic**.

Instead of writing database code directly inside controllers/services:

```
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

The repository acts as an abstraction over your data source.

1. The Problem

Without Repository Pattern, you might write:

```
public class UserService
{
    private readonly AppDbContext _context;

    public UserService(AppDbContext context)
    {
        _context = context;
    }

    public async Task<User?> GetUserAsync(int id)
    {
        return await _context.Users
            .FirstOrDefaultAsync(x => x.Id == id);
    }
}
```

Now **UserService** knows about:

- Entity Framework
- **DbContext**
- Database queries
- EF Core implementation details

This makes the service more tightly coupled to the database technology.

2. Repository Pattern Solution

Create an interface:

```
public interface IUserRepository
{
    Task<User?> GetByIdAsync(int id);
    Task<List<User>> GetAllAsync();
    Task AddAsync(User user);
    Task UpdateAsync(User user);
    Task DeleteAsync(int id);
}
```

Then create the implementation:

```
public class UserRepository : IUserRepository
{
    private readonly AppDbContext _context;

    public UserRepository(AppDbContext context)
    {
        _context = context;
    }

    public async Task<User?> GetByIdAsync(int id)
    {
        return await _context.Users
            .FirstOrDefaultAsync(x => x.Id == id);
    }

    public async Task<List<User>> GetAllAsync()
    {
        return await _context.Users
            .ToListAsync();
    }

    public async Task AddAsync(User user)
    {
        await _context.Users.AddAsync(user);
        await _context.SaveChangesAsync();
    }

    public async Task UpdateAsync(User user)
    {
        _context.Users.Update(user);
        await _context.SaveChangesAsync();
    }

    public async Task DeleteAsync(int id)
    {

```

```
{
    var user = await GetByIdAsync(id);

    if (user != null)
    {
        _context.Users.Remove(user);
        await _context.SaveChangesAsync();
    }
}
```

Now the database-related code is inside the repository.

3. Entity

For example:

```
public class User
{
    public int Id { get; set; }

    public string Name { get; set; } = string.Empty;

    public string Email { get; set; } = string.Empty;
}
```

4. DbContext

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<User> Users { get; set; }
}
```

5. Service Layer

The service doesn't need to know how the database works.

```
public class UserService
{
    private readonly IUserRepository _userRepository;

    public UserService(IUserRepository userRepository)
    {
        _userRepository = userRepository;
    }

    public async Task<User?> GetUserAsync(int id)
    {
        return await _userRepository.GetByIdAsync(id);
    }

    public async Task<List<User>> GetUsersAsync()
    {
        return await _userRepository.GetAllAsync();
    }
}
```

Notice:

```
UserService
  ↓
IUserRepository
  ↓
UserRepository
  ↓
AppDbContext
  ↓
Database
```

The service depends on the **interface**, not the concrete repository.

6. Register Dependency Injection

In `Program.cs`:

```
builder.Services.AddScoped<IUserRepository, UserRepository>();

builder.Services.AddScoped<UserService>();
```

Now ASP.NET Core automatically injects the repository.

7. Controller

```
[ApiController]
[Route("api/[controller]")]
public class UsersController : ControllerBase
{
    private readonly UserService _userService;

    public UsersController(UserService userService)
    {
        _userService = userService;
    }

    [HttpGet("{id}")]
    public async Task<IActionResult> GetUser(int id)
    {
        var user = await _userService.GetUserAsync(id);

        if (user == null)
            return NotFound();

        return Ok(user);
    }

    [HttpGet]
    public async Task<IActionResult> GetUsers()
    {
        var users = await _userService.GetUsersAsync();

        return Ok(users);
    }
}
```

The controller doesn't directly communicate with EF Core.

8. Complete Architecture

A typical ASP.NET Core application can look like:

```
MyApplication
├── Controllers
│   └── UsersController.cs
├── Services
│   └── UserService.cs
└──
```

```
├── Repositories
│   ├── IUserRepository.cs
│   └── UserRepository.cs
├── Models
│   └── User.cs
├── Data
│   └── AppDbContext.cs
└── Program.cs
```

And the request flow:

```
HTTP Request
  ↓
Controller
  ↓
Service
  ↓
Repository Interface
  ↓
Repository Implementation
  ↓
EF Core / DbContext
  ↓
SQL Server
```

9. Generic Repository

If you have many entities, you don't necessarily want to write the same CRUD methods repeatedly.

You can create:

```
public interface IRepository<T> where T : class
{
    Task<T?> GetByIdAsync(int id);

    Task<List<T>> GetAllAsync();

    Task AddAsync(T entity);

    void Update(T entity);

    void Delete(T entity);
}
```

```
        Task SaveChangesAsync();  
    }
```

Implementation:

```
public class Repository<T> : IRepository<T>  
    where T : class  
{  
    protected readonly AppDbContext _context;  
  
    public Repository(AppDbContext context)  
    {  
        _context = context;  
    }  
  
    public async Task<T?> GetByIdAsync(int id)  
    {  
        return await _context.Set<T>()  
            .FindAsync(id);  
    }  
  
    public async Task<List<T>> GetAllAsync()  
    {  
        return await _context.Set<T>()  
            .ToListAsync();  
    }  
  
    public async Task AddAsync(T entity)  
    {  
        await _context.Set<T>()  
            .AddAsync(entity);  
    }  
  
    public void Update(T entity)  
    {  
        _context.Set<T>().Update(entity);  
    }  
  
    public void Delete(T entity)  
    {  
        _context.Set<T>().Remove(entity);  
    }  
  
    public async Task SaveChangesAsync()  
    {  
        await _context.SaveChangesAsync();  
    }  
}
```

Then:

```
public interface IUserRepository : IRepository<User>
{
    Task<User?> GetByEmailAsync(string email);
}
```

Implementation:

```
public class UserRepository
    : Repository<User>, IUserRepository
{
    public UserRepository(AppDbContext context)
        : base(context)
    {
    }

    public async Task<User?> GetByEmailAsync(string email)
    {
        return await _context.Users
            .FirstOrDefaultAsync(x => x.Email == email);
    }
}
```

10. Repository vs Service

This distinction is very important.

Repository

Responsible for **data access**.

```
_repository.GetByIdAsync(id);
_repository.AddAsync(user);
_repository.Delete(user);
```

Think:

How do I get/store data?

Service

Responsible for **business logic**.

```
public async Task<bool> RegisterUser(User user)
{
```

```
var existingUser =  
    await _repository.GetByEmailAsync(user.Email);  
  
if (existingUser != null)  
    return false;  
  
// Business rules  
  
await _repository.AddAsync(user);  
  
return true;  
}
```

Think:

What should the application do?

11. Repository + Unit of Work

In larger applications, you may see:

```
Controller  
  ↓  
Service  
  ↓  
UnitOfWork  
  ↓  
Repositories  
  ↓  
DbContext  
  ↓  
Database
```

For example:

```
public interface IUnitOfWork  
{  
    IUserRepository Users { get; }  
  
    IOrderRepository Orders { get; }  
  
    Task<int> SaveChangesAsync();  
}
```

Then:


```
await _unitOfWork.Users.AddAsync(user);  
await _unitOfWork.Orders.AddAsync(order);  
  
await _unitOfWork.SaveChangesAsync();
```

This is useful when multiple repositories participate in **one transaction**.

12. Advantages

Separation of concerns

Database code is separated from business logic.

Testability

You can mock the repository:

```
var mockRepository = new Mock<IUserRepository>();  
  
mockRepository  
    .Setup(x => x.GetByIdAsync(1))  
    .ReturnsAsync(new User  
    {  
        Id = 1,  
        Name = "John"  
    });
```

Then test your service without connecting to SQL Server.

Maintainability

Database implementation can be changed more easily.

For example:

```
SQL Server  
↓  
EF Core
```

could potentially become:

```
PostgreSQL  
↓  
Dapper
```

while the service can continue using:

```
IUserRepository
```

13. Disadvantages

Repository Pattern isn't automatically better in every project.

With EF Core, `DbContext` and `DbSet<T>` already provide many repository/unit-of-work behaviors.

Therefore, creating:

```
Generic Repository
    ↓
EF Core DbSet
    ↓
EF Core DbContext
```

can sometimes add unnecessary abstraction.

For simple CRUD applications, this can be over-engineering.

14. When Should You Use It?

Use Repository Pattern when:

- Application has significant business logic
- You want strong separation of concerns
- You need isolated unit testing
- Data access is complex
- Multiple data sources are involved
- You want explicit domain/data-access boundaries
- The architecture benefits from repositories specific to business concepts

Avoid blindly creating a generic repository for every project.

Repository Pattern — Interview Definition

Repository Pattern is a structural/architectural pattern that abstracts data-access operations behind an interface, allowing business logic to work with data without being tightly coupled to the underlying database or ORM.

For an **ASP.NET Core interview**, remember this architecture:

```
Controller
  ↓
Service
  ↓
IRepository
  ↓
Repository
  ↓
DbContext
  ↓
Database
```

And the key distinction:

```
Repository = Data Access
Service    = Business Logic
Controller = HTTP/API Handling
DbContext  = Database/ORM Interaction
```

2. Unit of Work

Unit of Work Design Pattern in C#

The **Unit of Work (UoW)** pattern is a design pattern used to **group multiple database operations into a single transaction**.

Instead of saving every operation independently:

```
Add Customer → Save
Add Order    → Save
Update Stock → Save
```

Unit of Work lets you do:

```
Add Customer
Add Order
Update Stock
  ↓
Commit once
  ↓
Database
```

If something fails, you can **rollback the entire operation**.

1. Real-world example

Imagine an e-commerce application:

```
Place Order
├── Create Order
├── Add Order Items
├── Reduce Product Stock
├── Create Payment
└── Commit Transaction
```

Suppose payment creation fails.

Without Unit of Work:

Order created	☒
Order items created	☒
Stock reduced	☒
Payment failed	☒

Now your database is inconsistent.

With Unit of Work:

```
Order created      |
Order items created |
Stock reduced      |── Transaction
Payment failed     |
                   |
                   ↓
                Rollback
```

Everything is rolled back.

2. Unit of Work vs Repository

These two patterns are often used together.

Repository

Repository manages operations for **one type of entity**.

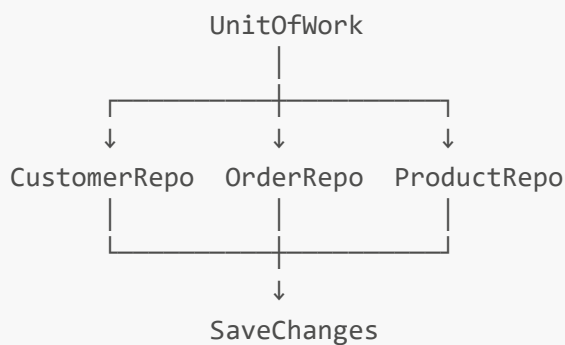
```
CustomerRepository  
OrderRepository  
ProductRepository
```

For example:

```
customerRepository.Add(customer);  
orderRepository.Add(order);  
productRepository.Update(product);
```

Unit of Work

Unit of Work manages the **transaction across repositories**.



So:

Repository = data access for an entity **Unit of Work = coordinate multiple repositories and commit them together**

3. Basic C# implementation

Let's create a simple example.

Entity

```
public class Customer  
{  
    public int Id { get; set; }  
  
    public string Name { get; set; } = string.Empty;  
}
```

```
public class Order
{
    public int Id { get; set; }

    public int CustomerId { get; set; }

    public decimal Amount { get; set; }
}
```

4. Repository interfaces

```
public interface ICustomerRepository
{
    void Add(Customer customer);

    Customer? GetById(int id);
}
```

```
public interface IOrderRepository
{
    void Add(Order order);

    Order? GetById(int id);
}
```

5. Unit of Work interface

```
public interface IUnitOfWork
{
    ICustomerRepository Customers { get; }

    IOrderRepository Orders { get; }

    Task<int> SaveChangesAsync();

    Task BeginTransactionAsync();

    Task CommitTransactionAsync();

    Task RollbackTransactionAsync();
}
```

Now our application can work with multiple repositories through one Unit of Work.

6. Entity Framework Core implementation

With EF Core, `DbContext` already behaves very much like a **Unit of Work**.

Example:

```
public class AppDbContext : DbContext
{
    public DbSet<Customer> Customers { get; set; }

    public DbSet<Order> Orders { get; set; }

    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }
}
```

Repositories:

```
public class CustomerRepository : ICustomerRepository
{
    private readonly AppDbContext _context;

    public CustomerRepository(AppDbContext context)
    {
        _context = context;
    }

    public void Add(Customer customer)
    {
        _context.Customers.Add(customer);
    }

    public Customer? GetById(int id)
    {
        return _context.Customers
            .FirstOrDefault(x => x.Id == id);
    }
}
```

Order repository:

```
public class OrderRepository : IOrderRepository
{
    private readonly AppDbContext _context;

    public OrderRepository(AppDbContext context)
    {
        _context = context;
    }

    public void Add(Order order)
    {
        _context.Orders.Add(order);
    }

    public Order? GetById(int id)
    {
        return _context.Orders
            .FirstOrDefault(x => x.Id == id);
    }
}
```

7. Unit of Work implementation

```
public class UnitOfWork : IUnitOfWork
{
    private readonly AppDbContext _context;

    public ICustomerRepository Customers { get; }

    public IOrderRepository Orders { get; }

    public UnitOfWork(
        AppDbContext context,
        ICustomerRepository customers,
        IOrderRepository orders)
    {
        _context = context;

        Customers = customers;
        Orders = orders;
    }

    public async Task<int> SaveChangesAsync()
    {
        return await _context.SaveChangesAsync();
    }

    public async Task BeginTransactionAsync()
```



```
{
    await _context.Database.BeginTransactionAsync();
}

public async Task CommitTransactionAsync()
{
    var transaction = _context.Database.CurrentTransaction;

    if (transaction != null)
    {
        await transaction.CommitAsync();
    }
}

public async Task RollbackTransactionAsync()
{
    var transaction = _context.Database.CurrentTransaction;

    if (transaction != null)
    {
        await transaction.RollbackAsync();
    }
}
}
```

8. Using Unit of Work in a Service

Suppose we want to create a customer and an order.

```
public class OrderService
{
    private readonly IUnitOfWork _unitOfWork;

    public OrderService(IUnitOfWork unitOfWork)
    {
        _unitOfWork = unitOfWork;
    }

    public async Task CreateOrderAsync()
    {
        await _unitOfWork.BeginTransactionAsync();

        try
        {
            var customer = new Customer
            {
                Name = "John"
            };
        }
```

```
        _unitOfWork.Customers.Add(customer);

        var order = new Order
        {
            CustomerId = customer.Id,
            Amount = 5000
        };

        _unitOfWork.Orders.Add(order);

        await _unitOfWork.SaveChangesAsync();

        await _unitOfWork.CommitTransactionAsync();
    }
    catch
    {
        await _unitOfWork.RollbackTransactionAsync();

        throw;
    }
}
```

There is one important issue here: `customer.Id` is normally generated by the database, so you would generally call `SaveChangesAsync()` after adding the customer if you need its generated ID before creating the order, or model the relationship through navigation properties.

9. Better EF Core approach

In modern ASP.NET Core applications, you often don't need to create a custom Unit of Work at all.

EF Core's:

```
DbContext
```

already provides:

- Change tracking
- Repository-like access through `DbSet<T>`
- `SaveChanges()`
- `SaveChangesAsync()`
- Transaction support

For example:

```
public class OrderService
{
    private readonly AppDbContext _context;

    public OrderService(AppDbContext context)
    {
        _context = context;
    }

    public async Task CreateOrderAsync()
    {
        var customer = new Customer
        {
            Name = "John"
        };

        var order = new Order
        {
            Amount = 5000,
            Customer = customer
        };

        _context.Orders.Add(order);

        await _context.SaveChangesAsync();
    }
}
```

`SaveChangesAsync()` commits the tracked changes as one unit.

10. Explicit transaction

When you have multiple operations that must succeed or fail together:

```
await using var transaction =
    await _context.Database.BeginTransactionAsync();

try
{
    // Operation 1
    _context.Customers.Add(customer);

    await _context.SaveChangesAsync();

    // Operation 2
    _context.Orders.Add(order);

    await _context.SaveChangesAsync();
}
```

```

        // Operation 3
        product.Stock--;

        await _context.SaveChangesAsync();

        await transaction.CommitAsync();
    }
    catch
    {
        await transaction.RollbackAsync();

        throw;
    }

```

The important idea is:

```

BEGIN TRANSACTION

    Customer operation
    Order operation
    Stock operation

COMMIT

```

or:

```

BEGIN TRANSACTION

    Customer operation
    Order operation
    Stock operation
    ↓
    ERROR

ROLLBACK

```

11. Dependency Injection

In ASP.NET Core:

```

builder.Services.AddScoped<AppDbContext>();

builder.Services.AddScoped<ICustomerRepository,
                        CustomerRepository>();

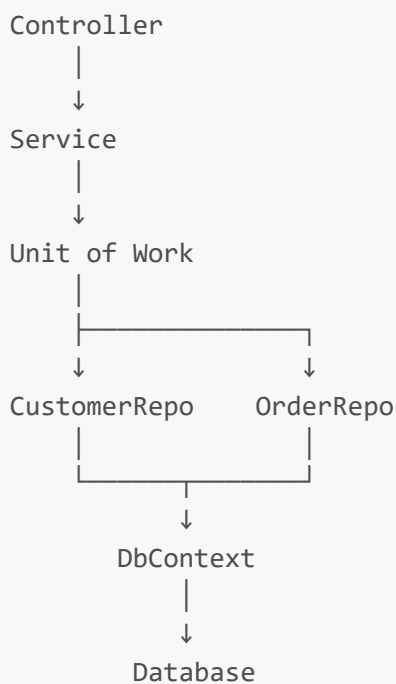
```

```
builder.Services.AddScoped<IOrderRepository,  
    OrderRepository>();  
  
builder.Services.AddScoped<IUnitOfWork,  
    UnitOfWork>();  
  
builder.Services.AddScoped<OrderService>();
```

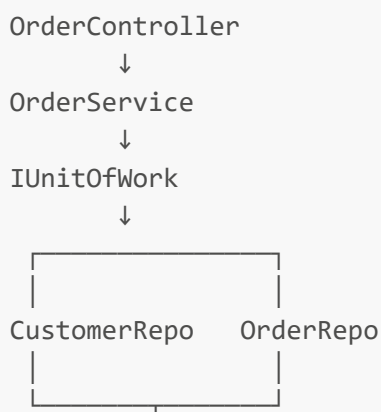
Scoped is important because a typical web request should use one **DbContext**/Unit of Work scope.

12. Repository + Unit of Work architecture

A common architecture looks like this:



Example:



```
↓  
DbContext  
↓  
SQL Server
```

13. When should you use Unit of Work?

Use it when:

Multiple repositories participate in one operation

```
Customer + Order + Payment
```

and they need to commit together.

You need transaction control

```
BEGIN  
  operation 1  
  operation 2  
  operation 3  
COMMIT
```

You have a complex data-access layer

For larger applications, explicit transaction boundaries can make business operations clearer.

14. When NOT to use it

If you're using EF Core:

```
DbContext
```

already provides much of the Unit of Work functionality.

Adding:

```
IUnitOfWork  
UnitOfWork
```

```

IRepository
Repository

```

for every simple CRUD operation can create unnecessary abstraction.

For example:

```

_context.Products.Add(product);

await _context.SaveChangesAsync();

```

may be better than creating:

```

IProductRepository
ProductRepository
IUnitOfWork
UnitOfWork

```

just to wrap two EF Core calls.

15. Unit of Work vs Transaction

They are related but not identical.

Transaction

Defines the database's atomic operation:

```

BEGIN
  INSERT
  UPDATE
  DELETE
  COMMIT / ROLLBACK

```

Unit of Work

Represents a collection of changes that should be treated as one logical operation.

```

UnitOfWork
|
|— Customer changes
|— Order changes
|— Product changes

```

|
└─ Save / Commit

Think:

Transaction = database mechanism

Unit of Work = application/data-access pattern

16. Interview answer

If an interviewer asks "**What is Unit of Work?**", you can say:

Unit of Work is a design pattern that coordinates multiple database operations and commits them as a single logical unit. It helps maintain consistency by ensuring that related changes are committed together or rolled back together. In EF Core, DbContext already implements many Unit of Work responsibilities through change tracking and SaveChanges().

Simple memory trick

```
Repository
  ↓
"How do I access this entity?"

Unit of Work
  ↓
"How do I commit all these entity changes together?"

Transaction
  ↓
"How do I ensure everything succeeds or everything rolls back?"
```

For **ASP.NET Core + EF Core**, this distinction is especially important because **DbContext** is already both **Repository-like and Unit-of-Work-like**, so you should avoid adding custom abstractions unless they provide a real architectural benefit.

3. Service Layer

Service Layer Design Pattern in C#

The **Service Layer Pattern** is an architectural/design pattern used to keep **business logic separate from controllers, UI, repositories, and database code**.

In an ASP.NET Core application, the typical flow is:


```
Client
  ↓
Controller
  ↓
Service Layer
  ↓
Repository
  ↓
Database
```

1. Why do we need a Service Layer?

Without a service layer, controllers often become like this:

```
[HttpPost]
public async Task<IActionResult> CreateOrder(Order order)
{
    if (order.Amount <= 0)
        return BadRequest();

    var customer = await _db.Customers
        .FirstOrDefaultAsync(x => x.Id == order.CustomerId);

    if (customer == null)
        return NotFound();

    order.CreatedDate = DateTime.UtcNow;

    _db.Orders.Add(order);
    await _db.SaveChangesAsync();

    return Ok(order);
}
```

The controller is now responsible for:

- Validation
- Business rules
- Database access
- Creating entities
- Saving data
- HTTP response handling

This becomes difficult to maintain and test.

2. With Service Layer

Move the business logic into a service:

```
Controller
  ↓
OrderService
  ↓
OrderRepository
  ↓
Database
```

The controller becomes simple:

```
[ApiController]
[Route("api/orders")]
public class OrderController : ControllerBase
{
    private readonly IOrderService _orderService;

    public OrderController(IOrderService orderService)
    {
        _orderService = orderService;
    }

    [HttpPost]
    public async Task<IActionResult> Create(OrderDto dto)
    {
        var order = await _orderService.CreateOrderAsync(dto);

        return Ok(order);
    }
}
```

The **Service** contains the business logic.

3. Basic Service Layer Example

Model

```
public class Order
{
    public int Id { get; set; }

    public int CustomerId { get; set; }

    public decimal Amount { get; set; }
```

```
    public DateTime CreatedDate { get; set; }  
}
```

DTO

```
public class CreateOrderDto  
{  
    public int CustomerId { get; set; }  
  
    public decimal Amount { get; set; }  
}
```

4. Repository

The repository handles data access.

```
public interface IOrderRepository  
{  
    Task<Order?> GetByIdAsync(int id);  
  
    Task AddAsync(Order order);  
  
    Task SaveChangesAsync();  
}
```

Implementation:

```
public class OrderRepository : IOrderRepository  
{  
    private readonly AppDbContext _context;  
  
    public OrderRepository(AppDbContext context)  
    {  
        _context = context;  
    }  
  
    public async Task<Order?> GetByIdAsync(int id)  
    {  
        return await _context.Orders  
            .FirstOrDefaultAsync(x => x.Id == id);  
    }  
  
    public async Task AddAsync(Order order)  
    {
```

```
        await _context.Orders.AddAsync(order);
    }

    public async Task SaveChangesAsync()
    {
        await _context.SaveChangesAsync();
    }
}
```

5. Service Interface

Create an abstraction for the business operations.

```
public interface IOrderService
{
    Task<Order> CreateOrderAsync(CreateOrderDto dto);

    Task<Order?> GetOrderAsync(int id);
}
```

6. Service Implementation

```
public class OrderService : IOrderService
{
    private readonly IOrderRepository _orderRepository;

    public OrderService(IOrderRepository orderRepository)
    {
        _orderRepository = orderRepository;
    }

    public async Task<Order> CreateOrderAsync(CreateOrderDto dto)
    {
        // Business validation
        if (dto.Amount <= 0)
        {
            throw new ArgumentException(
                "Order amount must be greater than zero.");
        }

        var order = new Order
        {
            CustomerId = dto.CustomerId,
            Amount = dto.Amount,
        }
    }
}
```

```

        CreatedDate = DateTime.UtcNow
    };

    await _orderRepository.AddAsync(order);
    await _orderRepository.SaveChangesAsync();

    return order;
}

public async Task<Order?> GetOrderAsync(int id)
{
    return await _orderRepository.GetByIdAsync(id);
}
}

```

Now the service owns the **business logic**.

7. Register with Dependency Injection

In **Program.cs**:

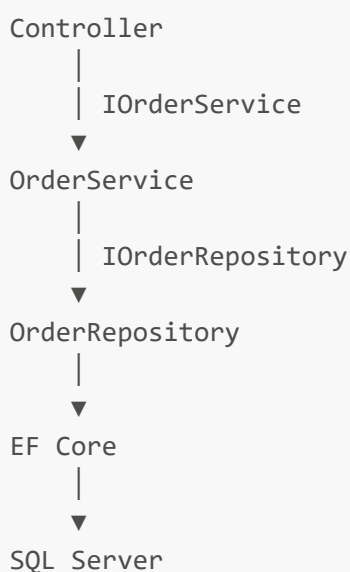
```

builder.Services.AddScoped<IOrderService, OrderService>();

builder.Services.AddScoped<IOrderRepository, OrderRepository>();

```

ASP.NET Core will automatically inject them.



8. What belongs in the Service Layer?

The service layer should generally contain **business/application logic**.

For example:

```
public async Task<bool> CancelOrderAsync(int orderId)
{
    var order = await _repository.GetByIdAsync(orderId);

    if (order == null)
        return false;

    if (order.Status == OrderStatus.Shipped)
    {
        throw new InvalidOperationException(
            "Shipped orders cannot be cancelled.");
    }

    order.Status = OrderStatus.Cancelled;

    await _repository.SaveChangesAsync();

    return true;
}
```

The rule:

```
Shipped order → Cannot cancel
```

is a **business rule**, so it belongs in the service rather than the controller.

9. Real-World Example

Suppose you are developing a banking application.

A withdrawal request might look like:

```
POST /api/accounts/100/withdraw
```

Controller:

```
[HttpPost("{accountId}/withdraw")]
public async Task<IActionResult> Withdraw(
    int accountId,
    decimal amount)
```

```
{
    await _accountService.WithdrawAsync(
        accountId,
        amount);

    return Ok();
}
```

The service handles the business rules:

```
public async Task WithdrawAsync(
    int accountId,
    decimal amount)
{
    var account =
        await _repository.GetByIdAsync(accountId);

    if (account == null)
        throw new Exception("Account not found.");

    if (amount <= 0)
        throw new Exception("Invalid amount.");

    if (account.Balance < amount)
        throw new Exception("Insufficient balance.");

    if (account.IsBlocked)
        throw new Exception("Account is blocked.");

    account.Balance -= amount;

    await _repository.SaveChangesAsync();
}
```

The controller doesn't need to know how the withdrawal works.

10. Service Layer vs Repository

This is very important for interviews.

Service	Repository
Contains business logic	Contains data access logic
Defines application operations	Performs CRUD/data queries
Coordinates multiple repositories	Usually works with a data source
Applies business rules	Doesn't normally decide business rules

Service	Repository
TransferMoney()	GetAccountById()
PlaceOrder()	AddOrder()
CancelOrder()	UpdateOrder()

For example:

```
TransferMoney()  
  |  
  ├── Get sender account  
  ├── Get receiver account  
  ├── Check balance  
  ├── Check account status  
  ├── Deduct money  
  ├── Add money  
  └── Save transaction
```

This is **service-layer responsibility**.

The repository might provide:

```
GetAccountById()  
UpdateAccount()  
AddTransaction()
```

11. Service Layer with Multiple Repositories

A service can coordinate multiple repositories.

```
public class PaymentService : IPaymentService  
{  
    private readonly IAccountRepository _accountRepository;  
    private readonly ITransactionRepository _transactionRepository;  
  
    public PaymentService(  
        IAccountRepository accountRepository,  
        ITransactionRepository transactionRepository)  
    {  
        _accountRepository = accountRepository;  
        _transactionRepository = transactionRepository;  
    }  
  
    public async Task TransferAsync(  

```



```

        int fromAccountId,
        int toAccountId,
        decimal amount)
    {
        var sender =
            await _accountRepository.GetByIdAsync(fromAccountId);

        var receiver =
            await _accountRepository.GetByIdAsync(toAccountId);

        if (sender == null || receiver == null)
            throw new Exception("Account not found.");

        if (sender.Balance < amount)
            throw new Exception("Insufficient balance.");

        sender.Balance -= amount;
        receiver.Balance += amount;

        await _transactionRepository.AddAsync(
            new Transaction
            {
                FromAccountId = fromAccountId,
                ToAccountId = toAccountId,
                Amount = amount,
                CreatedAt = DateTime.UtcNow
            });

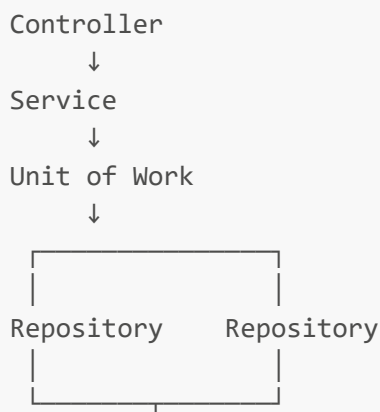
        await _accountRepository.SaveChangesAsync();
    }
}

```

This is one of the major reasons Service Layer is useful.

12. Service Layer + Unit of Work

In larger applications, you'll often see:



↓
Database

For example:

```
public async Task TransferAsync(
    int fromId,
    int toId,
    decimal amount)
{
    var sender =
        await _unitOfWork.Accounts.GetByIdAsync(fromId);

    var receiver =
        await _unitOfWork.Accounts.GetByIdAsync(toId);

    sender.Balance -= amount;
    receiver.Balance += amount;

    await _unitOfWork.Transactions.AddAsync(
        new Transaction
        {
            FromAccountId = fromId,
            ToAccountId = toId,
            Amount = amount
        });

    await _unitOfWork.SaveChangesAsync();
}
```

The service coordinates the operation while Unit of Work manages the transaction/persistence boundary.

13. Recommended ASP.NET Core Structure

A simple application:

```
MyApp
├── Controllers
│   └── OrderController.cs
├── Services
│   ├── Interfaces
│   │   └── IOrderService.cs
│   └── OrderService.cs
```

```
├── Repositories
│   ├── Interfaces
│   │   └── IOrderRepository.cs
│   └── OrderRepository.cs
├── Models
│   └── Order.cs
├── DTOs
│   └── CreateOrderDto.cs
├── Data
│   └── AppDbContext.cs
└── Program.cs
```

For a larger application:

```
MyApp
├── API
│   └── Controllers
├── Application
│   ├── Services
│   ├── DTOs
│   ├── Interfaces
│   └── Validators
├── Domain
│   ├── Entities
│   ├── ValueObjects
│   └── BusinessRules
└── Infrastructure
    ├── Repositories
    ├── EFCore
    └── ExternalServices
```

14. Service Layer and Dependency Injection

A good implementation uses interfaces:

```
public interface IUserService
{
```

```
Task<UserDto?> GetUserAsync(int id);  
}
```

```
public class UserService : IUserService  
{  
    private readonly IUserRepository _repository;  
  
    public UserService(IUserRepository repository)  
    {  
        _repository = repository;  
    }  
  
    public async Task<UserDto?> GetUserAsync(int id)  
    {  
        var user = await _repository.GetByIdAsync(id);  
  
        if (user == null)  
            return null;  
  
        return new UserDto  
        {  
            Id = user.Id,  
            Name = user.Name  
        };  
    }  
}
```

Register:

```
builder.Services.AddScoped<IUserService, UserService>();
```

15. Testing Becomes Easier

Because the controller depends on an interface:

```
IOrderService
```

you can mock it.

```
var mockService = new Mock<IOrderService>();  
  
mockService  
    .Setup(x => x.GetOrderAsync(1))
```

```
.ReturnsAsync(new Order
{
    Id = 1,
    Amount = 500
});
```

Then test the controller without accessing SQL Server.

Similarly, you can test `OrderService` by mocking:

```
IOrderRepository
```

instead of using a real database.

16. Common Mistake

Don't create a service that simply duplicates every repository method:

```
public Task<Order?> GetByIdAsync(int id)
{
    return _repository.GetByIdAsync(id);
}
```

If your entire service looks like this:

```
Service → Repository → Database
```

with no business/application behavior, you may just be adding unnecessary abstraction.

A service layer becomes particularly valuable when it **coordinates operations, enforces business rules, transforms data, or integrates multiple dependencies**.

17. Service Layer vs Controller

Bad

```
Controller
↓
Validation
↓
```

```
Business Rules
  ↓
Database
  ↓
Email
  ↓
Payment
```

Better

```
Controller
  ↓
Service
  ├── Repository
  ├── Payment Gateway
  ├── Email Service
  └── Notification Service
```

The controller focuses on HTTP:

```
Request
  ↓
Validate/request binding
  ↓
Call service
  ↓
HTTP response
```

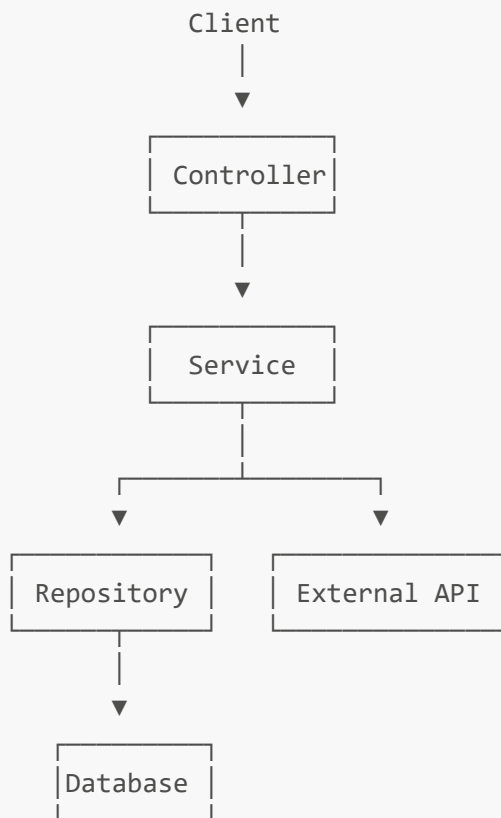
The service focuses on application/business behavior:

```
Business operation
  ↓
Business rules
  ↓
Repositories
  ↓
External services
  ↓
Persistence
```

18. Interview Definition

Service Layer is a pattern that encapsulates application and business logic behind a set of service operations, keeping controllers/UI separate from business rules and data-access concerns.

A typical ASP.NET Core architecture is:



Remember it simply

Controller = HTTP

Service = Business/Application logic

Repository = Data access

Database = Data storage

And when you combine this with the patterns you've been studying:

```
Controller
  ↓
Service Layer
  ↓
Unit of Work
  ↓
Repository
  ↓
EF Core / Dapper
  ↓
SQL Server
```

This combination is very common in **ASP.NET Core enterprise applications**.

4. Data Mapper

Data Mapper Design Pattern in C#

The **Data Mapper** is a **Data Access / Enterprise Application Architecture pattern** that keeps your **domain/business objects completely separate from database objects**.

The mapper is responsible for converting:

- **Database row** → **Domain object**
- **Domain object** → **Database row**

The important idea is:

The domain model should not know how data is stored in the database.

1. Problem without Data Mapper

Suppose you have a **Customer** domain object:

```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
}
```

You could directly put SQL/database logic inside it:

```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }

    public void Save(SqlConnection connection)
    {
        // SQL code
    }
}
```

This creates a problem:


```

Customer
|
+----- Business logic
|
+----- SQL
|
+----- Database knowledge

```

Now your domain object is tightly coupled to SQL Server.

2. Data Mapper Solution

With Data Mapper:

```

Application
|
v
Customer Domain
|
Data Mapper
|
v
Database

```

The **Customer** class only represents business data.

The mapper handles persistence.

```

Customer
|
| map
v
CustomerDataMapper
|
v
Database

```

3. Basic Example

Domain Model

```
public class Customer
{
    public int Id { get; set; }

    public string Name { get; set; }

    public string Email { get; set; }
}
```

Data Mapper Interface

```
public interface ICustomerMapper
{
    Customer GetById(int id);

    void Insert(Customer customer);

    void Update(Customer customer);

    void Delete(int id);
}
```

Implementation

```
public class CustomerMapper : ICustomerMapper
{
    private readonly string _connectionString;

    public CustomerMapper(string connectionString)
    {
        _connectionString = connectionString;
    }

    public Customer GetById(int id)
    {
        using SqlConnection connection =
            new SqlConnection(_connectionString);

        connection.Open();

        string sql = """
            SELECT Id, Name, Email
            FROM Customers
            WHERE Id = @Id
            """;

        using SqlCommand command =
            new SqlCommand(sql, connection);
```

```
        command.Parameters.AddWithValue("@Id", id);

        using SqlDataReader reader = command.ExecuteReader();

        if (!reader.Read())
            return null;

        return new Customer
        {
            Id = reader.GetInt32(0),
            Name = reader.GetString(1),
            Email = reader.GetString(2)
        };
    }

    public void Insert(Customer customer)
    {
        using SqlConnection connection =
            new SqlConnection(_connectionString);

        connection.Open();

        string sql = """
            INSERT INTO Customers (Name, Email)
            VALUES (@Name, @Email)
            """;

        using SqlCommand command =
            new SqlCommand(sql, connection);

        command.Parameters.AddWithValue("@Name", customer.Name);
        command.Parameters.AddWithValue("@Email", customer.Email);

        command.ExecuteNonQuery();
    }

    public void Update(Customer customer)
    {
        using SqlConnection connection =
            new SqlConnection(_connectionString);

        connection.Open();

        string sql = """
            UPDATE Customers
            SET Name = @Name,
                Email = @Email
            WHERE Id = @Id
            """;

        using SqlCommand command =
            new SqlCommand(sql, connection);
```

```

        command.Parameters.AddWithValue("@Id", customer.Id);
        command.Parameters.AddWithValue("@Name", customer.Name);
        command.Parameters.AddWithValue("@Email", customer.Email);

        command.ExecuteNonQuery();
    }

    public void Delete(int id)
    {
        using SqlConnection connection =
            new SqlConnection(_connectionString);

        connection.Open();

        string sql = ""
            DELETE FROM Customers
            WHERE Id = @Id
            "";

        using SqlCommand command =
            new SqlCommand(sql, connection);

        command.Parameters.AddWithValue("@Id", id);

        command.ExecuteNonQuery();
    }
}

```

The important separation is:

```

Customer
  ↓
Pure domain object

CustomerMapper
  ↓
Database operations

```

4. Mapping in Both Directions

Data Mapper essentially performs two types of conversion.

Database → Object

Suppose database contains:

Id	Name	Email
-----------	-------------	--------------

Id	Name	Email
10	Rahul	rahul@gmail.com

The mapper converts it:

```
Customer customer = new Customer
{
    Id = 10,
    Name = "Rahul",
    Email = "rahul@gmail.com"
};
```

This is:

```
Database
  ↓
Data Mapper
  ↓
Customer
```

Object → Database

Suppose:

```
Customer customer = new Customer
{
    Id = 10,
    Name = "Rahul",
    Email = "new@gmail.com"
};
```

The mapper converts this into database operations:

```
UPDATE Customers
SET Name = @Name,
    Email = @Email
WHERE Id = @Id
```

This is:

```
Customer
  ↓
```

```
Data Mapper
↓
Database
```

5. Separate Mapper Class

You can also separate the **mapping logic** from database operations.

```
public class CustomerDataMapper
{
    public Customer Map(SqlDataReader reader)
    {
        return new Customer
        {
            Id = reader.GetInt32(reader.GetOrdinal("Id")),
            Name = reader.GetString(reader.GetOrdinal("Name")),
            Email = reader.GetString(reader.GetOrdinal("Email"))
        };
    }
}
```

Then:

```
CustomerDataMapper mapper = new CustomerDataMapper();

Customer customer = mapper.Map(reader);
```

Now you have:

```
Database
↓
SqlDataReader
↓
CustomerDataMapper
↓
Customer
```

6. Data Mapper vs Repository

These two patterns are commonly confused.

Data Mapper

Responsible primarily for:

Mapping data between persistence representation and domain representation.

```
Database Row
  ↓
Data Mapper
  ↓
Domain Object
```

Repository

Responsible for:

Providing a collection-like interface for accessing domain objects.

```
Application
  ↓
Repository
  ↓
Data Mapper / ORM
  ↓
Database
```

For example:

```
public interface ICustomerRepository
{
    Customer GetById(int id);

    void Add(Customer customer);

    void Update(Customer customer);

    void Remove(Customer customer);
}
```

A repository might internally use a mapper:

```
CustomerService
  ↓
CustomerRepository
  ↓
CustomerMapper
```

↓
SQL Server

7. Data Mapper vs Active Record

This is an important interview question.

Active Record

The domain object itself knows how to save itself.

```
Customer customer = new Customer();  
  
customer.Save();  
customer.Delete();
```

Conceptually:

```
Customer  
├── Data  
├── Business Logic  
└── Database Logic
```

Data Mapper

The domain object doesn't know about the database.

```
Customer customer = customerMapper.GetById(10);
```

Conceptually:

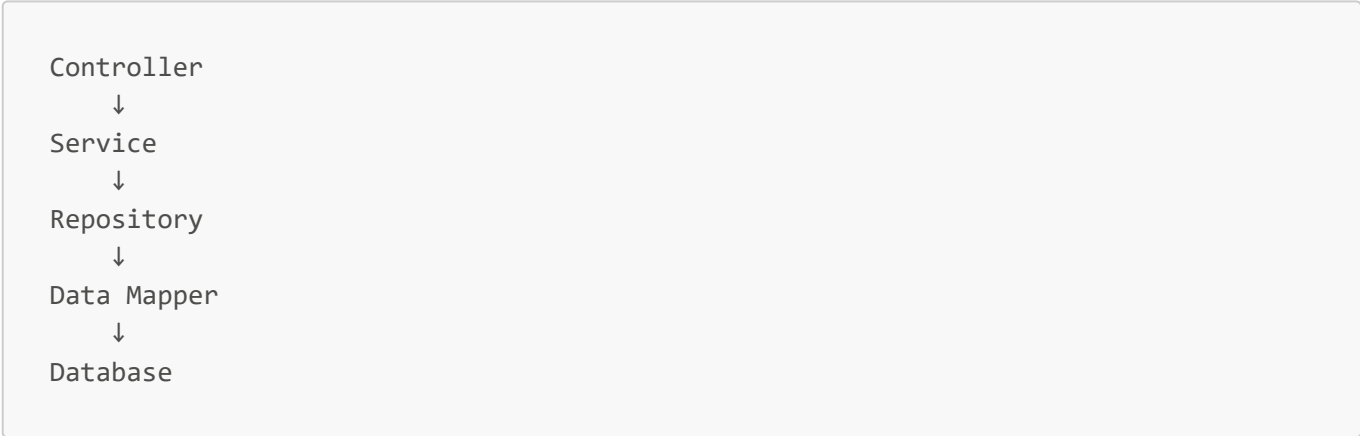
```
Customer  
  |  
  | independent  
  |  
Data Mapper  
  |  
  v  
Database
```

Comparison

Feature	Active Record	Data Mapper
DB logic	Entity	Mapper
Domain/DB coupling	High	Low
Simple CRUD	Excellent	Good
Complex domain	Less suitable	Excellent
Testability	Lower	Higher
Domain model	Persistence-aware	Persistence-ignorant

8. Data Mapper in ASP.NET Core

A typical ASP.NET Core architecture could look like:



Example:

```
public class CustomerService
{
    private readonly ICustomerRepository _repository;

    public CustomerService(ICustomerRepository repository)
    {
        _repository = repository;
    }

    public Customer GetCustomer(int id)
    {
        return _repository.GetById(id);
    }
}
```

Repository:

```
public class CustomerRepository : ICustomerRepository
{
    private readonly CustomerMapper _mapper;

    public CustomerRepository(CustomerMapper mapper)
    {
        _mapper = mapper;
    }

    public Customer GetById(int id)
    {
        return _mapper.GetById(id);
    }
}
```

Overall:

```
HTTP Request
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Data Mapper
  ↓
SQL Server
```

9. With DTOs

In a real ASP.NET Core application, you may have:

```
CustomerDto
Customer
CustomerEntity
```

For example:

```
public class CustomerDto
{
    public string Name { get; set; }
    public string Email { get; set; }
}
```

Domain:

```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
}
```

Database entity:

```
public class CustomerEntity
{
    public int CustomerId { get; set; }
    public string CustomerName { get; set; }
    public string EmailAddress { get; set; }
}
```

The mapper can translate:

```
API DTO
  ↓
Domain Model
  ↓
Database Entity
```

and back:

```
Database Entity
  ↓
Domain Model
  ↓
API DTO
```

This is especially useful when your database structure is different from your business model.

10. Advantages

1. Low coupling

Domain objects don't need to know about SQL Server.

2. Better testing

You can test business logic without requiring a database.

3. Flexible database design

Database schema can change without necessarily changing the domain model.

4. Supports complex domain models

Useful when the domain model doesn't map directly to database tables.

5. Separation of concerns

```
Domain
  → Business rules

Mapper
  → Persistence mapping

Repository
  → Data access abstraction
```

11. Disadvantages

More classes

You may have:

```
Customer
CustomerDto
CustomerEntity
CustomerMapper
CustomerRepository
CustomerService
```

For a simple CRUD application, this can feel like overengineering.

Mapping code

You have to maintain mappings:

```
Id      → CustomerId
Name    → CustomerName
Email   → EmailAddress
```

More complexity

For simple applications, an ORM such as Entity Framework Core can handle much of the mapping automatically.

12. Where EF Core Fits

EF Core itself uses concepts closely related to the **Data Mapper** approach.

You normally write:

```
var customer = await dbContext.Customers
    .FirstOrDefaultAsync(x => x.Id == id);
```

EF Core handles much of the conversion:

```
SQL row
  ↓
EF Core
  ↓
.NET object
```

So you don't necessarily need to manually write:

```
reader.GetInt32(...)
reader.GetString(...)
```

for every entity.

13. Interview Definition

If an interviewer asks:

What is the Data Mapper design pattern?

A good answer is:

Data Mapper is a persistence pattern that separates domain objects from database or persistence logic. A separate mapper is responsible for converting data between the database representation and the domain model. This reduces coupling between business logic and the database and improves testability and maintainability.

Remember this diagram

```
ASP.NET Core
|
Controller
|
Service
|
Repository
|
Data Mapper
|
Database
```

Key difference to remember:

Active Record:
Entity → saves itself

Data Mapper:
Mapper → saves entity

That distinction is one of the most important things to know for interviews.

5. Active Record

Active Record Design Pattern in C#

Active Record is a **data access design pattern** where an object represents a database row **and also contains the methods required to save, update, delete, or retrieve itself**.

In simple terms:

Entity + Database Operations = Active Record

For example, instead of having a separate `UserRepository`, the `User` object itself can perform database operations.

1. Basic Structure

Imagine a `Users` table:

```
Users
-----
Id
Name
```

```
Email  
Salary
```

With Active Record, we create:

```
public class User  
{  
    public int Id { get; set; }  
    public string Name { get; set; }  
    public string Email { get; set; }  
    public decimal Salary { get; set; }  
  
    public void Save()  
    {  
        // INSERT or UPDATE database  
    }  
  
    public void Delete()  
    {  
        // DELETE database record  
    }  
  
    public static User Find(int id)  
    {  
        // SELECT database record  
        return new User();  
    }  
}
```

The `User` class is responsible for both:

1. Representing the user
2. Performing database operations

2. Complete Example

Let's create a simple Active Record using ADO.NET.

```
using System;  
using Microsoft.Data.SqlClient;  
  
public class Employee  
{  
    private static readonly string connectionString =  
  
    "Server=localhost;Database=CompanyDb;Trusted_Connection=True;TrustServerCertificat  
e=True;";
```

```
public int Id { get; set; }
public string Name { get; set; }
public decimal Salary { get; set; }

// Create or Update
public void Save()
{
    using SqlConnection connection = new(connectionString);

    connection.Open();

    string sql;

    if (Id == 0)
    {
        sql = """
            INSERT INTO Employees (Name, Salary)
            OUTPUT INSERTED.Id
            VALUES (@Name, @Salary)
            """;

        using SqlCommand command = new(sql, connection);

        command.Parameters.AddWithValue("@Name", Name);
        command.Parameters.AddWithValue("@Salary", Salary);

        Id = (int)command.ExecuteScalar();
    }
    else
    {
        sql = """
            UPDATE Employees
            SET Name = @Name,
                Salary = @Salary
            WHERE Id = @Id
            """;

        using SqlCommand command = new(sql, connection);

        command.Parameters.AddWithValue("@Id", Id);
        command.Parameters.AddWithValue("@Name", Name);
        command.Parameters.AddWithValue("@Salary", Salary);

        command.ExecuteNonQuery();
    }
}

// Delete
public void Delete()
{
    using SqlConnection connection = new(connectionString);

    connection.Open();
```



```

        string sql = """
            DELETE FROM Employees
            WHERE Id = @Id
            """;

        using SqlCommand command = new(sql, connection);

        command.Parameters.AddWithValue("@Id", Id);

        command.ExecuteNonQuery();
    }

    // Find
    public static Employee? Find(int id)
    {
        using SqlConnection connection = new(connectionString);

        connection.Open();

        string sql = """
            SELECT Id, Name, Salary
            FROM Employees
            WHERE Id = @Id
            """;

        using SqlCommand command = new(sql, connection);

        command.Parameters.AddWithValue("@Id", id);

        using SqlDataReader reader = command.ExecuteReader();

        if (!reader.Read())
            return null;

        return new Employee
        {
            Id = reader.GetInt32(0),
            Name = reader.GetString(1),
            Salary = reader.GetDecimal(2)
        };
    }
}

```

Now the object itself controls persistence.

3. Using Active Record

Insert

```
Employee employee = new()
{
    Name = "Rahul",
    Salary = 50000
};

employee.Save();
```

The `Save()` method performs:

```
INSERT INTO Employees
(Name, Salary)
VALUES
('Rahul', 50000);
```

Update

```
Employee employee = Employee.Find(10);

if (employee != null)
{
    employee.Salary = 60000;

    employee.Save();
}
```

Because `Id != 0`, `Save()` performs an `UPDATE`.

Find

```
Employee employee = Employee.Find(10);

Console.WriteLine(employee?.Name);
```

Delete

```
Employee employee = Employee.Find(10);

if (employee != null)
{
```

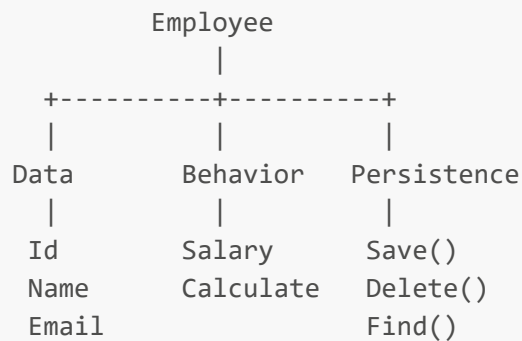
```

    employee.Delete();
}

```

4. Active Record Architecture

The important idea is:



The entity contains both **business/data information** and **persistence logic**.

5. Active Record vs Repository

This is particularly important because you've been looking at the Repository Pattern.

Active Record

```

Employee
|
+-- Properties
|
+-- Save()
+-- Delete()
+-- Find()

```

Repository

```

Employee
|
+-- Properties
|
+-- EmployeeRepository
|

```

```
+-- Add()  
+-- Update()  
+-- Delete()  
+-- GetById()
```

The key difference:

Active Record puts database operations inside the entity. Repository separates database operations from the entity.

6. Comparison

Feature	Active Record	Repository
Entity contains DB logic	Yes	No
Separate data-access class	Usually no	Yes
Simple CRUD	Excellent	Good
Complex business logic	Can become difficult	Better
Testing	More difficult	Easier
Separation of concerns	Lower	Higher
Large applications	Usually less suitable	Usually better
Small applications	Very suitable	May be overkill
ORM support	Common	Common

7. Active Record with Entity Framework

In modern .NET applications, you will often see something that **looks similar** to Active Record, but standard EF Core itself follows a different architectural approach.

For example:

```
public class Employee  
{  
    public int Id { get; set; }  
    public string Name { get; set; }  
    public decimal Salary { get; set; }  
}
```

Then:

```
public class AppDbContext : DbContext
{
    public DbSet<Employee> Employees { get; set; }
}
```

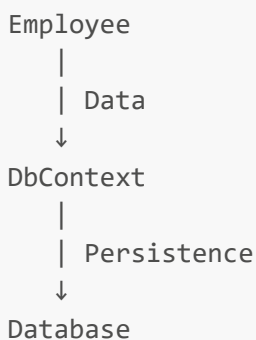
Operations happen through the **DbContext**:

```
Employee employee = new()
{
    Name = "Rahul",
    Salary = 50000
};

context.Employees.Add(employee);

context.SaveChanges();
```

Here:



So don't automatically call EF Core's entities "Active Record."

8. Active Record with Business Logic

A major characteristic of Active Record is that the entity can contain behavior.

For example:

```
public class BankAccount
{
    public int Id { get; set; }
    public decimal Balance { get; private set; }

    public void Deposit(decimal amount)
    {
```

```
        if (amount <= 0)
            throw new ArgumentException("Invalid amount");

        Balance += amount;
    }

    public void Withdraw(decimal amount)
    {
        if (amount > Balance)
            throw new InvalidOperationException("Insufficient balance");

        Balance -= amount;
    }

    public void Save()
    {
        // Save to database
    }
}
```

Usage:

```
var account = new BankAccount();

account.Deposit(10000);

account.Withdraw(2000);

account.Save();
```

The object contains:

```
BankAccount
├── Data
│   ├── Id
│   └── Balance
├── Business behavior
│   ├── Deposit()
│   └── Withdraw()
└── Persistence
    └── Save()
```

9. Advantages

1. Very simple

CRUD operations are easy:

```
employee.Save();  
employee.Delete();
```

2. Less boilerplate

You don't necessarily need:

```
IEmployeeRepository  
EmployeeRepository  
EmployeeService
```

for very simple applications.

3. Easy to understand

The object knows how to persist itself.

```
Object → Database
```

4. Good for simple CRUD applications

For applications with straightforward database operations, Active Record can be convenient.

10. Disadvantages

1. Violates separation of concerns

The entity is responsible for two things:

```
Business logic  
+  
Database logic
```

This can make the class large.

2. Harder to unit test

Consider:

```
employee.Save();
```

If `Save()` directly connects to SQL Server, testing the employee can require database infrastructure.

3. Complex queries become messy

Suppose you need:

```
Employees
  ↓
Departments
  ↓
Projects
  ↓
Salary
  ↓
Performance
  ↓
Permissions
```

Putting all this inside `Employee` can become difficult to maintain.

4. Tight coupling to database

The entity knows about:

```
SqlConnection
SqlCommand
SqlDataReader
```

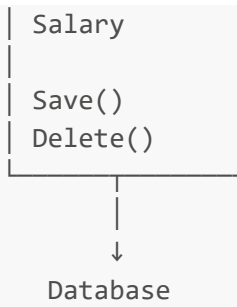
Therefore, your domain object becomes coupled to the persistence technology.

11. Active Record vs Data Mapper

Since you were also studying **Data Mapper**, this distinction is important.

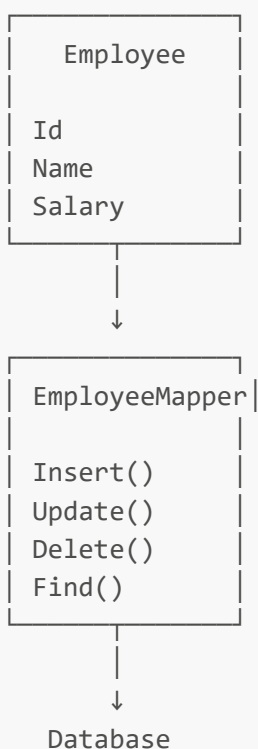
Active Record





The entity manages its own persistence.

Data Mapper



The entity doesn't know how it is stored.

12. When Should You Use Active Record?

Use Active Record when:

```
Small application
  ↓
Simple CRUD
  ↓
Simple business rules
  ↓
Few database relationships
```

↓
Active Record

Examples:

- Small internal tools
- Simple admin applications
- CRUD applications
- Prototypes
- Small applications where simplicity is more important than strict separation

For a large enterprise application, especially something with complicated business rules, **Repository/Data Mapper + Service Layer** is often a better architectural choice.

13. Interview Definition

If an interviewer asks:

What is Active Record?

A good answer is:

Active Record is a design pattern where an object represents a database record and also contains the methods required to perform persistence operations such as insert, update, delete, and retrieve. It is simple and useful for CRUD-oriented applications, but it couples domain objects with database logic, which can make large applications harder to maintain and test.

Remember it like this:

```
ACTIVE RECORD

Entity
+
Database Operations
=
Active Record
```

And the easiest comparison to remember:

```
Active Record:
Employee.Save()

Repository:
EmployeeRepository.Save(employee)
```

Entity owns persistence → Active Record **Separate object owns persistence** → Repository/Data Mapper

6. Specification Pattern

Specification Pattern in C#

The **Specification Pattern** is a behavioral/design pattern used to **encapsulate business rules or criteria into reusable objects**.

Instead of writing conditions everywhere like:

```
if (customer.IsActive && customer.Age >= 18 && customer.Country == "India")
{
    // ...
}
```

you create a **Specification** that represents the rule:

```
CustomerIsActive
    AND
CustomerIsAdult
    AND
CustomerIsFromIndia
```

This makes business rules **reusable, composable, testable, and easier to maintain**.

1. Basic idea

Suppose we have:

```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public int Age { get; set; }
    public bool IsActive { get; set; }
    public string Country { get; set; } = "";
}
```

Without Specification Pattern:

```
var customers = customers
    .Where(x => x.IsActive &&
        x.Age >= 18 &&
```

```
        x.Country == "India")  
    .ToList();
```

The problem becomes bigger when the same business rule is required in multiple places.

2. Create a Specification interface

A simple specification can look like this:

```
public interface ISpecification<T>  
{  
    bool IsSatisfiedBy(T entity);  
}
```

Now every business rule can implement this interface.

3. Create individual specifications

Active customer

```
public class ActiveCustomerSpecification  
    : ISpecification<Customer>  
{  
    public bool IsSatisfiedBy(Customer customer)  
    {  
        return customer.IsActive;  
    }  
}
```

Adult customer

```
public class AdultCustomerSpecification  
    : ISpecification<Customer>  
{  
    public bool IsSatisfiedBy(Customer customer)  
    {  
        return customer.Age >= 18;  
    }  
}
```

Indian customer

```
public class IndianCustomerSpecification
    : ISpecification<Customer>
{
    public bool IsSatisfiedBy(Customer customer)
    {
        return customer.Country == "India";
    }
}
```

Now each class represents **one business rule**.

4. Using the specifications

```
var activeSpec = new ActiveCustomerSpecification();
var adultSpec = new AdultCustomerSpecification();

foreach (var customer in customers)
{
    if (activeSpec.IsSatisfiedBy(customer) &&
        adultSpec.IsSatisfiedBy(customer))
    {
        Console.WriteLine(customer.Name);
    }
}
```

This works, but we can make the pattern much more powerful by allowing specifications to be combined.

5. Combining Specifications

We can create:

```
AND
OR
NOT
```

operations.

For example:

```
ActiveCustomer
AND
```

```
AdultCustomer
    AND
IndianCustomer
```

6. Base Specification

A common implementation is:

```
public abstract class Specification<T>
{
    public abstract bool IsSatisfiedBy(T entity);

    public Specification<T> And(
        Specification<T> other)
    {
        return new AndSpecification<T>(this, other);
    }

    public Specification<T> Or(
        Specification<T> other)
    {
        return new OrSpecification<T>(this, other);
    }

    public Specification<T> Not()
    {
        return new NotSpecification<T>(this);
    }
}
```

7. AND Specification

```
public class AndSpecification<T>
    : Specification<T>
{
    private readonly Specification<T> _left;
    private readonly Specification<T> _right;

    public AndSpecification(
        Specification<T> left,
        Specification<T> right)
    {
        _left = left;
        _right = right;
    }
}
```

```
    public override bool IsSatisfiedBy(T entity)
    {
        return _left.IsSatisfiedBy(entity)
            && _right.IsSatisfiedBy(entity);
    }
}
```

8. OR Specification

```
public class OrSpecification<T>
    : Specification<T>
{
    private readonly Specification<T> _left;
    private readonly Specification<T> _right;

    public OrSpecification(
        Specification<T> left,
        Specification<T> right)
    {
        _left = left;
        _right = right;
    }

    public override bool IsSatisfiedBy(T entity)
    {
        return _left.IsSatisfiedBy(entity)
            || _right.IsSatisfiedBy(entity);
    }
}
```

9. NOT Specification

```
public class NotSpecification<T>
    : Specification<T>
{
    private readonly Specification<T> _specification;

    public NotSpecification(
        Specification<T> specification)
    {
        _specification = specification;
    }

    public override bool IsSatisfiedBy(T entity)
```

```
    {  
        return !_specification.IsSatisfiedBy(entity);  
    }  
}
```

10. Customer specifications

Now our individual specifications inherit from `Specification<T>`.

```
public class ActiveCustomerSpecification  
    : Specification<Customer>  
{  
    public override bool IsSatisfiedBy(Customer customer)  
    {  
        return customer.IsActive;  
    }  
}
```

```
public class AdultCustomerSpecification  
    : Specification<Customer>  
{  
    public override bool IsSatisfiedBy(Customer customer)  
    {  
        return customer.Age >= 18;  
    }  
}
```

```
public class IndianCustomerSpecification  
    : Specification<Customer>  
{  
    public override bool IsSatisfiedBy(Customer customer)  
    {  
        return customer.Country == "India";  
    }  
}
```

11. Compose them

Now we can write:


```
var specification =  
    new ActiveCustomerSpecification()  
        .And(new AdultCustomerSpecification())  
        .And(new IndianCustomerSpecification());
```

This represents:

```
Active  
  AND  
Adult  
  AND  
Indian
```

Then:

```
foreach (var customer in customers)  
{  
    if (specification.IsSatisfiedBy(customer))  
    {  
        Console.WriteLine(customer.Name);  
    }  
}
```

12. OR example

Suppose we want:

```
Customer is from India  
OR  
Customer is from USA
```

We can write:

```
var specification =  
    new IndianCustomerSpecification()  
        .Or(new USCustomerSpecification());
```

Then:

```
if (specification.IsSatisfiedBy(customer))
{
    Console.WriteLine("Customer matched");
}
```

13. NOT example

Suppose we want:

NOT inactive customer

```
var specification =
    new ActiveCustomerSpecification().Not();
```

This actually means:

NOT Active

So it would match inactive customers.

If you want the positive rule "not inactive", you would normally define an `InactiveCustomerSpecification` and negate that:

```
var specification =
    new InactiveCustomerSpecification().Not();
```

14. More realistic example — Order

Consider an e-commerce system:

```
public class Order
{
    public int Id { get; set; }
    public decimal Amount { get; set; }
    public string Status { get; set; } = "";
    public bool IsPaid { get; set; }
}
```

We need a rule:

Order can be shipped when:

```
Amount >= 1000
AND
Status = "Confirmed"
AND
IsPaid = true
```

Create specifications.

```
public class MinimumAmountSpecification
    : Specification<Order>
{
    public override bool IsSatisfiedBy(Order order)
    {
        return order.Amount >= 1000;
    }
}
```

```
public class ConfirmedOrderSpecification
    : Specification<Order>
{
    public override bool IsSatisfiedBy(Order order)
    {
        return order.Status == "Confirmed";
    }
}
```

```
public class PaidOrderSpecification
    : Specification<Order>
{
    public override bool IsSatisfiedBy(Order order)
    {
        return order.IsPaid;
    }
}
```

Compose:

```
var canShip =  
    new MinimumAmountSpecification()  
        .And(new ConfirmedOrderSpecification())  
        .And(new PaidOrderSpecification());
```

Use:

```
if (canShip.IsSatisfiedBy(order))  
{  
    ShipOrder(order);  
}
```

This is much cleaner than putting a large condition inside `ShipOrder()`.

15. Specification with LINQ

In real ASP.NET Core applications, specifications are often used with databases.

Instead of only:

```
bool IsSatisfiedBy(T entity);
```

you can represent the rule as an expression:

```
Expression<Func<T, bool>>
```

For example:

```
public interface ISpecification<T>  
{  
    Expression<Func<T, bool>> Criteria { get; }  
}
```

Then:

```
public class ActiveCustomerSpecification  
    : ISpecification<Customer>  
{  
    public Expression<Func<Customer, bool>> Criteria =>
```

```
        customer => customer.IsActive;  
    }
```

Repository:

```
public async Task<List<Customer>> GetCustomers(  
    ISpecification<Customer> specification)  
{  
    return await _context.Customers  
        .Where(specification.Criteria)  
        .ToListAsync();  
}
```

The resulting query can be translated by EF Core into SQL.

Conceptually:

```
Specification  
    ↓  
Expression<Func<Customer,bool>>  
    ↓  
LINQ Where()  
    ↓  
EF Core  
    ↓  
SQL  
    ↓  
Database
```

16. Why use Specification Pattern?

Without it

You might have:

```
.Where(x => x.IsActive && x.Age >= 18)
```

in one service,

```
.Where(x => x.IsActive && x.Age >= 18)
```

in another service,

and:

```
.Where(x => x.IsActive && x.Age >= 18)
```

in a third place.

This creates duplicated business rules.

With Specification

You create:

```
AdultActiveCustomerSpecification
```

and reuse it:

```
repository.Get(specification);
```

17. Advantages

Benefit	Explanation
Reusable	Same business rule can be reused
Composable	Combine rules with AND/OR/NOT
Testable	Each rule can be unit tested
Maintainable	Business rules have dedicated classes
Separation of concerns	Rules aren't scattered throughout services
Database-friendly	Expression-based specifications can work with EF Core
Readable	Business rules become descriptive objects

18. Disadvantages

Don't create a specification for every tiny condition.

For example, this may be unnecessary:

```
class NameIsNotNullSpecification
```

for a trivial application.

Specification Pattern can introduce:

```
More classes
  ↓
More abstraction
  ↓
More complexity
```

So it is most useful when you have **complex, reusable business rules**.

19. Specification vs Strategy

These patterns are easy to confuse.

Specification

Answers:

Does this object satisfy a rule?

```
specification.IsSatisfiedBy(customer);
```

Result:

```
true / false
```

Strategy

Answers:

Which algorithm/behavior should I execute?

```
paymentStrategy.Pay(amount);
```

Result:

```
Different behavior/algorithm
```

Example:

```
Specification
    Customer is eligible for loan?

Strategy
    How should the loan interest be calculated?
```

20. Specification vs Repository

They solve different problems.

```
Specification
    ↓
Defines WHAT should match

Repository
    ↓
Defines HOW data is retrieved
```

Example:

```
var specification =
    new ActiveCustomerSpecification()
        .And(new AdultCustomerSpecification());

var customers =
    await repository.GetAsync(specification);
```

Here:

- **Specification** → defines the filtering/business rule
- **Repository** → retrieves the data

They work very well together.

21. Where you might use it in ASP.NET Core

A typical architecture could look like:


```
Controller
  ↓
Service Layer
  ↓
Repository
  ↓
Specification
  ↓
EF Core
  ↓
SQL Server
```

For example:

```
GET /customers/eligible
  ↓
CustomerService
  ↓
EligibleCustomerSpecification
  ↓
CustomerRepository
  ↓
EF Core
  ↓
SQL Server
```

22. Real-world fintech example

For a fintech application, you could have:

```
EligibleForLoanSpecification
EligibleForRewardSpecification
FraudFreeCustomerSpecification
KycCompletedSpecification
ActiveAccountSpecification
HighRiskCustomerSpecification
```

Then compose:

```
var eligible =
    new KycCompletedSpecification()
        .And(new ActiveAccountSpecification())
        .And(new FraudFreeCustomerSpecification())
        .And(new EligibleForLoanSpecification());
```

Conceptually:

```
KYC completed
  AND
Account active
  AND
No fraud
  AND
Loan eligible
  ↓
APPROVE
```

This is one of the situations where the Specification Pattern becomes particularly valuable because the business rules can become complicated and change frequently.

Key idea to remember

Think of **Specification** as a **reusable business-rule object**:

```
Specification
├── IsSatisfiedBy()
├── AND
├── OR
└── NOT
```

The simplest mental model is:

Specification Pattern = encapsulate a condition/business rule so that it can be reused and combined.

For ASP.NET Core + EF Core projects, the **Expression-based Specification** approach is usually more useful than a simple `bool IsSatisfiedBy()` implementation because the expression can be translated into SQL rather than filtering everything in memory.

7. Query Object

Query Object Pattern in C#

The **Query Object** pattern encapsulates a database query—including its filtering, sorting, paging, and other query parameters—inside a dedicated object.

It is useful when queries become complex and you don't want controllers/services filled with database-query logic.

1. The Problem

Imagine you have:

```
public async Task<List<Product>> GetProducts(  
    string? category,  
    decimal? minPrice,  
    decimal? maxPrice,  
    string? search,  
    bool sortByPrice)  
{  
    // Query logic becomes complicated...  
}
```

As requirements grow, this method can become difficult to maintain.

For example:

```
Category = Electronics  
Price >= 1000  
Price <= 50000  
Name contains "phone"  
Sort by price  
Only active products  
Page 2  
Page size 20
```

Instead of passing many parameters, create a **Query Object**.

2. Query Object

Create a class representing the query:

```
public class ProductQuery  
{  
    public string? Category { get; set; }  
  
    public decimal? MinPrice { get; set; }  
  
    public decimal? MaxPrice { get; set; }  
  
    public string? Search { get; set; }  
  
    public bool? IsActive { get; set; }  
}
```

```
public string? SortBy { get; set; }

public int Page { get; set; } = 1;

public int PageSize { get; set; } = 20;
}
```

Now instead of passing eight parameters:

```
GetProducts(
    "Electronics",
    1000,
    50000,
    "phone",
    true,
    true,
    2,
    20
);
```

you can pass one object:

```
var query = new ProductQuery
{
    Category = "Electronics",
    MinPrice = 1000,
    MaxPrice = 50000,
    Search = "phone",
    IsActive = true,
    SortBy = "price",
    Page = 2,
    PageSize = 20
};
```

3. Entity

Suppose we have:

```
public class Product
{
    public int Id { get; set; }

    public string Name { get; set; } = "";

    public string Category { get; set; } = "";
}
```

```
    public decimal Price { get; set; }

    public bool IsActive { get; set; }
}
```

And EF Core:

```
public class AppDbContext : DbContext
{
    public DbSet<Product> Products { get; set; }

    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }
}
```

4. Implementing the Query Object

The query object can contain the logic for building the query.

```
public class ProductQuery
{
    public string? Category { get; set; }

    public decimal? MinPrice { get; set; }

    public decimal? MaxPrice { get; set; }

    public string? Search { get; set; }

    public bool? IsActive { get; set; }

    public string? SortBy { get; set; }

    public int Page { get; set; } = 1;

    public int PageSize { get; set; } = 20;

    public IQueryable<Product> Apply(IQueryable<Product> query)
    {
        if (!string.IsNullOrEmpty(Category))
        {
            query = query.Where(x => x.Category == Category);
        }
    }
}
```

```
        if (MinPrice.HasValue)
        {
            query = query.Where(x => x.Price >= MinPrice.Value);
        }

        if (MaxPrice.HasValue)
        {
            query = query.Where(x => x.Price <= MaxPrice.Value);
        }

        if (!string.IsNullOrEmpty(Search))
        {
            query = query.Where(x =>
                x.Name.Contains(Search));
        }

        if (IsActive.HasValue)
        {
            query = query.Where(x =>
                x.IsActive == IsActive.Value);
        }

        if (SortBy == "price")
        {
            query = query.OrderBy(x => x.Price);
        }
        else if (SortBy == "name")
        {
            query = query.OrderBy(x => x.Name);
        }

        query = query
            .Skip((Page - 1) * PageSize)
            .Take(PageSize);

        return query;
    }
}
```

5. Repository

The repository becomes much cleaner:

```
public class ProductRepository
{
    private readonly AppDbContext _context;

    public ProductRepository(AppDbContext context)
    {
    }
```

```
        _context = context;
    }

    public async Task<List<Product>> GetProductsAsync(
        ProductQuery productQuery)
    {
        IQueryable<Product> query =
            _context.Products.AsQueryable();

        query = productQuery.Apply(query);

        return await query.ToListAsync();
    }
}
```

Usage:

```
var productQuery = new ProductQuery
{
    Category = "Electronics",
    MinPrice = 1000,
    MaxPrice = 50000,
    Search = "phone",
    IsActive = true,
    SortBy = "price",
    Page = 1,
    PageSize = 10
};

var products =
    await repository.GetProductsAsync(productQuery);
```

6. ASP.NET Core API Example

You can expose the query object directly from an API:

```
[ApiController]
[Route("api/products")]
public class ProductsController : ControllerBase
{
    private readonly ProductRepository _repository;

    public ProductsController(ProductRepository repository)
    {
        _repository = repository;
    }
}
```

```
[HttpGet]
public async Task<IActionResult> GetProducts(
    [FromQuery] ProductQuery query)
{
    var products =
        await _repository.GetProductsAsync(query);

    return Ok(products);
}
```

Then the client can call:

```
GET /api/products?
category=Electronics&minPrice=1000&maxPrice=50000&search=phone&page=1&pageSize=10
```

ASP.NET Core automatically binds the query-string values to:

ProductQuery

7. Better Architecture

In a larger application, I would separate the **HTTP request DTO** from the actual query logic.

For example:

```
Controller
  ↓
ProductQueryRequest
  ↓
ProductQuery
  ↓
Repository
  ↓
EF Core
  ↓
Database
```

You can also combine Query Object with the **Specification Pattern**.

Query Object

Usually represents:

"What does the caller want?"

Example:

```
new ProductQuery
{
    Category = "Electronics",
    MinPrice = 1000,
    Page = 2
};
```

Specification

Usually represents:

"What business/database rule should be applied?"

Example:

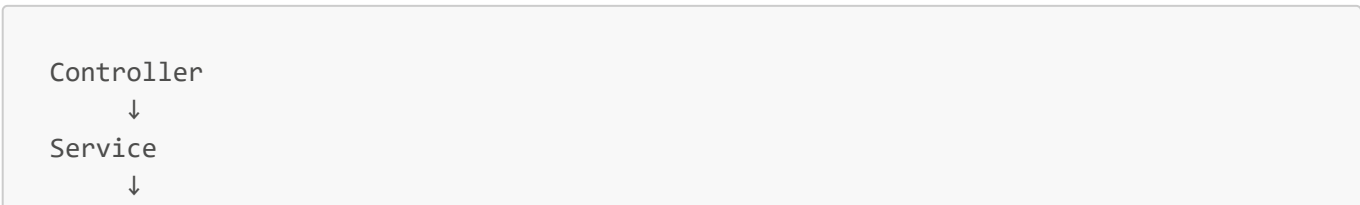
```
new ActiveProductsSpecification()
```

8. Query Object vs Repository

They solve different problems.

Pattern	Responsibility
Query Object	Represents query criteria
Repository	Provides access to persistence
Specification	Encapsulates reusable business/query rules
Unit of Work	Manages transaction boundary
Service Layer	Implements application/business operations

A common architecture is:



```
Repository
  ↓
Query Object / Specification
  ↓
EF Core
  ↓
Database
```

9. When to Use Query Object

Use it when you have:

- Many search/filter parameters
- Complex filtering
- Sorting
- Pagination
- Dynamic queries
- Reporting screens
- Admin dashboards
- Search APIs
- Frequently changing query requirements

For example:

```
Customer Search
  ↓
Name
Email
Country
Status
CreatedFrom
CreatedTo
MinimumBalance
MaximumBalance
Sort
Page
PageSize
```

Instead of:

```
SearchCustomers(
    name,
    email,
    country,
    status,
    createdFrom,
```

```
createdTo,  
minBalance,  
maxBalance,  
sort,  
page,  
pageSize);
```

use:

```
SearchCustomers(CustomerQuery query);
```

That's the main benefit of the **Query Object Pattern**: it turns a complicated set of query parameters and query logic into a reusable, maintainable object.

8. Identity Map

Identity Map Design Pattern in C#

The **Identity Map** is a data-access design pattern used to ensure that, within a particular context/session, **each database row is represented by only one object instance**.

In simple words:

If you load the same database record twice, Identity Map returns the **same C# object** instead of creating two different objects.

1. The problem

Suppose we have a **Customer** table:

```
Customer  
-----  
Id      Name  
1       Rahul  
2       Amit
```

Without Identity Map:

```
Customer customer1 = repository.GetById(1);  
Customer customer2 = repository.GetById(1);  
  
Console.WriteLine(customer1 == customer2);
```

Result:

```
False
```

Even though both represent:

```
Customer Id = 1
```

They are two different objects in memory.

```
customer1 —> Customer { Id = 1, Name = "Rahul" }  
customer2 —> Customer { Id = 1, Name = "Rahul" }
```

This can cause problems if one object is modified.

2. Identity Map solution

Identity Map maintains a dictionary:

```
Database ID → Object instance
```

For example:

```
1 → Customer object  
2 → Customer object  
3 → Customer object
```

When you request customer 1:

```
GetCustomer(1)  
  ↓  
Is ID 1 already in map?  
  ↓  
YES  
  ↓  
Return existing object
```

So:

```
var customer1 = repository.GetById(1);  
var customer2 = repository.GetById(1);  
  
Console.WriteLine(customer1 == customer2);
```

Result:

True

3. Basic C# implementation

Let's create a simple Identity Map.

Entity

```
public class Customer  
{  
    public int Id { get; set; }  
    public string Name { get; set; } = "";  
}
```

Identity Map

```
public class IdentityMap  
{  
    private readonly Dictionary<int, Customer> _customers = new();  
  
    public Customer? Get(int id)  
    {  
        _customers.TryGetValue(id, out var customer);  
        return customer;  
    }  
  
    public void Add(Customer customer)  
    {  
        _customers[customer.Id] = customer;  
    }  
  
    public bool Contains(int id)  
    {  
        return _customers.ContainsKey(id);  
    }  
}
```

```
    }

    public void Remove(int id)
    {
        _customers.Remove(id);
    }

    public void Clear()
    {
        _customers.Clear();
    }
}
```

4. Using Identity Map in Repository

Suppose we have a repository:

```
public class CustomerRepository
{
    private readonly IdentityMap _identityMap;

    public CustomerRepository(IdentityMap identityMap)
    {
        _identityMap = identityMap;
    }

    public Customer GetById(int id)
    {
        // 1. Check Identity Map
        var existingCustomer = _identityMap.Get(id);

        if (existingCustomer != null)
        {
            return existingCustomer;
        }

        // 2. Load from database
        var customer = LoadFromDatabase(id);

        // 3. Store in Identity Map
        _identityMap.Add(customer);

        return customer;
    }

    private Customer LoadFromDatabase(int id)
    {
        Console.WriteLine("Database query executed...");
    }
}
```

```
        return new Customer
        {
            Id = id,
            Name = "Rahul"
        };
    }
}
```

Now:

```
var identityMap = new IdentityMap();

var repository = new CustomerRepository(identityMap);

var customer1 = repository.GetById(1);

var customer2 = repository.GetById(1);

Console.WriteLine(customer1 == customer2);
```

Output:

```
Database query executed...
True
```

Notice that the database was accessed only once.

5. How it works

The first call:

```
var customer1 = repository.GetById(1);
```

Flow:

```
GetById(1)
  ↓
Identity Map
  ↓
ID 1 exists?
  ↓
NO
  ↓
```

```
Database
  ↓
Customer object created
  ↓
Add to Identity Map
  ↓
Return Customer
```

The second call:

```
var customer2 = repository.GetById(1);
```

Flow:

```
GetById(1)
  ↓
Identity Map
  ↓
ID 1 exists?
  ↓
YES
  ↓
Return existing Customer
```

Therefore:

```
customer1 == customer2
```

is:

```
True
```

6. Why is this useful?

Consider:

```
var customer1 = repository.GetById(1);

customer1.Name = "New Name";
```

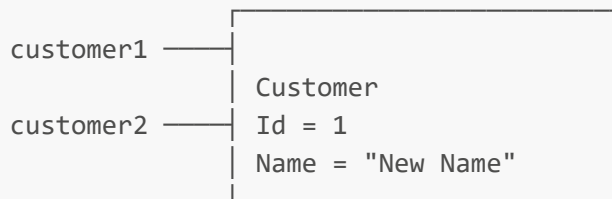


```
var customer2 = repository.GetById(1);  
  
Console.WriteLine(customer2.Name);
```

Because of Identity Map:

New Name

Both variables point to the same object.



Without Identity Map:

```
customer1 —> Customer #1  
             Name = "New Name"  
  
customer2 —> Customer #2  
             Name = "Rahul"
```

This can create inconsistent state.

7. Identity Map with multiple entities

A real application may have:

Customer
Order
Product
Payment

You could maintain separate maps:

```
public class IdentityMap  
{  
    private readonly Dictionary<int, Customer> _customers = new();
```

```
private readonly Dictionary<int, Order> _orders = new();
private readonly Dictionary<int, Product> _products = new();

// Customer methods
public Customer? GetCustomer(int id)
{
    _customers.TryGetValue(id, out var customer);
    return customer;
}

public void AddCustomer(Customer customer)
{
    _customers[customer.Id] = customer;
}

// Order methods
public Order? GetOrder(int id)
{
    _orders.TryGetValue(id, out var order);
    return order;
}

public void AddOrder(Order order)
{
    _orders[order.Id] = order;
}
}
```

8. Generic Identity Map

A cleaner approach is a generic implementation.

```
public class IdentityMap<TKey, TEntity>
    where TKey : notnull
{
    private readonly Dictionary<TKey, TEntity> _map = new();

    public TEntity? Get(TKey key)
    {
        _map.TryGetValue(key, out var entity);
        return entity;
    }

    public void Add(TKey key, TEntity entity)
    {
        _map[key] = entity;
    }

    public bool Contains(TKey key)
```

```
    {
        return _map.ContainsKey(key);
    }

    public void Remove(TKey key)
    {
        _map.Remove(key);
    }

    public void Clear()
    {
        _map.Clear();
    }
}
```

Usage:

```
var customerMap =
    new IdentityMap<int, Customer>();

var orderMap =
    new IdentityMap<int, Order>();
```

Then:

```
customerMap.Add(1, customer);

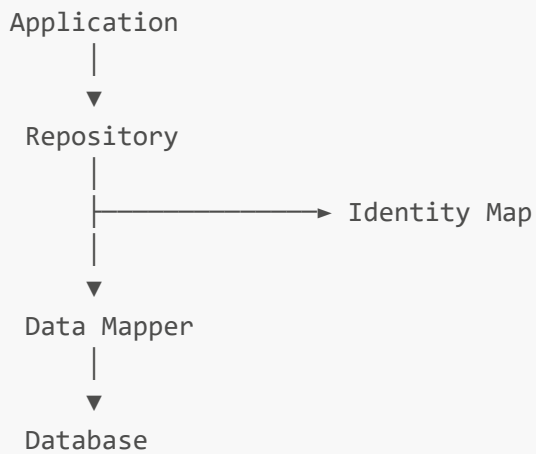
var customer = customerMap.Get(1);
```

9. Identity Map vs Repository

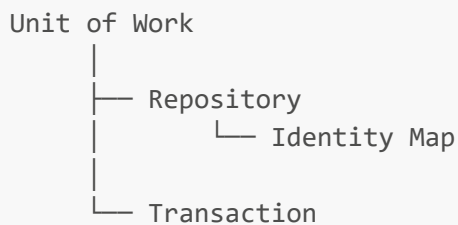
These two patterns are related but have different responsibilities.

Pattern	Responsibility
Repository	Provides access to entities
Identity Map	Ensures one object per identity
Unit of Work	Tracks changes and coordinates persistence
Data Mapper	Maps database records to objects
Cache	Stores data to avoid expensive retrieval

A typical architecture might look like:



And often:



10. Identity Map vs Cache

They look similar, but they solve different problems.

Cache

Main goal:

Avoid retrieving data repeatedly from the database.

Example:

```
Request 1 → Database
Request 2 → Cache
Request 3 → Cache
```

Identity Map

Main goal:

Ensure that the same database identity maps to the same object instance within a context.

Example:

```
Get Customer 1
  ↓
Customer object A

Get Customer 1
  ↓
Customer object A
```

So Identity Map is primarily about **object identity**, not simply performance.

11. Important: Identity Map is usually scoped

You generally don't want one global Identity Map for the entire application.

For example:

```
public class OrderService
{
    private readonly IdentityMap<int, Customer> _identityMap;

    public OrderService(IdentityMap<int, Customer> identityMap)
    {
        _identityMap = identityMap;
    }
}
```

In an ASP.NET Core application, it is commonly associated with a **request/unit-of-work scope**, depending on the architecture.

A global singleton can cause:

- memory growth
- stale objects
- concurrency issues
- unexpected state sharing

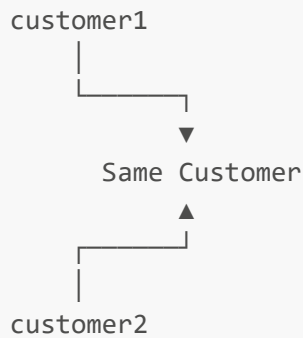
12. Identity Map + Unit of Work

This combination is especially useful.

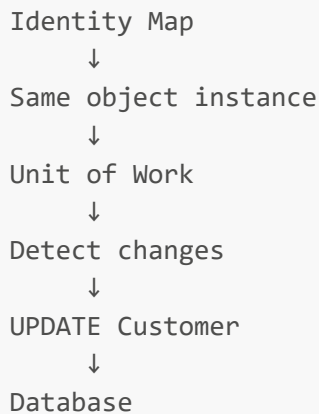
Imagine:

```
var customer1 = repository.GetById(10);  
  
customer1.Name = "Rahul";  
  
var customer2 = repository.GetById(10);
```

Identity Map ensures:



Then Unit of Work can detect that the entity was changed:



This is one reason Identity Map is frequently discussed alongside **Unit of Work** and **Data Mapper**.

13. Real-world example

Imagine an e-commerce application.

You execute:

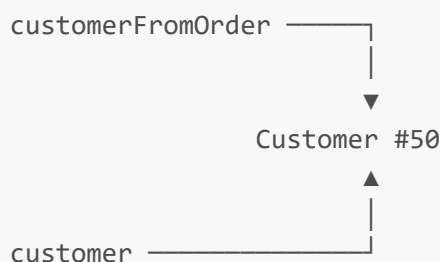
```
var order = orderRepository.GetById(100);  
  
var customerFromOrder =  
    customerRepository.GetById(order.CustomerId);
```

```
var customer =  
    customerRepository.GetById(50);
```

Suppose:

```
order.CustomerId = 50
```

With Identity Map:



Therefore:

```
customerFromOrder == customer
```

returns:

```
True
```

The application has one in-memory representation of Customer 50.

14. Interview definition

For an interview, you can say:

Identity Map is a data-access pattern that maintains a map of database identities to object instances within a particular context. When an entity with the same identity is requested again, the existing object instance is returned instead of creating another instance. This maintains object identity and helps prevent inconsistent in-memory state.

Remember the key idea

```
Database Identity
  ↓
Identity Map
  ↓
Same Object Instance
```

And the simplest implementation is essentially:

```
Dictionary<TKey, TEntity>
```

where the key represents the entity's unique database identity.

9. Lazy Load

Lazy Load Design Pattern in C#

Lazy Load is a design pattern used to **delay loading an object or data until it is actually needed**.

Instead of creating/loading everything immediately, we load it **only when the application first accesses it**.

Simple idea: "Don't load it now. Load it when someone asks for it."

1. Problem Without Lazy Loading

Suppose we have an **Employee** with a large **Profile**.

```
public class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }

    public EmployeeProfile Profile { get; set; }

    public Employee()
    {
        Profile = LoadProfile();
    }

    private EmployeeProfile LoadProfile()
    {
        Console.WriteLine("Loading profile from database...");

        return new EmployeeProfile
        {
            Address = "Mumbai",
            Phone = "9999999999"
        }
    }
}
```



```
    };  
  }  
}
```

Now:

```
Employee employee = new Employee();  
  
Console.WriteLine(employee.Name);
```

Even though we never use `Profile`, the profile is loaded.

Output:

```
Loading profile from database...
```

This can waste:

- Database calls
- Memory
- CPU
- Network resources
- Application startup time

2. Lazy Loading Solution

We can use C#'s built-in `Lazy<T>`.

```
public class Employee  
{  
    public int Id { get; set; }  
    public string Name { get; set; }  
  
    private Lazy<EmployeeProfile> _profile;  
  
    public Employee()  
    {  
        _profile = new Lazy<EmployeeProfile>(LoadProfile);  
    }  
  
    public EmployeeProfile Profile  
    {  
        get  
        {
```

```
        return _profile.Value;
    }
}

private EmployeeProfile LoadProfile()
{
    Console.WriteLine("Loading profile from database...");

    return new EmployeeProfile
    {
        Address = "Mumbai",
        Phone = "9999999999"
    };
}
```

Now:

```
Employee employee = new Employee();

Console.WriteLine(employee.Name);
```

The profile is **not loaded**.

But when we do:

```
Console.WriteLine(employee.Profile.Address);
```

the profile is loaded.

Output:

```
Loading profile from database...
Mumbai
```

3. How Lazy<T> Works

The important property is:

```
_profile.Value
```

The object is created when **Value** is accessed for the first time.

For example:

```
Lazy<EmployeeProfile> profile =  
    new Lazy<EmployeeProfile>(() =>  
    {  
        Console.WriteLine("Creating profile");  
  
        return new EmployeeProfile();  
    });
```

At this point:

```
EmployeeProfile NOT created
```

Then:

```
var p = profile.Value;
```

Now:

```
Creating profile  
EmployeeProfile CREATED
```

If we access it again:

```
var p2 = profile.Value;
```

The object isn't created again.

```
Creating profile
```

is printed only once.

4. Complete Example

```
public class EmployeeProfile  
{
```

```
        public string Address { get; set; }
        public string Phone { get; set; }
    }

    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }

        private readonly Lazy<EmployeeProfile> _profile;

        public Employee()
        {
            _profile = new Lazy<EmployeeProfile>(() =>
            {
                Console.WriteLine("Loading Employee Profile...");

                return new EmployeeProfile
                {
                    Address = "Mumbai",
                    Phone = "9999999999"
                };
            });
        }

        public EmployeeProfile Profile => _profile.Value;
    }
```

Usage:

```
Employee employee = new Employee
{
    Id = 1,
    Name = "Rahul"
};

Console.WriteLine("Employee created");

Console.WriteLine(employee.Name);

Console.WriteLine("Now accessing profile...");

Console.WriteLine(employee.Profile.Address);

Console.WriteLine(employee.Profile.Phone);
```

Output:

```
Employee created
Rahul
Now accessing profile...
Loading Employee Profile...
Mumbai
9999999999
```

Notice:

```
Employee created
```

doesn't load the profile.

Only this:

```
employee.Profile
```

causes the profile to load.

5. Lazy Loading in Entity Framework Core

Lazy loading is especially useful with related database entities.

For example:

```
public class Customer
{
    public int Id { get; set; }

    public string Name { get; set; }

    public virtual ICollection<Order> Orders { get; set; }
}
```

When lazy loading is configured, this:

```
Customer customer = db.Customers
    .First(c => c.Id == 1);
```

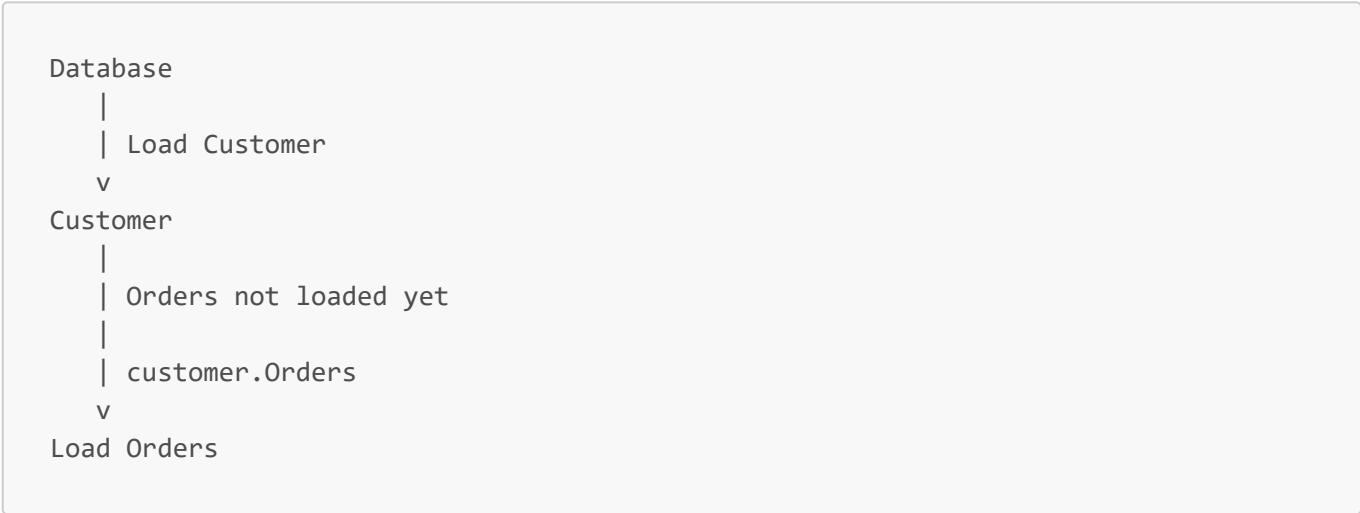
can initially load only the customer.

Then:

```
foreach (var order in customer.Orders)
{
    Console.WriteLine(order.Id);
}
```

can cause the orders to be loaded when the navigation property is accessed.

Conceptually:



Important

In modern ASP.NET Core applications, **explicit loading or eager loading is often preferred** when you want predictable database queries.

For example:

```
var customer = db.Customers
    .Include(c => c.Orders)
    .First(c => c.Id == 1);
```

This explicitly says:

Load Customer and Orders together.

6. Lazy Loading vs Eager Loading

Feature	Lazy Loading	Eager Loading
Loading	When needed	Immediately
Initial cost	Lower	Higher

Feature	Lazy Loading	Eager Loading
Memory usage	Usually lower initially	Usually higher
Database queries	Can happen later	Usually predictable
Risk	Unexpected queries	More data than needed
C# mechanism	<code>Lazy<T></code>	Normal initialization
EF Core	Navigation accessed later	<code>.Include()</code>

Lazy

```
var customer = GetCustomer();

var orders = customer.Orders;
```

Orders are obtained when needed.

Eager

```
var customer = db.Customers
    .Include(x => x.Orders)
    .First();
```

Customer and orders are loaded together.

7. Lazy Loading vs Lazy Initialization

These terms are closely related but can mean slightly different things.

Lazy initialization

Delay creation of an object:

```
private Lazy<ReportGenerator> _generator =
    new(() => new ReportGenerator());
```

Lazy loading

Delay retrieving data/resource:

```
public Orders Orders
{
    get
    {
        return LoadOrdersFromDatabase();
    }
}
```

So:

```
Lazy initialization
    ↓
Delay object creation

Lazy loading
    ↓
Delay data/resource loading
```

8. Lazy Loading a Database Repository

For example:

```
public class CustomerService
{
    private readonly Lazy<CustomerRepository> _repository;

    public CustomerService()
    {
        _repository = new Lazy<CustomerRepository>(
            () => new CustomerRepository());
    }

    public CustomerRepository Repository => _repository.Value;
}
```

The repository isn't created until:

```
service.Repository
```

is accessed.

This can be useful when the dependency is expensive to initialize.

9. Real-World Example

Imagine an e-commerce application:

```
Product
├── Basic information
├── Price
├── Images
├── Reviews
├── Specifications
└── Recommendations
```

Suppose the user only opens the product list.

You don't necessarily need:

```
Reviews
Images
Recommendations
Specifications
```

You can initially load:

```
Product
├── Id
├── Name
└── Price
```

and load expensive information only when the user opens the product details.

This is the basic idea behind **Lazy Load**.

10. Advantages

1. Better initial performance

Expensive resources aren't loaded unnecessarily.

2. Lower memory usage

Objects are created only when required.

3. Less unnecessary database work

You don't have to retrieve related data that isn't used.

4. Good for expensive objects

For example:

```
Large reports  
Database connections  
Large files  
Images  
Configuration  
External API data  
Complex calculations
```

11. Disadvantages

Lazy loading isn't always better.

1. Unexpected database queries

For example:

```
foreach (var customer in customers)  
{  
    Console.WriteLine(customer.Orders.Count);  
}
```

If **Orders** is lazy-loaded, this can potentially produce:

```
Query customers  
Query orders for customer 1  
Query orders for customer 2  
Query orders for customer 3  
...
```

This is the famous **N+1 query problem**.

2. Debugging can become harder

Accessing a property might unexpectedly perform I/O.

3. Performance becomes less predictable

You may think:

```
customer.Orders
```

is just property access, but it could actually execute a database query.

12. When Should You Use Lazy Load?

Use it when:

```
Object/resource is expensive
      +
It may not be needed
      +
Loading can safely be delayed
```

Good examples:

```
Large files
Expensive calculations
Large object graphs
Optional database relationships
Expensive services
Images
Reports
External API data
```

Avoid it when you need **predictable database access** or when lazy loading can easily create N+1 queries.

13. Interview Definition

For a C# interview, you can say:

Lazy Load is a design pattern where an object or resource is not loaded or initialized when the parent object is created. Instead, it is loaded only when it is actually accessed. In C#, `Lazy<T>` provides a convenient and thread-safe way to implement lazy initialization.

Key concept to remember

```
Normal Loading:

Create Object
  ↓
```

```
Load Everything
  ↓
Use Object

Lazy Loading:

Create Object
  ↓
Don't Load
  ↓
Need Resource?
  ↓
YES
  ↓
Load Resource
  ↓
Use Resource
```

Most important C# API:

```
Lazy<T>
```

and the key property:

```
.Value
```

So the pattern can be remembered as:

Lazy Load = Load expensive data only when it is actually needed.

10. Domain Model

Domain Model Design Pattern in C#

The **Domain Model** is a design pattern used in business applications where **business rules and behavior are placed inside domain objects**, rather than putting all logic in controllers, services, or database code.

It is commonly associated with **Domain-Driven Design (DDD)** and is especially useful for complex business systems such as banking, insurance, e-commerce, and fintech.

1. Basic idea

Instead of doing this:

```
public class OrderService
{
    public void CancelOrder(Order order)
    {
        if (order.Status == "Shipped")
            throw new Exception("Cannot cancel shipped order");

        order.Status = "Cancelled";
    }
}
```

The Domain Model puts the business rule inside `Order`:

```
public class Order
{
    public int Id { get; private set; }

    public OrderStatus Status { get; private set; }

    public void Cancel()
    {
        if (Status == OrderStatus.Shipped)
            throw new InvalidOperationException(
                "A shipped order cannot be cancelled.");

        Status = OrderStatus.Cancelled;
    }
}
```

Now the `Order` object **knows how an order can be cancelled**.

2. Real-world example

Suppose you are building a banking application.

A business rule says:

A customer cannot withdraw more money than their available balance.

A poor design might put this in a controller:

```
public IActionResult Withdraw(int accountId, decimal amount)
{
    var account = repository.GetAccount(accountId);

    if (account.Balance < amount)
        return BadRequest("Insufficient balance");
}
```

```
        account.Balance -= amount;

        repository.Save(account);

        return Ok();
    }
```

The controller now contains business logic.

With a Domain Model:

```
public class BankAccount
{
    public int Id { get; private set; }

    public decimal Balance { get; private set; }

    public void Deposit(decimal amount)
    {
        if (amount <= 0)
            throw new ArgumentException("Amount must be greater than zero.");

        Balance += amount;
    }

    public void Withdraw(decimal amount)
    {
        if (amount <= 0)
            throw new ArgumentException("Amount must be greater than zero.");

        if (amount > Balance)
            throw new InvalidOperationException("Insufficient balance.");

        Balance -= amount;
    }
}
```

Now:

```
account.Withdraw(5000);
```

The domain object itself enforces the business rules.

3. Typical Domain Model structure

A Domain Model can contain:

```
Domain
├── Entities
│   ├── Customer.cs
│   ├── Order.cs
│   └── BankAccount.cs
├── ValueObjects
│   ├── Money.cs
│   ├── Address.cs
│   └── Email.cs
├── Services
│   └── PaymentService.cs
├── Rules
│   └── WithdrawalRule.cs
└── Enums
    └── OrderStatus.cs
```

The important idea is that **business behavior lives close to the business data it operates on.**

4. Entity with behavior

Consider an e-commerce order.

```
public enum OrderStatus
{
    Pending,
    Confirmed,
    Shipped,
    Cancelled
}
```

```
public class Order
{
    private readonly List<OrderItem> _items = new();

    public int Id { get; private set; }

    public OrderStatus Status { get; private set; }

    public IReadOnlyCollection<OrderItem> Items => _items;
```

```
public decimal TotalAmount =>
    _items.Sum(x => x.Price * x.Quantity);

public void AddItem(Product product, int quantity)
{
    if (quantity <= 0)
        throw new ArgumentException("Quantity must be greater than zero.");

    _items.Add(new OrderItem(product.Id, product.Price, quantity));
}

public void Confirm()
{
    if (!_items.Any())
        throw new InvalidOperationException(
            "An empty order cannot be confirmed.");

    if (Status != OrderStatus.Pending)
        throw new InvalidOperationException(
            "Only pending orders can be confirmed.");

    Status = OrderStatus.Confirmed;
}

public void Cancel()
{
    if (Status == OrderStatus.Shipped)
        throw new InvalidOperationException(
            "Shipped order cannot be cancelled.");

    Status = OrderStatus.Cancelled;
}
}
```

Notice that the class contains:

- Data
- Calculations
- Validation
- Business rules
- Business operations

That's the essence of the **Domain Model**.

5. Domain Model vs Anemic Domain Model

This is an important interview concept.

Anemic Domain Model

The entity only contains properties:

```
public class BankAccount
{
    public int Id { get; set; }

    public decimal Balance { get; set; }
}
```

And business logic is somewhere else:

```
public class BankAccountService
{
    public void Withdraw(
        BankAccount account,
        decimal amount)
    {
        if (amount > account.Balance)
            throw new Exception("Insufficient balance");

        account.Balance -= amount;
    }
}
```

The entity has data but almost no behavior.

Rich Domain Model

```
public class BankAccount
{
    public decimal Balance { get; private set; }

    public void Withdraw(decimal amount)
    {
        if (amount > Balance)
            throw new InvalidOperationException(
                "Insufficient balance.");

        Balance -= amount;
    }
}
```

Now:

```
account.Withdraw(1000);
```

The entity controls its own state.

Comparison

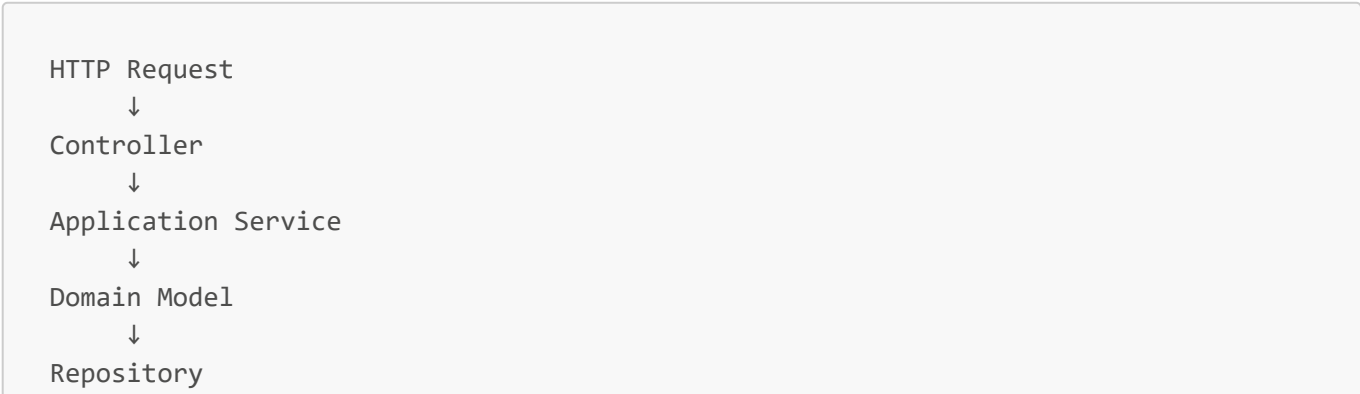
Anemic Model	Domain Model
Mostly properties	Properties + behavior
Logic in services	Business logic in domain
Easier initially	Better for complex business rules
Entities are passive	Entities are active
Can become service-heavy	Better encapsulation

6. Domain Model with ASP.NET Core

A typical application can look like:



The flow is:



↓
Database

For example:

```
public class OrderService
{
    private readonly IOrderRepository _repository;

    public OrderService(IOrderRepository repository)
    {
        _repository = repository;
    }

    public async Task CancelOrder(int orderId)
    {
        var order = await _repository.GetByIdAsync(orderId);

        if (order == null)
            throw new KeyNotFoundException("Order not found.");

        order.Cancel();

        await _repository.SaveAsync(order);
    }
}
```

The service coordinates the operation, while the domain object owns the business rule:

```
order.Cancel();
```

7. Domain Service

Sometimes a business rule doesn't naturally belong to one entity.

For example:

Transfer money between two bank accounts.

The operation involves **two accounts**, so a Domain Service can make sense.

```
public class MoneyTransferService
{
    public void Transfer(
        BankAccount from,
```

```
        BankAccount to,  
        decimal amount)  
    {  
        from.Withdraw(amount);  
        to.Deposit(amount);  
    }  
}
```

Then:

```
var transferService = new MoneyTransferService();  
  
transferService.Transfer(  
    sourceAccount,  
    destinationAccount,  
    5000);
```

The individual account rules remain inside `BankAccount`.

8. Value Objects

Domain Models often use **Value Objects**.

For example:

```
public record Money(decimal Amount, string Currency);
```

Then:

```
public class Product  
{  
    public int Id { get; private set; }  
  
    public Money Price { get; private set; }  
  
    public Product(Money price)  
    {  
        if (price.Amount < 0)  
            throw new ArgumentException("Price cannot be negative.");  
  
        Price = price;  
    }  
}
```

Another example:

```
public record Email
{
    public string Value { get; }

    public Email(string value)
    {
        if (string.IsNullOrEmpty(value))
            throw new ArgumentException("Email is required.");

        if (!value.Contains("@"))
            throw new ArgumentException("Invalid email.");

        Value = value;
    }

    public override string ToString() => Value;
}
```

Instead of:

```
customer.Email = "abc";
```

you can use:

```
customer.ChangeEmail(
    new Email("user@example.com"));
```

This makes invalid domain states harder to create.

9. Domain Model with Repository

The Repository handles persistence:

```
public interface IOrderRepository
{
    Task<Order?> GetByIdAsync(int id);

    Task SaveAsync(Order order);
}
```

The domain object doesn't need to know whether data is stored in:

```
SQL Server
  ↓
PostgreSQL
  ↓
MongoDB
  ↓
Redis
```

It simply represents the business concept.

10. Complete mini example

Domain

```
public enum AccountStatus
{
    Active,
    Frozen,
    Closed
}
```

```
public class BankAccount
{
    public int Id { get; private set; }

    public decimal Balance { get; private set; }

    public AccountStatus Status { get; private set; }

    public BankAccount(decimal initialBalance)
    {
        if (initialBalance < 0)
            throw new ArgumentException(
                "Initial balance cannot be negative.");

        Balance = initialBalance;
        Status = AccountStatus.Active;
    }

    public void Deposit(decimal amount)
    {
        EnsureActive();

        if (amount <= 0)
            throw new ArgumentException(
                "Deposit must be greater than zero.");
    }
}
```

```
        Balance += amount;
    }

    public void Withdraw(decimal amount)
    {
        EnsureActive();

        if (amount <= 0)
            throw new ArgumentException(
                "Withdrawal must be greater than zero.");

        if (amount > Balance)
            throw new InvalidOperationException(
                "Insufficient balance.");

        Balance -= amount;
    }

    public void Freeze()
    {
        if (Status == AccountStatus.Closed)
            throw new InvalidOperationException(
                "Closed account cannot be frozen.");

        Status = AccountStatus.Frozen;
    }

    private void EnsureActive()
    {
        if (Status != AccountStatus.Active)
            throw new InvalidOperationException(
                "Account is not active.");
    }
}
```

Usage:

```
var account = new BankAccount(10000);

account.Deposit(5000);

account.Withdraw(3000);

Console.WriteLine(account.Balance);
```

Output:

```
12000
```

The important point is that nobody can directly do:

```
account.Balance = -50000;
```

because:

```
public decimal Balance { get; private set; }
```

The domain object controls how its state changes.

11. When should you use Domain Model?

Use it when your application has **complex business rules**.

Good examples:

- Banking
- Fintech
- Insurance
- Trading
- E-commerce
- Order management
- Healthcare
- Billing
- Loan processing
- Rewards/fraud systems

For a simple CRUD application:

```
Create Customer  
Read Customer  
Update Customer  
Delete Customer
```

a rich Domain Model may be unnecessary.

But if you have rules like:

```
Customer can withdraw only within limits  
↓  
VIP customers have different limits  
↓
```



```
Fraud detection can block withdrawal
↓
Frozen accounts cannot transact
↓
Daily withdrawal limits apply
↓
Transaction fees depend on account type
```

then Domain Model becomes much more valuable.

12. Domain Model vs Service Layer

A useful way to remember the difference:

Domain Model

"What business rules does this object have?"

```
order.Cancel();
account.Withdraw(5000);
loan.Approve();
```

Application/Service Layer

"What steps should the application perform?"

```
var order = repository.GetById(id);

order.Cancel();

repository.Save(order);

notificationService.SendCancellationEmail(order);
```

So:

```
Controller
  ↓
Application Service
  ↓
Domain Model
  ↓
Repository
  ↓
Database
```

Interview definition

Domain Model is a design pattern in which business concepts, state, and business behavior are represented as objects in the domain layer. Instead of keeping business rules in controllers or procedural services, the domain entities and value objects encapsulate those rules and protect the consistency of the business state.

For your **ASP.NET Core/.NET backend interviews**, remember this key distinction:

Domain Model = business behavior + business state

Repository = persistence

Service Layer = application workflow/orchestration

Controller = HTTP/API handling

11. Transaction Script

Transaction Script Design Pattern in C#

The **Transaction Script** is an **Enterprise Application Design Pattern** where you organize business logic into **procedures/methods**, with each method handling **one complete business transaction**.

It is especially useful when the business logic is relatively simple and doesn't justify a complex domain model.

Basic idea

Instead of:

```
Controller
  ↓
Domain Objects
  ↓
Repositories
  ↓
Database
```

Transaction Script often looks like:

```
Controller
  ↓
Transaction Script / Service
  ↓
Database
```

Each method represents a business operation:

```
CreateOrder()  
CancelOrder()  
PayOrder()  
TransferMoney()  
RegisterCustomer()
```

1. Simple Example

Suppose we have a banking application.

A transaction is:

Transfer ₹1,000 from Account A to Account B.

The Transaction Script contains the entire business operation.

```
public class BankService  
{  
    private readonly BankDbContext _context;  
  
    public BankService(BankDbContext context)  
    {  
        _context = context;  
    }  
  
    public void TransferMoney(  
        int fromAccountId,  
        int toAccountId,  
        decimal amount)  
    {  
        var fromAccount = _context.Accounts  
            .FirstOrDefault(x => x.Id == fromAccountId);  
  
        var toAccount = _context.Accounts  
            .FirstOrDefault(x => x.Id == toAccountId);  
  
        if (fromAccount == null)  
            throw new Exception("Source account not found.");  
  
        if (toAccount == null)  
            throw new Exception("Destination account not found.");  
  
        if (amount <= 0)  
            throw new Exception("Invalid amount.");  
  
        if (fromAccount.Balance < amount)  
            throw new Exception("Insufficient balance.");  
    }  
}
```

```
        fromAccount.Balance -= amount;  
        toAccount.Balance += amount;  
  
        _context.SaveChanges();  
    }  
}
```

Here:

```
TransferMoney()
```

is the **Transaction Script**.

It contains the complete business workflow:

```
Find source account  
    ↓  
Find destination account  
    ↓  
Validate amount  
    ↓  
Check balance  
    ↓  
Debit source  
    ↓  
Credit destination  
    ↓  
Save transaction
```

2. ASP.NET Core Example

A more realistic ASP.NET Core application could look like:

```
MyApplication  
├── Controllers  
│   └── BankController.cs  
├── Services  
│   └── BankTransactionService.cs  
├── Models  
│   ├── Account.cs  
│   └── TransferRequest.cs
```

```
|  
└─ Data  
    └─ BankDbContext.cs
```

Model

```
public class Account  
{  
    public int Id { get; set; }  
  
    public string AccountNumber { get; set; } = "";  
  
    public decimal Balance { get; set; }  
}
```

Request DTO

```
public class TransferRequest  
{  
    public int FromAccountId { get; set; }  
  
    public int ToAccountId { get; set; }  
  
    public decimal Amount { get; set; }  
}
```

Transaction Script

```
public class BankTransactionService  
{  
    private readonly BankDbContext _context;  
  
    public BankTransactionService(BankDbContext context)  
    {  
        _context = context;  
    }  
  
    public async Task TransferAsync(TransferRequest request)  
    {  
        if (request.Amount <= 0)  
            throw new ArgumentException("Amount must be greater than zero.");  
  
        var source = await _context.Accounts  
            .FirstOrDefaultAsync(x => x.Id == request.FromAccountId);  
  
        var destination = await _context.Accounts
```

```
        .FirstOrDefaultAsync(x => x.Id == request.ToAccountId);

    if (source == null)
        throw new Exception("Source account not found.");

    if (destination == null)
        throw new Exception("Destination account not found.");

    if (source.Balance < request.Amount)
        throw new Exception("Insufficient balance.");

    source.Balance -= request.Amount;
    destination.Balance += request.Amount;

    await _context.SaveChangesAsync();
}
}
```

3. Controller

The controller simply calls the transaction script.

```
[ApiController]
[Route("api/bank")]
public class BankController : ControllerBase
{
    private readonly BankTransactionService _service;

    public BankController(BankTransactionService service)
    {
        _service = service;
    }

    [HttpPost("transfer")]
    public async Task<IActionResult> Transfer(
        TransferRequest request)
    {
        await _service.TransferAsync(request);

        return Ok(new
        {
            Message = "Money transferred successfully"
        });
    }
}
```

The controller doesn't contain the business workflow.

```
HTTP Request
  ↓
Controller
  ↓
TransferAsync()
  ↓
Validate
  ↓
Read Accounts
  ↓
Business Rules
  ↓
Update Accounts
  ↓
SaveChanges()
```

4. Multiple Transaction Scripts

You can have multiple business transactions:

```
public class OrderService
{
    public async Task CreateOrderAsync(...)
    {
        // Create order
    }

    public async Task CancelOrderAsync(...)
    {
        // Cancel order
    }

    public async Task PayOrderAsync(...)
    {
        // Pay order
    }

    public async Task ShipOrderAsync(...)
    {
        // Ship order
    }
}
```

Each method is a separate **Transaction Script**.

```
OrderService
|
|— CreateOrder()
|— CancelOrder()
|— PayOrder()
|— ShipOrder()
```

5. Real-World Example

Imagine an e-commerce application.

When a customer places an order:

```
PlaceOrder()
|
|— Validate customer
|
|— Get products
|
|— Check stock
|
|— Calculate total
|
|— Create order
|
|— Reduce inventory
|
|— Save order
|
|— Send confirmation
```

Transaction Script:

```
public async Task PlaceOrderAsync(
    int customerId,
    List<int> productIds)
{
    var customer = await GetCustomer(customerId);

    if (customer == null)
        throw new Exception("Customer not found.");

    var products = await GetProducts(productIds);

    foreach (var product in products)
    {
```



```

        if (product.Stock <= 0)
            throw new Exception(
                $"{product.Name} is out of stock.");
    }

    decimal total = products.Sum(x => x.Price);

    var order = new Order
    {
        CustomerId = customerId,
        TotalAmount = total,
        CreatedAt = DateTime.UtcNow
    };

    _context.Orders.Add(order);

    foreach (var product in products)
    {
        product.Stock--;
    }

    await _context.SaveChangesAsync();
}

```

The whole **Place Order** operation is implemented as one procedure.

6. Transaction Script vs Domain Model

This is an important distinction.

Transaction Script

Business logic is organized around **operations**.

```

CreateOrder()
CancelOrder()
PayOrder()
ShipOrder()

```

Domain Model

Business logic is organized around **objects/entities**.

```

Order
├─ AddItem()
├─ RemoveItem()
├─ CalculateTotal()

```

```
├─ Cancel()  
└─ Pay()
```

For example:

Transaction Script

```
public void CancelOrder(int orderId)  
{  
    var order = _context.Orders.Find(orderId);  
  
    if (order == null)  
        throw new Exception("Order not found.");  
  
    if (order.Status == "Shipped")  
        throw new Exception("Cannot cancel shipped order.");  
  
    order.Status = "Cancelled";  
  
    _context.SaveChanges();  
}
```

Domain Model

```
public class Order  
{  
    public OrderStatus Status { get; private set; }  
  
    public void Cancel()  
    {  
        if (Status == OrderStatus.Shipped)  
            throw new InvalidOperationException(  
                "Cannot cancel shipped order.");  
  
        Status = OrderStatus.Cancelled;  
    }  
}
```

The second approach puts the business rule **inside the domain object**.

7. When to Use Transaction Script

Transaction Script is a good choice when:

- Business logic is simple.

- CRUD operations dominate.
- Each operation has relatively independent rules.
- The application is small or medium-sized.
- You don't have complicated domain behavior.
- You want a simple implementation.

For example:

```
Customer Registration
Password Reset
Simple Payment
Simple CRUD
Report Generation
Simple Order Processing
```

8. Advantages

Simple

Easy to understand.

```
CreateCustomer()
UpdateCustomer()
DeleteCustomer()
```

Easy to implement

You don't need a complicated domain model.

Good for simple applications

Particularly useful for CRUD-heavy systems.

Easy to trace

You can follow one business transaction from beginning to end.

9. Disadvantages

The biggest problem is **duplicate business logic**.

For example:

```
CreateOrder()
```

might check:

```
if (customer.IsActive)
```

and later:

```
CancelOrder()
```

might also check:

```
if (customer.IsActive)
```

Eventually:

```
CreateOrder()
    ↓
Business Rule A

CancelOrder()
    ↓
Business Rule A

PayOrder()
    ↓
Business Rule A

ShipOrder()
    ↓
Business Rule A
```

The same business rules can become scattered across many methods.

This can lead to:

- duplication
- difficult maintenance
- large service classes
- difficult testing
- inconsistent business rules

10. Transaction Script vs Service Layer

They can look very similar in C#.

A **Transaction Script** focuses on implementing a **single business transaction**.

```
TransferMoney()
```

A **Service Layer** provides an application's boundary/API for business operations and may coordinate multiple domain objects, repositories, transactions, etc.

For example:

```
public class OrderService
{
    public Task CreateOrderAsync(...)
    public Task CancelOrderAsync(...)
    public Task PayOrderAsync(...)
}
```

The methods inside a service layer can themselves be implemented using Transaction Script.

So they aren't necessarily competing patterns.

11. Transaction Script + Repository + Unit of Work

In a modern ASP.NET Core application, you might combine them:

```
Controller
  ↓
Transaction Script / Service
  ↓
Repository
  ↓
Unit of Work
  ↓
EF Core
  ↓
Database
```

Example:

```
public async Task TransferAsync(
    int fromId,
    int toId,
    decimal amount)
{
    var source = await _accountRepository.GetByIdAsync(fromId);
    var destination = await _accountRepository.GetByIdAsync(toId);

    ValidateTransfer(source, destination, amount);

    source.Balance -= amount;
    destination.Balance += amount;

    await _unitOfWork.SaveChangesAsync();
}
```

Here:

- **Transaction Script** → `TransferAsync()`
- **Repository** → retrieves accounts
- **Unit of Work** → commits changes
- **EF Core** → persistence mechanism

12. Interview Definition

A good interview answer:

Transaction Script is an enterprise application pattern where business logic is organized into procedural methods, with each method handling a complete business transaction. It works well for simple business logic and CRUD-oriented applications, but can become difficult to maintain when the domain becomes complex because business rules may become duplicated across transaction scripts.

Remember it like this

```
Transaction Script
    ↓
One method = One business transaction
    ↓
Validate
    ↓
Execute business logic
    ↓
Persist changes
```

Example:

```
TransferMoney()  
PlaceOrder()  
CancelOrder()  
PayOrder()  
RegisterCustomer()
```

These are all natural candidates for Transaction Scripts.

Dependency & Infrastructure

12. **Dependency Injection**

Dependency Injection (DI) Design Pattern in C#

Dependency Injection (DI) is a design technique used to provide an object with the dependencies it needs **from outside**, instead of creating those dependencies inside the class.

It is one of the most important patterns in **ASP.NET Core**.

1. Problem Without Dependency Injection

Suppose we have an `OrderService` that needs an email service.

```
public class EmailService  
{  
    public void SendEmail(string message)  
    {  
        Console.WriteLine($"Email sent: {message}");  
    }  
}  
  
public class OrderService  
{  
    private EmailService _emailService;  
  
    public OrderService()  
    {  
        _emailService = new EmailService();  
    }  
  
    public void PlaceOrder()  
    {  
        // Place order logic  
  
        _emailService.SendEmail("Order placed successfully");  
    }  
}
```

Problem

`OrderService` directly creates:

```
_emailService = new EmailService();
```

Therefore `OrderService` is tightly coupled to `EmailService`.

If tomorrow you want:

```
EmailService  
  ↓  
SmsService
```

you need to modify `OrderService`.

2. Dependency Injection Solution

Instead of creating the dependency inside the class, pass it through the constructor.

First create an interface:

```
public interface INotificationService  
{  
    void Send(string message);  
}
```

Implement it:

```
public class EmailService : INotificationService  
{  
    public void Send(string message)  
    {  
        Console.WriteLine($"Email: {message}");  
    }  
}
```

Now inject it:

```
public class OrderService  
{  
    private readonly INotificationService _notificationService;
```



```
public OrderService(INotificationService notificationService)
{
    _notificationService = notificationService;
}

public void PlaceOrder()
{
    Console.WriteLine("Order placed");

    _notificationService.Send("Order placed successfully");
}
}
```

Now `OrderService` doesn't know whether the notification is sent through:

```
Email
SMS
WhatsApp
Push Notification
```

It only knows about:

```
INotificationService
```

3. Dependency Inversion

DI is closely related to the **Dependency Inversion Principle (DIP)**.

Instead of:

```
OrderService
  ↓
EmailService
```

we use:

```
OrderService
  ↓
INotificationService
  ↑
EmailService
```

The high-level class depends on an abstraction rather than a concrete implementation.

4. Types of Dependency Injection

There are three common types.

1. Constructor Injection

Most recommended.

```
public class OrderService
{
    private readonly INotificationService _notificationService;

    public OrderService(INotificationService notificationService)
    {
        _notificationService = notificationService;
    }
}
```

2. Property Injection

Dependency is supplied through a property.

```
public class OrderService
{
    public INotificationService NotificationService { get; set; }

    public void PlaceOrder()
    {
        NotificationService.Send("Order placed");
    }
}
```

This is less safe because the dependency can be missing.

3. Method Injection

Dependency is passed to a particular method.

```
public class OrderService
{
    public void PlaceOrder(INotificationService notificationService)
    {
        notificationService.Send("Order placed");
    }
}
```

```
}  
}
```

5. Dependency Injection in ASP.NET Core

ASP.NET Core has a built-in DI container.

Suppose we have:

```
public interface IProductService  
{  
    string GetProduct();  
}
```

Implementation:

```
public class ProductService : IProductService  
{  
    public string GetProduct()  
    {  
        return "Laptop";  
    }  
}
```

Register it in `Program.cs`:

```
var builder = WebApplication.CreateBuilder(args);  
  
builder.Services.AddScoped<IProductService, ProductService>();  
  
var app = builder.Build();  
  
app.Run();
```

Now inject it into a controller:

```
[ApiController]  
[Route("api/[controller]")]  
public class ProductController : ControllerBase  
{  
    private readonly IProductService _productService;  
  
    public ProductController(IProductService productService)
```

```
{
    _productService = productService;
}

[HttpGet]
public IActionResult Get()
{
    var product = _productService.GetProduct();

    return Ok(product);
}
```

ASP.NET Core automatically creates `ProductService` and provides it to `ProductController`.

6. DI Lifetimes in ASP.NET Core

There are three important lifetimes.

Transient

```
builder.Services.AddTransient<IProductService, ProductService>();
```

A new instance is created every time it is requested.

```
Request
├── ProductService → Instance A
├── ProductService → Instance B
└── ProductService → Instance C
```

Use for lightweight, stateless services.

Scoped

```
builder.Services.AddScoped<IProductService, ProductService>();
```

One instance is created per scope.

In a typical ASP.NET Core application:

```

HTTP Request
|
+----- ProductService
|
+----- OrderService
|
+----- Other Services

```

Services resolved within that request can share the same scoped instance.

`DbContext` is commonly registered as scoped.

Singleton

```
builder.Services.AddSingleton<IProductService, ProductService>();
```

Only one instance is created for the application's lifetime.

```

Application
|
+----- ProductService
|
+----- Same instance

```

Use carefully for shared, thread-safe, application-wide state/services.

7. Real-World ASP.NET Core Example

A common architecture is:

```

Controller
↓
Service
↓
Repository
↓
Database

```

For example:

```
public interface IProductRepository
{
    Product GetById(int id);
}
```

```
public class ProductRepository : IProductRepository
{
    public Product GetById(int id)
    {
        return new Product
        {
            Id = id,
            Name = "Laptop"
        };
    }
}
```

Service:

```
public interface IProductService
{
    Product GetProduct(int id);
}
```

```
public class ProductService : IProductService
{
    private readonly IProductRepository _repository;

    public ProductService(IProductRepository repository)
    {
        _repository = repository;
    }

    public Product GetProduct(int id)
    {
        return _repository.GetById(id);
    }
}
```

Controller:

```
[ApiController]
[Route("api/products")]
public class ProductController : ControllerBase
```

```
{  
    private readonly IProductService _productService;  
  
    public ProductController(IProductService productService)  
    {  
        _productService = productService;  
    }  
  
    [HttpGet("{id}")]  
    public IActionResult Get(int id)  
    {  
        return Ok(_productService.GetProduct(id));  
    }  
}
```

Register everything:

```
builder.Services.AddScoped<IProductRepository, ProductRepository>();  
builder.Services.AddScoped<IProductService, ProductService>();
```

The dependency graph becomes:

```
ProductController  
    ↓  
IProductService  
    ↓  
IProductRepository  
    ↓  
Database
```

8. DI Makes Unit Testing Easier

Without DI:

```
public class OrderService  
{  
    private EmailService _email = new EmailService();  
}
```

Testing becomes difficult because the class creates its own dependency.

With DI:

```
public class OrderService
{
    private readonly INotificationService _notification;

    public OrderService(INotificationService notification)
    {
        _notification = notification;
    }
}
```

You can inject a mock:

```
var mockNotification = new Mock<INotificationService>();

var service = new OrderService(mockNotification.Object);

service.PlaceOrder();

mockNotification.Verify(
    x => x.Send("Order placed successfully"),
    Times.Once);
```

This is one of the biggest advantages of DI.

9. DI vs Dependency Inversion Principle

These are related but not exactly the same.

Concept	Meaning
Dependency Injection	Technique for supplying dependencies
Dependency Inversion Principle	High-level modules should depend on abstractions
IoC	Control of object creation is transferred to an external mechanism
DI Container	Framework/component that creates and supplies dependencies

Think of it as:




```
DI Container
↓
Automates object creation and wiring
```

10. Advantages

Loose coupling

```
OrderService → Interface
```

instead of:

```
OrderService → Concrete class
```

Better testing

Mocks/fakes can easily be injected.

Maintainability

Implementations can be changed without modifying the consuming class.

Flexibility

For example:

```
INotificationService
```

can have:

```
EmailNotificationService
SmsNotificationService
PushNotificationService
```

Supports SOLID

Especially:

- **S** — Single Responsibility
- **D** — Dependency Inversion

11. Common Interview Question

Why use Dependency Injection?

A good interview answer:

Dependency Injection is a technique where a class receives its dependencies from an external source instead of creating them internally. It reduces tight coupling, improves testability and maintainability, and allows implementations to be replaced easily. ASP.NET Core provides a built-in DI container that supports Transient, Scoped, and Singleton lifetimes.

Simple Rule to Remember

Bad:

```
class OrderService
{
    EmailService email = new EmailService();
}
```

Good:

```
class OrderService
{
    private readonly IEmailService email;

    public OrderService(IEmailService email)
    {
        this.email = email;
    }
}
```

In ASP.NET Core, prefer constructor injection and register dependencies in **Program.cs**.

13. Dependency Inversion

Dependency Inversion Principle (DIP) in C#

Dependency Inversion is the **D** in **SOLID**.

It says:

High-level modules should not depend on low-level modules. Both should depend on abstractions.

And:

Abstractions should not depend on details. Details should depend on abstractions.

1. Problem without Dependency Inversion

Suppose an `OrderService` directly depends on `SqlOrderRepository`:

```
public class SqlOrderRepository
{
    public void SaveOrder(string order)
    {
        Console.WriteLine("Saving order to SQL Server");
    }
}

public class OrderService
{
    private SqlOrderRepository _repository;

    public OrderService()
    {
        _repository = new SqlOrderRepository();
    }

    public void PlaceOrder(string order)
    {
        _repository.SaveOrder(order);
    }
}
```

Here:

```
OrderService
  |
  v
SqlOrderRepository
```

`OrderService` is tightly coupled to SQL Server.

If tomorrow you want:

- MongoDB
- PostgreSQL
- Web API
- In-memory storage

you need to modify `OrderService`.

2. Apply Dependency Inversion

Create an abstraction:

```
public interface IOrderRepository
{
    void SaveOrder(string order);
}
```

The low-level implementation implements the abstraction:

```
public class SqlOrderRepository : IOrderRepository
{
    public void SaveOrder(string order)
    {
        Console.WriteLine("Saving order to SQL Server");
    }
}
```

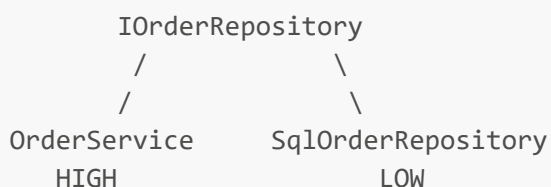
Now `OrderService` depends on the interface:

```
public class OrderService
{
    private readonly IOrderRepository _repository;

    public OrderService(IOrderRepository repository)
    {
        _repository = repository;
    }

    public void PlaceOrder(string order)
    {
        _repository.SaveOrder(order);
    }
}
```

Now the dependency looks like:



Both depend on the abstraction.

3. Using Dependency Injection

We can inject the implementation:

```
IOrderRepository repository = new SqlOrderRepository();

OrderService service = new OrderService(repository);

service.PlaceOrder("Order-101");
```

The important point is that `OrderService` doesn't know or care that SQL Server is being used.

4. Add another implementation

Suppose we want MongoDB:

```
public class MongoOrderRepository : IOrderRepository
{
    public void SaveOrder(string order)
    {
        Console.WriteLine("Saving order to MongoDB");
    }
}
```

We don't need to modify `OrderService`.

```
IOrderRepository repository =
    new MongoOrderRepository();

OrderService service =
    new OrderService(repository);

service.PlaceOrder("Order-101");
```

This is the major benefit of DIP.

5. DIP with ASP.NET Core

This is where DIP becomes very useful.

Interface

```
public interface IOrderRepository
{
    Task SaveAsync(Order order);
}
```

Implementation

```
public class SqlOrderRepository : IOrderRepository
{
    public async Task SaveAsync(Order order)
    {
        // Save to database
    }
}
```

Service

```
public class OrderService
{
    private readonly IOrderRepository _repository;

    public OrderService(IOrderRepository repository)
    {
        _repository = repository;
    }

    public async Task CreateOrder(Order order)
    {
        // Business logic

        await _repository.SaveAsync(order);
    }
}
```

Register in Program.cs

```
builder.Services.AddScoped<IOrderRepository, SqlOrderRepository>();
builder.Services.AddScoped<OrderService>();
```

ASP.NET Core's DI container now creates:

```
OrderController
  |
  v
OrderService
  |
  v
IOrderRepository
  |
  v
SqlOrderRepository
```

6. DIP vs Dependency Injection

These two are related but **not the same**.

Dependency Inversion Principle

A **design principle**:

```
High-level code
  |
  v
Abstraction
  ^
  |
Low-level implementation
```

Dependency Injection

A **technique used to implement DIP**.

For example:

```
public OrderService(IOrderRepository repository)
{
    _repository = repository;
}
```

The dependency is injected from outside.

So:

```
DIP = Principle
DI  = Technique
```

7. Bad vs Good Design

✗ Without DIP

```
public class PaymentService
{
    private StripePayment _payment = new StripePayment();

    public void Pay(decimal amount)
    {
        _payment.Process(amount);
    }
}
```

`PaymentService` is tightly coupled to Stripe.

☑ With DIP

```
public interface IPaymentGateway
{
    void Process(decimal amount);
}
```

```
public class StripePayment : IPaymentGateway
{
    public void Process(decimal amount)
    {
        Console.WriteLine("Stripe payment");
    }
}
```

```
public class RazorpayPayment : IPaymentGateway
{
    public void Process(decimal amount)
    {
        Console.WriteLine("Razorpay payment");
    }
}
```



```
public class PaymentService
{
    private readonly IPaymentGateway _paymentGateway;

    public PaymentService(IPaymentGateway paymentGateway)
    {
        _paymentGateway = paymentGateway;
    }

    public void Pay(decimal amount)
    {
        _paymentGateway.Process(amount);
    }
}
```

Now:



You can change the payment provider without changing `PaymentService`.

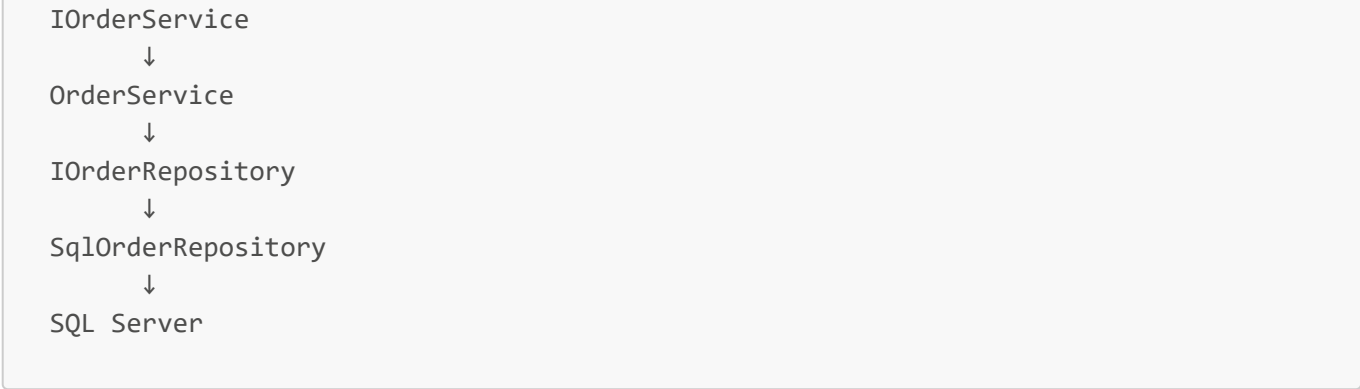
8. Real-world ASP.NET Core example

A common architecture is:

```
Controller
  ↓
Service
  ↓
Interface
  ↓
Repository
  ↓
Database
```

For example:

```
OrderController
  ↓
```



Each layer can depend on abstractions rather than concrete implementations.

9. Benefits of DIP

Benefit	Explanation
Loose coupling	Classes don't depend directly on concrete classes
Testability	Easy to replace dependencies with mocks
Maintainability	Implementation can change independently
Flexibility	SQL → MongoDB, Stripe → Razorpay, etc.
Extensibility	New implementations can be added easily
SOLID compliance	Helps achieve clean architecture

Interview answer

If an interviewer asks **"What is Dependency Inversion Principle?"**, you can say:

Dependency Inversion Principle states that high-level modules should not depend directly on low-level modules. Both should depend on abstractions. In C#, we normally achieve this using interfaces and dependency injection. For example, a service should depend on `IRepository` rather than directly on `SqlRepository`, which makes the application loosely coupled, testable, and easier to maintain.

Important: DIP is a **SOLID principle**, not technically a GoF design pattern.

14. Service Locator

Service Locator Design Pattern in C#

The **Service Locator** pattern provides a central object that knows how to locate and return services/dependencies when they are needed.

It is essentially a **registry of services**.

1. Basic idea

Instead of a class receiving its dependency through the constructor:

```
OrderService
  ↓
Service Locator
  ↓
IPaymentService
```

The class asks the Service Locator for the dependency.

Simple C# Example

Step 1: Create an interface

```
public interface IPaymentService
{
    void Pay(decimal amount);
}
```

Step 2: Implement the service

```
public class PaymentService : IPaymentService
{
    public void Pay(decimal amount)
    {
        Console.WriteLine($"Payment of {amount} processed.");
    }
}
```

Step 3: Create Service Locator

```
public static class ServiceLocator
{
    private static readonly Dictionary<Type, object> _services = new();

    public static void Register<T>(T service)
    {
        _services[typeof(T)] = service!;
    }

    public static T Get<T>()
    {
        if (_services.TryGetValue(typeof(T), out var service))
        {

```

```
        return (T)service;
    }

    throw new InvalidOperationException(
        $"Service {typeof(T).Name} is not registered.");
}
}
```

Step 4: Register the service

```
ServiceLocator.Register<IPaymentService>(
    new PaymentService()
);
```

Step 5: Retrieve the service

```
var paymentService =
    ServiceLocator.Get<IPaymentService>();

paymentService.Pay(1000);
```

Output:

```
Payment of 1000 processed.
```

Using Service Locator inside another class

```
public class OrderService
{
    public void CreateOrder(decimal amount)
    {
        var paymentService =
            ServiceLocator.Get<IPaymentService>();

        paymentService.Pay(amount);

        Console.WriteLine("Order created.");
    }
}
```

Usage:

```
var orderService = new OrderService();  
  
orderService.CreateOrder(500);
```

The dependency is hidden inside `OrderService`.

Service Locator vs Dependency Injection

This distinction is **very important in ASP.NET Core interviews**.

Service Locator

```
public class OrderService  
{  
    public void CreateOrder()  
    {  
        var payment =  
            ServiceLocator.Get<IPaymentService>();  
  
        payment.Pay(1000);  
    }  
}
```

The class **looks like it has no dependencies**, but internally it depends on `IPaymentService`.

Dependency Injection

```
public class OrderService  
{  
    private readonly IPaymentService _paymentService;  
  
    public OrderService(IPaymentService paymentService)  
    {  
        _paymentService = paymentService;  
    }  
  
    public void CreateOrder()  
    {  
        _paymentService.Pay(1000);  
    }  
}
```

Now the dependency is explicit.

```
Service Locator

OrderService
  |
  | secretly asks
  ↓
ServiceLocator
  ↓
IPaymentService
```

```
Dependency Injection

OrderService
  ↑
  |
IPaymentService
```

Service Locator in ASP.NET Core

ASP.NET Core already has a built-in DI container:

```
builder.Services.AddScoped<IPaymentService, PaymentService>();
```

Normally, you should use constructor injection:

```
public class OrderService
{
    private readonly IPaymentService _paymentService;

    public OrderService(IPaymentService paymentService)
    {
        _paymentService = paymentService;
    }
}
```

However, you can manually access the container:

```
var service =
    serviceProvider.GetRequiredService<IPaymentService>();
```

For example:

```
public class OrderService
{
    private readonly IServiceProvider _serviceProvider;

    public OrderService(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
    }

    public void CreateOrder()
    {
        var payment =
            _serviceProvider.GetRequiredService<IPaymentService>();

        payment.Pay(1000);
    }
}
```

This is effectively using the **Service Locator approach**.

Advantages

1. Centralized service lookup

Services can be registered and retrieved from one place.

2. Loose coupling to concrete implementations

The caller can request:

IPaymentService

rather than:

PaymentService

3. Useful for legacy systems

It can be useful when modifying an old application where introducing constructor injection everywhere would be difficult.

4. Dynamic service resolution

Sometimes the service type is determined at runtime.

Disadvantages

The biggest problem is **hidden dependencies**.

Consider:

```
public class OrderService
{
    public void CreateOrder()
    {
        var payment =
            ServiceLocator.Get<IPaymentService>();

        var logger =
            ServiceLocator.Get<ILogger>();

        var repository =
            ServiceLocator.Get<IOrderRepository>();
    }
}
```

From the constructor, you might think:

```
new OrderService();
```

requires nothing.

But internally it actually requires:

```
OrderService
├── IPaymentService
├── ILogger
└── IOrderRepository
```

This makes the code harder to understand and test.

Other problems

- Hidden dependencies
- Harder unit testing
- Runtime errors if a service isn't registered
- Dependency graph becomes difficult to understand
- Can turn into a global service registry
- Often considered an **anti-pattern** when used instead of normal DI

Service Locator vs Factory

They can look similar but solve different problems.

Factory

A Factory is responsible for **creating/selecting objects**.

```
var payment = PaymentFactory.Create("CreditCard");
```

Service Locator

A Service Locator is responsible for **finding an already registered service**.

```
var payment =  
    ServiceLocator.Get<IPaymentService>();
```

Think:

```
Factory  
↓  
"Which object should I create?"  
  
Service Locator  
↓  
"Where can I get this service?"
```

When should you use it?

In modern C# / ASP.NET Core applications:

Prefer Dependency Injection.

Use Service Locator only when there is a specific reason, such as:

- Legacy code
- Dynamic runtime service resolution
- Framework/infrastructure code
- Situations where constructor injection genuinely cannot be used

For normal application services, prefer:

```
public OrderService(  
    IPaymentService paymentService,  
    IOrderRepository repository)  
{  
    ...  
}
```

rather than:

```
public OrderService()  
{  
    var payment =  
        ServiceLocator.Get<IPaymentService>();  
}
```

Interview definition

Service Locator is a design pattern that provides a centralized mechanism for locating and retrieving dependencies at runtime. Although it can reduce explicit wiring, it hides dependencies and is generally discouraged in modern C# applications in favor of Dependency Injection.

15. Composition Root

Composition Root Design Pattern in C#

Composition Root is a design principle/pattern where **all object creation and dependency wiring is centralized in one place**, usually at the **entry point of the application**.

It is closely related to **Dependency Injection (DI)**.

Simple idea: Business classes should not decide *what dependencies to create*. The Composition Root decides that.

1. Problem without a Composition Root

Suppose we have:

```
public class EmailService  
{  
    public void Send(string message)  
    {  
        Console.WriteLine($"Email: {message}");  
    }  
}
```

And:

```
public class OrderService
{
    private readonly EmailService _emailService;

    public OrderService()
    {
        _emailService = new EmailService();
    }

    public void PlaceOrder()
    {
        Console.WriteLine("Order placed");

        _emailService.Send("Order confirmation");
    }
}
```

Here `OrderService` is responsible for creating `EmailService`.

```
OrderService
|
└─ creates EmailService
```

This creates **tight coupling**.

If tomorrow you want:

```
EmailService
  ↓
SmsService
```

you have to modify `OrderService`.

2. Using Dependency Injection

First create an abstraction:

```
public interface INotificationService
{
    void Send(string message);
}
```

Implementation:

```
public class EmailNotificationService : INotificationService
{
    public void Send(string message)
    {
        Console.WriteLine($"Email: {message}");
    }
}
```

Now `OrderService` depends only on the abstraction:

```
public class OrderService
{
    private readonly INotificationService _notificationService;

    public OrderService(INotificationService notificationService)
    {
        _notificationService = notificationService;
    }

    public void PlaceOrder()
    {
        Console.WriteLine("Order placed");

        _notificationService.Send("Order confirmation");
    }
}
```

Notice something important:

OrderService

doesn't know whether the implementation is:

EmailNotificationService
SmsNotificationService
PushNotificationService

It only knows:

INotificationService

3. Where does the dependency get created?

This is where the **Composition Root** comes in.

For a console application:

```
class Program
{
    static void Main()
    {
        INotificationService notificationService =
            new EmailNotificationService();

        OrderService orderService =
            new OrderService(notificationService);

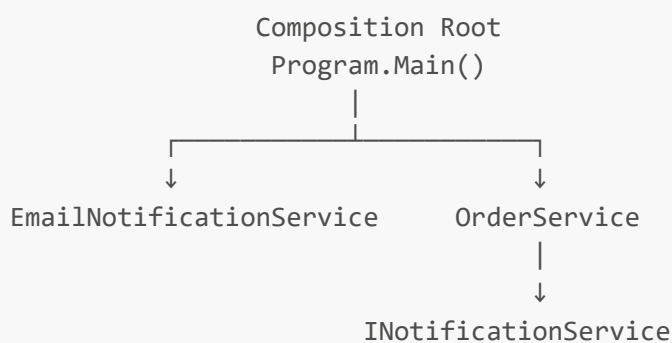
        orderService.PlaceOrder();
    }
}
```

`Program.Main()` is the **Composition Root**.

It is responsible for:

```
Create objects
    ↓
Connect dependencies
    ↓
Start application
```

So:



4. Composition Root in ASP.NET Core

In modern ASP.NET Core, `Program.cs` is typically the Composition Root.

For example:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddScoped<IOrderService, OrderService>();
builder.Services.AddScoped<INotificationService, EmailNotificationService>();

var app = builder.Build();

app.MapControllers();

app.Run();
```

The important part is:

```
builder.Services.AddScoped<IOrderService, OrderService>();

builder.Services.AddScoped<
    INotificationService,
    EmailNotificationService>();
```

The application is saying:

Whenever someone asks for `IOrderService`, create `OrderService`.

And:

Whenever someone asks for `INotificationService`, create `EmailNotificationService`.

ASP.NET Core's DI container then builds the object graph.

5. Complete ASP.NET Core example

Interface

```
public interface INotificationService
{
    Task SendAsync(string message);
}
```

Implementation

```
public class EmailNotificationService : INotificationService
{
    public Task SendAsync(string message)
    {
        Console.WriteLine($"Email sent: {message}");

        return Task.CompletedTask;
    }
}
```

Order service

```
public interface IOrderService
{
    Task PlaceOrderAsync();
}
```

```
public class OrderService : IOrderService
{
    private readonly INotificationService _notificationService;

    public OrderService(
        INotificationService notificationService)
    {
        _notificationService = notificationService;
    }

    public async Task PlaceOrderAsync()
    {
        Console.WriteLine("Order placed");

        await _notificationService
            .SendAsync("Order confirmation");
    }
}
```

Controller

```
[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly IOrderService _orderService;

    public OrdersController(IOrderService orderService)
    {
    }
```

```
        _orderService = orderService;
    }

    [HttpPost]
    public async Task<IActionResult> Create()
    {
        await _orderService.PlaceOrderAsync();

        return Ok();
    }
}
```

Composition Root — Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

builder.Services.AddScoped<IOrderService, OrderService>();

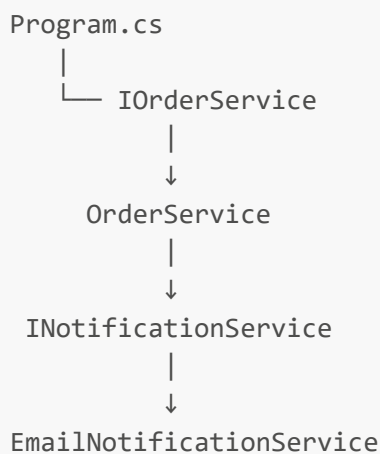
builder.Services.AddScoped<
    INotificationService,
    EmailNotificationService>();

var app = builder.Build();

app.MapControllers();

app.Run();
```

The dependency graph becomes:



6. Why is this useful?

Without Composition Root

You may see:

```
public class OrderService
{
    public OrderService()
    {
        var email = new EmailService();
        var logger = new FileLogger();
        var repository = new SqlOrderRepository();
    }
}
```

Now your business class controls object creation.

This makes testing and changing implementations harder.

With Composition Root

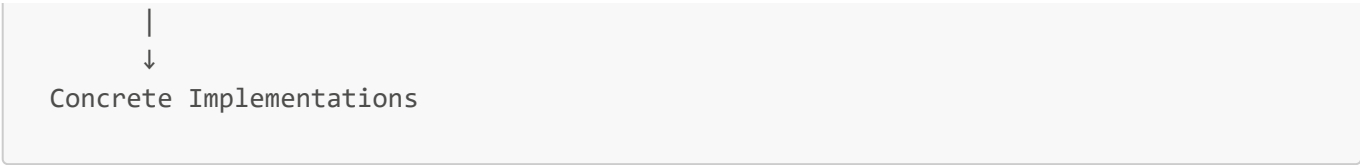
```
public class OrderService
{
    public OrderService(
        IEmailService email,
        ILogger logger,
        IOrderRepository repository)
    {
        ...
    }
}
```

And `Program.cs` handles the wiring:

```
builder.Services.AddScoped<IEmailService, EmailService>();
builder.Services.AddScoped<ILogger, FileLogger>();
builder.Services.AddScoped<IOrderRepository, SqlOrderRepository>();
```

This gives you:

```
Business Logic
  |
  ↓
Abstractions
  |
  ↑
Composition Root
```



7. Composition Root vs Dependency Injection

These are related but **not the same thing**.

Concept	Purpose
Dependency Injection	Supplies dependencies to objects
Composition Root	Defines where and how dependencies are wired
DI Container	Automates creation and resolution of dependencies

For example:

```
builder.Services.AddScoped<IOrderService, OrderService>();
```

This is **DI configuration**.

The location:

```
Program.cs
```

is the **Composition Root**.

8. Composition Root vs Service Locator

This distinction is important.

Composition Root

```
// Program.cs
var payment = new PaymentService();
var order = new OrderService(payment);
```

Dependencies are explicitly constructed and passed in.

Service Locator

```
public class OrderService
{
    public void PlaceOrder()
    {
        var payment =
            ServiceLocator.Get<IPaymentService>();
    }
}
```

The class asks a global service locator for its dependency.

That's generally undesirable because the dependency is **hidden**.

With Composition Root + DI:

```
public OrderService(IPaymentService payment)
{
    _payment = payment;
}
```

The dependency is obvious.

9. Testing becomes easier

You can create a fake implementation:

```
public class FakeNotificationService : INotificationService
{
    public Task SendAsync(string message)
    {
        Console.WriteLine("Fake notification");

        return Task.CompletedTask;
    }
}
```

Then:

```
var notification =
    new FakeNotificationService();

var orderService =
    new OrderService(notification);
```

```
await orderService.PlaceOrderAsync();
```

No real email system is required.

10. Real-world ASP.NET Core structure

A clean application might look like:

```
MyApplication
├── Controllers
│   └── OrdersController.cs
├── Services
│   ├── IOrderService.cs
│   └── OrderService.cs
├── Repositories
│   ├── IOrderRepository.cs
│   └── OrderRepository.cs
├── Infrastructure
│   ├── EmailService.cs
│   └── SqlConnectionFactory.cs
└── Program.cs      ← Composition Root
```

Program.cs:

```
builder.Services.AddScoped<IOrderService, OrderService>();
builder.Services.AddScoped<IOrderRepository, OrderRepository>();
builder.Services.AddScoped<IEmailService, EmailService>();
```

The rest of the application should generally **not be responsible for assembling the object graph**.

11. Key principle

A very useful rule to remember:

"Application composition should happen as close as possible to the application's entry point."

In ASP.NET Core:

```
Program.cs
  ↓
Composition Root
  ↓
DI Container
  ↓
Controllers
  ↓
Services
  ↓
Repositories
  ↓
Database
```

Interview answer

If an interviewer asks "**What is a Composition Root?**", you can say:

Composition Root is the single location in an application where concrete implementations are created and dependencies are wired together. In ASP.NET Core, `Program.cs` commonly acts as the Composition Root by configuring the built-in DI container. Business classes receive dependencies through constructors instead of creating them themselves. This reduces coupling and makes the application easier to test and maintain.

Relationship to the patterns you've been studying:

```
Composition Root
  ↓
Dependency Injection
  ↓
Dependency Inversion
  ↓
Loose Coupling
  ↓
Testable / Maintainable Code
```

So, **Composition Root is not usually a GoF pattern** like Factory, Strategy, or Observer. It is primarily an architectural/DI principle for **centralizing dependency composition**.

6. ASP.NET Core Common Patterns

For an ASP.NET Core developer, learn these particularly well:

1. **Dependency Injection**
2. **Repository**
3. **Unit of Work**

4. **Factory**
5. **Strategy**
6. **Decorator**
7. **Adapter**
8. **Facade**
9. **Mediator**
10. **CQRS**

CQRS (Command Query Responsibility Segregation) in C#

CQRS stands for **Command Query Responsibility Segregation**.

The core idea is simple:

Separate operations that change data (Commands) from operations that read data (Queries).

Instead of one service handling both reads and writes, CQRS creates separate models/handlers for each responsibility.

1. Without CQRS

A traditional service might look like this:

```
public class OrderService
{
    public void CreateOrder(Order order)
    {
        // Insert order
    }

    public Order GetOrder(int id)
    {
        // Get order
        return new Order();
    }

    public List<Order> GetOrders()
    {
        // Get all orders
        return new List<Order>();
    }

    public void CancelOrder(int id)
    {
        // Update order
    }
}
```

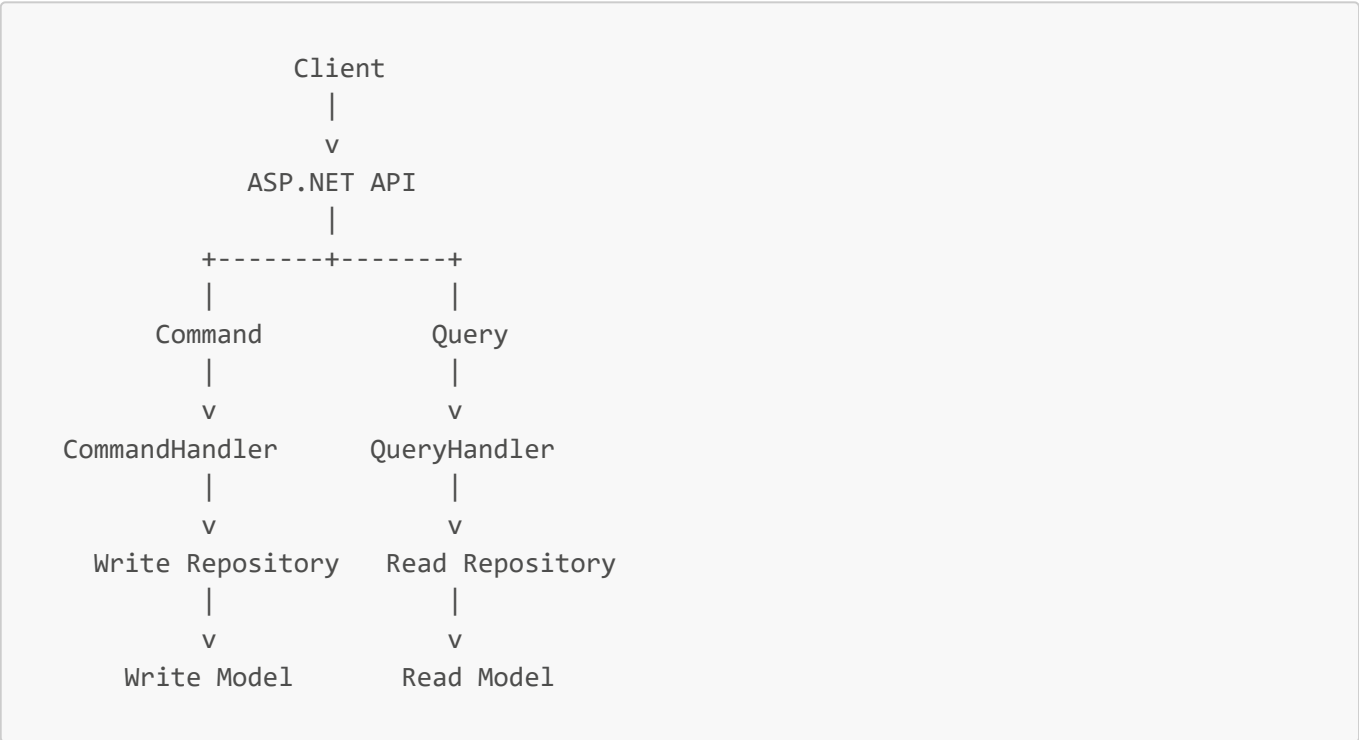
The same service handles:

- Creating data
- Updating data
- Deleting data
- Reading data

With CQRS, we separate these responsibilities.

2. CQRS Architecture

A simple CQRS architecture looks like:



Command

A **Command** says:

"I want to change something."

Examples:

```
CreateOrder
UpdateOrder
CancelOrder
DeleteOrder
ApprovePayment
RegisterCustomer
```

Query

A **Query** says:

```
"I want to retrieve something."
```

Examples:

```
GetOrderById  
GetCustomer  
GetOrders  
GetTransactionHistory  
GetPaymentDetails
```

3. Command Example

Suppose we want to create an order.

Command

```
public record CreateOrderCommand(  
    int CustomerId,  
    decimal Amount  
);
```

The command contains the information required to perform the operation.

It does **not** perform the operation itself.

Command Handler

```
public class CreateOrderCommandHandler  
{  
    private readonly IOrderRepository _repository;  
  
    public CreateOrderCommandHandler(IOrderRepository repository)  
    {  
        _repository = repository;  
    }  
  
    public async Task<int> Handle(CreateOrderCommand command)  
    {  
        var order = new Order  
        {  
            CustomerId = command.CustomerId,  

```



```
        Amount = command.Amount,  
        CreatedAt = DateTime.UtcNow  
    };  
  
    await _repository.AddAsync(order);  
  
    return order.Id;  
}  
}
```

The handler contains the logic for executing the command.

4. Query Example

Now suppose we want to retrieve an order.

Query

```
public record GetOrderQuery(int OrderId);
```

Query Handler

```
public class GetOrderQueryHandler  
{  
    private readonly IOrderReadRepository _repository;  
  
    public GetOrderQueryHandler(IOrderReadRepository repository)  
    {  
        _repository = repository;  
    }  
  
    public async Task<OrderDto?> Handle(GetOrderQuery query)  
    {  
        return await _repository.GetByIdAsync(query.OrderId);  
    }  
}
```

Notice that the query handler doesn't modify anything.

It only retrieves data.

5. Command vs Query

Command	Query
Changes state	Reads state
Create	Get
Update	Search
Delete	List
Cancel	Report
Approve	Get details
Usually returns result/ID	Returns data
Should not be treated as a read	Should not modify state

A useful rule:

Command → Change something

Query → Tell me something

6. CQRS with ASP.NET Core

For example:

```
[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly CreateOrderCommandHandler _commandHandler;
    private readonly GetOrderQueryHandler _queryHandler;

    public OrdersController(
        CreateOrderCommandHandler commandHandler,
        GetOrderQueryHandler queryHandler)
    {
        _commandHandler = commandHandler;
        _queryHandler = queryHandler;
    }

    [HttpPost]
    public async Task<IActionResult> Create(CreateOrderCommand command)
    {
        var orderId = await _commandHandler.Handle(command);

        return Ok(orderId);
    }
}
```

```
[HttpGet("{id}")]
public async Task<IActionResult> Get(int id)
{
    var order = await _queryHandler.Handle(
        new GetOrderQuery(id));

    if (order == null)
        return NotFound();

    return Ok(order);
}
```

Now the controller clearly separates:

```
POST → Command
GET  → Query
```

7. CQRS with MediatR

In real-world .NET applications, CQRS is often implemented using **MediatR** or a similar mediator library.

The command can implement `IRequest<T>`:

```
public record CreateOrderCommand(
    int CustomerId,
    decimal Amount
) : IRequest<int>;
```

Handler:

```
public class CreateOrderCommandHandler
    : IRequestHandler<CreateOrderCommand, int>
{
    private readonly AppDbContext _context;

    public CreateOrderCommandHandler(AppDbContext context)
    {
        _context = context;
    }

    public async Task<int> Handle(
        CreateOrderCommand request,
        CancellationToken cancellationToken)
    {
        // ...
    }
}
```

```
{
    var order = new Order
    {
        CustomerId = request.CustomerId,
        Amount = request.Amount,
        CreatedAt = DateTime.UtcNow
    };

    _context.Orders.Add(order);

    await _context.SaveChangesAsync(cancellationToken);

    return order.Id;
}
```

Query:

```
public record GetOrderQuery(int OrderId)
    : IRequest<OrderDto?>;
```

Handler:

```
public class GetOrderQueryHandler
    : IRequestHandler<GetOrderQuery, OrderDto?>
{
    private readonly AppDbContext _context;

    public GetOrderQueryHandler(AppDbContext context)
    {
        _context = context;
    }

    public async Task<OrderDto?> Handle(
        GetOrderQuery request,
        CancellationToken cancellationToken)
    {
        return await _context.Orders
            .Where(x => x.Id == request.OrderId)
            .Select(x => new OrderDto
            {
                Id = x.Id,
                CustomerId = x.CustomerId,
                Amount = x.Amount
            })
            .FirstOrDefaultAsync(cancellationToken);
    }
}
```

The controller becomes very thin:

```
[HttpPost]
public async Task<IActionResult> Create(
    CreateOrderCommand command)
{
    var id = await _mediator.Send(command);

    return Ok(id);
}

[HttpGet("{id}")]
public async Task<IActionResult> Get(int id)
{
    var result = await _mediator.Send(
        new GetOrderQuery(id));

    return result == null
        ? NotFound()
        : Ok(result);
}
```

8. Folder Structure

A common CQRS structure in ASP.NET Core is:

```
MyApplication
├── Domain
│   ├── Entities
│   │   └── Order.cs
│   └── Interfaces
├── Application
│   ├── Orders
│   │   ├── Commands
│   │   │   ├── CreateOrderCommand.cs
│   │   │   ├── CreateOrderCommandHandler.cs
│   │   │   ├── CancelOrderCommand.cs
│   │   │   └── CancelOrderCommandHandler.cs
│   │   └── Queries
│   │       ├── GetOrderQuery.cs
│   │       ├── GetOrderQueryHandler.cs
│   │       ├── GetOrdersQuery.cs
│   │       └── GetOrdersQueryHandler.cs
```

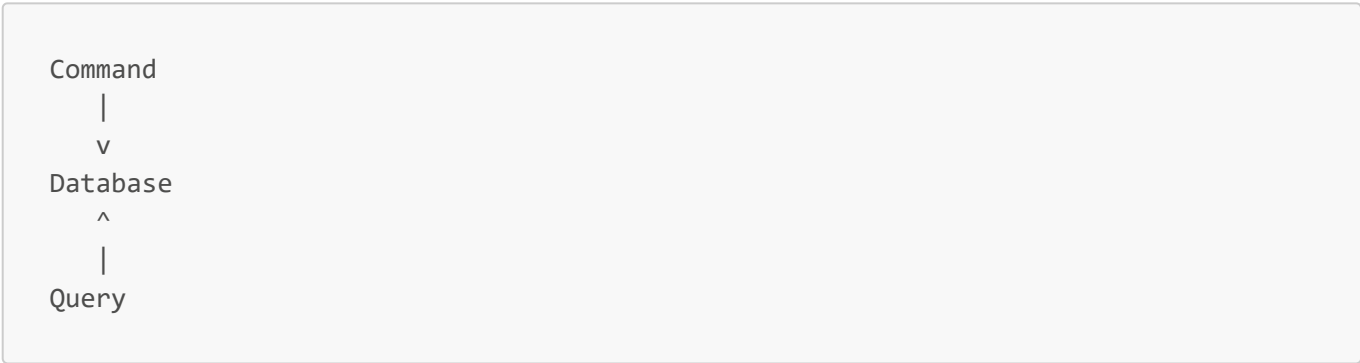


This becomes especially useful as the application grows.

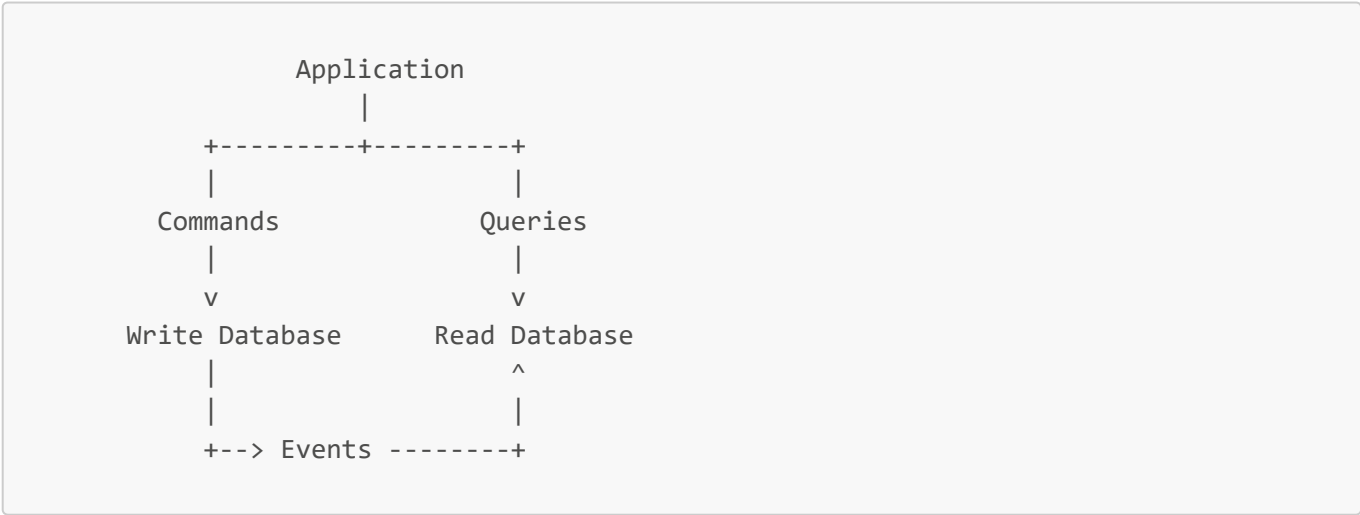
9. Advanced CQRS: Separate Read and Write Databases

CQRS can go further.

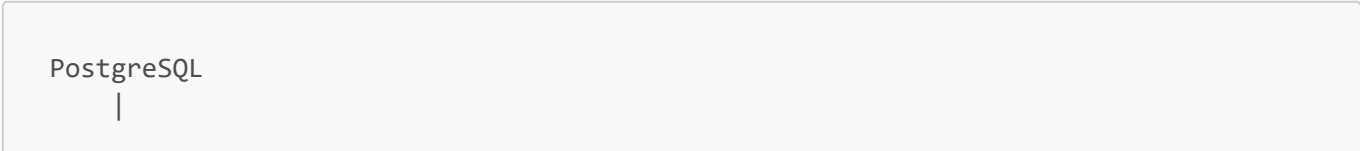
Instead of:



you can have:



For example:



```

    | Write
    v
Order Service
    |
    | OrderCreated event
    v
Message Broker
    |
    v
Read Model
    |
    v
Redis / Elasticsearch / SQL

```

This is useful when the application has:

- Very high read traffic
- Complex reporting
- Large datasets
- Different read/write performance requirements
- Event-driven architecture
- Microservices

10. CQRS + Event Sourcing

CQRS is often combined with **Event Sourcing**, but they are **not the same pattern**.

CQRS:

Separate Commands and Queries

Event Sourcing:

Store state changes as events

For example, instead of storing:

```

Order
-----
Id = 10
Status = Completed
Amount = 5000

```

you might store:

```
OrderCreated
Amount = 5000

PaymentReceived
Amount = 5000

OrderCompleted
```

Current state can then be reconstructed from the events.

So:

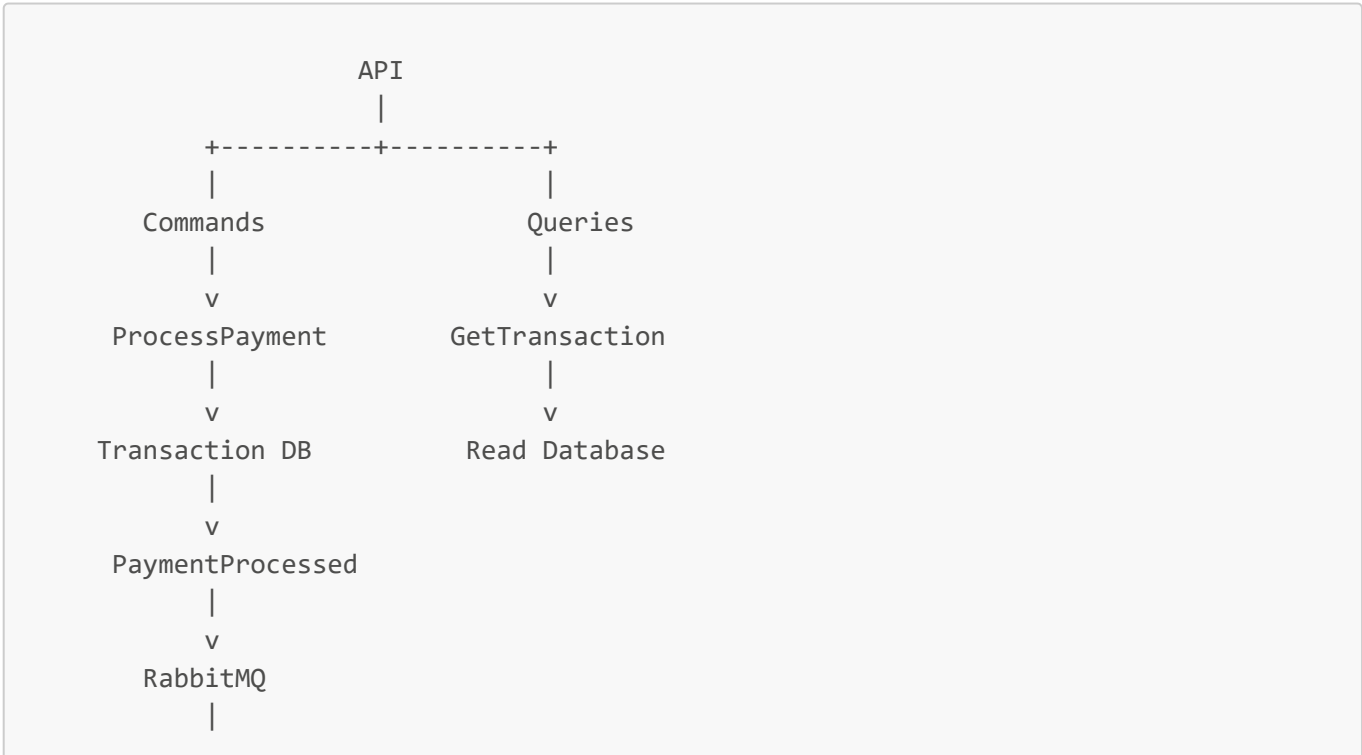
```
CQRS
+
Event Sourcing
=
Powerful event-driven architecture
```

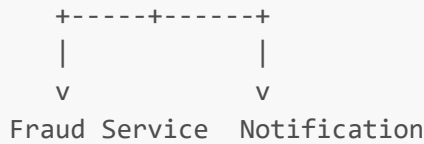
But you don't need Event Sourcing to use CQRS.

11. CQRS in a Fintech Application

CQRS is particularly useful for systems involving transactions, payments, rewards, fraud detection, etc.

For example:





A payment command could be:

```

public record ProcessPaymentCommand(
    int CustomerId,
    decimal Amount
) : IRequest<PaymentResult>;

```

The query might be:

```

public record GetTransactionHistoryQuery(
    int CustomerId
) : IRequest<List<TransactionDto>>;

```

The important separation is:

```

ProcessPaymentCommand
    ↓
Changes financial state

GetTransactionHistoryQuery
    ↓
Only reads financial state

```

12. Advantages

1. Separation of concerns

Commands and queries have different responsibilities.

2. Easier scaling

You can independently scale read and write workloads.

```

100 writes/sec
10,000 reads/sec

```

The read side can be optimized separately.

3. Better performance

Read models can be optimized specifically for queries.

For example:

```
Write DB → normalized  
Read DB  → denormalized
```

4. Easier maintenance

Instead of one giant service:

```
OrderService  
  3000 lines
```

you might have:

```
CreateOrderCommandHandler  
CancelOrderCommandHandler  
UpdateOrderCommandHandler  
  
GetOrderQueryHandler  
GetOrdersQueryHandler  
GetOrderHistoryQueryHandler
```

5. Better testing

You can test each handler independently.

13. Disadvantages

CQRS isn't automatically better.

It introduces additional complexity:

```
Simple CRUD  
  ↓  
CQRS  
  ↓  
More classes  
More handlers
```

More infrastructure
Potential synchronization issues
Potentially multiple databases

For a simple application:

Customer CRUD
Product CRUD
Employee CRUD

CQRS may be unnecessary.

For a complex system:

Payments
Orders
Fraud Detection
Rewards
Transactions
Reporting
Events
High read traffic

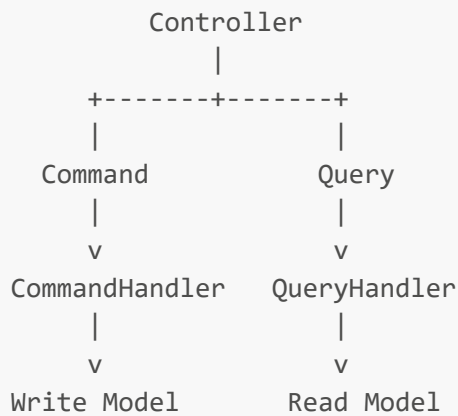
CQRS can be very valuable.

14. CQRS vs CRUD

Traditional CRUD

```
graph TD
    Controller --> Service
    Service --> Repository
    Repository --> Database
```

CQRS



15. CQRS vs Repository Pattern

These patterns solve different problems.

Repository:

How do I abstract persistence/data access?

CQRS:

How do I separate operations that change state from operations that read state?

They can be used together:

```

CreateOrderCommand
  ↓
CommandHandler
  ↓
OrderRepository
  ↓
Database

```

and:

```

GetOrderQuery
  ↓
QueryHandler
  ↓
ReadRepository
  ↓
Database

```

16. CQRS vs Mediator

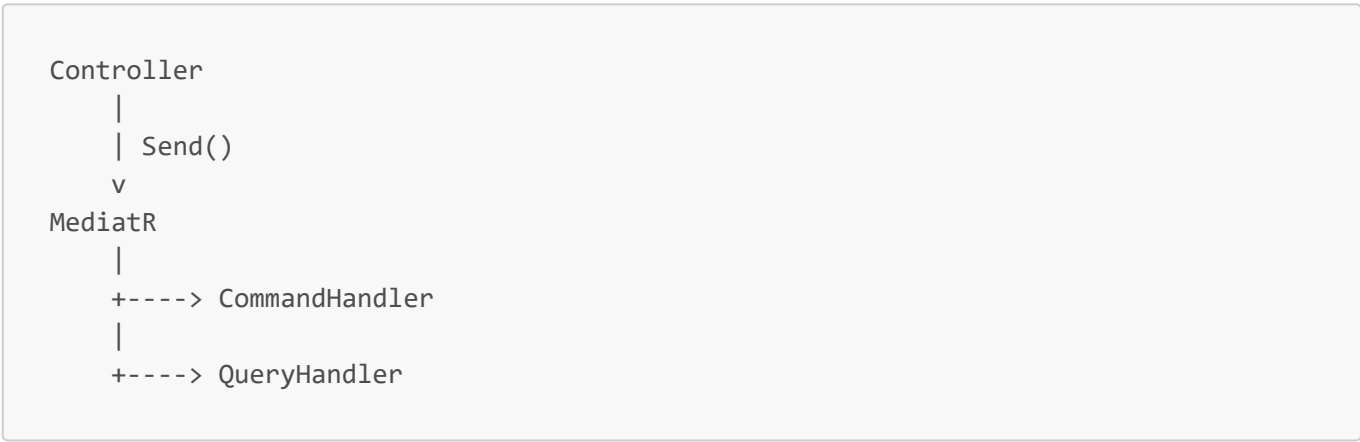
They are also different.

CQRS defines the architectural separation:

```
Commands ≠ Queries
```

Mediator defines how objects communicate without directly depending on each other.

MediatR can be used to implement CQRS:



So **MediatR is not CQRS**.

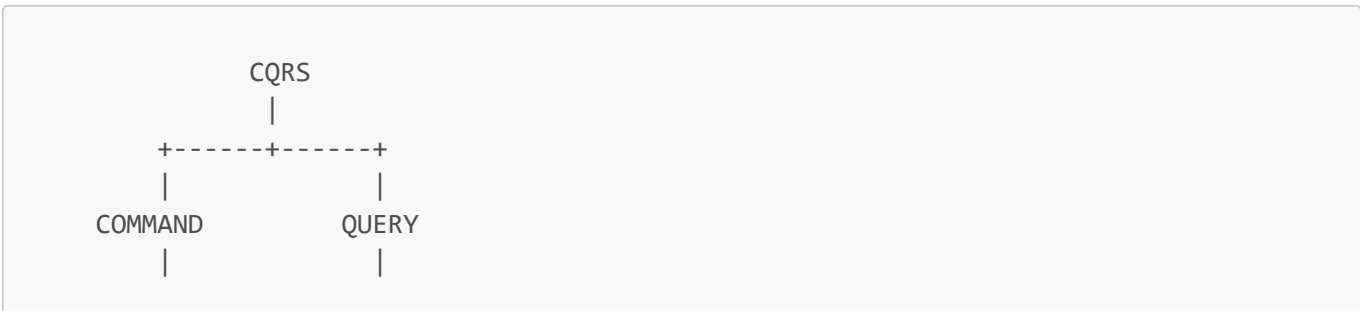
It is simply a common tool for implementing the command/query dispatching mechanism.

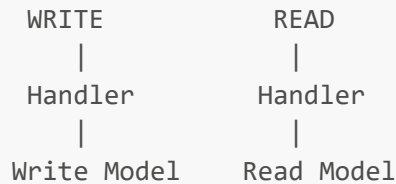
17. Interview Definition

If an interviewer asks "**What is CQRS?**", a good answer is:

CQRS, or Command Query Responsibility Segregation, is a design pattern that separates operations that modify application state from operations that retrieve data. Commands are responsible for state changes, while Queries are responsible for reading data. This separation allows the read and write sides to be independently optimized, scaled, and modeled. CQRS can be implemented with a single database or with separate read/write data stores, and it is often used with MediatR, messaging, and event-driven architectures.

Remember it like this:





The most important distinction:

Command = "Do something." Query = "Give me something."

11. **Specification**
12. **Options Pattern**

Options Pattern in C#

The **Options Pattern** is a design pattern commonly used in **ASP.NET Core** to represent configuration settings as strongly typed C# classes.

Instead of scattering configuration values throughout your application:

```

configuration["Database:ConnectionString"]
configuration["Jwt:SecretKey"]
configuration["Jwt:Issuer"]

```

you create a strongly typed options class:

```

public class JwtOptions
{
    public string SecretKey { get; set; } = "";
    public string Issuer { get; set; } = "";
    public int ExpirationMinutes { get; set; }
}

```

and inject it where required.

1. Why use the Options Pattern?

Suppose your `appsettings.json` contains:

```

{
  "Jwt": {
    "SecretKey": "my-secret-key",
    "Issuer": "MyApplication",
    "ExpirationMinutes": 60
  }
}

```

```
}  
}
```

Without Options Pattern:

```
public class AuthService  
{  
    private readonly IConfiguration _configuration;  
  
    public AuthService(IConfiguration configuration)  
    {  
        _configuration = configuration;  
    }  
  
    public void Login()  
    {  
        var secret = _configuration["Jwt:SecretKey"];  
        var issuer = _configuration["Jwt:Issuer"];  
    }  
}
```

This works, but configuration access is:

- string-based
- not strongly typed
- scattered across the application
- harder to validate
- harder to test

With Options Pattern:

```
public class JwtOptions  
{  
    public string SecretKey { get; set; } = "";  
    public string Issuer { get; set; } = "";  
    public int ExpirationMinutes { get; set; }  
}
```

Now your service receives `JwtOptions` instead.

2. Register the Options

In `Program.cs`:

```
builder.Services.Configure<JwtOptions>(  
    builder.Configuration.GetSection("Jwt"));
```

This tells ASP.NET Core:

Bind the `Jwt` section of configuration to `JwtOptions`.

3. Inject `IOptions<T>`

```
using Microsoft.Extensions.Options;  
  
public class AuthService  
{  
    private readonly JwtOptions _jwtOptions;  
  
    public AuthService(IOptions<JwtOptions> jwtOptions)  
    {  
        _jwtOptions = jwtOptions.Value;  
    }  
  
    public void Login()  
    {  
        Console.WriteLine(_jwtOptions.SecretKey);  
        Console.WriteLine(_jwtOptions.Issuer);  
        Console.WriteLine(_jwtOptions.ExpirationMinutes);  
    }  
}
```

The important part is:

```
IOptions<JwtOptions>
```

and:

```
jwtOptions.Value
```

4. Complete ASP.NET Core Example

appsettings.json


```
{
  "Jwt": {
    "SecretKey": "super-secret-key",
    "Issuer": "MyAPI",
    "ExpirationMinutes": 60
  }
}
```

Options class

```
public class JwtOptions
{
    public string SecretKey { get; set; } = "";
    public string Issuer { get; set; } = "";
    public int ExpirationMinutes { get; set; }
}
```

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.Configure<JwtOptions>(
    builder.Configuration.GetSection("Jwt"));

builder.Services.AddScoped<AuthService>();

var app = builder.Build();

app.Run();
```

Service

```
using Microsoft.Extensions.Options;

public class AuthService
{
    private readonly JwtOptions _options;

    public AuthService(IOptions<JwtOptions> options)
    {
        _options = options.Value;
    }

    public void GenerateToken()
    {
        Console.WriteLine($"Issuer: {_options.Issuer}");
    }
}
```

```
        Console.WriteLine($"Expiration: {_options.ExpirationMinutes}");
    }
}
```

5. IOptions vs IOptionsSnapshot vs IOptionsMonitor

This is an important interview topic.

Interface	Configuration reload	Lifetime/use
IOptions<T>	No	Singleton-style access
IOptionsSnapshot<T>	Yes	Scoped
IOptionsMonitor<T>	Yes	Singleton-friendly
IOptionsMonitor<T>.CurrentValue	Gets current value	Runtime

IOptions

```
public AuthService(IOptions<JwtOptions> options)
{
    var config = options.Value;
}
```

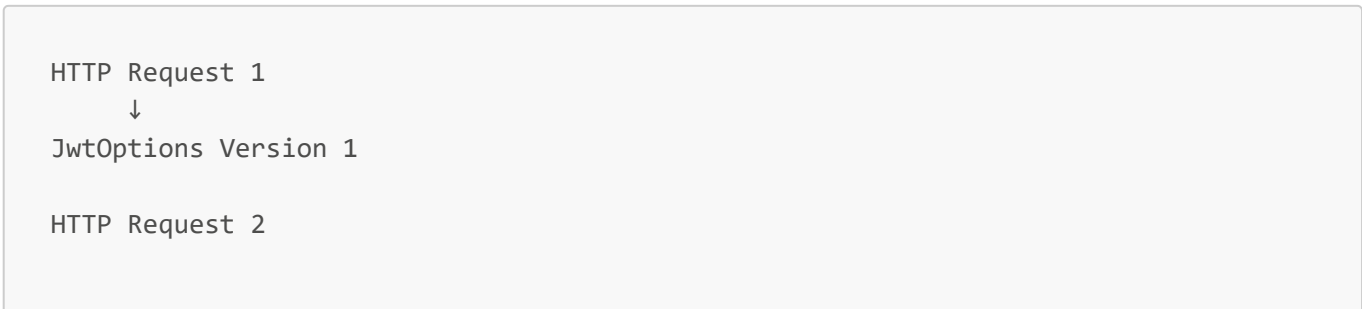
Use when configuration doesn't need to change during application execution.

IOptionsSnapshot

```
public AuthService(IOptionsSnapshot<JwtOptions> options)
{
    var config = options.Value;
}
```

It is useful for **scoped services** when configuration can be reloaded.

For example:



↓
JwtOptions Version 2

IOptionsMonitor

Useful when you need to detect configuration changes dynamically.

```
public class AuthService
{
    private readonly IOptionsMonitor<JwtOptions> _options;

    public AuthService(IOptionsMonitor<JwtOptions> options)
    {
        _options = options;
    }

    public void GenerateToken()
    {
        var config = _options.CurrentValue;

        Console.WriteLine(config.SecretKey);
    }
}
```

You can also listen for changes:

```
_options.OnChange(options =>
{
    Console.WriteLine("JWT configuration changed!");
});
```

6. Named Options

Sometimes you have multiple configurations of the same type.

For example:

```
{
    "Email": {
        "Host": "smtp.company.com",
        "Port": 587
    }
}
```

and another email configuration:

```
BackupEmail
```

You can register named options:

```
builder.Services.Configure<EmailOptions>(  
    "Primary",  
    builder.Configuration.GetSection("Email"));
```

Then retrieve:

```
public class EmailService  
{  
    private readonly IOptionsMonitor<EmailOptions> _options;  
  
    public EmailService(IOptionsMonitor<EmailOptions> options)  
    {  
        _options = options;  
    }  
  
    public void Send()  
    {  
        var email = _options.Get("Primary");  
  
        Console.WriteLine(email.Host);  
    }  
}
```

7. Options Validation

One major advantage is configuration validation.

```
public class JwtOptions  
{  
    public string SecretKey { get; set; } = "";  
  
    public string Issuer { get; set; } = "";  
  
    public int ExpirationMinutes { get; set; }  
}
```

You can use data annotations:

```
using System.ComponentModel.DataAnnotations;

public class JwtOptions
{
    [Required]
    public string SecretKey { get; set; } = "";

    [Required]
    public string Issuer { get; set; } = "";

    [Range(1, 1440)]
    public int ExpirationMinutes { get; set; }
}
```

Register:

```
builder.Services
    .AddOptions<JwtOptions>()
    .Bind(builder.Configuration.GetSection("Jwt"))
    .ValidateDataAnnotations();
```

Now invalid configuration can be detected rather than allowing bad values to propagate into your application.

8. Validate on Startup

You can force validation when the application starts:

```
builder.Services
    .AddOptions<JwtOptions>()
    .Bind(builder.Configuration.GetSection("Jwt"))
    .ValidateDataAnnotations()
    .ValidateOnStart();
```

This is particularly useful for production applications.

Instead of:

```
Application starts
    ↓
Requests arrive
    ↓
JWT configuration used
```

↓
Error occurs

you get:

Application starts
↓
Configuration validated
↓
Invalid configuration → application fails immediately

9. Options Pattern with a Custom Validator

For complex validation:

```
public class JwtOptionsValidator
    : IValidateOptions<JwtOptions>
{
    public ValidateOptionsResult Validate(
        string? name,
        JwtOptions options)
    {
        if (string.IsNullOrEmpty(options.SecretKey))
        {
            return ValidateOptionsResult.Fail(
                "JWT SecretKey is required.");
        }

        if (options.ExpirationMinutes <= 0)
        {
            return ValidateOptionsResult.Fail(
                "Expiration must be greater than zero.");
        }

        return ValidateOptionsResult.Success;
    }
}
```

Register it:

```
builder.Services.AddSingleton<
    IValidateOptions<JwtOptions>,
    JwtOptionsValidator>();
```

10. Options Pattern in a Real Project

For example, in a fintech application you might have:

```
Configuration
├── Jwt
├── Redis
├── RabbitMQ
├── Apigee
├── Database
├── Cloudflare
└── FraudDetection
```

Create separate classes:

```
public class RedisOptions
{
    public string ConnectionString { get; set; } = "";
}

public class RabbitMqOptions
{
    public string Host { get; set; } = "";
    public int Port { get; set; }
    public string Username { get; set; } = "";
    public string Password { get; set; } = "";
}
```

Then:

```
builder.Services.Configure<RedisOptions>(
    builder.Configuration.GetSection("Redis"));

builder.Services.Configure<RabbitMqOptions>(
    builder.Configuration.GetSection("RabbitMQ"));
```

Your services remain clean:

```
public class FraudService
{
    private readonly RedisOptions _redis;

    public FraudService(IOptions<RedisOptions> redis)
    {
```

```
        _redis = redis.Value;  
    }  
}
```

11. Options Pattern vs IConfiguration

IConfiguration

```
var connectionString =  
    configuration["Database:ConnectionString"];
```

Options Pattern

```
public class DatabaseOptions  
{  
    public string ConnectionString { get; set; } = "";  
}
```

Then:

```
public DatabaseService(IOptions<DatabaseOptions> options)  
{  
    _connectionString = options.Value.ConnectionString;  
}
```

The second approach gives you:

Strong typing

```
options.Value.ConnectionString
```

instead of:

```
configuration["Database:ConnectionString"]
```

Centralized configuration


```
DatabaseOptions  
JwtOptions  
RedisOptions  
RabbitMqOptions
```

Validation

```
[Required]  
[Range(...)]
```

Better testability

```
var options = Options.Create(new JwtOptions  
{  
    SecretKey = "test",  
    Issuer = "TestAPI",  
    ExpirationMinutes = 60  
});
```

12. When should you use it?

Use the **Options Pattern** when:

- you have multiple related configuration values
- configuration belongs to a specific component
- you want strongly typed configuration
- you need configuration validation
- you want cleaner dependency injection
- you want to avoid repeated **IConfiguration** lookups

For example:

```
IConfiguration  
  ↓  
Bind  
  ↓  
JwtOptions  
  ↓  
AuthService
```

Interview answer

If an interviewer asks **"What is the Options Pattern in ASP.NET Core?"**, you can answer:

The Options Pattern is a way to represent related configuration settings as strongly typed C# classes. We bind a configuration section, such as `Jwt`, to an options class and inject it using `IOptions<T>`, `IOptionsSnapshot<T>`, or `IOptionsMonitor<T>`. It provides better type safety, separation of configuration concerns, validation, and testability compared with directly accessing `IConfiguration`.

Remember this hierarchy

```
appsettings.json
    ↓
IConfiguration
    ↓
Bind / Configure<T>()
    ↓
Options class
    ↓
IOptions<T>
    ↓
Service / Controller
```

Most commonly used:

```
builder.Services.Configure<JwtOptions>(
    builder.Configuration.GetSection("Jwt"));
```

and:

```
public MyService(IOptions<JwtOptions> options)
{
    var jwt = options.Value;
}
```

That is the core of the **Options Pattern** in C#.

13. Middleware Pattern

Middleware Pattern in C#

The **Middleware Pattern** is a behavioral design pattern where a request passes through a **pipeline of components**, and each component can perform work **before and/or after** passing the request to the next component.

It is one of the most important patterns in **ASP.NET Core**.

Simple idea

Think of a request pipeline like:

```
Client
  ↓
Logging Middleware
  ↓
Authentication Middleware
  ↓
Authorization Middleware
  ↓
Exception Middleware
  ↓
Controller / Endpoint
  ↓
Response
```

Each middleware can inspect or modify the request and response.

1. Basic Middleware Example

Suppose we want to log every incoming request.

```
public class LoggingMiddleware
{
    private readonly RequestDelegate _next;

    public LoggingMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        Console.WriteLine($"Request: {context.Request.Path}");

        await _next(context);

        Console.WriteLine($"Response: {context.Response.StatusCode}");
    }
}
```

Here:

```
await _next(context);
```

means:

```
"Pass the request to the next middleware."
```

The middleware can therefore execute code **before and after** the next component.

2. Register Middleware

In `Program.cs`:

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.UseMiddleware<LoggingMiddleware>();

app.MapControllers();

app.Run();
```

Now requests will go through `LoggingMiddleware`.

3. Middleware with Request Blocking

Middleware doesn't have to call `_next`.

For example, we can block requests from a particular IP:

```
public class IpBlockingMiddleware
{
    private readonly RequestDelegate _next;

    public IpBlockingMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        var ip = context.Connection.RemoteIpAddress?.ToString();

        if (ip == "192.168.1.100")
        {
            context.Response.StatusCode = StatusCodes.Status403Forbidden;
            await context.Response.WriteAsync("Access denied");
            return;
        }

        _next(context);
    }
}
```

```
    }  
  
    await _next(context);  
  }  
}
```

The pipeline becomes:

```
Request  
↓  
IP Blocking Middleware  
↓  
├── Blocked → 403 Response  
└── Allowed  
    ↓  
    Next Middleware
```

4. Middleware Using `HttpContext`

Middleware can access almost everything related to the HTTP request.

Request

```
context.Request.Method  
context.Request.Path  
context.Request.Headers  
context.Request.Query  
context.Request.Cookies
```

Response

```
context.Response.StatusCode  
context.Response.Headers
```

User

```
context.User
```

For example:

```
public async Task InvokeAsync(HttpContext context)
{
    var method = context.Request.Method;
    var path = context.Request.Path;

    Console.WriteLine($"{method} {path}");

    await _next(context);
}
```

5. Middleware Pipeline

Suppose we have three middleware components:

```
app.UseMiddleware<MiddlewareA>();
app.UseMiddleware<MiddlewareB>();
app.UseMiddleware<MiddlewareC>();

app.MapControllers();
```

The execution looks like:

```
Request
  ↓
Middleware A
  ↓
Middleware B
  ↓
Middleware C
  ↓
Controller
  ↓
Middleware C
  ↓
Middleware B
  ↓
Middleware A
  ↓
Response
```

For example:

```
public async Task InvokeAsync(HttpContext context)
{
```

```
    Console.WriteLine("A - Before");

    await _next(context);

    Console.WriteLine("A - After");
}
```

Output:

```
A - Before
B - Before
C - Before
Controller
C - After
B - After
A - After
```

This **onion-like execution** is a key characteristic of middleware.

6. Inline Middleware

You don't always need to create a class.

You can write middleware directly:

```
app.Use(async (context, next) =>
{
    Console.WriteLine("Before request");

    await next();

    Console.WriteLine("After request");
});
```

Then:

```
app.MapGet("/", () => "Hello World");
```

7. Middleware for Exception Handling

A common real-world use is centralized exception handling.

```
public class ExceptionMiddleware
{
    private readonly RequestDelegate _next;

    public ExceptionMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch (Exception ex)
        {
            context.Response.StatusCode = 500;
            context.Response.ContentType = "application/json";

            await context.Response.WriteAsJsonAsync(new
            {
                message = "Something went wrong"
            });
        }
    }
}
```

Register it early:

```
app.UseMiddleware<ExceptionMiddleware>();
```

Then:

```
Request
↓
Exception Middleware
↓
Authentication
↓
Authorization
↓
Controller
↓
Exception occurs
↓
Exception Middleware
```


↓

500 Response

8. Middleware with Dependency Injection

Middleware can use services through dependency injection.

```
public class AuditMiddleware
{
    private readonly RequestDelegate _next;

    public AuditMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(
        HttpContext context,
        IAuditService auditService)
    {
        await auditService.LogAsync(
            context.Request.Path
        );

        await _next(context);
    }
}
```

Register:

```
builder.Services.AddScoped<IAuditService, AuditService>();
```

ASP.NET Core resolves the dependency when invoking the middleware.

9. Middleware vs Filter

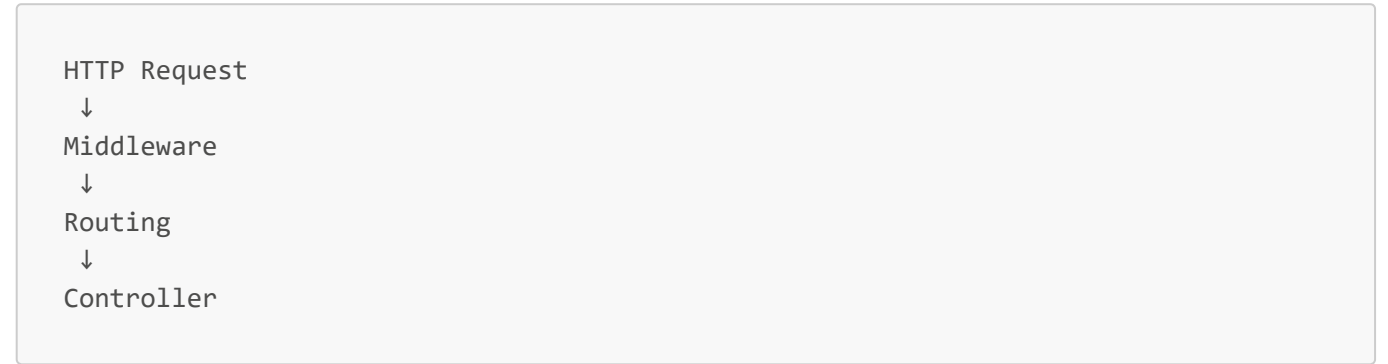
This is an important interview question.

Middleware	Filter
Works at HTTP pipeline level	Usually works around MVC/controller execution

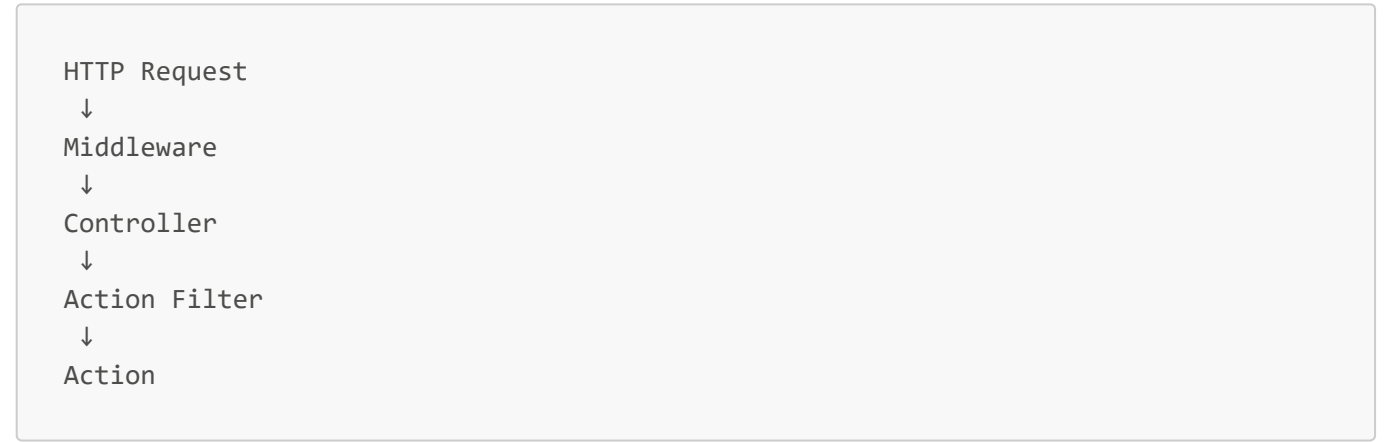
Middleware	Filter
Can run before routing/authentication depending on position	Runs within MVC/Razor processing
Applies broadly	Can target controllers/actions
<code>HttpContext</code> focused	Action/controller focused
Good for logging, exception handling, authentication	Good for action-specific concerns

Example:

Middleware



MVC Filter

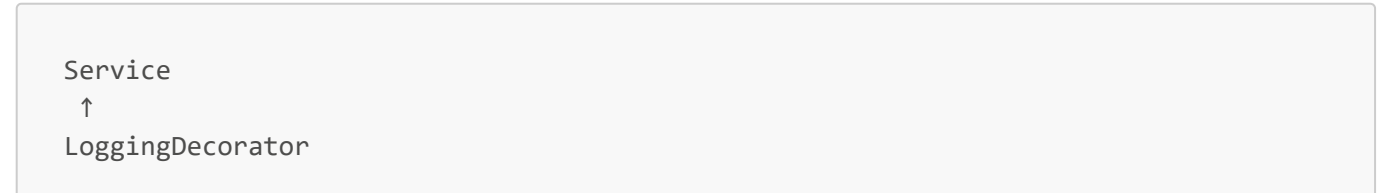


10. Middleware vs Decorator

They are related conceptually but aren't the same.

Decorator

Wraps an object:



↑
CachingDecorator

Example:

IOrderService
↓
LoggingOrderService
↓
RealOrderService

Middleware

Wraps a **request pipeline**:

Request
↓
Logging
↓
Authentication
↓
Authorization
↓
Endpoint

Both use a similar **chain/wrapping concept**, but middleware is specifically designed around request processing pipelines.

11. When Should You Use Middleware?

Middleware is appropriate for **cross-cutting concerns**.

Common examples:

- Exception handling
- Request logging
- Authentication
- Authorization
- CORS
- Rate limiting
- Security headers
- Request/response transformation
- Correlation IDs
- Performance measurement

- Audit logging
- IP filtering
- API key validation

For example, in a fintech API:

```
Client
↓
Correlation ID Middleware
↓
Exception Middleware
↓
Logging Middleware
↓
Authentication
↓
Authorization
↓
Rate Limiting
↓
Controller
↓
Service
↓
Database
```

12. Important: Middleware Order Matters

This is one of the most important things to understand in ASP.NET Core.

For example:

```
app.UseExceptionHandler("/error");

app.UseHttpsRedirection();

app.UseAuthentication();

app.UseAuthorization();

app.MapControllers();
```

Changing the order can change the behavior of the application.

For example:

```
Exception Handler
  ↓
Authentication
  ↓
Authorization
  ↓
Controller
```

is different from:

```
Authentication
  ↓
Exception Handler
  ↓
Authorization
  ↓
Controller
```

because middleware executes according to its position in the pipeline.

13. Custom Middleware Extension Method

A common professional approach is to create an extension method.

```
public static class MiddlewareExtensions
{
    public static IApplicationBuilder UseAuditMiddleware(
        this IApplicationBuilder app)
    {
        return app.UseMiddleware<AuditMiddleware>();
    }
}
```

Then:

```
app.UseAuditMiddleware();
```

This makes `Program.cs` cleaner.

14. Complete Example

Middleware

```
public class RequestTimingMiddleware
{
    private readonly RequestDelegate _next;

    public RequestTimingMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        var start = DateTime.UtcNow;

        await _next(context);

        var duration = DateTime.UtcNow - start;

        Console.WriteLine(
            $"{context.Request.Path} took {duration.TotalMilliseconds} ms"
        );
    }
}
```

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

var app = builder.Build();

app.UseMiddleware<RequestTimingMiddleware>();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();

app.Run();
```

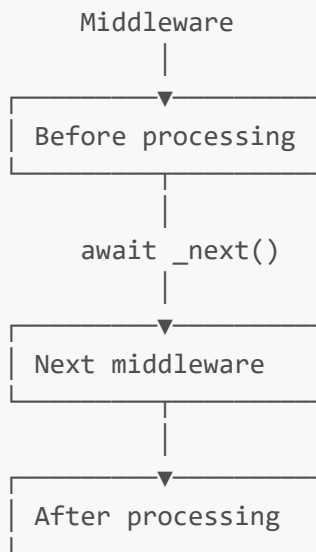
Now every request is measured.

Middleware Pattern — Interview Definition

Middleware is a behavioral pipeline pattern in which an HTTP request passes through a sequence of components, where each component can process the request, pass it to the next

component, and optionally process the response on the way back.

Remember this structure



Key interview points: `RequestDelegate`, `HttpContext`, `_next`, pipeline ordering, short-circuiting, before/after execution, and cross-cutting concerns.

14. Pipeline Pattern

Pipeline Pattern in C#

The **Pipeline Pattern** is a design pattern where a task is broken into a sequence of **independent processing stages**. Each stage receives data, performs some operation, and passes the result to the next stage.

Think of it like an assembly line:

```
Input
↓
Validate
↓
Authenticate
↓
Transform
↓
Log
↓
Output
```

Each step in the pipeline has a single responsibility.

1. Basic Example

Suppose we are processing an HTTP request:

```
Request
↓
Validate Request
↓
Authenticate User
↓
Check Authorization
↓
Process Request
↓
Response
```

Instead of putting everything into one large method, we create separate pipeline stages.

Pipeline stage interface

```
public interface IPipelineStep<T>
{
    T Execute(T input);
}
```

Now create individual steps.

Validation

```
public class ValidationStep : IPipelineStep<string>
{
    public string Execute(string input)
    {
        Console.WriteLine("Validating...");

        if (string.IsNullOrEmpty(input))
            throw new ArgumentException("Input cannot be empty");

        return input;
    }
}
```

Transformation

```
public class TransformationStep : IPipelineStep<string>
{
    public string Execute(string input)
```



```
    {  
        Console.WriteLine("Transforming...");  
  
        return input.ToUpper();  
    }  
}
```

Logging

```
public class LoggingStep : IPipelineStep<string>  
{  
    public string Execute(string input)  
    {  
        Console.WriteLine($"Logging: {input}");  
  
        return input;  
    }  
}
```

2. Creating the Pipeline

```
public class Pipeline<T>  
{  
    private readonly List<IPipelineStep<T>> _steps = new();  
  
    public Pipeline<T> AddStep(IPipelineStep<T> step)  
    {  
        _steps.Add(step);  
        return this;  
    }  
  
    public T Execute(T input)  
    {  
        T result = input;  
  
        foreach (var step in _steps)  
        {  
            result = step.Execute(result);  
        }  
  
        return result;  
    }  
}
```

Usage:

```
var pipeline = new Pipeline<string>()
    .AddStep(new ValidationStep())
    .AddStep(new TransformationStep())
    .AddStep(new LoggingStep());

var result = pipeline.Execute("hello");

Console.WriteLine(result);
```

Output:

```
Validating...
Transforming...
Logging: HELLO
HELLO
```

The important part is that each stage doesn't need to know about the other stages.

3. Real-World Example

Imagine a payment processing system:

```
Payment Request
  ↓
Validate Payment
  ↓
Check Fraud
  ↓
Authorize Payment
  ↓
Save Transaction
  ↓
Send Notification
```

Each stage could be:

```
public interface IPaymentStep
{
    PaymentContext Execute(PaymentContext context);
}
```

Context:

```
public class PaymentContext
{
    public decimal Amount { get; set; }
    public string CardNumber { get; set; } = "";
    public bool IsFraud { get; set; }
    public bool IsAuthorized { get; set; }
}
```

Validation:

```
public class ValidatePaymentStep : IPaymentStep
{
    public PaymentContext Execute(PaymentContext context)
    {
        if (context.Amount <= 0)
            throw new Exception("Invalid amount");

        return context;
    }
}
```

Fraud check:

```
public class FraudCheckStep : IPaymentStep
{
    public PaymentContext Execute(PaymentContext context)
    {
        Console.WriteLine("Checking fraud...");

        context.IsFraud = context.Amount > 100000;

        if (context.IsFraud)
            throw new Exception("Fraud detected");

        return context;
    }
}
```

Authorization:

```
public class AuthorizationStep : IPaymentStep
{
    public PaymentContext Execute(PaymentContext context)
    {
        Console.WriteLine("Authorizing payment...");
    }
}
```

```
        context.IsAuthorized = true;

        return context;
    }
}
```

Pipeline:

```
var pipeline = new Pipeline<PaymentContext>()
    .AddStep(new ValidatePaymentStep())
    .AddStep(new FraudCheckStep())
    .AddStep(new AuthorizationStep());

var payment = new PaymentContext
{
    Amount = 5000,
    CardNumber = "123456789"
};

var result = pipeline.Execute(payment);
```

4. Pipeline vs Chain of Responsibility

These patterns look very similar.

Pipeline	Chain of Responsibility
Usually all stages execute	A handler may stop processing
Data flows through stages	Request flows through handlers
Focuses on processing sequence	Focuses on finding a handler
Common in middleware	Common for request handling
Every stage can transform data	Handler may handle or forward

For example:

Pipeline

```
A → B → C → D
```

All stages normally execute.

Chain of Responsibility

```
A → B → C → D
```

But if **B** handles the request:

```
A → B → STOP
```

5. Pipeline in ASP.NET Core

ASP.NET Core middleware is a very good real-world example of a pipeline.

```
app.UseMiddleware<LoggingMiddleware>();  
  
app.UseMiddleware<AuthenticationMiddleware>();  
  
app.UseMiddleware<AuthorizationMiddleware>();  
  
app.UseMiddleware<ErrorHandlingMiddleware>();
```

Conceptually:

```
HTTP Request  
  ↓  
Logging Middleware  
  ↓  
Authentication Middleware  
  ↓  
Authorization Middleware  
  ↓  
Controller  
  ↓  
HTTP Response
```

A middleware looks like:

```
public class LoggingMiddleware  
{  
    private readonly RequestDelegate _next;  
  
    public LoggingMiddleware(RequestDelegate next)  
    {  
        _next = next;  
    }  
}
```

```
public async Task InvokeAsync(HttpContext context)
{
    Console.WriteLine("Request received");

    await _next(context);

    Console.WriteLine("Response sent");
}
```

`_next(context)` passes execution to the next pipeline stage.

6. Async Pipeline

In real ASP.NET Core applications, pipeline stages are often asynchronous.

```
public interface IPipelineStep<T>
{
    Task<T> ExecuteAsync(T input);
}
```

Example:

```
public class ValidationStep : IPipelineStep<PaymentContext>
{
    public async Task<PaymentContext> ExecuteAsync(
        PaymentContext context)
    {
        await Task.Delay(10);

        if (context.Amount <= 0)
            throw new Exception("Invalid amount");

        return context;
    }
}
```

Pipeline:

```
public class AsyncPipeline<T>
{
    private readonly List<IPipelineStep<T>> _steps = new();

    public AsyncPipeline<T> AddStep(IPipelineStep<T> step)
```

```
{
    _steps.Add(step);
    return this;
}

public async Task<T> ExecuteAsync(T input)
{
    var result = input;

    foreach (var step in _steps)
    {
        result = await step.ExecuteAsync(result);
    }

    return result;
}
```

Usage:

```
var pipeline = new AsyncPipeline<PaymentContext>()
    .AddStep(new ValidatePaymentStep())
    .AddStep(new FraudCheckStep())
    .AddStep(new AuthorizationStep());

var result = await pipeline.ExecuteAsync(payment);
```

7. Pipeline with Dependency Injection

In an ASP.NET Core application, this becomes much more useful with DI.

```
builder.Services.AddScoped<IPaymentStep, ValidatePaymentStep>();
builder.Services.AddScoped<IPaymentStep, FraudCheckStep>();
builder.Services.AddScoped<IPaymentStep, AuthorizationStep>();
```

Then inject:

```
public class PaymentPipeline
{
    private readonly IEnumerable<IPaymentStep> _steps;

    public PaymentPipeline(IEnumerable<IPaymentStep> steps)
    {
        _steps = steps;
    }
}
```

```
public PaymentContext Execute(PaymentContext context)
{
    foreach (var step in _steps)
    {
        context = step.Execute(context);
    }

    return context;
}
```

This is particularly useful because adding a new stage doesn't require modifying the existing stages.

8. When Should You Use Pipeline Pattern?

Use it when you have:

- Multiple sequential processing steps
- A request that needs several transformations
- Validation → processing → logging type workflows
- Payment processing
- Data processing
- Authentication/authorization
- ETL/data transformation
- Request processing
- Message processing
- Middleware-like behavior

For example:

```
Order
↓
Validate
↓
Calculate Price
↓
Apply Discount
↓
Calculate Tax
↓
Save Order
↓
Send Email
```

Each operation can become an independent pipeline step.

9. Advantages

Single Responsibility

Each stage does one thing.

```
ValidationStep  
FraudCheckStep  
AuthorizationStep
```

Easy to extend

You can add:

```
RateLimitStep  
  ↓  
ValidationStep  
  ↓  
FraudCheckStep  
  ↓  
AuthorizationStep
```

without rewriting the existing steps.

Testability

You can unit-test each step independently.

```
var step = new FraudCheckStep();  
  
var result = step.Execute(payment);
```

Reusability

A step can potentially be reused in different pipelines.

10. Disadvantages

Pipeline isn't always the right choice.

It can introduce:

- More classes

- More abstraction
- Debugging complexity
- Ordering dependencies
- Extra overhead for very simple workflows

For a simple operation like:

```
var total = price * quantity;
```

a pipeline would be overengineering.

11. Pipeline vs Middleware vs Chain of Responsibility

A useful way to remember them:

```
Pipeline
  ↓
Sequential processing stages

Middleware
  ↓
Pipeline specifically designed around request/response processing

Chain of Responsibility
  ↓
Pass responsibility until a handler handles it
```

In modern C#/.NET, **ASP.NET Core middleware is one of the most important practical examples of the Pipeline Pattern.**

Interview definition

The Pipeline Pattern breaks a process into a sequence of independent processing stages, where each stage performs a specific operation and passes the result to the next stage. It improves separation of concerns, extensibility, and testability.

15. Result Pattern

Result Pattern in C#

The **Result Pattern** is a way to represent the outcome of an operation explicitly as either:

- **Success** → contains a value
- **Failure** → contains an error
- Optionally, **Success/Failure without a value**

It is commonly used in modern C# applications to avoid using exceptions for **expected business failures** such as validation errors, insufficient balance, duplicate records, or unauthorized operations.

1. The problem

Without a Result Pattern, you might write:

```
public User GetUser(int id)
{
    var user = _repository.GetById(id);

    if (user == null)
        throw new Exception("User not found");

    return user;
}
```

The caller has to use exception handling:

```
try
{
    var user = service.GetUser(10);
}
catch (Exception ex)
{
    // Handle failure
}
```

This can be problematic because **"user not found" is often an expected business outcome**, not an exceptional system failure.

2. Basic Result Pattern

We can create a `Result<T>` class.

```
public class Result<T>
{
    public bool IsSuccess { get; }
    public T? Value { get; }
    public string? Error { get; }

    private Result(bool isSuccess, T? value, string? error)
    {
        IsSuccess = isSuccess;
    }
}
```

```
        Value = value;
        Error = error;
    }

    public static Result<T> Success(T value)
    {
        return new Result<T>(true, value, null);
    }

    public static Result<T> Failure(string error)
    {
        return new Result<T>(false, default, error);
    }
}
```

Now a service can return a result instead of throwing an exception.

```
public Result<User> GetUser(int id)
{
    var user = _repository.GetById(id);

    if (user == null)
        return Result<User>.Failure("User not found");

    return Result<User>.Success(user);
}
```

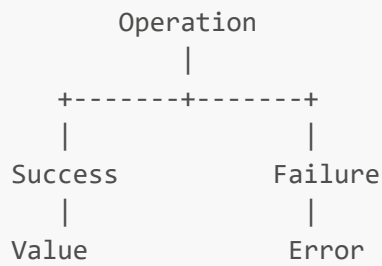
3. Consuming the Result

```
var result = userService.GetUser(10);

if (result.IsSuccess)
{
    Console.WriteLine(result.Value!.Name);
}
else
{
    Console.WriteLine(result.Error);
}
```

The caller is forced to consider both outcomes.

Conceptually:



4. Result Pattern in ASP.NET Core

This pattern is particularly useful in Web APIs.

For example:

```

[HttpGet("{id}")]
public IActionResult GetUser(int id)
{
    var result = _userService.GetUser(id);

    if (!result.IsSuccess)
        return NotFound(result.Error);

    return Ok(result.Value);
}
  
```

The service doesn't need to know anything about HTTP.

```

Controller
  |
  v
Service
  |
  v
Result<T>
  / \
Success Failure
  
```

The controller translates the Result into an HTTP response.

5. Better Result with Error Codes

Instead of returning only a string, define an error.

```
public record Error(  
    string Code,  
    string Message);
```

Then:

```
public class Result<T>  
{  
    public bool IsSuccess { get; }  
    public T? Value { get; }  
    public Error? Error { get; }  
  
    private Result(bool isSuccess, T? value, Error? error)  
    {  
        IsSuccess = isSuccess;  
        Value = value;  
        Error = error;  
    }  
  
    public static Result<T> Success(T value)  
        => new(true, value, null);  
  
    public static Result<T> Failure(Error error)  
        => new(false, default, error);  
}
```

Usage:

```
return Result<User>.Failure(  
    new Error(  
        "User.NotFound",  
        "The requested user was not found."));
```

Now the API can make decisions based on the error code.

```
if (result.Error?.Code == "User.NotFound")  
{  
    return NotFound(result.Error);  
}
```

6. Real-world example: Payment

Imagine a fintech application.

```
public Result<Payment> ProcessPayment(
    decimal amount,
    decimal balance)
{
    if (amount <= 0)
    {
        return Result<Payment>.Failure(
            new Error(
                "Payment.InvalidAmount",
                "Payment amount must be greater than zero."));
    }

    if (balance < amount)
    {
        return Result<Payment>.Failure(
            new Error(
                "Payment.InsufficientFunds",
                "Insufficient funds."));
    }

    var payment = new Payment
    {
        Amount = amount,
        Status = "Completed"
    };

    return Result<Payment>.Success(payment);
}
```

Caller:

```
var result = paymentService.ProcessPayment(500, 300);

if (!result.IsSuccess)
{
    Console.WriteLine(result.Error!.Message);
    return;
}

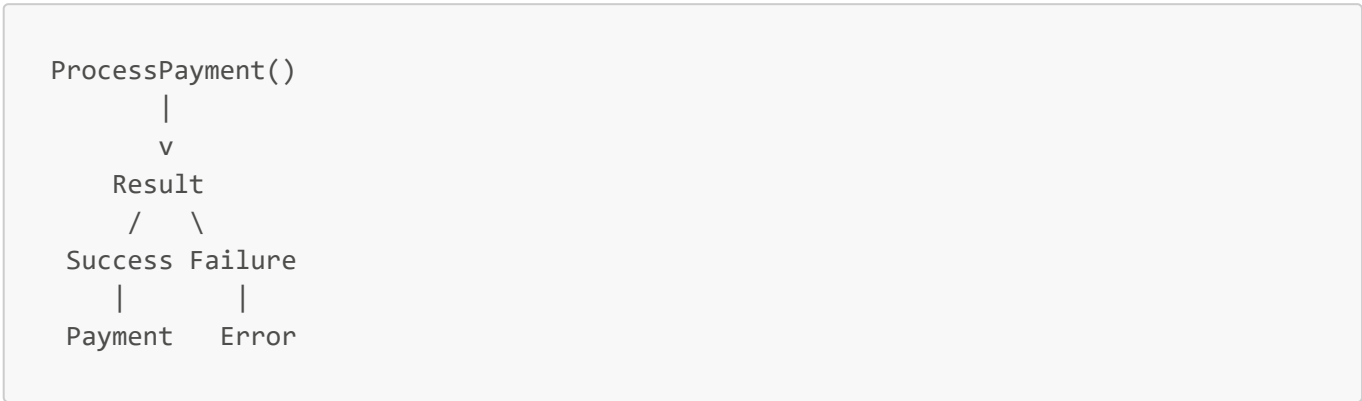
Console.WriteLine("Payment successful");
```

Instead of:

```
ProcessPayment()
|
+---- throws exception
|
```

```
      v
try/catch
```

we have:



7. Result Pattern vs Exceptions

Result Pattern	Exceptions
Expected failures	Unexpected failures
Validation failure	Database connection failure
User not found	NullPointerException
Insufficient funds	Network failure
Duplicate business operation	Infrastructure failure
Invalid state	Programming bugs

A useful rule is:

Use Results for expected business outcomes; use exceptions for exceptional/unexpected failures.

For example:

```
// Result is appropriate
return Result<User>.Failure(
    new Error("User.NotFound", "User doesn't exist."));
```

But a database outage is different:

```
// Exception is appropriate
throw new DatabaseException(
    "Database connection failed.");
```

8. Result without a return value

Sometimes an operation succeeds but doesn't need to return an object.

For example:

```
public Result DeleteUser(int id)
{
    var user = _repository.GetById(id);

    if (user == null)
        return Result.Failure(
            new Error(
                "User.NotFound",
                "User was not found."));

    _repository.Delete(user);

    return Result.Success();
}
```

A non-generic `Result` can be:

```
public class Result
{
    public bool IsSuccess { get; }
    public Error? Error { get; }

    protected Result(bool isSuccess, Error? error)
    {
        IsSuccess = isSuccess;
        Error = error;
    }

    public static Result Success()
        => new(true, null);

    public static Result Failure(Error error)
        => new(false, error);
}
```

9. Result Pattern with `async`

This is very common in ASP.NET Core.

```
public async Task<Result<User>> GetUserAsync(int id)
{
    var user = await _repository.GetByIdAsync(id);

    if (user == null)
    {
        return Result<User>.Failure(
            new Error(
                "User.NotFound",
                "User was not found."));
    }

    return Result<User>.Success(user);
}
```

Controller:

```
[HttpGet("{id}")]
public async Task<IActionResult> GetUser(int id)
{
    var result = await _userService.GetUserAsync(id);

    if (!result.IsSuccess)
        return NotFound(result.Error);

    return Ok(result.Value);
}
```

10. Result Pattern with validation

Suppose we have:

```
public Result RegisterUser(string email, string password)
{
    if (string.IsNullOrEmpty(email))
    {
        return Result.Failure(
            new Error(
                "User.InvalidEmail",
                "Email is required."));
    }

    if (password.Length < 8)
    {
        return Result.Failure(
            new Error(
                "User.InvalidPassword",
                "Password must be at least 8 characters long."));
    }

    // ... rest of the logic
}
```

```

        "User.WeakPassword",
        "Password must contain at least 8 characters.));
    }

    // Create user...

    return Result.Success();
}

```

The application flow becomes:

```

Request
|
v
Controller
|
v
Service
|
+---- Validation failure ----> Result.Failure
|
+---- Business failure -----> Result.Failure
|
+---- Success -----> Result.Success

```

11. Result Pattern with multiple errors

For validation, you may want multiple errors.

```

public record ValidationError(
    string Code,
    string Message);

```

Then:

```

public class Result<T>
{
    public bool IsSuccess { get; }
    public T? Value { get; }
    public IReadOnlyList<ValidationError> Errors { get; }

    private Result(
        bool isSuccess,
        T? value,
        IReadOnlyList<ValidationError> errors)
    {
        IsSuccess = isSuccess;
        Value = value;
        Errors = errors;
    }
}

```

```

    {
        IsSuccess = isSuccess;
        Value = value;
        Errors = errors;
    }

    public static Result<T> Success(T value)
        => new(true, value, []);

    public static Result<T> Failure(
        params ValidationError[] errors)
        => new(false, default, errors);
}

```

Usage:

```

return Result<User>.Failure(
    new ValidationError(
        "Email.Required",
        "Email is required."),

    new ValidationError(
        "Password.Required",
        "Password is required.));

```

This is especially useful for APIs.

12. Result Pattern vs Either

You may also encounter the functional programming concept **Either**.

Conceptually:

```

Either<Error, User>
  /      \
Error    User

```

A **Result<T>** is essentially a domain-oriented version of this idea:

```

Result<User>
  /      \
Failure  Success
Error    User

```

Libraries such as `FluentResults` and `ErrorOr` provide more sophisticated implementations, but you can implement the core pattern yourself.

13. Where Result Pattern fits in Clean Architecture

A typical architecture:



Result flows upward:



This keeps business logic independent of HTTP.

14. Result Pattern vs Repository Pattern

They solve completely different problems.

Repository Pattern

How do I abstract data access?

```
IUserRepository
```

Result Pattern

How do I explicitly represent success/failure?

```
Result<User>
```

They can be used together:

```
public async Task<Result<User>> GetUserAsync(int id)
{
    var user = await _repository.GetByIdAsync(id);

    if (user == null)
        return Result<User>.Failure(
            new Error("User.NotFound", "User not found."));

    return Result<User>.Success(user);
}
```

15. When should you use it?

Good use cases

- Validation
- Business rule violations
- Not found
- Duplicate records
- Insufficient funds
- Invalid state transitions
- Authorization/business decisions

- Expected failures from application operations

Don't use it for everything

Avoid turning every unexpected infrastructure problem into:

```
Result.Failure("Something went wrong");
```

For example:

```
Database unavailable
Redis unavailable
Unexpected NullReferenceException
Out-of-memory
Programming bug
```

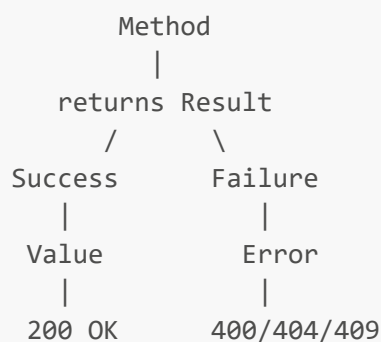
These may need exceptions, logging, retries, circuit breakers, etc.

Interview definition

If asked "**What is the Result Pattern?**", a strong answer is:

The Result Pattern is a pattern for explicitly representing the outcome of an operation as either success or failure. Instead of using exceptions for expected business failures, a method returns a Result object containing either the successful value or structured error information. This makes error handling explicit, improves readability, and keeps business logic separate from transport-specific concerns such as HTTP status codes.

Simple mental model



For **ASP.NET Core + Clean Architecture**, the Result Pattern is particularly useful at the **Application/Domain boundary**, while the API layer translates the result into HTTP responses.

7. Distributed Systems / Microservices Patterns

For advanced .NET backend development:

1. API Gateway

API Gateway Design Pattern in C#

The **API Gateway Pattern** provides a **single entry point** for clients to access multiple backend services.

Instead of a client directly calling:

```
Client
|
|→ User Service
|→ Order Service
|→ Payment Service
|→ Notification Service
```

the client calls one gateway:

```
Client → API Gateway → Order Service
                        |
                        |→ User Service
                        |→ Payment Service
                        |→ Notification Service
```

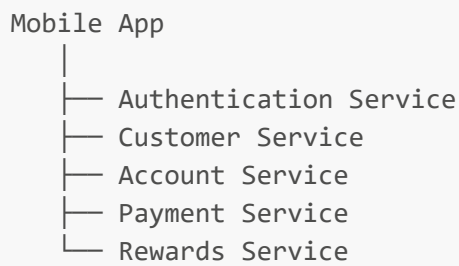
The gateway can handle **authentication, authorization, routing, rate limiting, logging, caching, aggregation, and load balancing**.

1. Why use API Gateway?

Imagine a fintech application with these microservices:

```
Authentication Service
Customer Service
Account Service
Payment Service
Rewards Service
Notification Service
```

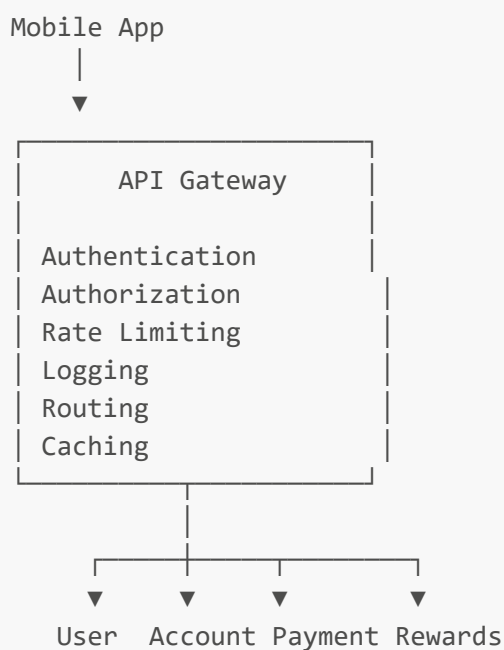

Without a gateway:



The mobile application needs to know:

- service URLs
- authentication mechanisms
- API versions
- retry behavior
- error handling
- potentially different protocols

With an API Gateway:



The client only needs to know the gateway.

2. Simple C# Example

Suppose we have:

```
/api/users  
/api/orders  
/api/payments
```

The gateway can route requests to different services.

A simple ASP.NET Core gateway controller might look like:

```
[ApiController]  
[Route("api/gateway")]  
public class GatewayController : ControllerBase  
{  
    private readonly HttpClient _httpClient;  
  
    public GatewayController(HttpClient httpClient)  
    {  
        _httpClient = httpClient;  
    }  
  
    [HttpGet("users/{id}")]  
    public async Task<IActionResult> GetUser(int id)  
    {  
        var response = await _httpClient.GetAsync(  
            $"http://user-service/api/users/{id}");  
  
        if (!response.IsSuccessStatusCode)  
            return StatusCode((int)response.StatusCode);  
  
        var content = await response.Content.ReadAsStringAsync();  
  
        return Content(content, "application/json");  
    }  
  
    [HttpGet("orders/{id}")]  
    public async Task<IActionResult> GetOrder(int id)  
    {  
        var response = await _httpClient.GetAsync(  
            $"http://order-service/api/orders/{id}");  
  
        if (!response.IsSuccessStatusCode)  
            return StatusCode((int)response.StatusCode);  
  
        var content = await response.Content.ReadAsStringAsync();  
  
        return Content(content, "application/json");  
    }  
}
```

The client calls:

```
GET /api/gateway/users/10
```

instead of:

```
GET http://user-service/api/users/10
```

3. Real API Gateway using Ocelot

For .NET applications, **Ocelot** is a commonly used API Gateway library.

Install:

```
dotnet add package Ocelot
```

Configure it in `Program.cs`:

```
using Ocelot.DependencyInjection;
using Ocelot.Middleware;

var builder = WebApplication.CreateBuilder(args);

builder.Configuration.AddJsonFile(
    "ocelot.json",
    optional: false,
    reloadOnChange: true);

builder.Services.AddOcelot(builder.Configuration);

var app = builder.Build();

await app.UseOcelot();

app.Run();
```

Create:

```
ocelot.json
```

Example:

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/users/{id}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "user-service",
          "Port": 5001
        }
      ],
      "UpstreamPathTemplate": "/gateway/users/{id}",
      "UpstreamHttpMethod": [ "Get" ]
    }
  ]
}
```

Now:

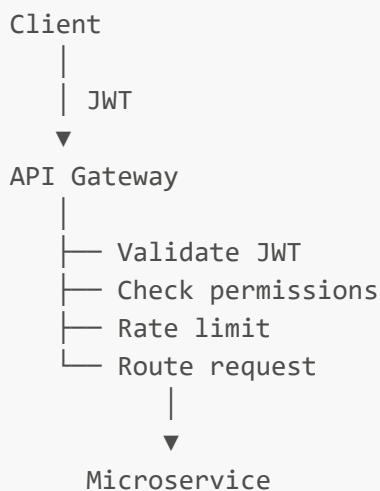
```
GET /gateway/users/10
```

is routed to:

```
http://user-service:5001/api/users/10
```

4. API Gateway with Authentication

A gateway is a good place to validate JWT tokens.



For example:

```
builder.Services
    .AddAuthentication("Bearer")
    .AddJwtBearer("Bearer", options =>
    {
        options.Authority = "https://identity-server";
        options.Audience = "payment-api";
    });
```

Then:

```
app.UseAuthentication();
app.UseAuthorization();
```

The gateway can reject an unauthorized request before it reaches the downstream service.

5. API Gateway for Rate Limiting

This is particularly useful for public APIs.

Example:

```
Client
|
| 10,000 requests
▼
API Gateway
|
| Rate Limit: 100/minute
▼
Payment Service
```

The gateway might return:

```
HTTP/1.1 429 Too Many Requests
```

instead of allowing excessive traffic to reach the payment service.

This can help protect against:

- accidental traffic spikes
- abusive clients

- API flooding
- some denial-of-service scenarios

6. API Gateway for Aggregation

One of the most useful features is **request aggregation**.

Suppose the frontend dashboard needs:

```
Customer
Account
Rewards
Transactions
```

Without aggregation:

```
Frontend
|
|→ Customer Service
|→ Account Service
|→ Rewards Service
|→ Transaction Service
```

With a gateway:

```
Frontend
|
▼
API Gateway
|
|→ Customer
|→ Account
|→ Rewards
|→ Transactions
```

The gateway returns:

```
{
  "customer": {
    "id": 10,
    "name": "John"
  },
  "account": {
    "balance": 50000
  }
}
```

```
    },  
    "rewards": {  
      "points": 1250  
    },  
  },  
  "transactions": [  
    {  
      "amount": 500  
    }  
  ]  
}
```

This reduces the number of calls the frontend needs to make.

7. Parallel aggregation in C#

You can execute downstream requests concurrently:

```
[HttpGet("dashboard/{customerId}")]  
public async Task<IActionResult> Dashboard(int customerId)  
{  
    var customerTask =  
        _httpClient.GetStringAsync(  
            $"http://customer-service/api/customers/{customerId}");  
  
    var accountTask =  
        _httpClient.GetStringAsync(  
            $"http://account-service/api/accounts/{customerId}");  
  
    var rewardsTask =  
        _httpClient.GetStringAsync(  
            $"http://rewards-service/api/rewards/{customerId}");  
  
    await Task.WhenAll(  
        customerTask,  
        accountTask,  
        rewardsTask);  
  
    return Ok(new  
    {  
        Customer = await customerTask,  
        Account = await accountTask,  
        Rewards = await rewardsTask  
    });  
}
```

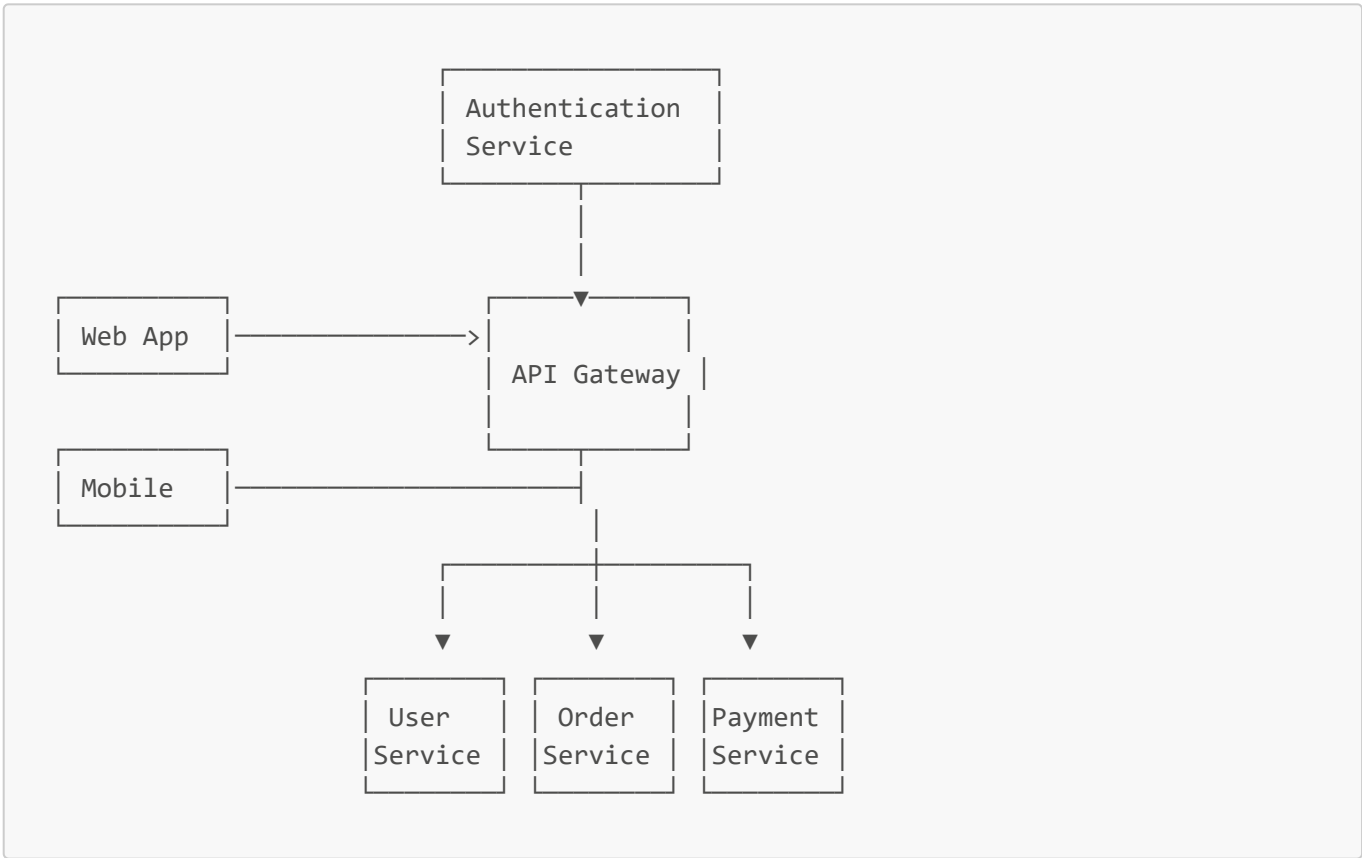
This is much better than:

```
var customer = await GetCustomer();
var account = await GetAccount();
var rewards = await GetRewards();
```

because the independent requests can execute concurrently.

8. API Gateway in a Microservices Architecture

A typical architecture could be:



The gateway becomes the **entry point into the microservices ecosystem**.

9. What should an API Gateway handle?

Common responsibilities:

Responsibility	Example
Routing	/users → User Service
Authentication	Validate JWT
Authorization	Check roles/scopes
Rate limiting	100 requests/minute

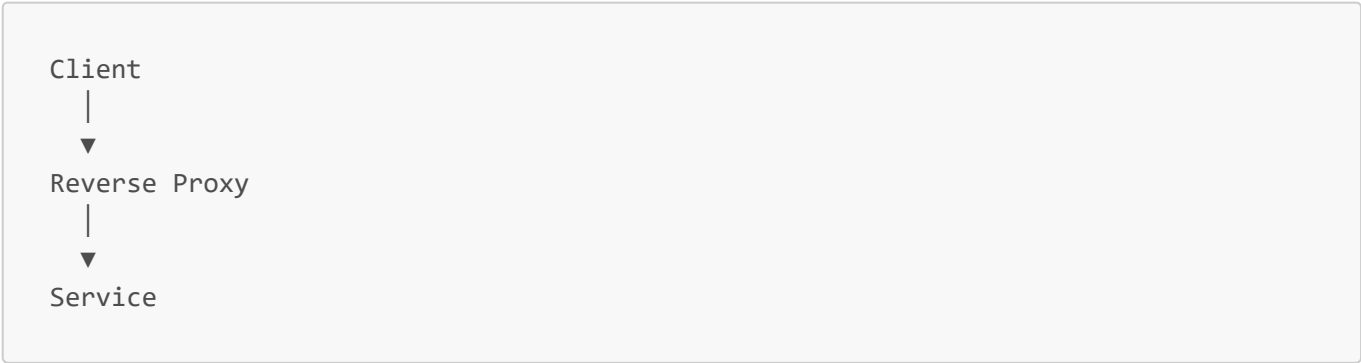
Responsibility	Example
Logging	Request/response logging
Monitoring	Metrics/tracing
Caching	Cache frequently requested data
Load balancing	Distribute requests
SSL/TLS termination	HTTPS handling
Request transformation	Modify headers/body
Response transformation	Combine/modify responses
Aggregation	Combine multiple services
Versioning	/v1/users, /v2/users
Resilience	Timeout/retry/circuit breaker

10. API Gateway vs Reverse Proxy

They are related but not identical.

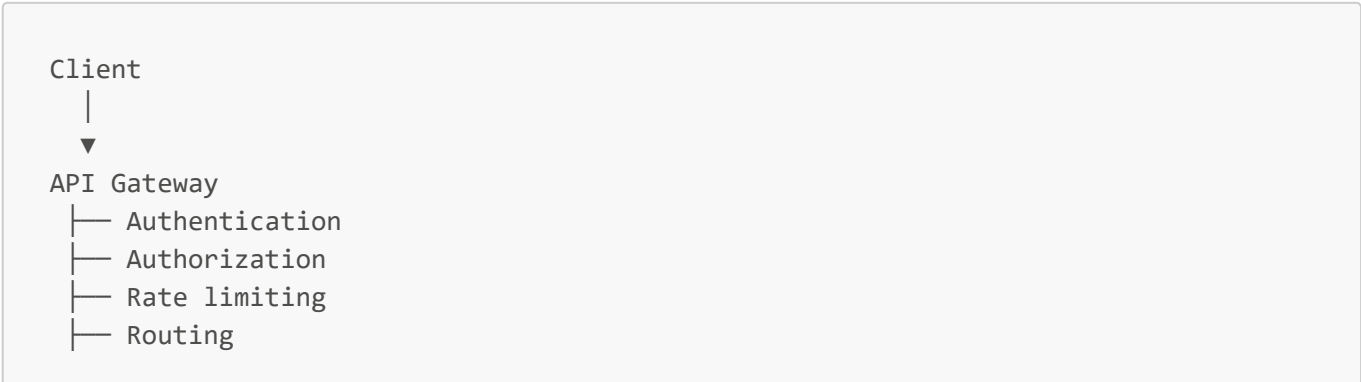
Reverse Proxy

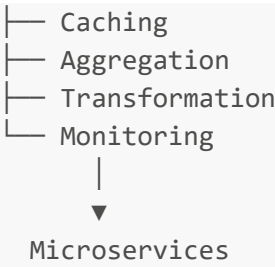
Primarily forwards traffic:



API Gateway

Usually provides additional API-specific capabilities:





Technologies such as **YARP** can be used to build high-performance reverse-proxy/gateway infrastructure in .NET.

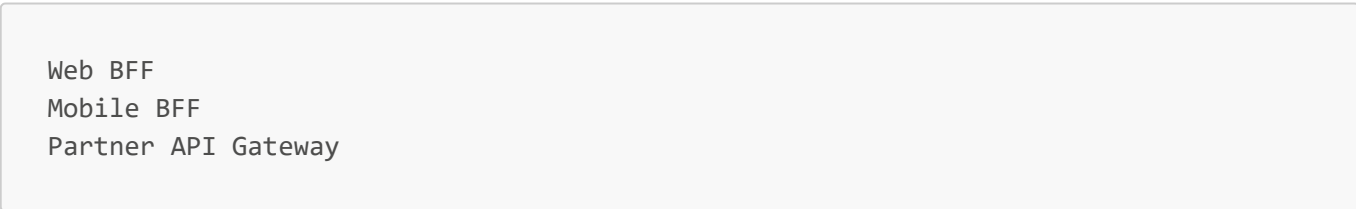
11. API Gateway vs BFF

A **Backend for Frontend (BFF)** is a specialized form of gateway designed for a particular client.

For example:



You might have:

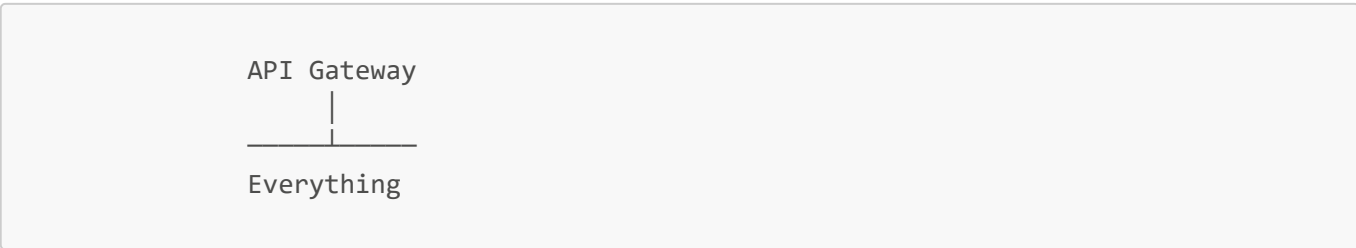


because each client may need different APIs and response shapes.

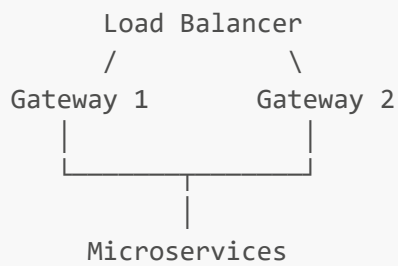
12. Important disadvantage

The API Gateway can become a **single bottleneck** or **single point of failure** if designed badly.

Bad:



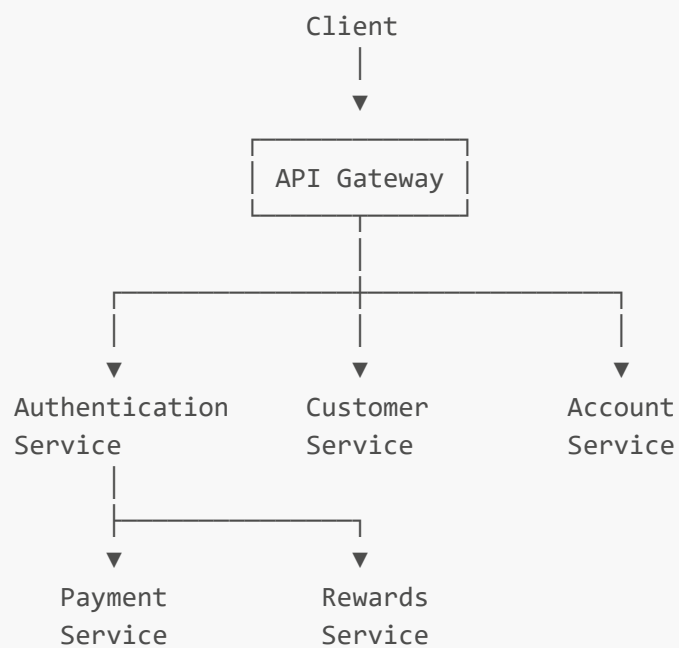
Better:



Run multiple gateway instances and put them behind a load balancer.

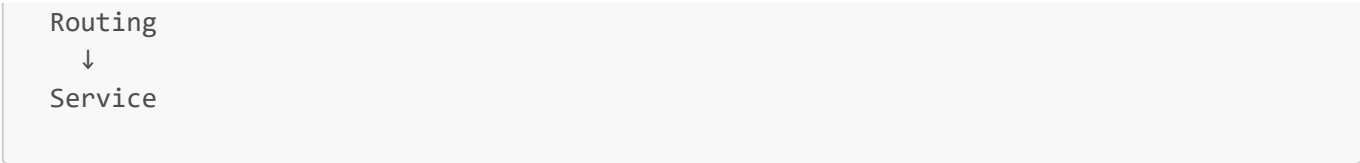
13. API Gateway in your .NET/fintech architecture

For a fintech-style system, you could have:

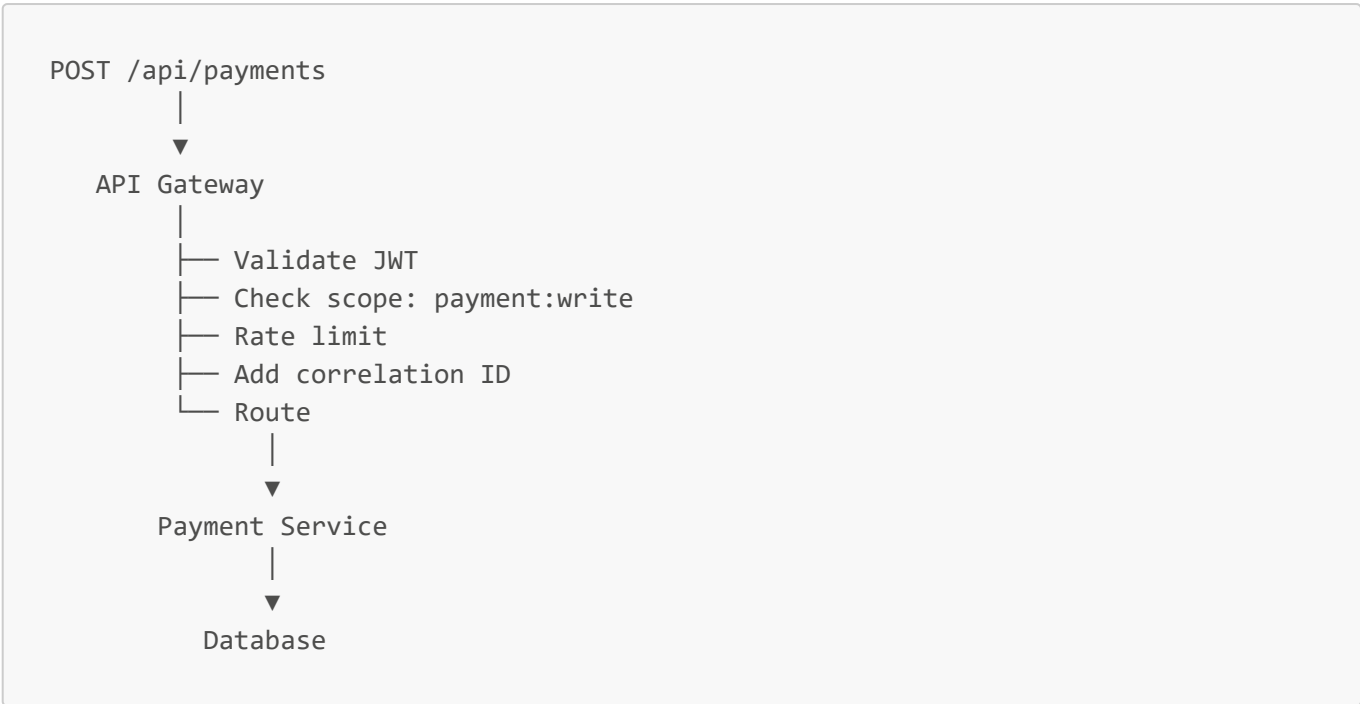


The gateway can perform:

```
HTTPS
↓
JWT validation
↓
Rate limiting
↓
Authorization
↓
Request logging
↓
```



For example:



When to use it

Use an API Gateway when you have **multiple backend services**, especially microservices, and need centralized concerns such as routing, authentication, rate limiting, observability, or aggregation.

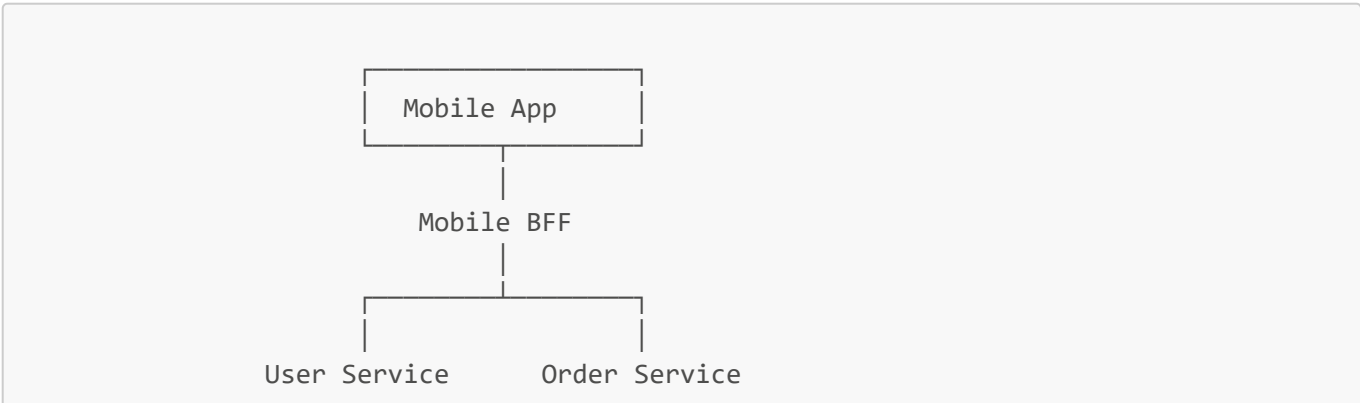
For a simple monolithic ASP.NET Core application, an API Gateway is often **unnecessary complexity**.

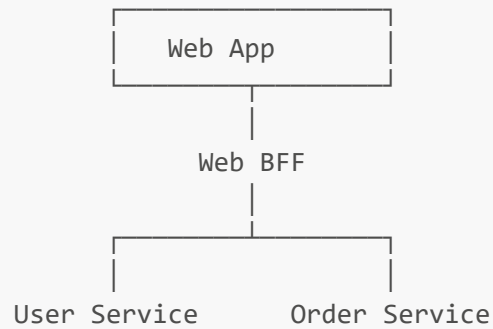
2. Backend for Frontend (BFF)

Backend for Frontend (BFF) Pattern in C#

The **Backend for Frontend (BFF)** pattern creates a **separate backend/API layer specifically for each type of frontend**.

For example:





Instead of making the frontend communicate directly with many microservices, the BFF acts as an **API layer tailored to that frontend**.

1. Why use BFF?

Imagine your system has:

```
User Service
Order Service
Payment Service
Notification Service
Product Service
```

A mobile application might need:

```
GET /users/123
GET /orders/123
GET /products/456
GET /notifications/123
```

That's a lot of network requests.

With BFF:

```
Mobile App
  |
  | GET /mobile/home
  ↓
Mobile BFF
  |
  ├── User Service
  ├── Order Service
  ├── Product Service
  └── Notification Service
```

The mobile application makes **one request**, while the BFF orchestrates the backend calls.

2. Simple ASP.NET Core example

Suppose we have:

```
Frontend
  ↓
BFF
  ↓
User API
Order API
Product API
```

User service

```
public record UserDto(
    int Id,
    string Name,
    string Email
);
```

Order service

```
public record OrderDto(
    int Id,
    decimal Total,
    string Status
);
```

3. BFF endpoint

```
[ApiController]
[Route("api/mobile")]
public class MobileBffController : ControllerBase
{
    private readonly IUserService _userService;
    private readonly IOrderService _orderService;

    public MobileBffController(
```

```

        IUserService userService,
        IOrderService orderService)
    {
        _userService = userService;
        _orderService = orderService;
    }

    [HttpGet("home")]
    public async Task<IActionResult> GetHome()
    {
        var user = await _userService.GetUserAsync(123);
        var orders = await _orderService.GetOrdersAsync(123);

        return Ok(new
        {
            User = user,
            Orders = orders
        });
    }
}

```

The mobile application only needs:

```
GET /api/mobile/home
```

The BFF internally calls:

```
User Service
Order Service
```

and combines their responses.

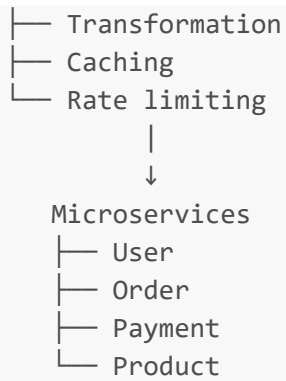
4. A better architecture

In a real application, avoid putting all orchestration directly inside the controller.

```

Mobile App
  |
  ↓
Mobile BFF
  |
  ├── Authentication
  ├── Authorization
  ├── Request validation
  └── Aggregation

```



For example:

```

public interface IMobileHomeService
{
    Task<MobileHomeResponse> GetHomeAsync(int userId);
}

```

Implementation:

```

public class MobileHomeService : IMobileHomeService
{
    private readonly IUserService _userService;
    private readonly IOrderService _orderService;

    public MobileHomeService(
        IUserService userService,
        IOrderService orderService)
    {
        _userService = userService;
        _orderService = orderService;
    }

    public async Task<MobileHomeResponse> GetHomeAsync(int userId)
    {
        var userTask = _userService.GetUserAsync(userId);
        var ordersTask = _orderService.GetOrdersAsync(userId);

        await Task.WhenAll(userTask, ordersTask);

        return new MobileHomeResponse
        {
            User = await userTask,
            Orders = await ordersTask
        };
    }
}

```


DTO:

```
public class MobileHomeResponse
{
    public UserDto User { get; set; } = default!;
    public IEnumerable<OrderDto> Orders { get; set; }
        = Enumerable.Empty<OrderDto>();
}
```

Controller:

```
[ApiController]
[Route("api/mobile")]
public class MobileController : ControllerBase
{
    private readonly IMobileHomeService _homeService;

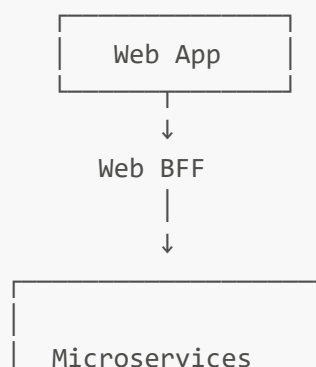
    public MobileController(IMobileHomeService homeService)
    {
        _homeService = homeService;
    }

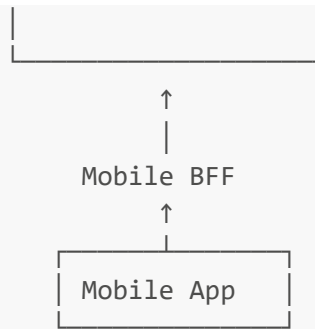
    [HttpGet("home/{userId}")]
    public async Task<IActionResult> Home(int userId)
    {
        var response =
            await _homeService.GetHomeAsync(userId);

        return Ok(response);
    }
}
```

5. Different BFFs for different clients

This is where the pattern becomes particularly useful.





The web application may need:

```
{
  "user": {},
  "orders": [],
  "recommendations": [],
  "recentPayments": []
}
```

Mobile might need a smaller response:

```
{
  "user": {},
  "orders": []
}
```

The BFF can tailor the response to each client.

6. BFF with HttpClient

Often the BFF calls actual microservice APIs.

Register clients:

```
builder.Services.AddHttpClient<IUserApiClient, UserApiClient>(
    client =>
    {
        client.BaseAddress =
            new Uri("https://user-service");
    });

builder.Services.AddHttpClient<IOrderApiClient, OrderApiClient>(
    client =>
    {
        client.BaseAddress =
```

```
        new Uri("https://order-service");  
    });
```

User API client:

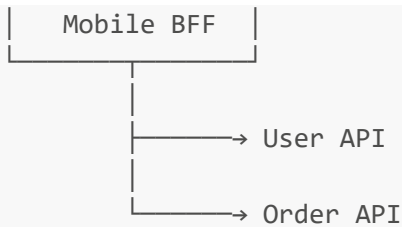
```
public class UserApiClient : IUserApiClient  
{  
    private readonly HttpClient _httpClient;  
  
    public UserApiClient(HttpClient httpClient)  
    {  
        _httpClient = httpClient;  
    }  
  
    public async Task<UserDto?> GetUserAsync(int id)  
    {  
        return await _httpClient  
            .GetFromJsonAsync<UserDto>($"api/users/{id}");  
    }  
}
```

Order API client:

```
public class OrderApiClient : IOrderApiClient  
{  
    private readonly HttpClient _httpClient;  
  
    public OrderApiClient(HttpClient httpClient)  
    {  
        _httpClient = httpClient;  
    }  
  
    public async Task<List<OrderDto?>> GetOrdersAsync(int userId)  
    {  
        return await _httpClient  
            .GetFromJsonAsync<List<OrderDto>>(  
                $"api/orders/user/{userId}");  
    }  
}
```

Then:





7. BFF can transform data

One important responsibility of a BFF is **response transformation**.

Microservice:

```
{
  "firstName": "John",
  "lastName": "Smith",
  "emailAddress": "john@example.com"
}
```

Mobile BFF can return:

```
{
  "name": "John Smith",
  "email": "john@example.com"
}
```

The frontend doesn't need to understand the internal microservice model.

```
return new MobileUserResponse
{
    Name = $"{user.FirstName} {user.LastName}",
    Email = user.EmailAddress
};
```

This also prevents internal backend DTOs from leaking into your frontend contract.

8. Parallel calls

BFFs frequently aggregate multiple services.

Don't unnecessarily do this:

```
var user = await GetUserAsync();  
var orders = await GetOrdersAsync();  
var products = await GetProductsAsync();
```

These calls execute sequentially.

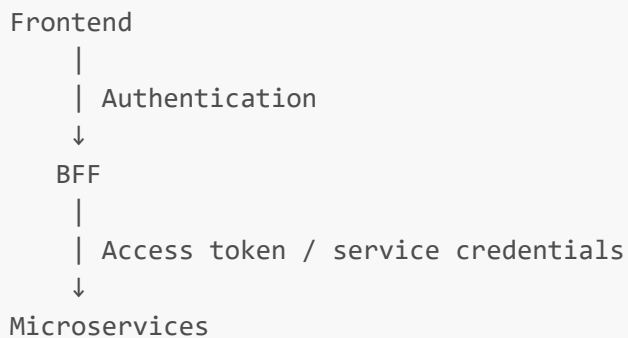
Instead:

```
var userTask = GetUserAsync();  
var ordersTask = GetOrdersAsync();  
var productsTask = GetProductsAsync();  
  
await Task.WhenAll(  
    userTask,  
    ordersTask,  
    productsTask);  
  
var response = new  
{  
    User = await userTask,  
    Orders = await ordersTask,  
    Products = await productsTask  
};
```

This can significantly reduce overall latency when the calls are independent.

9. BFF authentication

A BFF can also handle authentication.



For example:

```
[Authorize]  
[HttpGet("home")]
```

```
public async Task<IActionResult> Home()
{
    ...
}
```

The BFF validates the user's authentication and can propagate the appropriate identity/token to downstream services.

For browser applications, a BFF can also be used with a **server-side session/cookie approach**, reducing the amount of token-handling logic exposed to browser JavaScript.

10. BFF vs API Gateway

These are related but different concepts.

API Gateway

Usually provides infrastructure-level concerns:

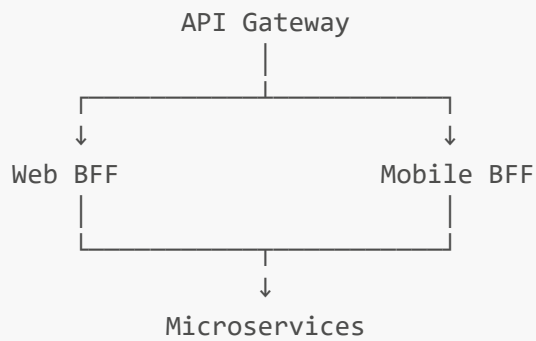
```
Client
  ↓
API Gateway
  ├── Routing
  ├── Authentication
  ├── Rate limiting
  ├── Logging
  └── Load balancing
      ↓
Microservices
```

BFF

Provides **frontend-specific backend behavior**:

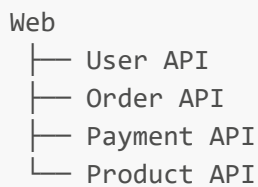
```
Mobile
  ↓
Mobile BFF
  ├── Aggregate APIs
  ├── Transform responses
  ├── Mobile-specific logic
  └── Mobile-specific caching
      ↓
Microservices
```

They can be combined:

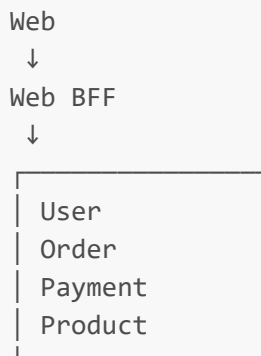


11. BFF vs traditional backend

Without BFF:



With BFF:



The frontend has a **simpler API contract**.

12. Advantages

1. Frontend-specific APIs

Mobile and web can have different requirements.

2. API aggregation

One request can retrieve data from multiple services.

3. Reduced frontend complexity

The frontend doesn't need to understand your entire microservice architecture.

4. Response optimization

The BFF can return only the data needed by the client.

5. Security boundary

The BFF can centralize authentication and authorization-related concerns.

6. Backend independence

Microservices can change internally without forcing every frontend to change.

13. Disadvantages

Additional service

You now have:

```
Web BFF
Mobile BFF
```

instead of directly calling APIs.

More deployment complexity

Each BFF needs:

- deployment
- monitoring
- logging
- scaling
- testing

Possible business-logic duplication

Be careful not to put core business rules into BFFs.

A good rule is:

```
BFF
↓
```



```
Orchestration / presentation logic
```

```
Domain Service
```

```
↓
```

```
Business rules
```

14. What should NOT go into a BFF?

Avoid putting important business rules here:

```
// Bad BFF responsibility
if (user.Age < 18)
{
    // Complex business decision
}
```

Core business logic should generally remain in the domain/application services.

The BFF should primarily handle:

```
Client-specific API contract
    ↓
Aggregation
    ↓
Transformation
    ↓
Authentication boundary
    ↓
Calling backend services
```

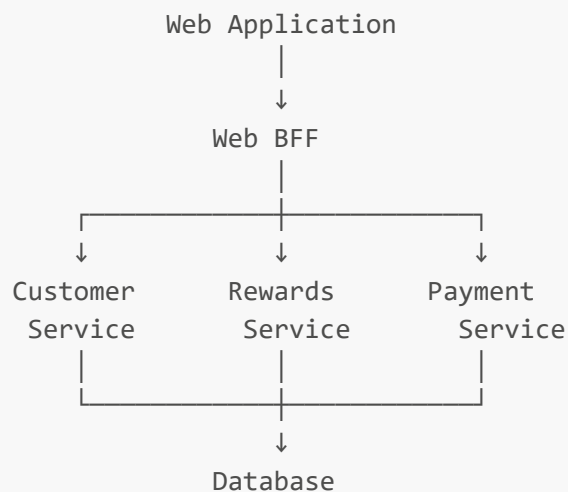
15. Typical ASP.NET Core project structure

```
MobileBff/
|
|— Controllers/
|   └─ HomeController.cs
|
|— Services/
|   └─ MobileHomeService.cs
|
|— Clients/
|   └─ UserApiClient.cs
```

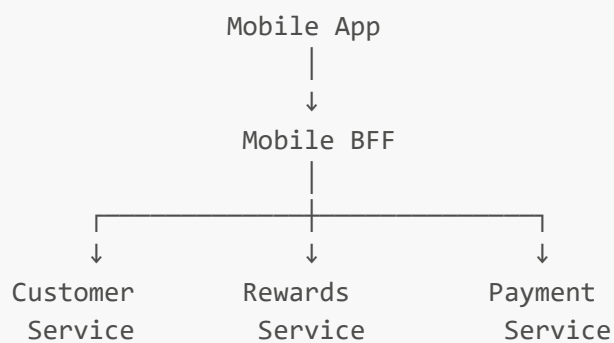
```
|
|   | OrderApiClient.cs
|   | ProductApiClient.cs
|
|   | DTOS/
|   |   | UserDto.cs
|   |   | OrderDto.cs
|   |   | MobileHomeResponse.cs
|
|   | Authentication/
|
|   | Middleware/
|
|   | Program.cs
|   | appsettings.json
```

16. BFF in a real fintech architecture

For a fintech application, you could have:



For mobile:



The mobile BFF might expose:

```
GET /api/mobile/dashboard
```

and return:

```
{
  "customer": {
    "name": "John"
  },
  "rewards": {
    "points": 1250
  },
  "payment": {
    "balance": 50000
  }
}
```

Internally it could make three service calls.

BFF in one sentence

Backend for Frontend is a client-specific backend layer that aggregates, transforms, and orchestrates backend services to provide an API tailored to a particular frontend.

For a **C#/.NET backend interview**, remember this distinction:

```
API Gateway
  = "How do I route/protect backend APIs?"

BFF
  = "What API does this particular frontend need?"

Microservice
  = "What business capability does the system provide?"
```

And a common production architecture is:

```
Mobile App → Mobile BFF
Web App → Web BFF
Admin UI → Admin BFF
└──┬──┘
    |
    | → API Gateway → Microservices
```

3. CQRS

4. Event Sourcing

Event Sourcing Design Pattern in C#

Event Sourcing is an architectural/design pattern where you **store every change to an entity as an event**, rather than storing only its current state.

Instead of:

```
Account
-----
Balance = ₹5,000
```

you store the history:

```
AccountOpened    ₹10,000
MoneyDeposited   ₹2,000
MoneyWithdrawn   ₹3,000
```

The current balance is reconstructed by replaying these events.

1. Core idea

Traditional approach:

```
Command
  ↓
Update Database
  ↓
Current State
```

Event Sourcing:

```
Command
  ↓
Domain Logic
  ↓
Event
  ↓
Event Store
  ↓
Replay Events
  ↓
Current State
```

The **event store becomes the source of truth**.

2. Example: Bank Account

Suppose a customer performs:

Open Account	₹10,000
Deposit	₹5,000
Withdraw	₹2,000

Instead of updating:

Balance = 13,000

we store:

AccountOpened
MoneyDeposited
MoneyWithdrawn

Each event represents something that **already happened**.

3. Events in C#

Define an event interface:

```
public interface IDomainEvent
{
    Guid EventId { get; }
    DateTime OccurredOn { get; }
}
```

Create events:

```
public record AccountOpened(
    Guid EventId,
    Guid AccountId,
```

```
    decimal InitialBalance,  
    DateTime OccurredOn) : IDomainEvent;  
  
public record MoneyDeposited(  
    Guid EventId,  
    Guid AccountId,  
    decimal Amount,  
    DateTime OccurredOn) : IDomainEvent;  
  
public record MoneyWithdrawn(  
    Guid EventId,  
    Guid AccountId,  
    decimal Amount,  
    DateTime OccurredOn) : IDomainEvent;
```

Notice the naming:

```
MoneyDeposited  
MoneyWithdrawn  
AccountOpened
```

These are past-tense statements because they represent facts.

4. Aggregate

The aggregate contains the business rules and can rebuild itself from events.

```
public class BankAccount  
{  
    public Guid Id { get; private set; }  
    public decimal Balance { get; private set; }  
  
    private readonly List<IDomainEvent> _events = new();  
  
    public IReadOnlyCollection<IDomainEvent> Events => _events;  
  
    public void Open(Guid accountId, decimal initialBalance)  
    {  
        if (initialBalance < 0)  
            throw new InvalidOperationException(  
                "Initial balance cannot be negative.");  
  
        var @event = new AccountOpened(  
            Guid.NewGuid(),  
            accountId,  
            initialBalance,  
            DateTime.UtcNow);
```

```

        Apply(@event);

        _events.Add(@event);
    }

    public void Deposit(decimal amount)
    {
        if (amount <= 0)
            throw new InvalidOperationException(
                "Deposit must be greater than zero.");

        var @event = new MoneyDeposited(
            Guid.NewGuid(),
            Id,
            amount,
            DateTime.UtcNow);

        Apply(@event);

        _events.Add(@event);
    }

    public void Withdraw(decimal amount)
    {
        if (amount <= 0)
            throw new InvalidOperationException(
                "Withdrawal must be greater than zero.");

        if (Balance < amount)
            throw new InvalidOperationException(
                "Insufficient balance.");

        var @event = new MoneyWithdrawn(
            Guid.NewGuid(),
            Id,
            amount,
            DateTime.UtcNow);

        Apply(@event);

        _events.Add(@event);
    }

```

Then define event application:

```

private void Apply(IDomainEvent @event)
{
    switch (@event)
    {
        case AccountOpened e:
            Id = e.AccountId;

```

```

        Balance = e.InitialBalance;
        break;

    case MoneyDeposited e:
        Balance += e.Amount;
        break;

    case MoneyWithdrawn e:
        Balance -= e.Amount;
        break;
    }
}

```

5. Rebuilding the aggregate

This is one of the most important concepts in Event Sourcing.

Suppose the event store contains:

AccountOpened	₹10,000
MoneyDeposited	₹5,000
MoneyWithdrawn	₹2,000

We replay them:

```

var account = new BankAccount();

foreach (var @event in events)
{
    account.Apply(@event);
}

```

Conceptually:

```

Initial State
Balance = 0

    ↓ AccountOpened ₹10,000

Balance = ₹10,000

    ↓ MoneyDeposited ₹5,000

Balance = ₹15,000

```


↓ MoneyWithdrawn ₹2,000

Balance = ₹13,000

The current state is therefore:

Balance = ₹13,000

6. Event Store

A very simple in-memory event store:

```
public interface IEventStore
{
    Task AppendAsync(
        Guid aggregateId,
        IEnumerable<IDomainEvent> events);

    Task<IReadOnlyList<IDomainEvent>> GetEventsAsync(
        Guid aggregateId);
}
```

Implementation:

```
public class InMemoryEventStore : IEventStore
{
    private readonly Dictionary<Guid, List<IDomainEvent>> _store = new();

    public Task AppendAsync(
        Guid aggregateId,
        IEnumerable<IDomainEvent> events)
    {
        if (!_store.ContainsKey(aggregateId))
            _store[aggregateId] = new List<IDomainEvent>();

        _store[aggregateId].AddRange(events);

        return Task.CompletedTask;
    }

    public Task<IReadOnlyList<IDomainEvent>> GetEventsAsync(
        Guid aggregateId)
    {
        if (!_store.TryGetValue(aggregateId, out var events))
            return Task.FromResult<IReadOnlyList<IDomainEvent>>(<IReadOnlyList<IDomainEvent>>());

        return Task.FromResult<IReadOnlyList<IDomainEvent>>(<IReadOnlyList<IDomainEvent>>().Concat(events).ToList());
    }
}
```

```
        return Task.FromResult<IReadOnlyList<IDomainEvent>>(
            new List<IDomainEvent>());

        return Task.FromResult<IReadOnlyList<IDomainEvent>>(
            events);
    }
}
```

In a real application, the event store could be backed by:

- SQL Server
- PostgreSQL
- EventStoreDB
- MongoDB
- Cosmos DB
- Kafka + persistent event storage

7. Saving events

Example application service:

```
public class AccountService
{
    private readonly IEventStore _eventStore;

    public AccountService(IEventStore eventStore)
    {
        _eventStore = eventStore;
    }

    public async Task CreateAccount(
        Guid accountId,
        decimal initialBalance)
    {
        var account = new BankAccount();

        account.Open(accountId, initialBalance);

        await _eventStore.AppendAsync(
            accountId,
            account.Events);
    }
}
```

For deposit:

```

public async Task Deposit(
    Guid accountId,
    decimal amount)
{
    var events =
        await _eventStore.GetEventsAsync(accountId);

    var account = RebuildAccount(events);

    account.Deposit(amount);

    await _eventStore.AppendAsync(
        accountId,
        account.Events);
}

```

The important idea is:

```

Load events
  ↓
Rebuild aggregate
  ↓
Execute command
  ↓
Generate new events
  ↓
Append events

```

8. Event Sourcing vs traditional CRUD

Traditional CRUD

Database:

```

Accounts
-----
Id      Balance
1       13000

```

If the balance changes:

```

UPDATE Accounts
SET Balance = 13000
WHERE Id = 1;

```

The previous state is lost unless you have separate auditing.

Event Sourcing

Database:

Events

```
-----  
AccountOpened      ₹10,000  
MoneyDeposited     ₹5,000  
MoneyWithdrawn     ₹2,000
```

Nothing is overwritten.

You have the complete history.

9. Why Event Sourcing is useful

1. Complete audit history

You know exactly what happened.

```
10:00 AccountOpened ₹10,000  
10:05 MoneyDeposited ₹5,000  
10:15 MoneyWithdrawn ₹2,000
```

This is particularly useful in:

- Banking
- Fintech
- Payments
- Insurance
- Trading
- Order management

2. Time travel

You can reconstruct the state at a previous point in time.

For example:

```
Current balance = ₹13,000
```

But you can ask:

What was the balance yesterday at 10 AM?

Replay only events that occurred before that time.

3. Debugging

If something goes wrong, you can inspect the exact sequence:

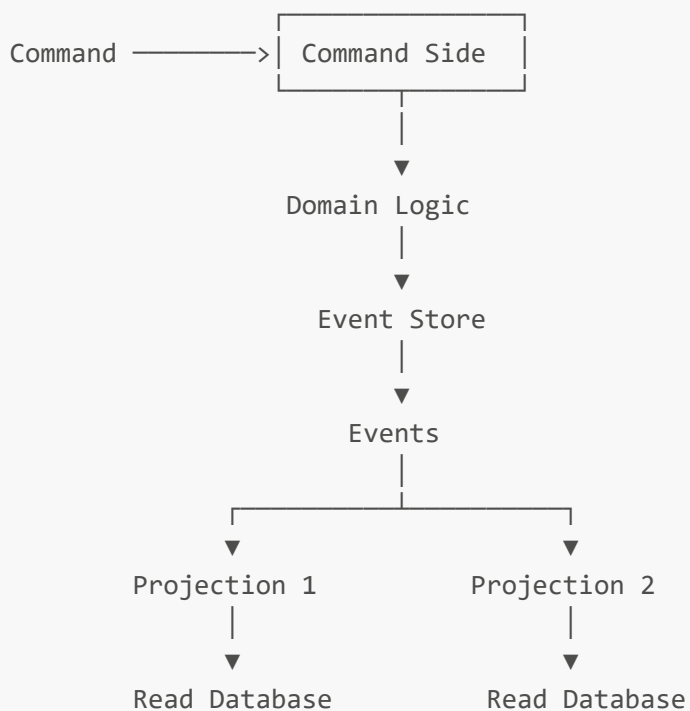
```
OrderCreated
PaymentInitiated
PaymentFailed
PaymentRetried
PaymentSucceeded
OrderConfirmed
```

This is much easier to investigate than a single row saying:

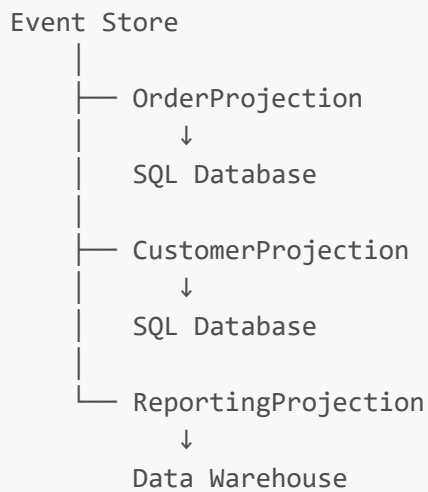
```
OrderStatus = Completed
```

10. Event Sourcing + CQRS

Event Sourcing is frequently combined with **CQRS**.



For example:



The same events can produce multiple read models.

11. Projection

A projection converts events into a read-friendly model.

For example:

```
public class AccountProjection
{
    public Guid AccountId { get; set; }
    public decimal Balance { get; set; }
}
```

When:

MoneyDeposited

arrives:

```
projection.Balance += @event.Amount;
```

When:

MoneyWithdrawn

arrives:

projection.Balance -= @event.Amount;

The projection database might contain:

AccountId	Balance

A1	₹13,000

The event store still contains:

AccountOpened
MoneyDeposited
MoneyWithdrawn

12. Event Store vs State Store

State store

Stores:

Current state

Example:

Balance = ₹13,000

Event store

Stores:

History of state changes

Example:

```
AccountOpened ₹10,000
MoneyDeposited ₹5,000
MoneyWithdrawn ₹2,000
```

The difference is fundamental:

```
CRUD:
"Here is what the entity looks like now."

Event Sourcing:
"Here is everything that happened to the entity."
```

13. Important concept: Events are immutable

Once an event has been stored:

```
MoneyDeposited ₹5,000
```

you generally **do not modify it**.

You append another event instead.

Bad:

```
MoneyDeposited ₹5,000
  ↓
change it to ₹7,000
```

Good:

```
MoneyDeposited ₹5,000
DepositCorrected ₹2,000
```

The history remains intact.

14. Event versioning

Your event schema will evolve.

Version 1:

```
public record MoneyDepositedV1(  
    Guid AccountId,  
    decimal Amount);
```

Later:

```
public record MoneyDepositedV2(  
    Guid AccountId,  
    decimal Amount,  
    string Currency);
```

You need strategies for handling old events:

- Upcasting
- Event versioning
- Backward compatibility
- Schema evolution

This is one of the major complexities of Event Sourcing.

15. Snapshots

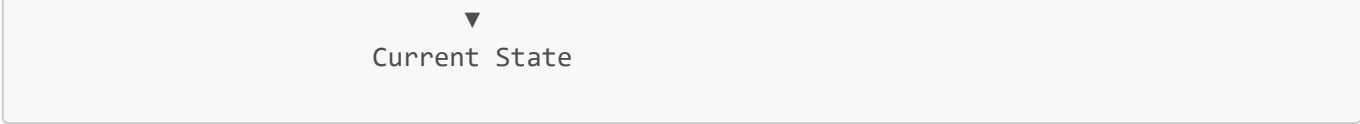
Imagine an account has:

10 million events

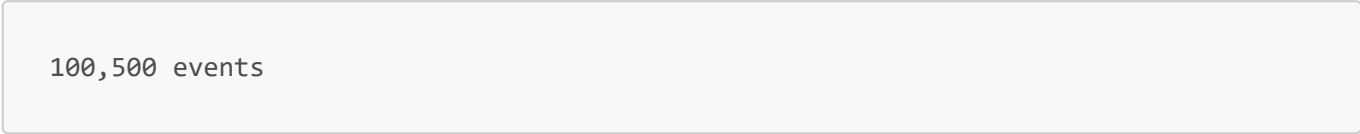
Replaying all of them every time would be expensive.

Use a snapshot:

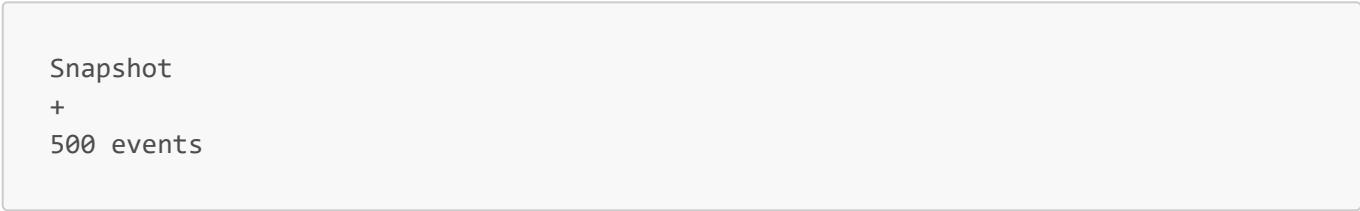




Instead of replaying:



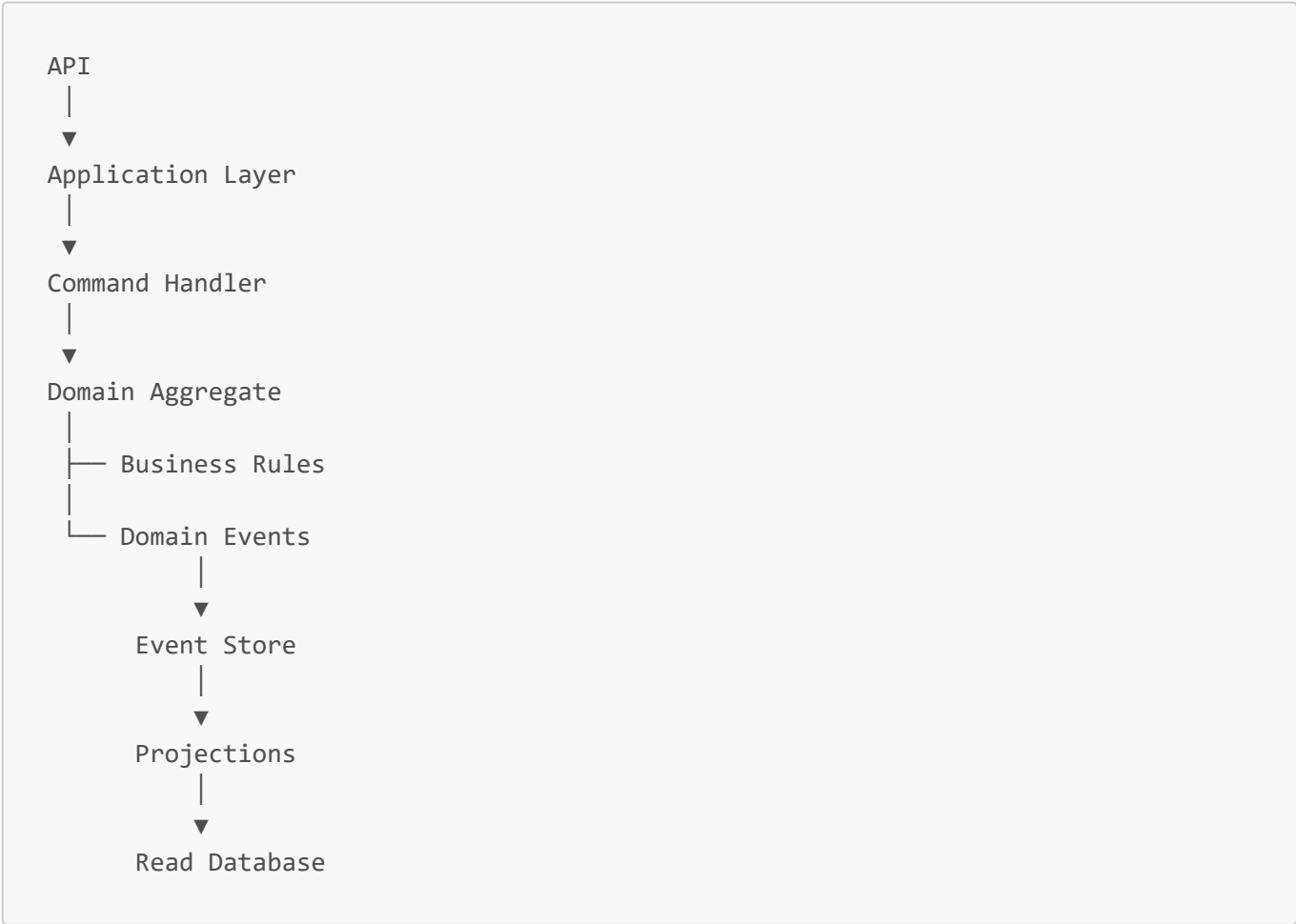
you load:



This makes aggregate reconstruction much faster.

16. Event Sourcing in an ASP.NET Core application

A typical architecture might look like:



Example:



17. Advantages

Advantage	Explanation
Auditability	Complete history
Debugging	See exactly what happened
Time travel	Reconstruct previous states
Multiple read models	Events can feed many projections
Scalability	CQRS/read projections can scale independently
Recovery	State can be rebuilt from events
Business history	Domain events represent business facts

18. Disadvantages

Event Sourcing is **not automatically better than CRUD**.

Major disadvantages:

Complexity

You introduce:

Event store
Event handlers

```
Projections
Snapshots
Event versioning
Replay mechanisms
Consistency handling
```

Storage growth

Events are never normally deleted or overwritten, so storage grows.

Eventual consistency

With CQRS/projections:

```
Command
  ↓
Event Store
  ↓
Projection
  ↓
Read DB
```

The read model may temporarily lag behind the event store.

Harder schema changes

Changing an ordinary SQL table can be relatively simple.

Changing millions of historical events can be much more complicated.

19. When should you use Event Sourcing?

Good candidates:

```
Banking
Payment systems
Financial transactions
Trading
Order management
Inventory
Insurance
Workflow systems
Audit-heavy applications
```

Especially when:

The history of changes is as important as the current state.

Avoid it for simple CRUD applications such as:

```
Basic employee management
Simple blog
Contact management
Simple admin dashboard
Basic configuration management
```

For those, normal EF Core + SQL CRUD is often much simpler.

20. Event Sourcing vs CQRS

They are related but **not the same thing**.

CQRS

Separates:

```
Commands
```

from:

```
Queries
```

Event Sourcing

Stores:

```
Events
```

as the source of truth.

You can have:

```
CQRS without Event Sourcing
```

and:

Event Sourcing without full CQRS

But they are frequently combined.

Interview definition

If asked "**What is Event Sourcing?**", a strong answer is:

Event Sourcing is a pattern where an application's state is persisted as a sequence of immutable domain events rather than storing only the latest state. The current state can be reconstructed by replaying those events. It provides a complete audit history and works particularly well with CQRS, but introduces complexity around event versioning, projections, snapshots, and eventual consistency.

The mental model to remember

Traditional CRUD

```
graph TD
    Entity --> CurrentState[Current State]
    CurrentState --> Database
```

Event Sourcing

```
graph TD
    Command --> DomainAggregate[Domain Aggregate]
    DomainAggregate --> Event
    Event --> EventStore[Event Store]
    EventStore --> EventHistory[Event History]
    EventHistory --> ReplayProjection[Replay / Projection]
    ReplayProjection --> CurrentReadModel[Current Read Model]
```

The key idea: Don't store only what the object is. Store what happened to the object.

5. Saga

Saga Design Pattern in C#

The **Saga Pattern** is a distributed transaction pattern used in **microservices** when a business operation spans multiple services and you cannot use a single database transaction across all of them.

Instead of:

One big transaction → commit everything or rollback everything

Saga uses:

A sequence of local transactions + compensating actions if something fails.

1. Real-world example

Imagine an e-commerce order:

```
Create Order
  ↓
Reserve Inventory
  ↓
Process Payment
  ↓
Create Shipment
```

These operations might belong to different microservices:

```
Order Service
Inventory Service
Payment Service
Shipping Service
```

You cannot easily do:

```
BEGIN TRANSACTION
  Order DB
  Inventory DB
  Payment DB
  Shipping DB
COMMIT
```

because each service owns its own database.

Instead, Saga does:

```
Create Order
  ↓
```

```
Reserve Inventory
  ↓
Charge Payment
  ↓
Create Shipment
```

If payment fails:

```
Payment failed
  ↓
Release Inventory
  ↓
Cancel Order
```

The operations `ReleaseInventory()` and `CancelOrder()` are **compensating transactions**.

2. Saga structure

A Saga consists of:

```
Transaction 1 → Transaction 2 → Transaction 3
    ↓             ↓             ↓
Compensate 1 ← Compensate 2 ← Compensate 3
```

For example:

```
CreateOrder
  ↓
ReserveInventory
  ↓
ChargePayment
  ↓
CreateShipment
```

If `ChargePayment` fails:

```
ChargePayment ✕

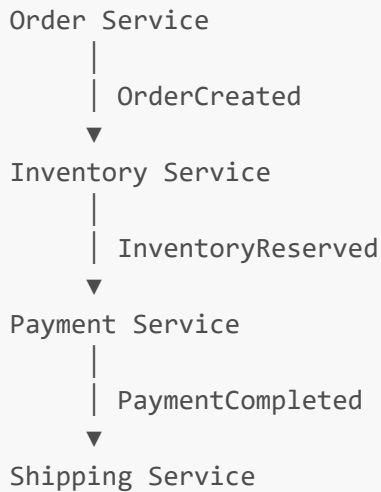
Compensate:
  ReleaseInventory
  CancelOrder
```


3. Two types of Saga

There are two major implementations.

Choreography

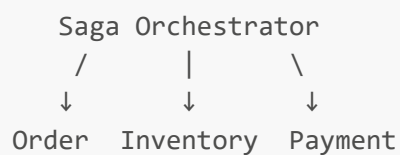
Services communicate through events.



There is **no central coordinator**.

Orchestration

A central **Saga Orchestrator** controls the workflow.



The orchestrator says:

1. Create order
2. Reserve inventory
3. Charge payment
4. Create shipment

If something fails:

Payment failed
↓

```
Orchestrator
  ↓
Release inventory
  ↓
Cancel order
```

For learning and interview purposes, **orchestration is generally easier to understand and demonstrate in C#**.

4. Simple C# Saga example

Let's model an order-processing Saga.

```
public interface ISagaStep
{
    Task ExecuteAsync();
    Task CompensateAsync();
}
```

Create individual steps.

```
public class CreateOrderStep : ISagaStep
{
    public async Task ExecuteAsync()
    {
        Console.WriteLine("Order created");
    }

    public async Task CompensateAsync()
    {
        Console.WriteLine("Order cancelled");
    }
}
```

Inventory:

```
public class ReserveInventoryStep : ISagaStep
{
    public async Task ExecuteAsync()
    {
        Console.WriteLine("Inventory reserved");
    }

    public async Task CompensateAsync()
    {
```

```
        Console.WriteLine("Inventory released");
    }
}
```

Payment:

```
public class ProcessPaymentStep : ISagaStep
{
    public async Task ExecuteAsync()
    {
        Console.WriteLine("Payment processed");
    }

    public async Task CompensateAsync()
    {
        Console.WriteLine("Payment refunded");
    }
}
```

5. Saga orchestrator

Now create the orchestrator.

```
public class OrderSaga
{
    private readonly List<ISagaStep> _steps = new();

    public void AddStep(ISagaStep step)
    {
        _steps.Add(step);
    }

    public async Task ExecuteAsync()
    {
        var completedSteps = new Stack<ISagaStep>();

        try
        {
            foreach (var step in _steps)
            {
                await step.ExecuteAsync();
                completedSteps.Push(step);
            }

            Console.WriteLine("Saga completed successfully.");
        }
        catch
        {
            // Handle exception
        }
    }
}
```

```

        {
            Console.WriteLine("Saga failed. Starting compensation...");

            while (completedSteps.Count > 0)
            {
                var step = completedSteps.Pop();

                await step.CompensateAsync();
            }

            throw;
        }
    }
}

```

Usage:

```

var saga = new OrderSaga();

saga.AddStep(new CreateOrderStep());
saga.AddStep(new ReserveInventoryStep());
saga.AddStep(new ProcessPaymentStep());

await saga.ExecuteAsync();

```

The normal flow is:

```

Create Order
  ↓
Reserve Inventory
  ↓
Process Payment
  ↓
Saga completed

```

6. What happens when something fails?

Suppose:

```

public class ProcessPaymentStep : ISagaStep
{
    public async Task ExecuteAsync()
    {
        Console.WriteLine("Processing payment...");
    }
}

```

```
        throw new Exception("Payment failed");
    }

    public async Task CompensateAsync()
    {
        Console.WriteLine("Refund payment");
    }
}
```

Execution becomes:



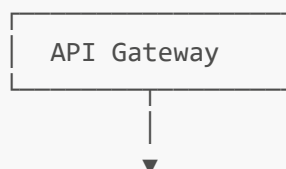
Output conceptually:

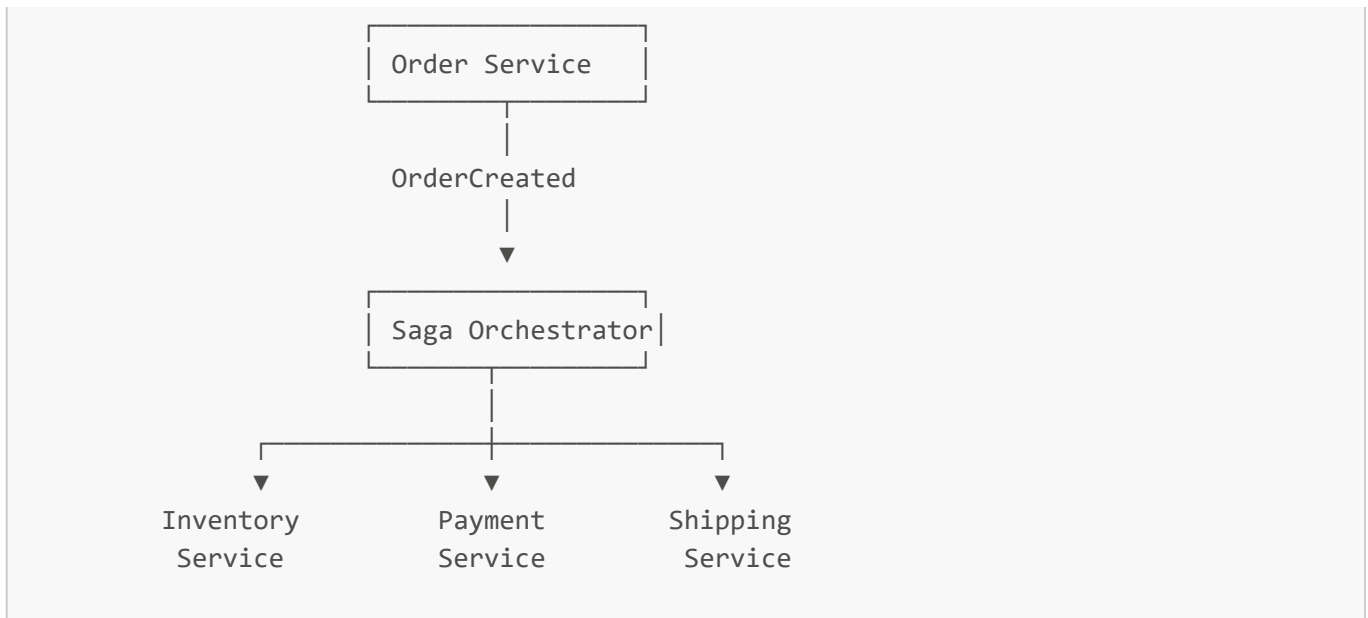
```
Order created
Inventory reserved
Processing payment...
Saga failed. Starting compensation...
Inventory released
Order cancelled
```

Notice that payment's compensation isn't executed because the payment step itself never successfully completed.

7. Saga in microservices

A real system would look more like:





Communication could use:

- RabbitMQ
- Azure Service Bus
- Kafka
- AWS SNS/SQS
- MassTransit
- NServiceBus

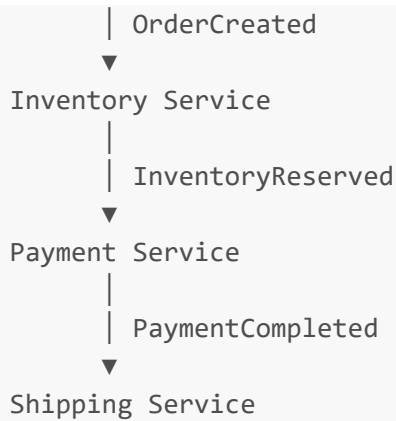
8. Saga with events

For example, define events:

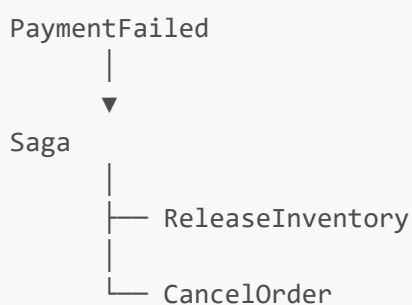
```
public record OrderCreated(  
    Guid OrderId,  
    decimal Amount);  
  
public record InventoryReserved(  
    Guid OrderId);  
  
public record PaymentCompleted(  
    Guid OrderId);  
  
public record PaymentFailed(  
    Guid OrderId);
```

The flow becomes:

```
Order Service  
|
```



If payment fails:



9. Choreography example

In choreography, there isn't a central `OrderSaga` class.

Each service reacts to events.

Order Service

```

public async Task CreateOrderAsync(Order order)
{
    await _repository.AddAsync(order);

    await _eventBus.PublishAsync(
        new OrderCreated(order.Id));
}
  
```

Inventory listens:

```

public async Task Handle(OrderCreated message)
{
    await _inventory.ReserveAsync(message.OrderId);
}
  
```

```
        await _eventBus.PublishAsync(
            new InventoryReserved(message.OrderId));
    }
```

Payment listens:

```
public async Task Handle(InventoryReserved message)
{
    var success = await _payment.ProcessAsync(message.OrderId);

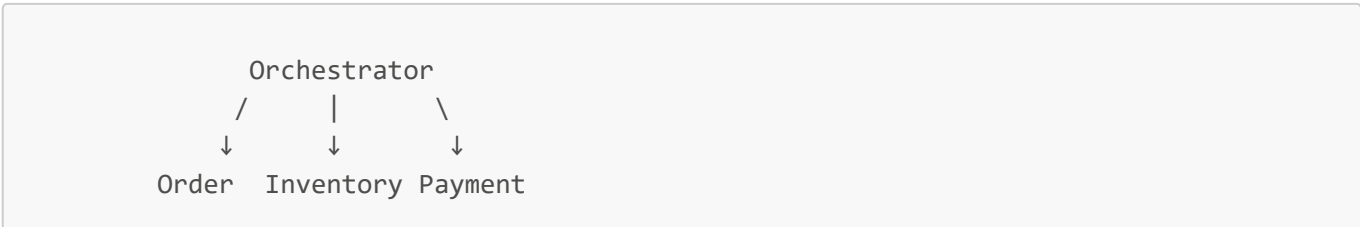
    if (success)
    {
        await _eventBus.PublishAsync(
            new PaymentCompleted(message.OrderId));
    }
    else
    {
        await _eventBus.PublishAsync(
            new PaymentFailed(message.OrderId));
    }
}
```

Order or inventory services can react to `PaymentFailed`.

10. Orchestration vs choreography

Feature	Orchestration	Choreography
Coordinator	Yes	No
Communication	Commands/events	Events
Workflow visibility	Easy	Harder
Coupling	Central coordinator	Event-based
Debugging	Easier	More difficult
Suitable for	Complex workflows	Simple workflows
Failure handling	Centralized	Distributed

Orchestration



Choreography



11. Saga vs database transaction

A normal transaction:

```
BEGIN
  Insert Order
  Update Inventory
  Process Payment
COMMIT
```

If something fails:

```
ROLLBACK
```

Saga:

```
Create Order
  ↓
Reserve Inventory
  ↓
Process Payment
  ↓
Failure
  ↓
Release Inventory
```

↓
Cancel Order

So Saga provides **eventual consistency**, not traditional ACID atomicity across services.

12. Important concept: compensation isn't always rollback

This is an important interview point.

Suppose:

Payment successful

You cannot literally undo the transaction like a database rollback.

Instead:

Payment successful
↓
Refund payment

So:

Transaction:
 ChargePayment()

Compensation:
 RefundPayment()

Similarly:

ReserveInventory()
↓
ReleaseInventory()

and:

CreateOrder()
↓

```
CancelOrder()
```

13. Real ASP.NET Core architecture

A production implementation might look like:

```
src/  
├── OrderService/  
│   ├── Controllers/  
│   ├── Domain/  
│   ├── Application/  
│   ├── Infrastructure/  
│   └── Program.cs  
├── InventoryService/  
├── PaymentService/  
├── ShippingService/  
└── Saga/  
    ├── OrderSaga.cs  
    ├── Commands/  
    ├── Events/  
    └── Handlers/
```

You would typically combine Saga with:

```
ASP.NET Core  
+  
RabbitMQ / Kafka  
+  
Outbox Pattern  
+  
Idempotency  
+  
Retry  
+  
Dead Letter Queue  
+  
Saga
```

14. Saga + Outbox Pattern

This is particularly important in real microservices.

Suppose:

```
await db.SaveChangesAsync();

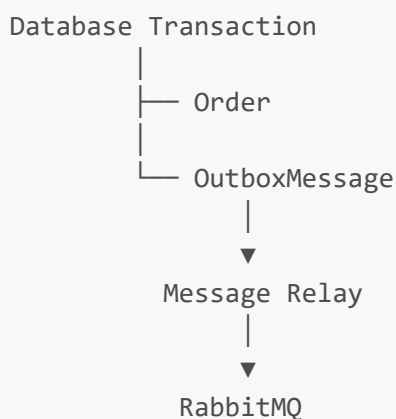
await messageBus.PublishAsync(
    new OrderCreated(order.Id));
```

What if:

DB save	✓
Message	X

Now your system is inconsistent.

The **Transactional Outbox Pattern** solves this by storing the event in the same database transaction:



This is commonly paired with Saga.

15. Saga + idempotency

Messages can be delivered more than once.

For example:

```
PaymentCompleted
PaymentCompleted
PaymentCompleted
```

Your handler shouldn't charge the customer three times.

Use an idempotency key:

```
public async Task Handle(PaymentCommand command)
{
    if (await _processedMessages.ExistsAsync(command.MessageId))
        return;

    await _payment.ProcessAsync(command.OrderId);

    await _processedMessages.AddAsync(command.MessageId);
}
```

This is extremely important in production Saga implementations.

16. Saga state

A real Saga often needs persistent state.

```
public class OrderSagaState
{
    public Guid SagaId { get; set; }

    public Guid OrderId { get; set; }

    public bool OrderCreated { get; set; }

    public bool InventoryReserved { get; set; }

    public bool PaymentCompleted { get; set; }

    public bool ShippingCreated { get; set; }

    public string Status { get; set; } = "Started";
}
```

For example:

```
SagaId: 123

OrderCreated: true
InventoryReserved: true
PaymentCompleted: false
ShippingCreated: false

Status: PaymentFailed
```

The state can be stored in:

- SQL Server
- PostgreSQL
- Redis
- MongoDB

depending on the architecture.

17. When should you use Saga?

Use Saga when:

- A business transaction crosses multiple microservices.
- Each service owns its own database.
- You need distributed workflow coordination.
- You need compensating actions.
- Eventual consistency is acceptable.

Example:

```
Order
↓
Inventory
↓
Payment
↓
Shipping
```

Saga is a strong candidate.

When NOT to use Saga

Don't introduce Saga simply because you have multiple classes or database tables.

If everything is inside one service/database:

```
Order
Inventory
Payment
```

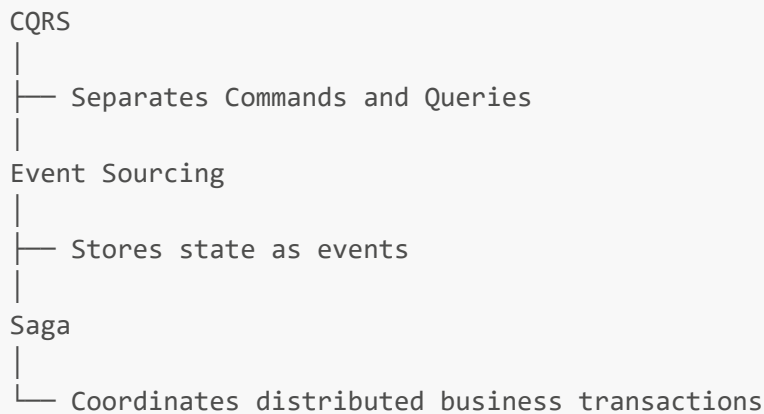
and can safely use:

```
using var transaction = await db.Database.BeginTransactionAsync();
```

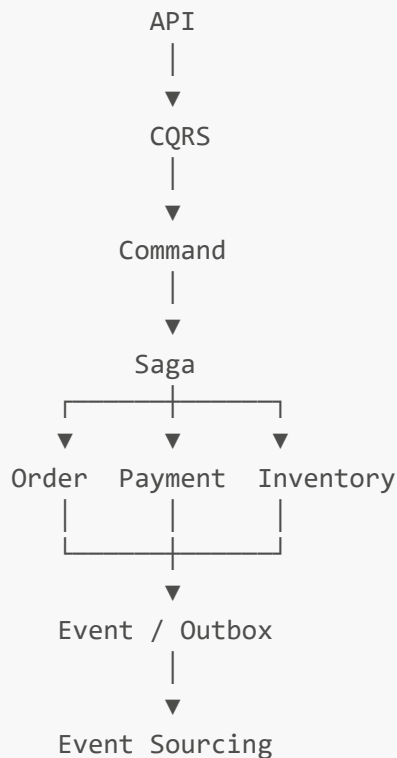
then a normal database transaction is usually much simpler.

18. Saga vs CQRS vs Event Sourcing

Since you're studying these patterns together, the distinction is important:



They can be combined:



But **Saga does not require CQRS or Event Sourcing**.

19. Interview definition

A good interview answer is:

The Saga Pattern is a distributed transaction pattern used in microservice architectures. It breaks a long-running business transaction into a sequence of local transactions. If a step fails, previously completed steps are undone through compensating transactions. Saga can be implemented using orchestration, where a central coordinator controls the workflow, or choreography, where services communicate through events.

Key words to remember

```
Saga
↓
Distributed transaction
↓
Local transactions
↓
Compensating transactions
↓
Eventual consistency
↓
Orchestration / Choreography
```

Most important production concepts: Saga + Outbox + Idempotency + Retry + Dead Letter Queue.

6. Outbox Pattern

Outbox Pattern in C#

The **Outbox Pattern** is a reliability pattern used in distributed systems and microservices to ensure that **database changes and event/message publishing happen reliably together.**

It solves a common problem:

"What happens if I save data to the database successfully, but the application crashes before publishing the event?"

1. The problem

Imagine an Order service:

```
Client
|
v
Order API
|
+----> Save Order to SQL Database
```



```
|
+----> Publish OrderCreated to RabbitMQ
```

Suppose this happens:

- 1. Save Order ☒
- 2. Application crashes ☒
- 3. Publish event ☒

The order exists in the database, but other services never receive **OrderCreated**.

This creates **inconsistent state**.

2. Outbox Pattern solution

Instead of immediately publishing the event, save the event into an **Outbox table** in the same database transaction as the business data.



The key idea is:

Store the business change and the event in the same database transaction. Publish the event later.

3. Example

Suppose we create an order.

Order entity

```
public class Order
{
    public Guid Id { get; set; }

    public Guid CustomerId { get; set; }

    public decimal Amount { get; set; }
}
```

Outbox entity

```
public class OutboxMessage
{
    public Guid Id { get; set; }

    public string EventType { get; set; } = null!;

    public string Payload { get; set; } = null!;

    public DateTime CreatedAt { get; set; }

    public DateTime? ProcessedAt { get; set; }

    public int RetryCount { get; set; }
}
```

4. Event

Create an event representing the business action.

```
public record OrderCreatedEvent(
    Guid OrderId,
```

```
Guid customerId,  
decimal amount);
```

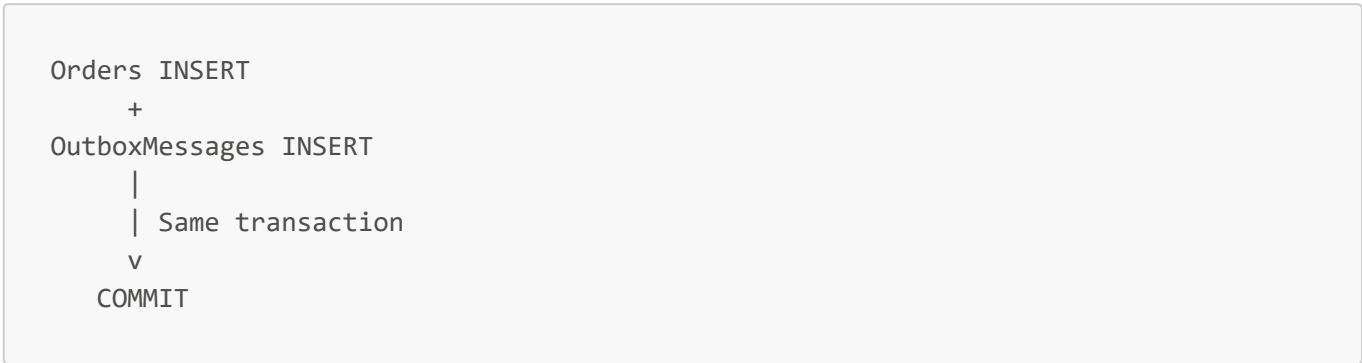
5. Creating the order

The important part is that both the order and outbox message are saved in **one transaction**.

```
public async Task<Guid> CreateOrderAsync(  
    Guid customerId,  
    decimal amount)  
{  
    var order = new Order  
    {  
        Id = Guid.NewGuid(),  
        CustomerId = customerId,  
        Amount = amount  
    };  
  
    var orderCreatedEvent = new OrderCreatedEvent(  
        order.Id,  
        order.CustomerId,  
        order.Amount);  
  
    var outboxMessage = new OutboxMessage  
    {  
        Id = Guid.NewGuid(),  
        EventType = nameof(OrderCreatedEvent),  
        Payload = JsonSerializer.Serialize(orderCreatedEvent),  
        CreatedAt = DateTime.UtcNow  
    };  
  
    await using var transaction =  
        await _db.Database.BeginTransactionAsync();  
  
    try  
    {  
        _db.Orders.Add(order);  
  
        _db.OutboxMessages.Add(outboxMessage);  
  
        await _db.SaveChangesAsync();  
  
        await transaction.CommitAsync();  
  
        return order.Id;  
    }  
    catch  
    {  
        await transaction.RollbackAsync();  
    }  
}
```

```
        throw;
    }
}
```

Now:



Either both succeed or both fail.

6. What does the Outbox table look like?

After creating an order:

Id	EventType	Payload	CreatedAt	ProcessedAt
101	OrderCreatedEvent	{...}	10:00:01	NULL
102	OrderCreatedEvent	{...}	10:00:05	NULL

ProcessedAt = NULL means:

This message has not yet been published successfully.

7. Background worker

Now we need something that continuously reads the Outbox table and publishes messages.

In ASP.NET Core, a `BackgroundService` is commonly used.

```
public class OutboxProcessor : BackgroundService
{
    private readonly IServiceScopeFactory _scopeFactory;

    public OutboxProcessor(IServiceScopeFactory scopeFactory)
    {
        _scopeFactory = scopeFactory;
    }
}
```

```
protected override async Task ExecuteAsync(
    CancellationToken stoppingToken)
{
    while (!stoppingToken.IsCancellationRequested)
    {
        using var scope =
            _scopeFactory.CreateScope();

        var db = scope.ServiceProvider
            .GetRequiredService<AppDbContext>();

        var messages = await db.OutboxMessages
            .Where(x => x.ProcessedAt == null)
            .OrderBy(x => x.CreatedAt)
            .Take(100)
            .ToListAsync(stoppingToken);

        foreach (var message in messages)
        {
            try
            {
                await PublishMessageAsync(message);

                message.ProcessedAt = DateTime.UtcNow;

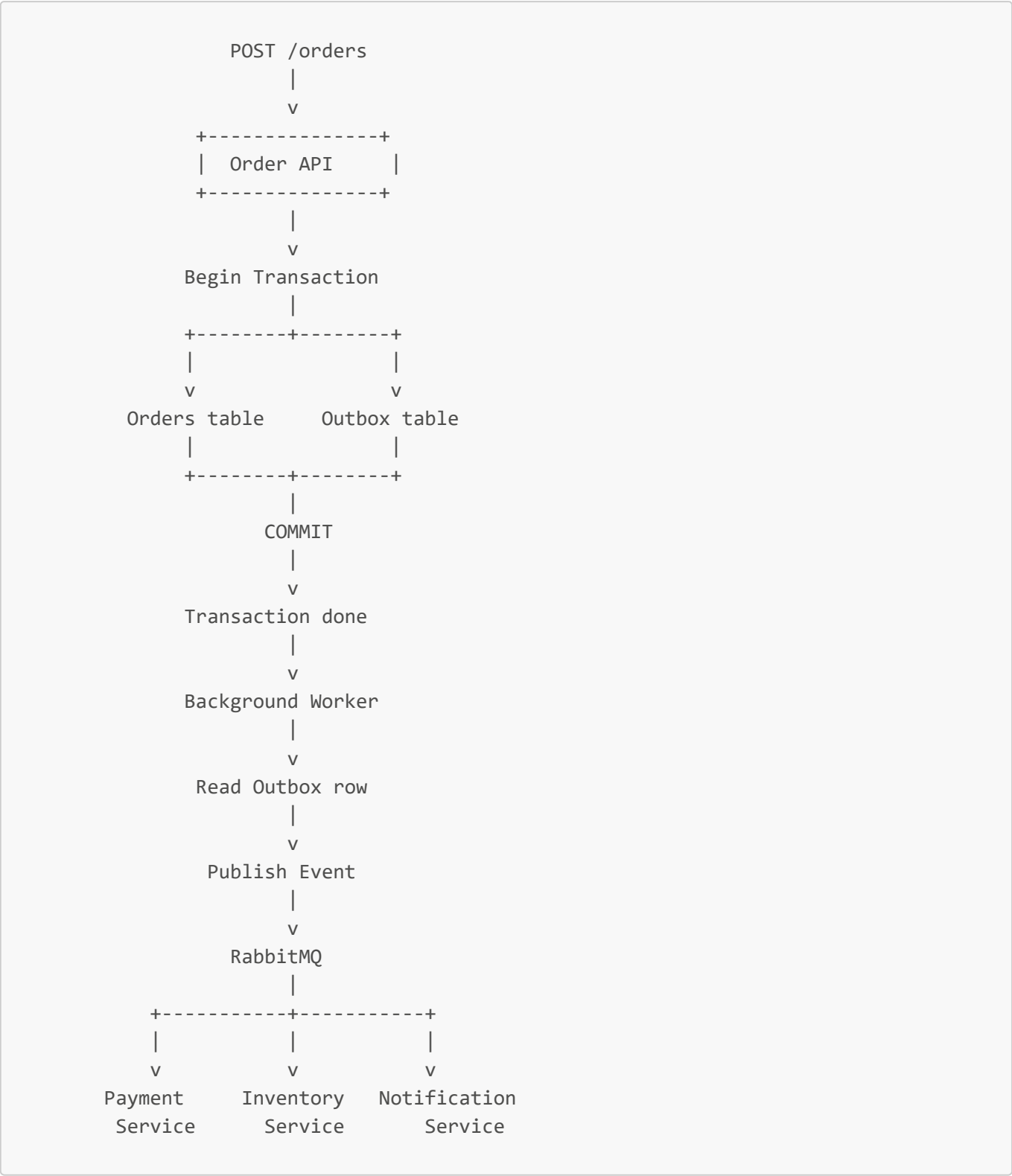
                await db.SaveChangesAsync(stoppingToken);
            }
            catch (Exception)
            {
                // Keep message unprocessed.
                // It will be retried later.
            }
        }

        await Task.Delay(
            TimeSpan.FromSeconds(5),
            stoppingToken);
    }
}

private async Task PublishMessageAsync(
    OutboxMessage message)
{
    // Publish to RabbitMQ / Kafka / Azure Service Bus
    await Task.CompletedTask;
}
}
```

8. Complete flow

The complete architecture becomes:



9. What if RabbitMQ is down?

This is where the pattern becomes powerful.

Suppose:

Order saved	☒
Outbox saved	☒
RabbitMQ publish	☒

The outbox message remains:

```
ProcessedAt = NULL
```

The worker tries again later.

Attempt 1 → RabbitMQ	☒
Attempt 2 → RabbitMQ	☒
Attempt 3 → RabbitMQ	☒
Attempt 4 → RabbitMQ	☒

Then:

```
ProcessedAt = 2026-08-14 16:40:20
```

So the event isn't lost just because the message broker was temporarily unavailable.

10. Retry mechanism

In production, you generally want explicit retry information.

```
public class OutboxMessage
{
    public Guid Id { get; set; }

    public string EventType { get; set; } = null!;

    public string Payload { get; set; } = null!;

    public DateTime CreatedAt { get; set; }

    public DateTime? ProcessedAt { get; set; }

    public int RetryCount { get; set; }

    public DateTime? NextAttemptAt { get; set; }
```

```
    public string? Error { get; set; }  
}
```

Then:

```
RetryCount = 0  
  |  
  v  
Publish  
  |  
  X  
  |  
RetryCount = 1  
NextAttempt = +5 sec  
  |  
  v  
Publish  
  |  
  X  
  |  
RetryCount = 2  
NextAttempt = +30 sec  
  |  
  v  
Publish
```

This is generally implemented using **exponential backoff**.

11. Important problem: duplicate messages

There is an important limitation.

Imagine:

- | | |
|------------------------|-------------------------------------|
| 1. Read Outbox message | ☒ |
| 2. Publish to RabbitMQ | ☒ |
| 3. Application crashes | ☒ |
| 4. Mark ProcessedAt | ☒ |

When the worker restarts:

```
ProcessedAt == NULL
```

So it publishes the same event again.


```
RabbitMQ:  
  
OrderCreated  
OrderCreated    ← duplicate
```

Therefore:

Outbox Pattern normally provides at-least-once delivery, not exactly-once delivery.

12. Idempotent consumers

Consumers should therefore be **idempotent**.

For example, use the Outbox message ID as an event ID.

```
public record OrderCreatedEvent(  
    Guid EventId,  
    Guid OrderId,  
    Guid CustomerId,  
    decimal Amount);
```

Consumer:

```
public async Task HandleAsync(OrderCreatedEvent message)  
{  
    var alreadyProcessed =  
        await _db.ProcessedMessages  
            .AnyAsync(x => x.EventId == message.EventId);  
  
    if (alreadyProcessed)  
        return;  
  
    // Process event  
  
    _db.ProcessedMessages.Add(  
        new ProcessedMessage  
        {  
            EventId = message.EventId,  
            ProcessedAt = DateTime.UtcNow  
        });  
  
    await _db.SaveChangesAsync();  
}
```

Now:

```
Event ID: ABC123

First delivery
  ↓
Process
  ↓
Save ABC123

Second delivery
  ↓
ABC123 already exists
  ↓
Ignore
```

13. Outbox vs normal publishing

Without Outbox

```
await _db.SaveChangesAsync();

await _rabbitMq.PublishAsync(orderCreated);
```

Potential failure:

```
Database ☒
RabbitMQ ☒
```

Result:

```
Database says:
Order exists

Other services say:
Order doesn't exist
```

With Outbox

```
_db.Orders.Add(order);

_db.OutboxMessages.Add(message);
```

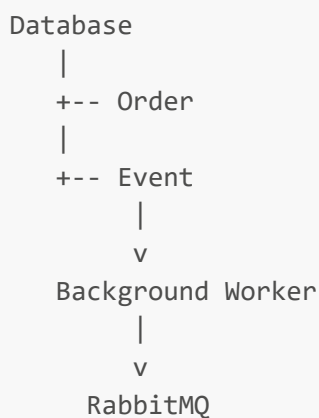
```
await _db.SaveChangesAsync();
```

Both happen in the same database transaction.

Later:

```
await _rabbitMq.PublishAsync(message);
```

So:



14. Outbox Pattern with EF Core

A more realistic EF Core configuration:

```

public class AppDbContext : DbContext
{
    public DbSet<Order> Orders => Set<Order>();

    public DbSet<OutboxMessage> OutboxMessages =>
        Set<OutboxMessage>();

    public AppDbContext(
        DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    protected override void OnModelCreating(
        modelBuilder modelBuilder)
    {
        modelBuilder.Entity<OutboxMessage>()
            .HasKey(x => x.Id);
    }
}

```

```
        modelBuilder.Entity<OutboxMessage>()  
            .HasIndex(x => new  
            {  
                x.ProcessedAt,  
                x.CreatedAt  
            });  
    }  
}
```

The index is useful because the worker frequently asks:

```
WHERE ProcessedAt IS NULL  
ORDER BY CreatedAt
```

15. Typical SQL table

```
CREATE TABLE OutboxMessages  
(  
    Id UNIQUEIDENTIFIER NOT NULL PRIMARY KEY,  
  
    EventType NVARCHAR(200) NOT NULL,  
  
    Payload NVARCHAR(MAX) NOT NULL,  
  
    CreatedAt DATETIME2 NOT NULL,  
  
    ProcessedAt DATETIME2 NULL,  
  
    RetryCount INT NOT NULL DEFAULT 0,  
  
    NextAttemptAt DATETIME2 NULL,  
  
    Error NVARCHAR(MAX) NULL  
);
```

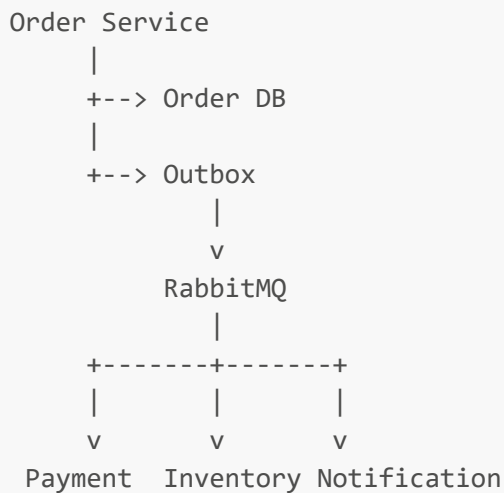
16. Where is it useful?

Outbox is particularly useful in:

- **Microservices**
- Payment systems
- Order processing

- Banking/fintech
- Inventory systems
- Event-driven architectures
- RabbitMQ systems
- Kafka systems
- Azure Service Bus systems
- Distributed workflows

For example:



17. Outbox vs Saga

These two patterns are often used together, but solve different problems.

Outbox

Solves:

How do I reliably publish an event after changing my database?

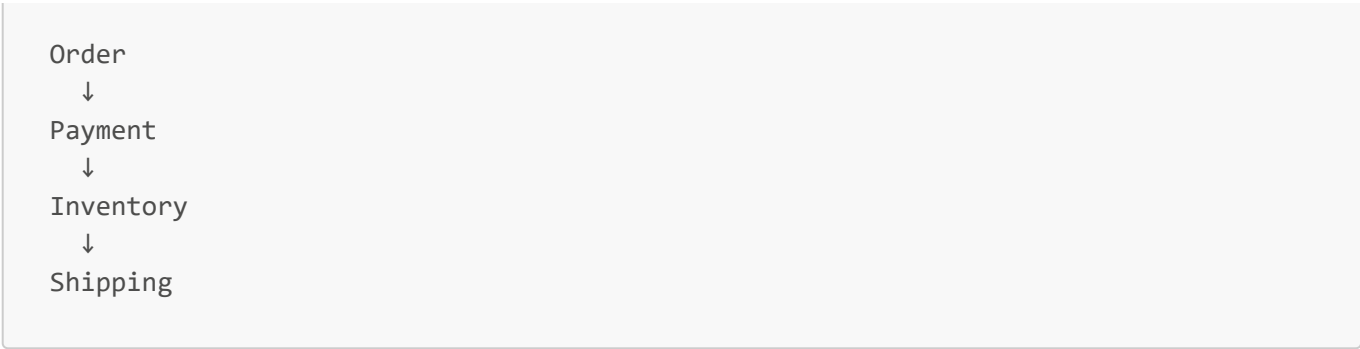
```

DB transaction
+
Event publishing
  
```

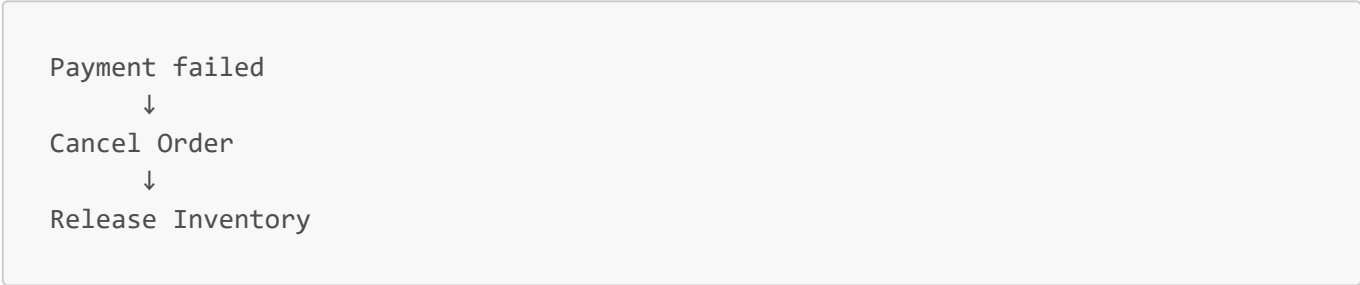
Saga

Solves:

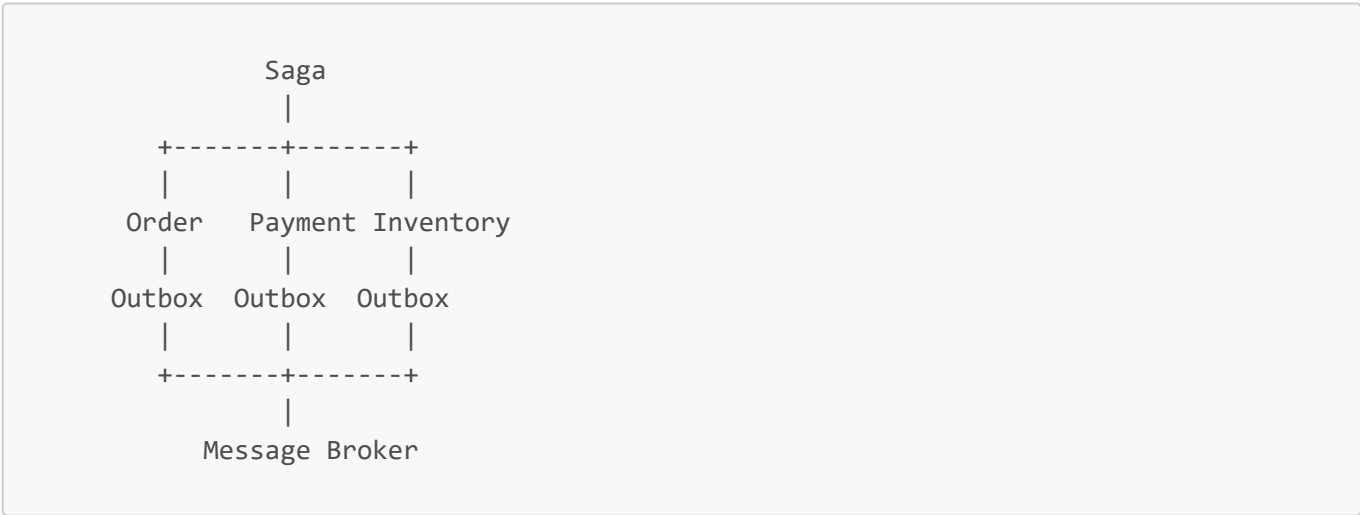
How do I coordinate a distributed business transaction across multiple services?



If payment fails:

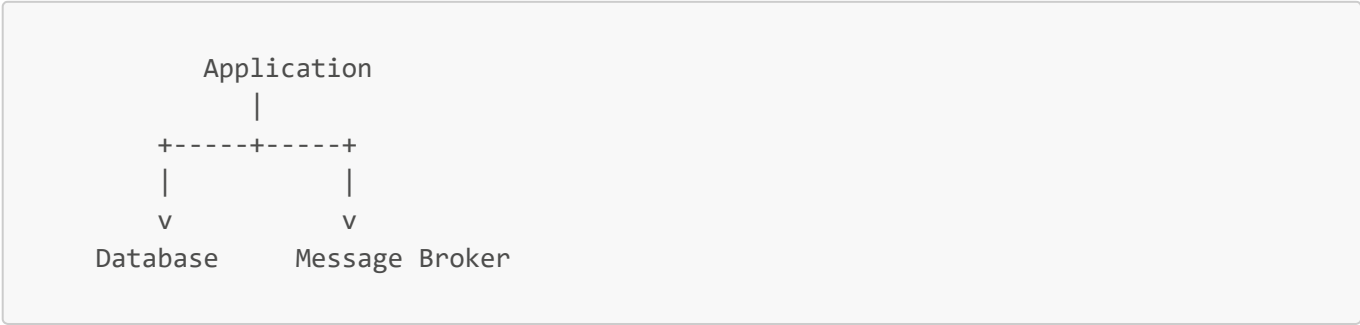


A common architecture is therefore:



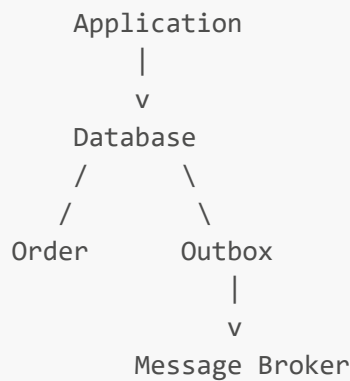
18. Outbox vs Transactional Messaging

The fundamental problem is the **dual-write problem**.



You cannot normally make these two independent systems participate in one simple ACID transaction.

The Outbox pattern changes this to:



Only the database operation needs to be transactional.

19. Interview answer

If an interviewer asks:

What is the Outbox Pattern?

A strong answer is:

"The Outbox Pattern solves the dual-write problem in distributed systems. Instead of updating the database and publishing a message independently, we store the business entity and an event in an Outbox table within the same database transaction. A background worker then reads the Outbox table and publishes those events to a message broker such as RabbitMQ or Kafka. This prevents events from being lost when the application or message broker fails. Since publishing can result in duplicates, consumers should be idempotent."

20. Key points to remember

```
Outbox Pattern
|
+-- Solves dual-write problem
|
+-- Business data + event saved atomically
|
+-- Background worker publishes events
|
+-- Supports retries
|
+-- Events aren't easily lost
|
+-- Usually at-least-once delivery
|
```

```
+-- Consumers should be idempotent
|
+-- Common with RabbitMQ/Kafka
|
+-- Very useful in microservices
```

In one sentence:

Outbox Pattern = Save the database change and the event together, then publish the event asynchronously and reliably.

7. Inbox Pattern

Inbox Pattern in C#

The **Inbox Pattern** is a messaging pattern used in distributed systems and microservices to make **message processing idempotent**.

The main idea is:

Store every received message ID before processing it, so the same message is not processed twice.

This is especially useful with **RabbitMQ, Azure Service Bus, Kafka, AWS SQS**, etc.

Why do we need it?

Imagine an Order Service receives:

```
MessageId = 123
Event = PaymentCompleted
```

It processes the payment event and updates the database.

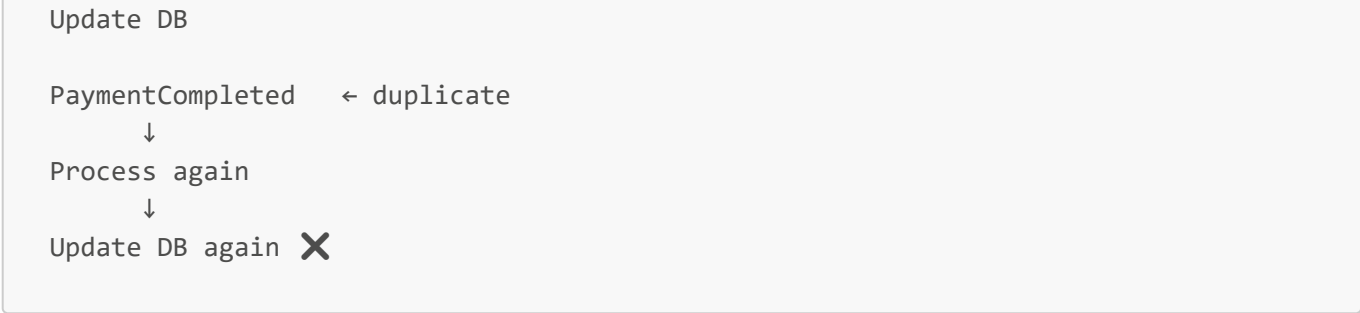
But the message broker doesn't receive the acknowledgment because of a network failure.

The broker sends the same message again:

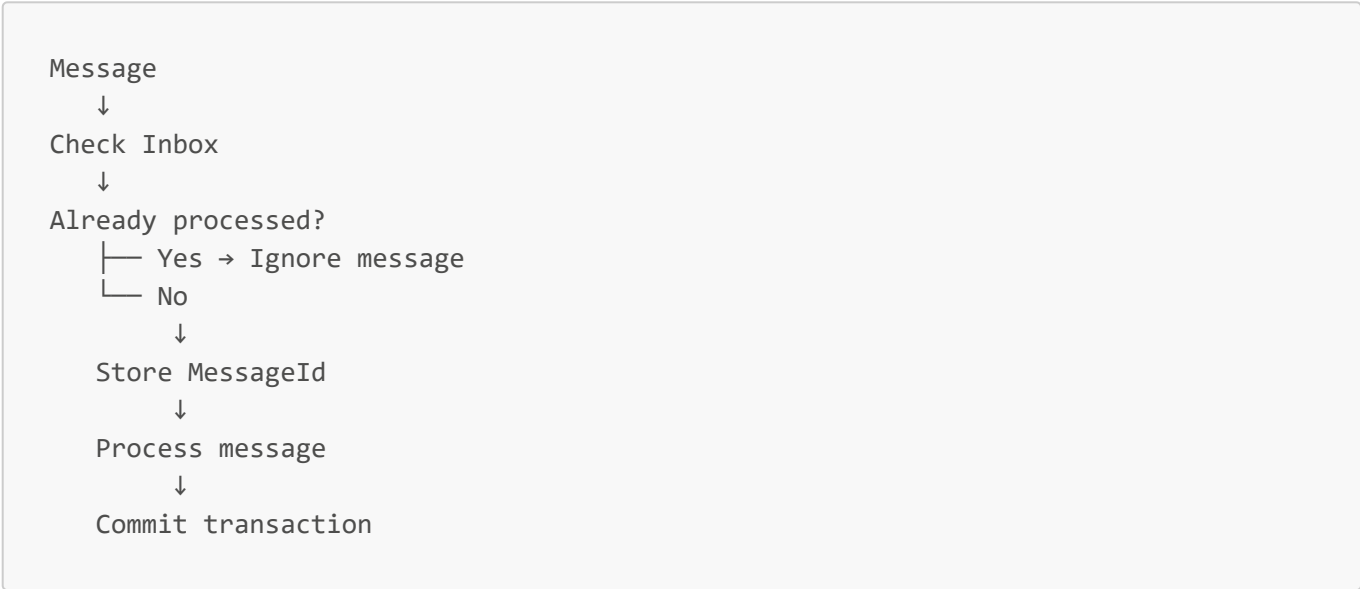
```
MessageId = 123
Event = PaymentCompleted
```

Without an Inbox Pattern:

```
PaymentCompleted
  ↓
Process
  ↓
```

With the Inbox Pattern:



1. Inbox Table

Typically you create an `InboxMessages` table.

```
public class InboxMessage
{
    public Guid MessageId { get; set; }

    public string MessageType { get; set; } = string.Empty;

    public DateTime ReceivedAt { get; set; }

    public DateTime? ProcessedAt { get; set; }
}
```

Database:

InboxMessages	

MessageId	MessageType

123	PaymentCompleted
456	OrderCreated
789	CustomerCreated

Make **MessageId** **unique**.

This is important because the database itself can prevent duplicate messages.

2. Entity Framework Configuration

```
public class AppDbContext : DbContext
{
    public DbSet<InboxMessage> InboxMessages => Set<InboxMessage>();

    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<InboxMessage>()
            .HasKey(x => x.MessageId);
    }
}
```

3. Message

Suppose RabbitMQ sends this event:

```
public record PaymentCompleted(
    Guid MessageId,
    Guid OrderId,
    decimal Amount);
```

4. Consumer

A simple consumer can look like this:

```
public class PaymentCompletedConsumer
{
    private readonly AppDbContext _db;

    public PaymentCompletedConsumer(AppDbContext db)
    {
        _db = db;
    }

    public async Task HandleAsync(PaymentCompleted message)
    {
        var alreadyProcessed =
            await _db.InboxMessages
                .AnyAsync(x => x.MessageId == message.MessageId);

        if (alreadyProcessed)
        {
            return;
        }

        var inboxMessage = new InboxMessage
        {
            MessageId = message.MessageId,
            MessageType = nameof(PaymentCompleted),
            ReceivedAt = DateTime.UtcNow
        };

        _db.InboxMessages.Add(inboxMessage);

        // Business operation
        var order = await _db.Orders
            .FirstAsync(x => x.Id == message.OrderId);

        order.Status = "Paid";

        inboxMessage.ProcessedAt = DateTime.UtcNow;

        await _db.SaveChangesAsync();
    }
}
```

Now if the same message arrives again:

```
MessageId = ABC123
```

the consumer finds:

```
AnyAsync(x => x.MessageId == message.MessageId)
```

and returns without processing it.

5. Important: Use a Database Transaction

The real power of the Inbox Pattern comes from doing the **Inbox insert and business operation in the same transaction**.

```
public async Task HandleAsync(PaymentCompleted message)
{
    await using var transaction =
        await _db.Database.BeginTransactionAsync();

    var exists = await _db.InboxMessages
        .AnyAsync(x => x.MessageId == message.MessageId);

    if (exists)
    {
        await transaction.RollbackAsync();
        return;
    }

    _db.InboxMessages.Add(new InboxMessage
    {
        MessageId = message.MessageId,
        MessageType = nameof(PaymentCompleted),
        ReceivedAt = DateTime.UtcNow
    });

    var order = await _db.Orders
        .FirstAsync(x => x.Id == message.OrderId);

    order.Status = "Paid";

    await _db.SaveChangesAsync();

    await transaction.CommitAsync();
}
```

The important property is:

```
Inbox Insert
    +
Business Update
    ↓
Same Database Transaction
```

Either **both succeed**, or **both fail**.

6. What happens if processing fails?

Suppose:

```
Receive message
  ↓
Insert Inbox record
  ↓
Update Order
  ↓
Exception ✕
```

If both operations are inside the same transaction:

```
Transaction Rollback
  ↓
Inbox record removed
  ↓
Order update removed
```

The broker can then redeliver the message.

On retry:

```
Message
  ↓
Inbox doesn't contain ID
  ↓
Process
  ↓
Success
  ↓
Commit
```

7. Race Condition

There is an important problem with this code:

```
if (!await InboxExists(message.MessageId))
{
    AddInbox(message.MessageId);
}
```

Two consumers could receive the same message simultaneously:

Consumer A	Consumer B
Check Inbox	Check Inbox
→ Not found	→ Not found
Insert	Insert
+----- duplicate -----+	

Therefore, you should have a **unique constraint/index** on `MessageId`.

For example:

```
modelBuilder.Entity<InboxMessage>()
    .HasIndex(x => x.MessageId)
    .IsUnique();
```

Then the database guarantees:

```
MessageId = 123
MessageId = 123 ✗ duplicate
```

8. Generic Inbox Handler

In a real application, you can create a reusable Inbox service.

```
public interface IInboxService
{
    Task<bool> HasBeenProcessedAsync(Guid messageId);

    Task MarkAsProcessedAsync(
        Guid messageId,
        string messageType);
}
```

Implementation:

```
public class InboxService : IInboxService
{
    private readonly AppDbContext _db;

    public InboxService(AppDbContext db)
    {
        _db = db;
    }

    public async Task<bool> HasBeenProcessedAsync(Guid messageId)
    {
        return await _db.InboxMessages
            .AnyAsync(x => x.MessageId == messageId);
    }

    public async Task MarkAsProcessedAsync(
        Guid messageId,
        string messageType)
    {
        _db.InboxMessages.Add(new InboxMessage
        {
            MessageId = messageId,
            MessageType = messageType,
            ReceivedAt = DateTime.UtcNow,
            ProcessedAt = DateTime.UtcNow
        });

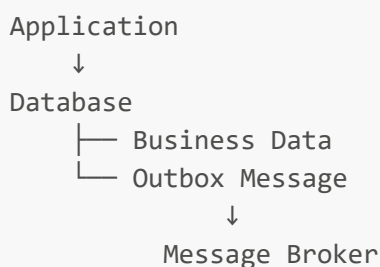
        await _db.SaveChangesAsync();
    }
}
```

9. Inbox vs Outbox

Since you are learning **Outbox, Saga, CQRS, Event Sourcing**, this distinction is important.

Outbox

Outbox protects outgoing messages.

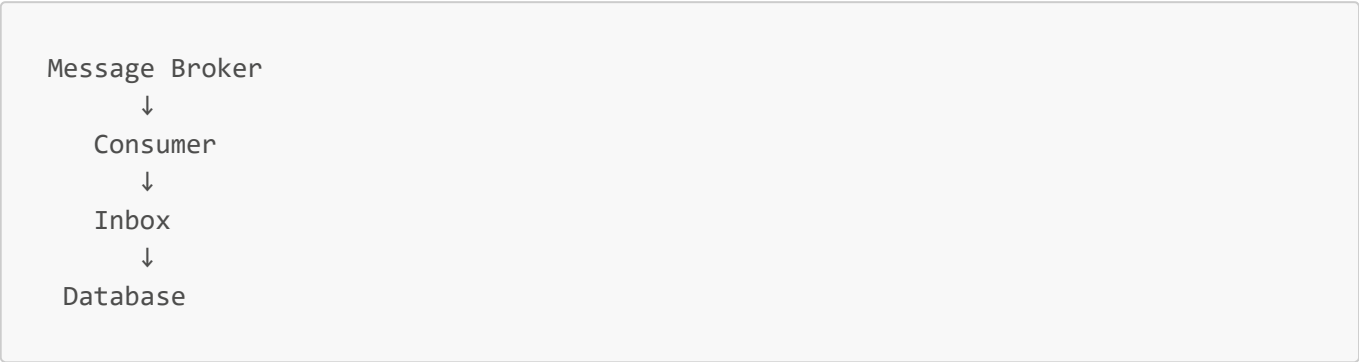


It solves:

"How do I reliably publish an event after changing my database?"

Inbox

Inbox protects incoming messages.

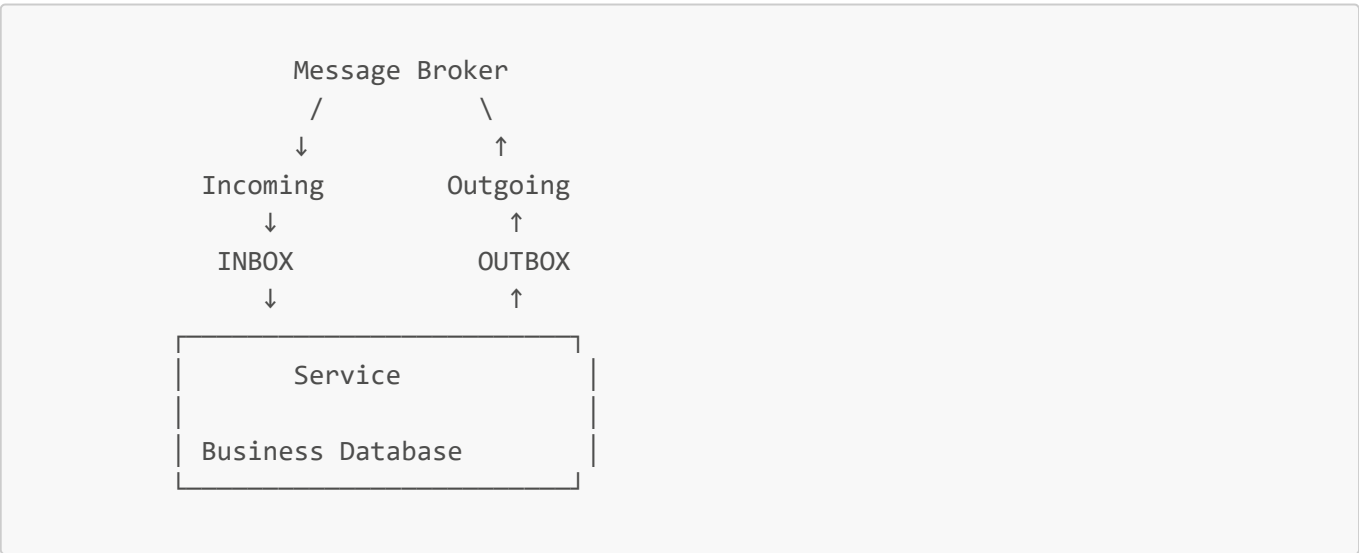


It solves:

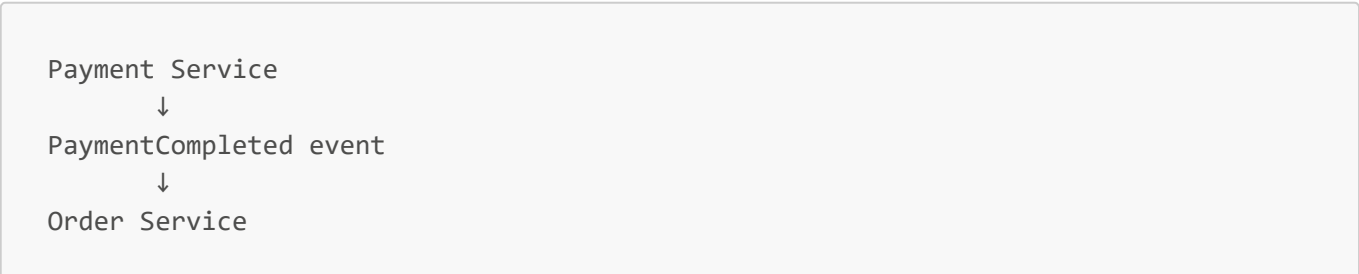
"How do I make sure I don't process the same message twice?"

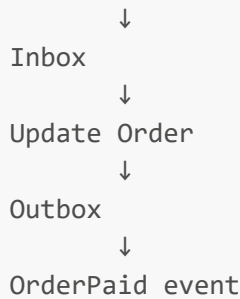
10. Inbox + Outbox Together

In microservices, you often use both:



For example:





This gives you a much more reliable event-driven architecture.

When should you use Inbox Pattern?

Use it when:

- Messages can be delivered more than once.
- You're building microservices.
- You're using RabbitMQ/Kafka/Azure Service Bus/etc.
- Processing a duplicate message could cause incorrect results.
- You need reliable **at-least-once delivery**.
- Your business operation is not naturally idempotent.

Key interview answer

The Inbox Pattern provides idempotent message consumption by recording processed message IDs in a persistent store. When a message is received, the consumer checks whether its ID has already been processed. If it has, the message is ignored; otherwise, the message is processed and the Inbox record is persisted atomically with the business transaction.

Outbox = reliable publishing. Inbox = reliable/idempotent consumption.

8. Retry Pattern

Retry Pattern in C#

The **Retry Pattern** is a resilience design pattern where an application **automatically retries a failed operation** when the failure is likely to be temporary.

Typical transient failures:

- Network timeout
- Temporary database connection failure
- HTTP 503 Service Unavailable
- HTTP 429 Too Many Requests
- Temporary cloud-service failure
- Message broker temporarily unavailable

The basic idea is:

```
Operation
  ↓
Fails?
  ↓ Yes
Wait
  ↓
Retry
  ↓
Fails?
  ↓ Yes
Wait longer
  ↓
Retry
  ↓
Success / Give up
```

1. Simple Retry in C#

```
public async Task<string> GetDataAsync()
{
    int maxRetries = 3;

    for (int attempt = 1; attempt <= maxRetries; attempt++)
    {
        try
        {
            return await CallApiAsync();
        }
        catch (HttpRequestException) when (attempt < maxRetries)
        {
            await Task.Delay(1000);
        }
    }

    throw new Exception("Operation failed after retries.");
}

private async Task<string> CallApiAsync()
{
    // Simulate API call
    return await Task.FromResult("Success");
}
```

Here:

```
Attempt 1 → Failure
  ↓
```

```
Wait 1 sec
↓
Attempt 2 → Failure
↓
Wait 1 sec
↓
Attempt 3 → Success
```

2. Retry with Exponential Backoff

A better implementation doesn't retry immediately.

Instead:

```
Attempt 1 → wait 1 second
Attempt 2 → wait 2 seconds
Attempt 3 → wait 4 seconds
Attempt 4 → wait 8 seconds
```

This is called **exponential backoff**.

```
public async Task<string> GetDataAsync()
{
    int maxRetries = 5;

    for (int attempt = 1; attempt <= maxRetries; attempt++)
    {
        try
        {
            return await CallApiAsync();
        }
        catch (HttpRequestException) when (attempt < maxRetries)
        {
            int delay = (int)Math.Pow(2, attempt - 1) * 1000;

            await Task.Delay(delay);
        }
    }

    throw new Exception("Operation failed.");
}
```

The delays are approximately:

Attempt	Delay
---------	-------

Attempt	Delay
1	1 sec
2	2 sec
3	4 sec
4	8 sec
5	16 sec

3. Retry with Jitter

In distributed systems, thousands of clients could fail at the same time.

If all clients retry simultaneously:

```
Service fails
  ↓
1000 clients fail
  ↓
All retry after 2 seconds
  ↓
1000 requests arrive together
  ↓
Service becomes overloaded again
```

Jitter adds randomness to the delay.

```
private static readonly Random Random = new();

private static int GetRetryDelay(int attempt)
{
    int exponentialDelay =
        (int)Math.Pow(2, attempt - 1) * 1000;

    int jitter = Random.Next(0, 500);

    return exponentialDelay + jitter;
}
```

Then:

```
await Task.Delay(GetRetryDelay(attempt));
```

Instead of everyone retrying at exactly 2 seconds:

```
Client A → 2.1 sec  
Client B → 2.4 sec  
Client C → 2.2 sec  
Client D → 2.7 sec
```

This spreads the load.

4. Retry with HttpClient

For an ASP.NET Core application, you commonly want retries around outgoing HTTP calls.

A simple implementation:

```
public class PaymentClient  
{  
    private readonly HttpClient _httpClient;  
  
    public PaymentClient(HttpClient httpClient)  
    {  
        _httpClient = httpClient;  
    }  
  
    public async Task<string> GetPaymentAsync(  
        string paymentId)  
    {  
        const int maxRetries = 3;  
  
        for (int attempt = 1; attempt <= maxRetries; attempt++)  
        {  
            try  
            {  
                var response = await _httpClient.GetAsync(  
                    $"/api/payments/{paymentId}");  
  
                response.EnsureSuccessStatusCode();  
  
                return await response.Content.ReadAsStringAsync();  
            }  
            catch (HttpRequestException)  
            when (attempt < maxRetries)  
            {  
                await Task.Delay(  
                    TimeSpan.FromSeconds(attempt * 2));  
            }  
        }  
    }  
}
```

```
        throw new Exception(  
            "Payment service unavailable.");  
    }  
}
```

5. Important: Don't Retry Everything

This is one of the most important Retry Pattern concepts.

You should retry **transient failures**, not every exception.

Usually retry

```
HTTP 408 Request Timeout  
HTTP 429 Too Many Requests  
HTTP 502 Bad Gateway  
HTTP 503 Service Unavailable  
HTTP 504 Gateway Timeout  
Network timeout  
Temporary connection failure
```

Usually don't retry

```
HTTP 400 Bad Request  
HTTP 401 Unauthorized  
HTTP 403 Forbidden  
HTTP 404 Not Found  
Invalid input  
Business validation failure
```

For example:

```
POST /payments  
Amount = -5000
```

Retrying this won't magically make **-5000** valid.

6. Retry + CancellationToken

Production code should support cancellation.

```
public async Task<string> GetDataAsync(
    CancellationToken cancellationToken)
{
    const int maxRetries = 3;

    for (int attempt = 1; attempt <= maxRetries; attempt++)
    {
        try
        {
            return await CallApiAsync(cancellationToken);
        }
        catch (HttpRequestException)
            when (attempt < maxRetries)
        {
            var delay = TimeSpan.FromSeconds(attempt);

            await Task.Delay(
                delay,
                cancellationToken);
        }
    }

    throw new Exception("Operation failed.");
}
```

This allows the caller to stop the retry process.

7. Retry Pattern with Polly

In modern .NET applications, **Polly** is commonly used for resilience policies.

Conceptually:

```
var pipeline = new ResiliencePipelineBuilder()
    .AddRetry(new RetryStrategyOptions
    {
        MaxRetryAttempts = 3,
        Delay = TimeSpan.FromSeconds(2),
        BackoffType = DelayBackoffType.Exponential
    })
    .Build();
```

Then:

```
var result = await pipeline.ExecuteAsync(
    async cancellationToken =>
```

```
{  
    return await CallApiAsync(cancellationToken);  
},  
cancellationToken);
```

This separates your retry logic from your business logic.

8. Retry Pattern in ASP.NET Core

A common architecture is:

```
Controller  
  ↓  
Service  
  ↓  
HttpClient  
  ↓  
External API
```

Instead of putting retry logic in every service:

```
PaymentService  
└─ retry  
  
CustomerService  
└─ retry  
  
OrderService  
└─ retry
```

you can centralize resilience around the HTTP client:

```
PaymentService ┌  
CustomerService ┤ ─> Resilience/Retry ─> External API  
OrderService └
```

This avoids duplicating retry code.

9. Retry Pattern vs Circuit Breaker

These patterns are frequently used together.

Retry

```
Request
↓
Failure
↓
Wait
↓
Retry
```

Retry says:

"The failure might be temporary. Let's try again."

Circuit Breaker

```
Request
↓
Failure
↓
Failure
↓
Failure
↓
Circuit OPEN
↓
Stop sending requests
```

Circuit Breaker says:

"This service appears unhealthy. Stop hitting it for a while."

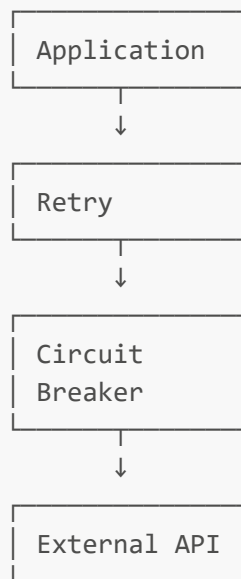
Together:

```
Application
↓
Retry
↓
Circuit Breaker
↓
External Service
```

This is very common in **microservices**.

10. Retry + Circuit Breaker

A typical resilience strategy:



For example:

```
API call
  ↓
503
  ↓
Retry
  ↓
503
  ↓
Retry
  ↓
503
  ↓
Circuit opens
  ↓
Fail fast
```

11. Retry Pattern in a Fintech System

Suppose your payment service calls a bank API:

```
Payment API
  ↓
Payment Service
  ↓
Bank API
```

The bank API temporarily returns:

```
503 Service Unavailable
```

A retry policy can do:

```
Request
  ↓
503
  ↓
Wait 1 sec
  ↓
Retry
  ↓
503
  ↓
Wait 2 sec
  ↓
Retry
  ↓
200 OK
```

But there's an important issue with **payments**.

You should **not blindly retry non-idempotent operations** such as:

```
POST /payments
```

because the first request might have succeeded on the server even though the response was lost.

You could accidentally create:

```
Payment ₹10,000
Payment ₹10,000
```

instead of one payment.

Use an **idempotency key**:

```
POST /payments

Idempotency-Key: 8f92c1...
```

Then retries can safely refer to the same logical operation.

12. Retry Pattern with Idempotency

```
public async Task<PaymentResult> ProcessPaymentAsync(
    PaymentRequest request,
    Cancellation_token cancellationToken)
{
    var idempotencyKey = Guid.NewGuid().ToString();

    for (int attempt = 1; attempt <= 3; attempt++)
    {
        try
        {
            using var httpRequest = new HttpRequestMessage(
                HttpMethod.Post,
                "/api/payments");

            httpRequest.Headers.Add(
                "Idempotency-Key",
                idempotencyKey);

            httpRequest.Content =
                JsonConvert.Create(request);

            var response = await _httpClient.SendAsync(
                httpRequest,
                cancellationToken);

            response.EnsureSuccessStatusCode();

            return await response.Content
                .ReadFromJsonAsync<PaymentResult>(
                    cancellationToken: cancellationToken)
                ?? throw new Exception("Invalid response.");
        }
        catch (HttpRequestException)
        {
            when (attempt < 3)
            {
                await Task.Delay(
                    TimeSpan.FromSeconds(attempt),
                    cancellationToken);
            }
        }
    }

    throw new Exception("Payment failed.");
}
```

The same idempotency key is used for every retry.

13. Retry Pattern vs Timeout

These are different resilience patterns.

Timeout

```
Request
  ↓
Wait 5 seconds
  ↓
No response
  ↓
Cancel
```

Retry

```
Request
  ↓
Failure
  ↓
Wait
  ↓
Try again
```

Usually you combine them:

```
Timeout
  ↓
Retry
  ↓
Circuit Breaker
  ↓
External Service
```

14. Retry Pattern vs Outbox Pattern

Since you're studying distributed-system patterns, this distinction is important.

Retry handles a failed operation:

```
Call Service
  ↓
Failure
  ↓
Try again
```

Outbox handles reliable event/message publishing:

```
Database Transaction
  ↓
Save business data
  +
Save event in Outbox
  ↓
Background Worker
  ↓
Publish Message
  ↓
Retry if publishing fails
```

So Retry is often **inside larger patterns** such as:

- Outbox
- Saga
- Circuit Breaker
- API Gateway
- Message processing
- Microservices communication

15. Advantages

Advantages

- Handles temporary failures
- Improves reliability
- Useful for network communication
- Simple to implement
- Works well with microservices
- Can be combined with exponential backoff
- Can be combined with circuit breakers

Disadvantages

- Adds latency
- Can increase load on an unhealthy service

- Can cause duplicate operations
- Incorrect retry policies can make outages worse
- Requires careful handling of idempotency

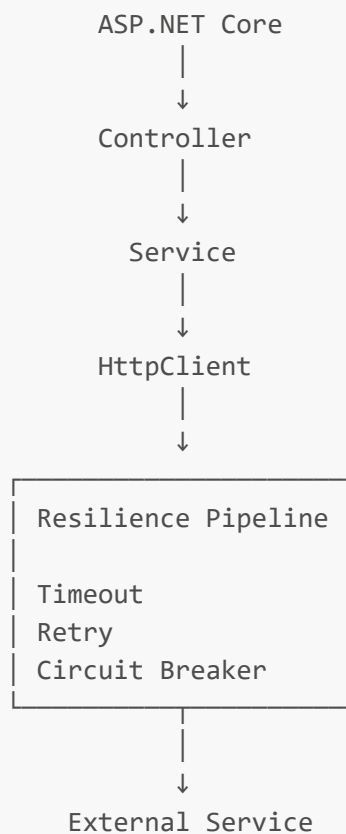
16. Interview Definition

Retry Pattern is a resilience pattern that automatically repeats a failed operation when the failure is considered transient, usually with a maximum retry count and a delay such as exponential backoff with jitter.

A strong interview answer would mention:

```
Retry
├── Maximum attempts
├── Delay
├── Exponential backoff
├── Jitter
├── Transient-error filtering
├── Cancellation
├── Idempotency
└── Circuit Breaker
```

Real-world .NET architecture



Key rule: Retry only when the failure is likely to be transient, and for operations that may be repeated, make sure the operation is **idempotent**.

9. **Circuit Breaker**

The **Circuit Breaker** pattern in C# is a resilience pattern used to prevent an application from repeatedly calling a failing or unavailable dependency.

Think of it like an **electrical circuit breaker**:

If a downstream service keeps failing, stop sending requests to it temporarily. After some time, test it again. If it works, resume normal traffic.

Why use it?

Suppose your application calls:

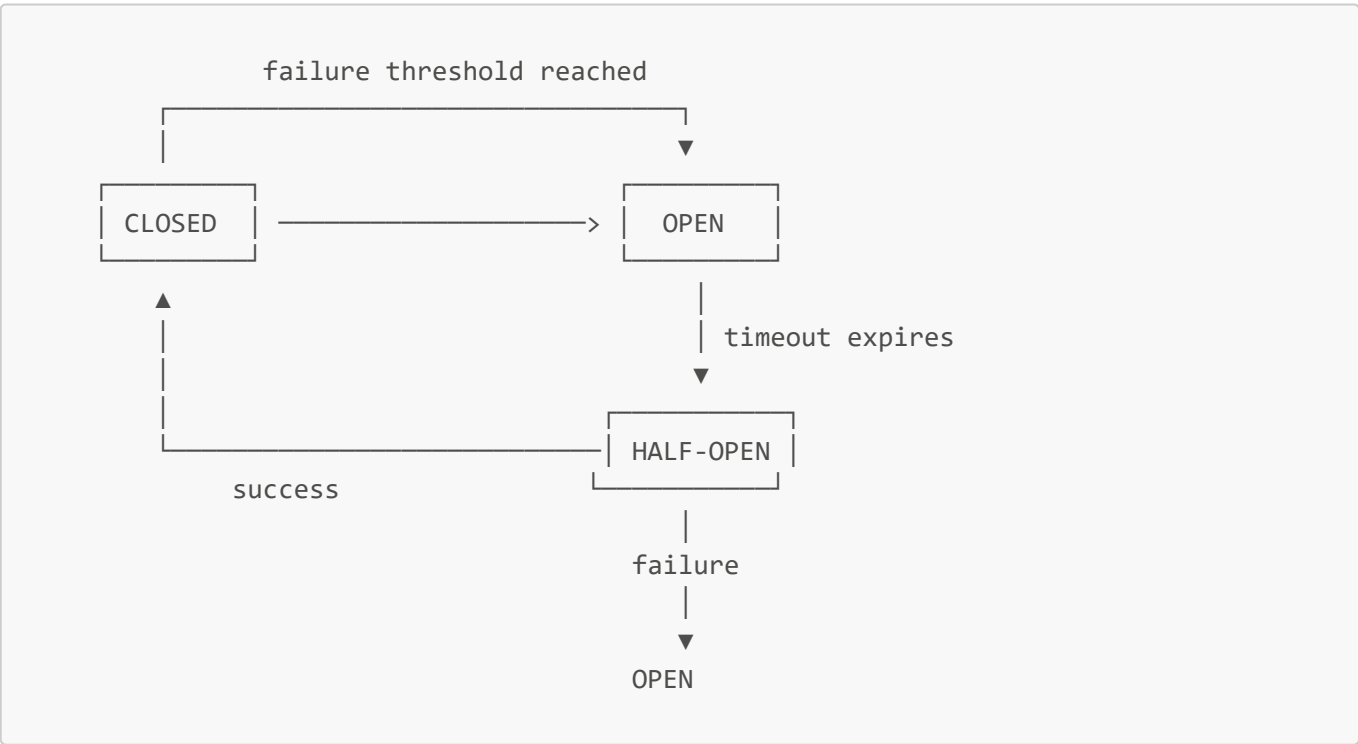
Your API → Payment Service

If the Payment Service is down and you keep calling it:

- Every request waits for a timeout.
- Threads/connections get consumed.
- Response times increase.
- The failure can cascade through your application.

A circuit breaker stops this by **failing fast**.

Three states



Closed

- Requests flow normally.
- Failures are counted.
- If failures exceed a threshold, the circuit opens.

Open

- Requests are immediately rejected.
- The dependency isn't called.
- After a configured delay, the circuit moves to half-open.

Half-Open

- A small number of test requests are allowed.
- If they succeed → **Closed**.
- If they fail → **Open** again.

Simple C# implementation

For example, suppose we want to protect an HTTP call:

```
public class CircuitBreaker
{
    private int failureCount = 0;
    private readonly int failureThreshold = 3;
    private readonly TimeSpan breakDuration = TimeSpan.FromSeconds(10);

    private DateTime openedAt;
    private bool isOpen = false;

    public async Task<string> ExecuteAsync(
        Func<Task<string>> action)
    {
        // Circuit is OPEN
        if (isOpen)
        {
            if (DateTime.UtcNow - openedAt < breakDuration)
            {
                throw new Exception("Circuit is OPEN");
            }

            // Move to HALF-OPEN
            isOpen = false;
        }

        try
        {
            var result = await action();

            // Successful request
            failureCount = 0;
        }
    }
}
```

```

        return result;
    }
    catch
    {
        failureCount++;

        if (failureCount >= failureThreshold)
        {
            isOpen = true;
            openedAt = DateTime.UtcNow;
        }

        throw;
    }
}

```

Usage:

```

var circuitBreaker = new CircuitBreaker();

try
{
    var response = await circuitBreaker.ExecuteAsync(async () =>
    {
        using var client = new HttpClient();

        return await client.GetStringAsync(
            "https://payment-service/api/payment");
    });

    Console.WriteLine(response);
}
catch (Exception ex)
{
    Console.WriteLine($"Request failed: {ex.Message}");
}

```

After **3 consecutive failures**, the circuit opens. Subsequent calls fail immediately for 10 seconds instead of calling the Payment Service.

In real-world ASP.NET Core

You generally **shouldn't implement the circuit breaker yourself**. A resilience library such as **Polly** is commonly used.

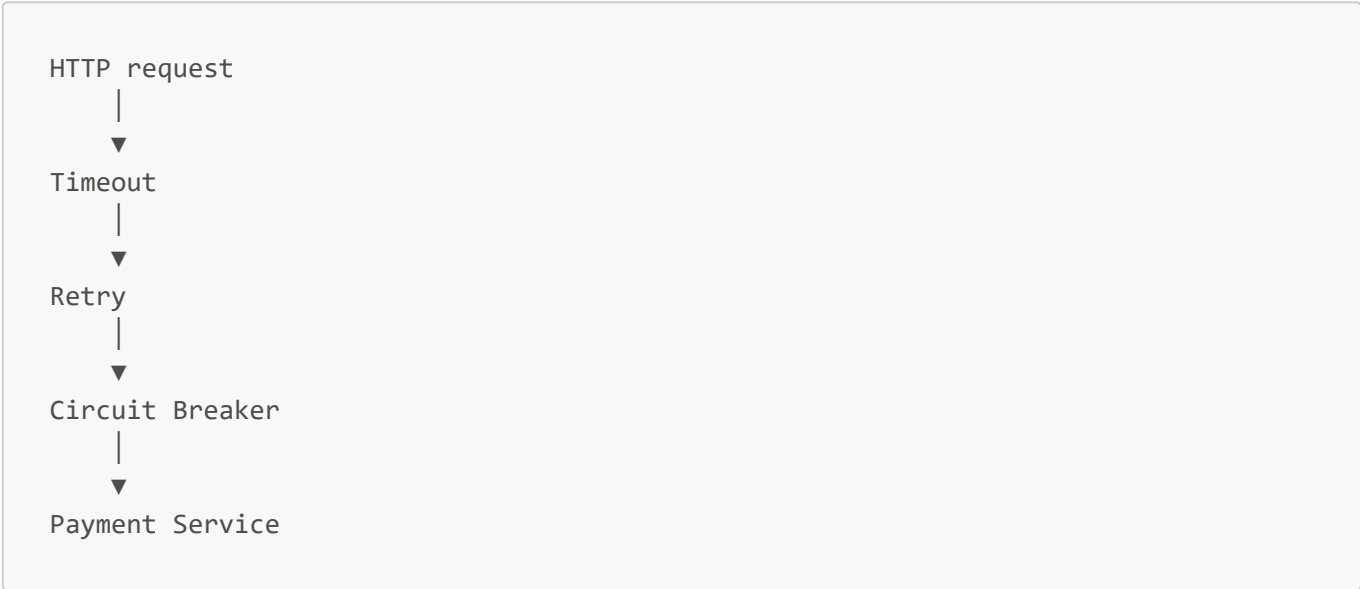
With modern .NET, you can also use the built-in resilience support around [HttpClient](#).

For example, conceptually:

```
builder.Services
    .AddHttpClient<PaymentService>()
    .AddStandardResilienceHandler();
```

This gives you a production-oriented resilience pipeline rather than maintaining state yourself.

A typical resilience pipeline can combine:

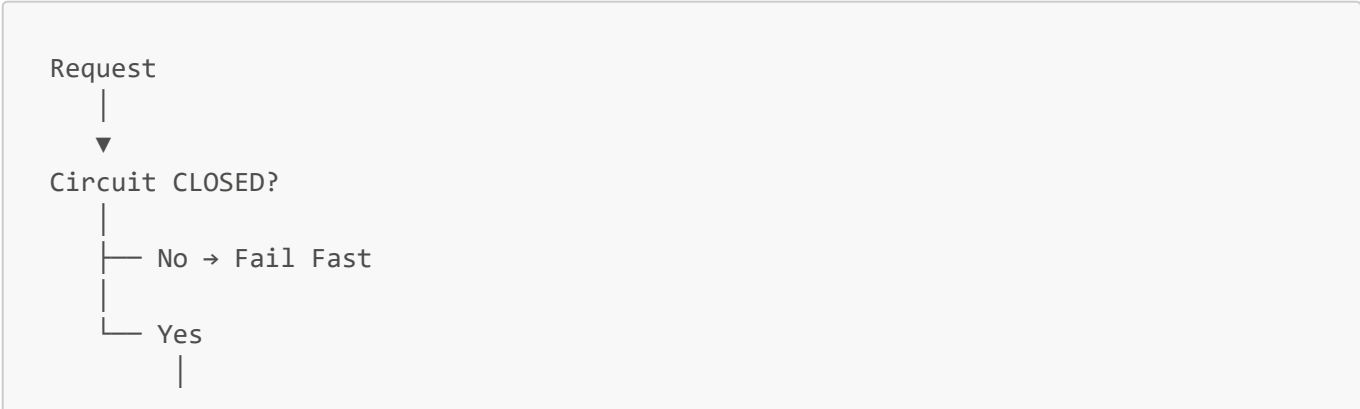


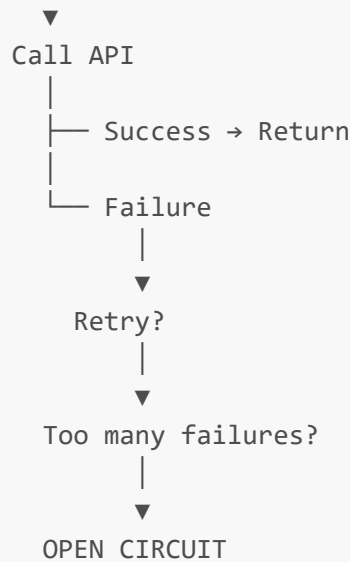
Circuit Breaker vs Retry

This distinction is important in interviews:

Pattern	Purpose
Retry	Try the failed operation again
Circuit Breaker	Stop calling a dependency that's repeatedly failing
Timeout	Stop waiting too long
Fallback	Return an alternative response
Bulkhead	Limit concurrent calls to prevent resource exhaustion

For example:





Interview definition:

The Circuit Breaker pattern prevents repeated calls to a failing downstream service by temporarily stopping requests after a failure threshold is reached. It has three states—Closed, Open, and Half-Open—and allows the system to recover without causing cascading failures.

10. Bulkhead

The **Bulkhead Design Pattern** in C# is a resilience pattern used to **isolate parts of an application so that failure or overload in one part doesn't bring down the entire application**.

Think of a ship's bulkheads: if one compartment floods, the others remain protected.

Simple example

Suppose your application calls three external services:

- Payment Service
- Email Service
- Inventory Service

Without a bulkhead, all requests might share the same thread/concurrency resources. If the Payment Service becomes slow, it can consume all available resources and prevent Email and Inventory from working.

With a bulkhead, each service gets its **own concurrency limit**.

```

using Polly;
using Polly.Bulkhead;

var paymentPolicy = Policy.BulkheadAsync(
    maxParallelization: 10,
    maxQueueingActions: 20);

var emailPolicy = Policy.BulkheadAsync(
    maxParallelization: 5,

```

```

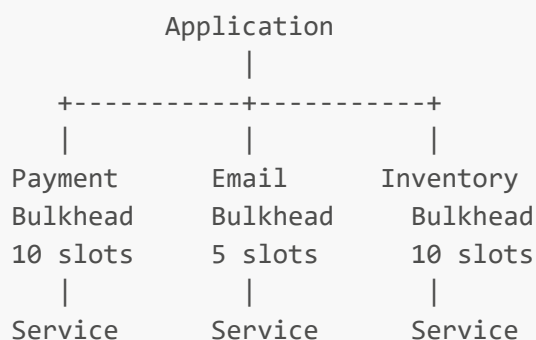
    maxQueueingActions: 10);

    async Task ProcessPayment()
    {
        await paymentPolicy.ExecuteAsync(async () =>
        {
            Console.WriteLine("Processing payment...");
            await Task.Delay(2000);
        });
    }

    async Task SendEmail()
    {
        await emailPolicy.ExecuteAsync(async () =>
        {
            Console.WriteLine("Sending email...");
            await Task.Delay(1000);
        });
    }
}

```

Here:



If Payment gets overloaded and all **10 payment slots** are occupied, new payment requests are rejected/queued according to the policy.

Email is unaffected, because it has its own bulkhead.

A more modern .NET approach

With modern .NET applications, you can implement bulkheads using [SemaphoreSlim](#):

```

public class PaymentService
{
    private readonly SemaphoreSlim _bulkhead = new(
        initialCount: 10,
        maxCount: 10);

    public async Task ProcessPaymentAsync()
    {

```

```

        if (!await _bulkhead.WaitAsync(TimeSpan.FromSeconds(1)))
        {
            throw new Exception("Payment service is overloaded.");
        }

        try
        {
            Console.WriteLine("Processing payment...");

            await Task.Delay(2000);

            Console.WriteLine("Payment completed.");
        }
        finally
        {
            _bulkhead.Release();
        }
    }
}

```

The important part is:

```
await _bulkhead.WaitAsync(...);
```

Only **10 operations can execute concurrently**.

If 100 requests arrive:

```

100 requests
  |
  v
+-----+
| Payment      |
| Bulkhead     |
|              |
| 10 concurrent|
+-----+
  |
  v
Payment Service

```

The remaining requests don't get unlimited access to the service. Depending on your implementation, they can wait for a slot or be rejected.

Bulkhead vs Circuit Breaker

They're often used together:

Pattern	Purpose
Bulkhead	Limits how many resources a failing/slow dependency can consume
Circuit Breaker	Stops calling a dependency when it is known to be failing
Retry	Attempts a failed operation again
Timeout	Prevents an operation from running indefinitely

For example:

```
Request
  |
  v
Timeout
  |
  v
Bulkhead
  |
  v
Circuit Breaker
  |
  v
Retry
  |
  v
External Service
```

A good mental model is:

Bulkhead controls how much damage a dependency can cause; Circuit Breaker stops sending requests when the dependency is failing.

If you're learning **.NET Web API / microservices**, the most useful next step is to see how to implement a **Bulkhead + Circuit Breaker + Retry** using **Microsoft.Extensions.Http.Resilience** and **HttpClient**.

11. Timeout

The **Timeout design pattern** in C# is used when you want an operation to stop waiting after a specified amount of time instead of waiting indefinitely.

1. Simple timeout with **CancellationToken**

This is the preferred modern approach for async code:

```
public async Task<string> GetDataAsync(CancellationToken cancellationToken)
{
    using var client = new HttpClient();

    return await client.GetStringAsync(
```

```
        "https://example.com",  
        cancellationToken);  
    }
```

The caller can impose a timeout:

```
using var cts = new CancellationTokenSource(  
    TimeSpan.FromSeconds(5));  
  
try  
{  
    string result = await GetDataAsync(cts.Token);  
    Console.WriteLine(result);  
}  
catch (OperationCanceledException)  
{  
    Console.WriteLine("Operation timed out.");  
}
```

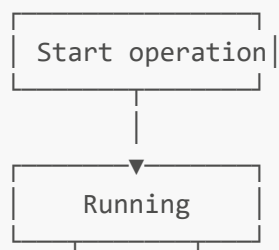
Here, the operation is allowed to run for **5 seconds**. After that, the cancellation token is triggered.

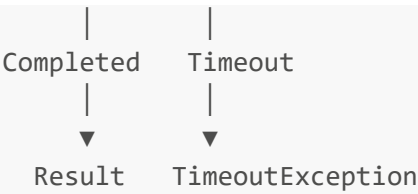
2. `Task.WaitAsync` — very convenient

In modern .NET, `WaitAsync` makes applying a timeout particularly easy:

```
try  
{  
    var result = await GetDataAsync()  
        .WaitAsync(TimeSpan.FromSeconds(5));  
  
    Console.WriteLine(result);  
}  
catch (TimeoutException)  
{  
    Console.WriteLine("Operation timed out.");  
}
```

Conceptually:





Important: `WaitAsync` stops *waiting* for the task; it doesn't necessarily stop the underlying operation. If the underlying operation needs to actually stop, combine it with cancellation.

3. Timeout + cancellation together

This is often the best production pattern:

```
public async Task<string> GetDataAsync(
    CancellationToken cancellationToken)
{
    using var client = new HttpClient();

    return await client.GetStringAsync(
        "https://example.com",
        cancellationToken);
}

public async Task<string> ExecuteAsync(
    CancellationToken cancellationToken)
{
    using var timeoutCts =
        CancellationTokenSource.CreateLinkedTokenSource(
            cancellationToken);

    timeoutCts.CancelAfter(TimeSpan.FromSeconds(5));

    return await GetDataAsync(timeoutCts.Token);
}
```

Now there are **two ways to cancel**:

- The caller cancels the operation.
- The 5-second timeout expires.

4. When to use which approach

Approach	Use when
<code>CancellationTokenSource.CancelAfter()</code>	You need the underlying operation to actually cancel
<code>Task.WaitAsync(timeout)</code>	You simply need to stop waiting after a timeout

Approach	Use when
Both together	You need a robust timeout that also cancels the underlying operation

A useful rule is:

Timeout controls how long you're willing to wait; cancellation controls whether the operation should stop.

If you're preparing for a **C#/.NET interview**, the key distinction between **Task.Delay timeout**, **CancellationToken**, **WaitAsync**, and **Polly timeout/retry policies** is especially worth knowing.

12. Rate Limiting

Rate Limiting Design Pattern in C#

Rate limiting controls how many requests a client, user, API key, or service can make within a given period.

For example:

Allow a user to make **100 API requests per minute**. Request #101 should be rejected or delayed.

It is commonly used to protect APIs from abuse, traffic spikes, and accidental overload.

1. Fixed Window

The simplest approach is:

```

10:00:00 ————— 10:00:59
           max 100 requests

10:01:00 ————— 10:01:59
           max 100 requests

```

In modern .NET, you can use the built-in **System.Threading.RateLimiting** namespace.

```

using System.Threading.RateLimiting;

var limiter = new FixedWindowRateLimiter(
    new FixedWindowRateLimiterOptions
    {
        PermitLimit = 100,
        Window = TimeSpan.FromMinutes(1),
        QueueLimit = 0
    });

var lease = await limiter.AcquireAsync(1);

if (lease.IsAcquired)

```

```
{  
    // Process request  
}  
else  
{  
    // HTTP 429 Too Many Requests  
}
```

2. ASP.NET Core — Recommended Approach

For an ASP.NET Core API, the built-in rate-limiting middleware is usually preferable.

Configure it in `Program.cs`:

```
using System.Threading.RateLimiting;  
  
var builder = WebApplication.CreateBuilder(args);  
  
builder.Services.AddRateLimiter(options =>  
{  
    options.AddFixedWindowLimiter("api", limiterOptions =>  
    {  
        limiterOptions.PermitLimit = 100;  
        limiterOptions.Window = TimeSpan.FromMinutes(1);  
        limiterOptions.QueueLimit = 0;  
    });  
  
    options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;  
});  
  
var app = builder.Build();  
  
app.UseRateLimiter();  
  
app.MapGet("/api/products", () =>  
{  
    return Results.Ok("Products");  
})  
    .RequireRateLimiting("api");  
  
app.Run();
```

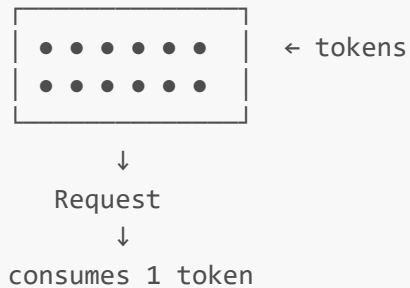
Now the endpoint allows approximately:

```
100 requests  
    ↓  
per 1 minute  
    ↓  
additional requests → HTTP 429
```

3. Token Bucket

A **Token Bucket** is useful when you want to allow controlled bursts.

Imagine a bucket containing 100 tokens:



Example:

```
options.AddTokenBucketLimiter("api", limiterOptions =>
{
    limiterOptions.TokenLimit = 100;
    limiterOptions.TokensPerPeriod = 20;
    limiterOptions.ReplenishmentPeriod = TimeSpan.FromSeconds(10);
    limiterOptions.AutoReplenishment = true;
    limiterOptions.QueueLimit = 0;
});
```

This means the system can handle an initial burst of up to 100 requests, while tokens are replenished over time.

4. Sliding Window

A sliding window avoids some of the boundary problems of fixed windows.

For example, instead of:

```
10:00 ————— 10:01
      100

10:01 ————— 10:02
      100
```

the limiter continuously evaluates a moving time window.

```
options.AddSlidingWindowLimiter("api", limiterOptions =>
{
```

```
limiterOptions.PermitLimit = 100;
limiterOptions.Window = TimeSpan.FromMinutes(1);
limiterOptions.SegmentsPerWindow = 6;
limiterOptions.QueueLimit = 0;
});
```

Which algorithm should you choose?

Algorithm	Good for
Fixed Window	Simple APIs and straightforward limits
Sliding Window	Smoother request distribution
Token Bucket	APIs that need controlled bursts
Concurrency Limiter	Limiting simultaneous operations

A useful distinction is:

Rate limit

"You can make 100 requests per minute."

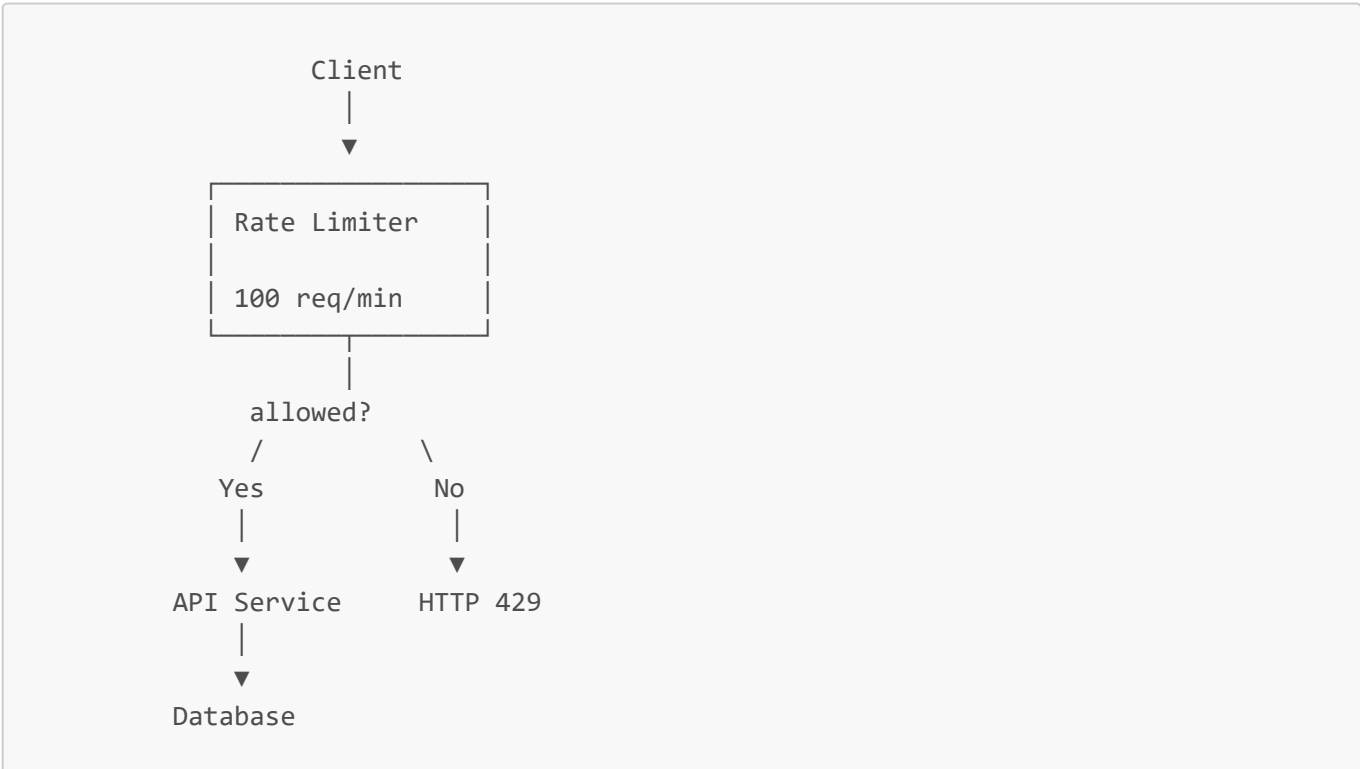
Concurrency limit

"Only 10 requests can execute at the same time."

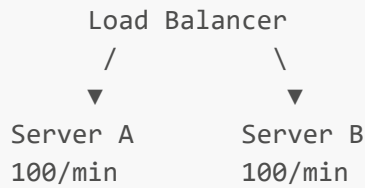
These solve different problems.

Real-world design

For a production API, you might have:



For distributed applications with **multiple API instances**, be careful with an in-memory limiter. Each server has its own counter:



A client could effectively get **200 requests/minute**.

For a distributed rate limit, you generally need shared state—for example, Redis—or a rate limiter at the API gateway/load-balancer layer.

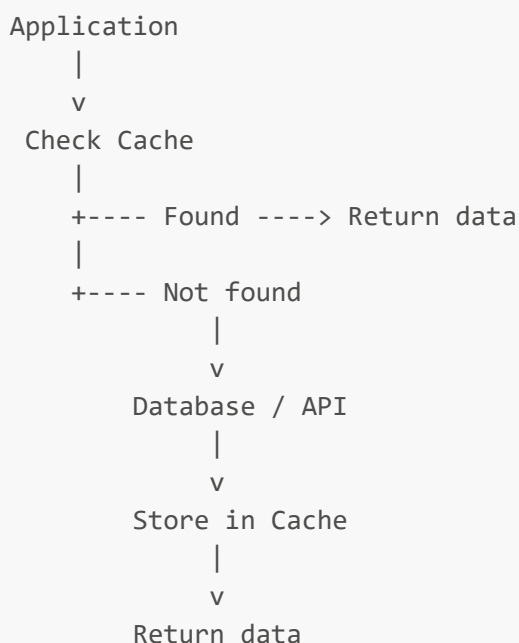
Interview takeaway: In modern C#, know `System.Threading.RateLimiting`, the four common algorithms (fixed window, sliding window, token bucket, concurrency), HTTP **429**, and the difference between **local** and **distributed** rate limiting.

13. Cache-Aside

The **Cache-Aside** pattern (also called **Lazy Loading**) is a common caching pattern in C#/.NET where the application:

1. Checks the cache first.
2. If the data exists → returns it.
3. If not → retrieves it from the database/API.
4. Stores the retrieved data in the cache.
5. Returns the data.

Basic flow



C# example with `IMemoryCache`

Suppose we have a `ProductService`:

```
public class ProductService
{
    private readonly IMemoryCache _cache;
    private readonly IProductRepository _repository;

    public ProductService(
        IMemoryCache cache,
        IProductRepository repository)
    {
        _cache = cache;
        _repository = repository;
    }

    public async Task<Product?> GetProductAsync(int id)
    {
        string cacheKey = $"product:{id}";

        // 1. Check cache
        if (_cache.TryGetValue(cacheKey, out Product? product))
        {
            return product;
        }

        // 2. Cache miss -> get from database
        product = await _repository.GetByIdAsync(id);

        if (product == null)
        {
            return null;
        }

        // 3. Store in cache
        var cacheOptions = new MemoryCacheEntryOptions
        {
            AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(10)
        };

        _cache.Set(cacheKey, product, cacheOptions);

        // 4. Return data
        return product;
    }
}
```

What happens?

For the first request:

```
GetProduct(10)
  ↓
Cache: product:10 ✗
  ↓
Database
  ↓
Product
  ↓
Cache: product:10 ✓
  ↓
Return Product
```

For subsequent requests:

```
GetProduct(10)
  ↓
Cache: product:10 ✓
  ↓
Return Product
```

So the database isn't queried every time.

Cache invalidation

The important part of Cache-Aside is that **the application is responsible for updating/invalidating the cache**.

For example, when updating a product:

```
public async Task UpdateProductAsync(Product product)
{
    await _repository.UpdateAsync(product);

    string cacheKey = $"product:{product.Id}";

    _cache.Remove(cacheKey);
}
```

The next `GetProductAsync()` will get the updated value from the database and put it back into the cache.

You can also update the cache directly:


```
await _repository.UpdateAsync(product);

_cache.Set(
    $"product:{product.Id}",
    product,
    TimeSpan.FromMinutes(10));
```

Whether you **remove** or **update** the cache depends on the application and consistency requirements.

Using Redis / `IDistributedCache`

For a distributed application with multiple API instances, you'd typically use Redis rather than `IMemoryCache`.

```
public async Task<Product?> GetProductAsync(int id)
{
    string cacheKey = $"product:{id}";

    // 1. Cache
    var cachedJson = await _cache.GetStringAsync(cacheKey);

    if (cachedJson != null)
    {
        return JsonSerializer.Deserialize<Product>(cachedJson);
    }

    // 2. Database
    var product = await _repository.GetByIdAsync(id);

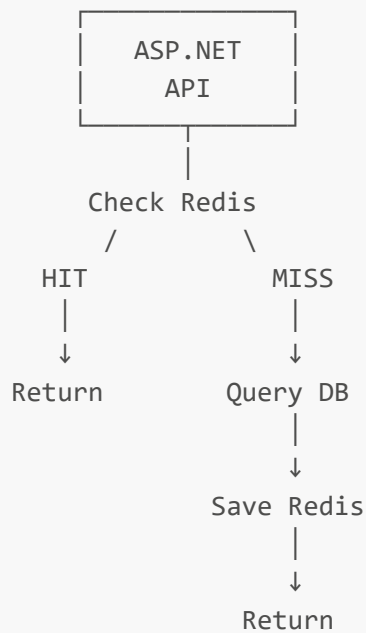
    if (product == null)
    {
        return null;
    }

    // 3. Cache
    var json = JsonSerializer.Serialize(product);

    await _cache.SetStringAsync(
        cacheKey,
        json,
        new DistributedCacheEntryOptions
        {
            AbsoluteExpirationRelativeToNow =
                TimeSpan.FromMinutes(10)
        });

    return product;
}
```

Here the architecture is typically:



Advantages

- **Faster reads** — cache is usually much faster than a database.
- **Reduced database load.**
- **Scales well** for read-heavy applications.
- The cache only contains data that the application actually requests.

Disadvantages

- **Cache invalidation** can be difficult.
- First request after expiration is slower.
- Cached data can become stale.
- With high concurrency, multiple requests can simultaneously miss the cache and hit the database (**cache stampede**).

Cache-Aside vs Read-Through

The key distinction is:

Cache-Aside:

```
Application → Cache
Application → Database
Application → Cache
```

The application explicitly manages the cache.

Read-Through:

Application → Cache → Database

The cache layer itself knows how to load missing data.

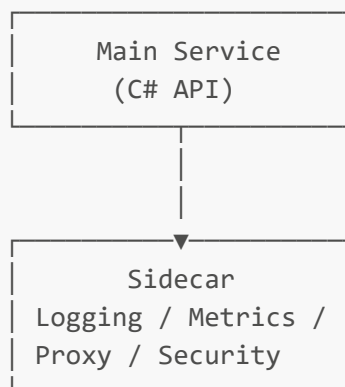
In most ASP.NET Core applications, **Cache-Aside is straightforward and very commonly used**, especially with Redis or `IMemoryCache`.

14. Sidecar

The **Sidecar pattern** in C# is an architectural pattern where you place a **separate component alongside your main application/service** to provide supporting functionality without putting that logic directly into the application.

It is especially common in **microservices, containers, and distributed systems**.

Basic idea



The sidecar runs **alongside** the main application and handles infrastructure-related concerns.

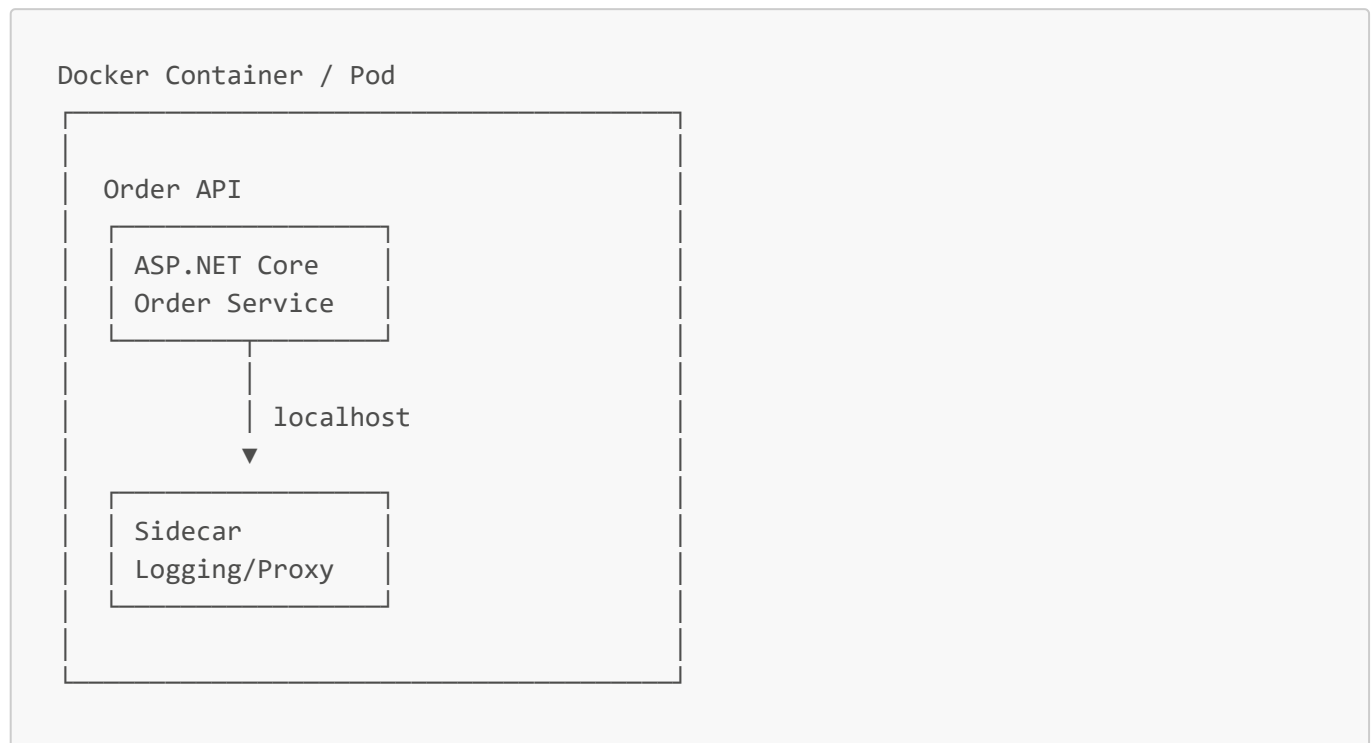
Example in C#

Suppose you have an order service:

```
public class OrderService
{
    public async Task CreateOrderAsync(Order order)
    {
        // Business logic
        await SaveOrderAsync(order);
    }
}
```

Instead of putting telemetry, logging, authentication, service discovery, etc. into `OrderService`, a sidecar can handle those concerns.

For example:



A concrete example is **Envoy** running as a sidecar to a .NET microservice. The C# application doesn't need to implement things such as retries, traffic routing, TLS handling, or service-to-service communication policies itself.

Why use Sidecar?

Common responsibilities include:

- **Logging**
- **Monitoring and metrics**
- **Distributed tracing**
- **Authentication/authorization**
- **Service discovery**
- **Network proxying**
- **Retries and circuit breaking**
- **Configuration management**
- **Secrets handling**

C# example using a local sidecar

Imagine your C# application needs to send telemetry:

```
public class OrderService
{
    private readonly HttpClient _client;

    public OrderService(HttpClient client)
    {
        _client = client;
    }
}
```

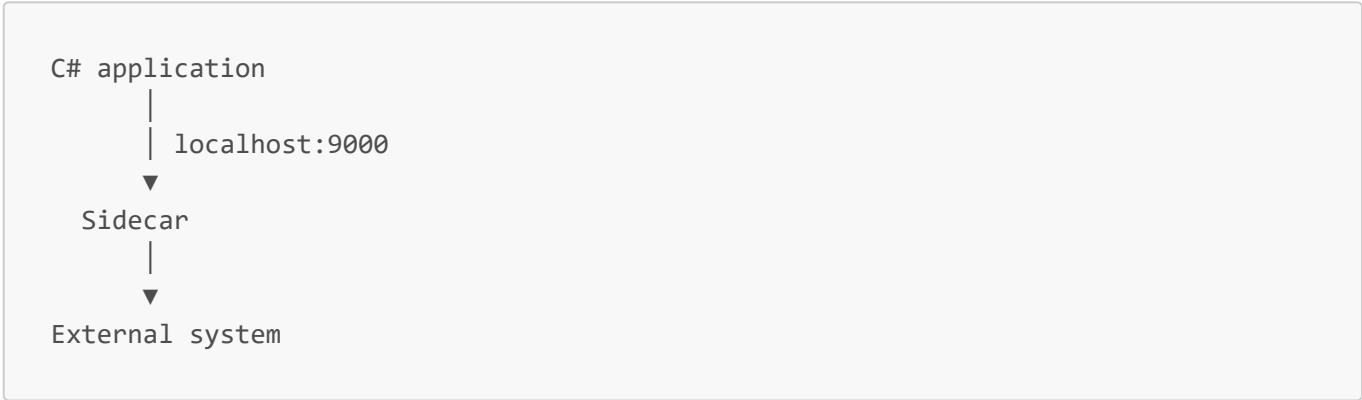
```
    }

    public async Task CreateOrderAsync(Order order)
    {
        await SaveOrderAsync(order);

        await _client.PostAsJsonAsync(
            "http://localhost:9000/events",
            new
            {
                Type = "OrderCreated",
                OrderId = order.Id
            });
    }
}
```

Here `localhost:9000` could be a sidecar responsible for forwarding events to a centralized messaging or telemetry system.

The important point is that:



The C# service doesn't need to know how the external system works.

Sidecar vs Decorator

They're conceptually similar but operate at different levels:

Pattern	Purpose
Decorator	Adds behavior to an object/class
Middleware	Adds behavior to an HTTP/request pipeline
Sidecar	Adds supporting functionality to an application/service externally
Adapter	Makes incompatible interfaces work together

So, if you're preparing for a **C#/.NET interview**, a good one-line definition is:

The Sidecar pattern deploys a separate companion process alongside an application to provide common infrastructure capabilities without coupling those capabilities to the application's business logic.

A classic .NET/cloud example is **ASP.NET Core service + Envoy/Dapr sidecar**, particularly in Kubernetes.

15. Strangler Fig

The **Strangler Fig pattern** is an incremental modernization pattern used when you want to **replace a legacy system gradually** instead of rewriting it all at once.

The name comes from the strangler fig tree: a new structure grows around the old one until the old system can eventually be removed.

Basic idea

Suppose you have a legacy C# application:

```
Client
|
v
Legacy ASP.NET Application
|
+-- Customer
+-- Orders
+-- Payments
+-- Reports
```

Instead of rewriting everything:

```
Client
|
v
New API / Facade
|
+----> New Customer Service
|
+----> New Orders Service
|
+----> Legacy Application
      |
      +-- Payments
      +-- Reports
```

You **move functionality one piece at a time**.

Example in C#

Imagine the old application has this controller:

```
[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    [HttpGet("{id}")]
    public IActionResult GetOrder(int id)
    {
        // Legacy implementation
        var order = LegacyOrderRepository.GetOrder(id);

        return Ok(order);
    }
}
```

You want to replace the order functionality with a new implementation.

Step 1 — Introduce a facade

The facade decides whether a request goes to the new or legacy implementation:

```
[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly INewOrderService _newOrderService;
    private readonly ILegacyOrderService _legacyOrderService;

    public OrdersController(
        INewOrderService newOrderService,
        ILegacyOrderService legacyOrderService)
    {
        _newOrderService = newOrderService;
        _legacyOrderService = legacyOrderService;
    }

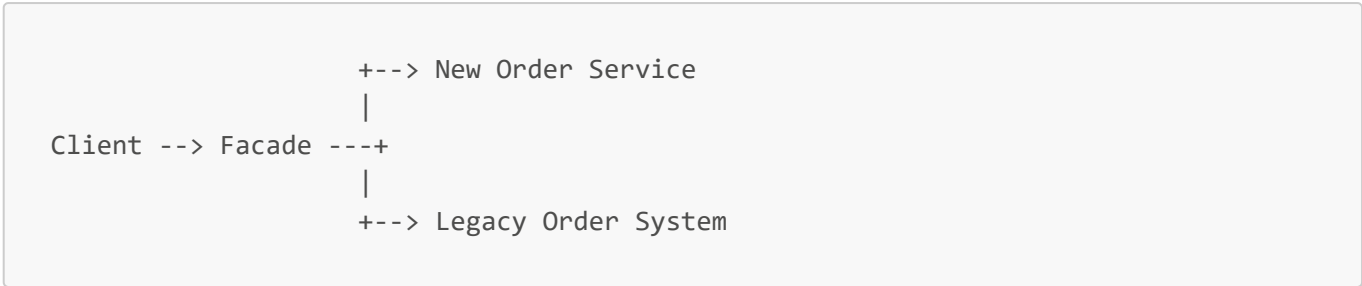
    [HttpGet("{id}")]
    public async Task<IActionResult> GetOrder(int id)
    {
        if (UseNewOrderService(id))
        {
            var order = await _newOrderService.GetOrderAsync(id);
            return Ok(order);
        }

        var legacyOrder = await _legacyOrderService.GetOrderAsync(id);
        return Ok(legacyOrder);
    }

    private bool UseNewOrderService(int id)
    {
        // Could be feature flag, percentage rollout,
    }
}
```

```
        // customer-based routing, etc.
        return id % 2 == 0;
    }
}
```

Now you can gradually migrate traffic.



Step 2 — Move functionality gradually

Initially:

Customer	--> Legacy
Orders	--> Legacy
Payments	--> Legacy
Reports	--> Legacy

Then migrate Orders:

Customer	--> Legacy
Orders	--> NEW
Payments	--> Legacy
Reports	--> Legacy

Then Payments:

Customer	--> Legacy
Orders	--> NEW
Payments	--> NEW
Reports	--> Legacy

Eventually:

Customer	--> NEW
Orders	--> NEW
Payments	--> NEW
Reports	--> NEW

At that point, the legacy application can be removed.

A more realistic ASP.NET Core example

A common approach is to put the routing logic into a service rather than the controller:

```
public interface IOrderService
{
    Task<Order> GetOrderAsync(int id);
}
```

The implementation can initially delegate to the legacy system:

```
public class OrderService : IOrderService
{
    private readonly ILegacyOrderService _legacy;

    public OrderService(ILegacyOrderService legacy)
    {
        _legacy = legacy;
    }

    public Task<Order> GetOrderAsync(int id)
    {
        return _legacy.GetOrderAsync(id);
    }
}
```

You then create the new implementation:

```
public class NewOrderService : IOrderService
{
    private readonly AppDbContext _db;

    public NewOrderService(AppDbContext db)
    {
        _db = db;
    }

    public async Task<Order> GetOrderAsync(int id)
    {
        return await _db.Orders
            .FirstAsync(x => x.Id == id);
    }
}
```

```
    }
}
```

And use a strangler/router service:

```
public class StranglerOrderService : IOrderService
{
    private readonly NewOrderService _newService;
    private readonly ILegacyOrderService _legacyService;
    private readonly IFeatureFlagService _featureFlags;

    public StranglerOrderService(
        NewOrderService newService,
        ILegacyOrderService legacyService,
        IFeatureFlagService featureFlags)
    {
        _newService = newService;
        _legacyService = legacyService;
        _featureFlags = featureFlags;
    }

    public async Task<Order> GetOrderAsync(int id)
    {
        if (_featureFlags.IsEnabled("new-orders"))
        {
            return await _newService.GetOrderAsync(id);
        }

        return await _legacyService.GetOrderAsync(id);
    }
}
```

Your controller doesn't need to know whether the implementation is legacy or new:

```
[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly IOrderService _orders;

    public OrdersController(IOrderService orders)
    {
        _orders = orders;
    }

    [HttpGet("{id}")]
    public async Task<IActionResult> Get(int id)
    {
        var order = await _orders.GetOrderAsync(id);
    }
}
```

```
        return Ok(order);  
    }  
}
```

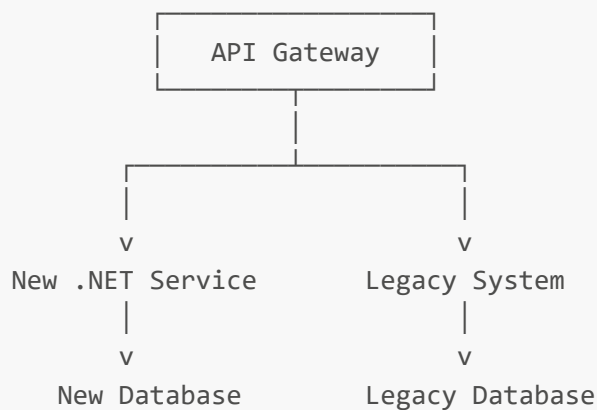
That's an important benefit: **the migration logic is isolated from business logic.**

Where the pattern is useful

The Strangler Fig pattern works particularly well when:

- The legacy application is large.
- A big-bang rewrite would take months/years.
- You need to keep the existing system running.
- You want to migrate module-by-module.
- You need gradual production rollout.
- You want the ability to roll back new functionality.

Typical migration:



Over time, more requests move to the left.

Important consideration: data migration

The hardest part is often **not the code**—it's the data.

For example, if the legacy application owns:

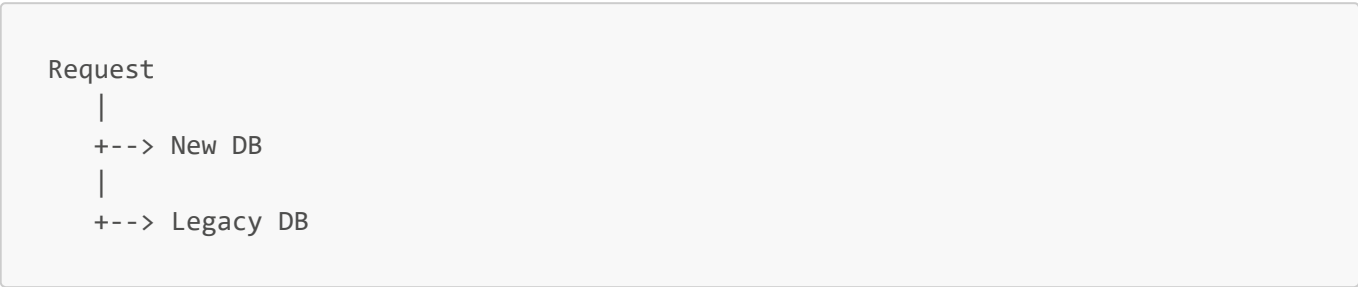
```
Legacy DB  
├── Customers  
├── Orders  
└── Payments
```

and your new Orders service needs its own database, you need a migration strategy.

Possible approaches include:

- 1. **Shared database temporarily**
- 2. **Database replication/CDC**
- 3. **Dual writes**
- 4. **Event-based synchronization**
- 5. **One-time data migration followed by a cutover**

Be particularly careful with dual writes:



A failure between the two writes can leave your systems inconsistent. An outbox/event-driven approach is often safer.

Strangler Fig vs Big-Bang Rewrite

Big-Bang Rewrite	Strangler Fig
Replace everything at once	Replace incrementally
Large upfront effort	Smaller migration steps
High deployment risk	Lower risk
Long period before value	Value delivered continuously
Difficult rollback	Easier rollback
Usually requires "freeze"	Legacy can continue operating
Data migration happens largely at once	Data can migrate gradually

Simple way to remember it

Don't replace the legacy system. Surround it, redirect functionality piece by piece, and eventually remove what's left.

In a modern **C#/.NET** environment, this is often implemented using **ASP.NET Core APIs + a facade/API gateway + feature flags + separate services/modules + gradual data migration**.

16. Ambassador

The **Ambassador Design Pattern** in C# is a structural pattern where you introduce an **intermediary object (the Ambassador)** between a client and another component/service.

The Ambassador handles things such as:

- Logging
- Metrics
- Retry logic
- Authentication
- Caching
- Performance monitoring
- Error handling

The key idea is that the client doesn't directly deal with the underlying service.

Simple example

Suppose you have a slow or remote service:

```
public interface IPaymentService
{
    void ProcessPayment(decimal amount);
}

public class PaymentService : IPaymentService
{
    public void ProcessPayment(decimal amount)
    {
        Console.WriteLine($"Processing payment: {amount}");
    }
}
```

Instead of calling it directly:

```
var service = new PaymentService();
service.ProcessPayment(100);
```

we introduce an **Ambassador**:

```
public class PaymentServiceAmbassador : IPaymentService
{
    private readonly IPaymentService _paymentService;

    public PaymentServiceAmbassador(IPaymentService paymentService)
    {
        _paymentService = paymentService;
    }

    public void ProcessPayment(decimal amount)
    {
        var stopwatch = System.Diagnostics.Stopwatch.StartNew();
```

```

        try
        {
            Console.WriteLine("Starting payment...");

            _paymentService.ProcessPayment(amount);

            Console.WriteLine("Payment successful.");
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Payment failed: {ex.Message}");
            throw;
        }
        finally
        {
            stopwatch.Stop();
            Console.WriteLine(
                $"Payment took {stopwatch.ElapsedMilliseconds} ms");
        }
    }
}

```

Now the client uses the Ambassador:

```

IPaymentService paymentService =
    new PaymentServiceAmbassador(
        new PaymentService());

paymentService.ProcessPayment(100);

```

How it works

```

Client
|
v
PaymentServiceAmbassador
|
|-- Logging
|-- Metrics
|-- Error handling
|-- Retry
|
v
PaymentService

```

The **Ambassador doesn't replace the actual service**. It provides a controlled interface to it.

Ambassador vs Decorator

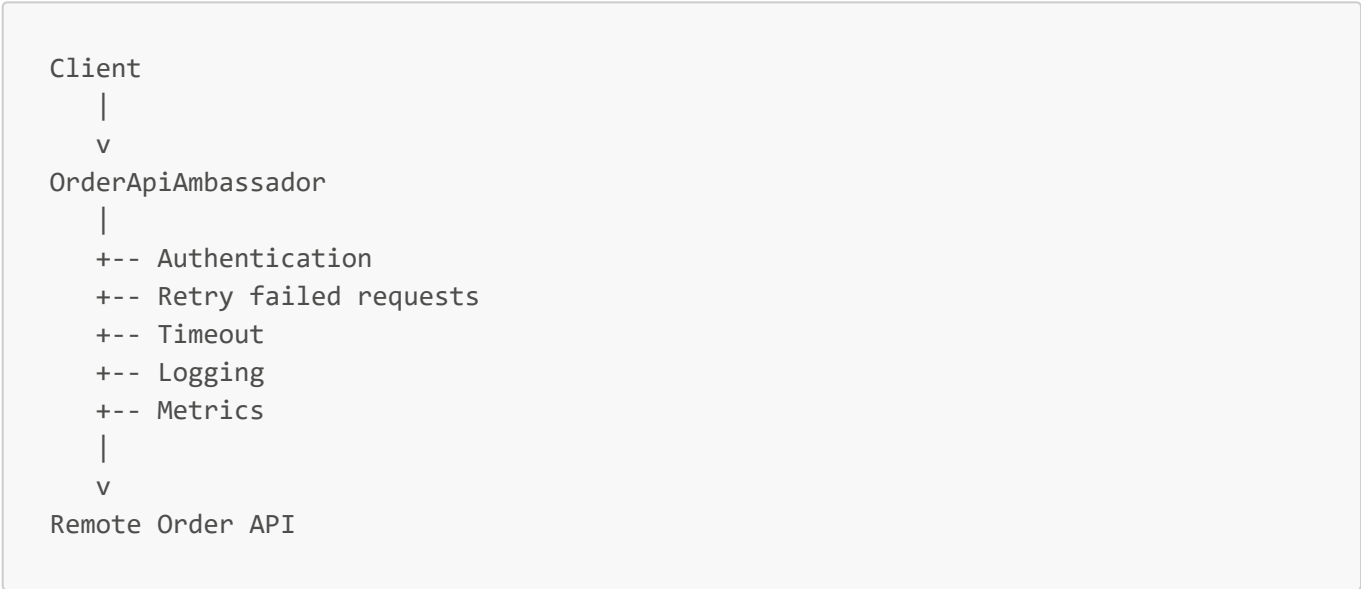
They can look very similar in C# because both commonly use **composition**.

Ambassador	Decorator
Acts as an intermediary for another component	Adds behavior to an object
Often used for remote/expensive operations	Usually used to extend functionality
Commonly handles monitoring, retries, security, etc.	Commonly adds features such as caching/logging
Pattern focuses on communication/control	Pattern focuses on dynamically adding responsibilities

In fact, an Ambassador can be implemented using the same basic technique as a Decorator.

Real-world example

Imagine a client calling a remote API:



The client simply does:

```
var order = ambassador.GetOrder(123);
```

while the Ambassador takes care of the infrastructure concerns.

In short:

The Ambassador Pattern provides a local intermediary that controls and manages access to another object or service, especially when that service is remote, expensive, or unreliable.

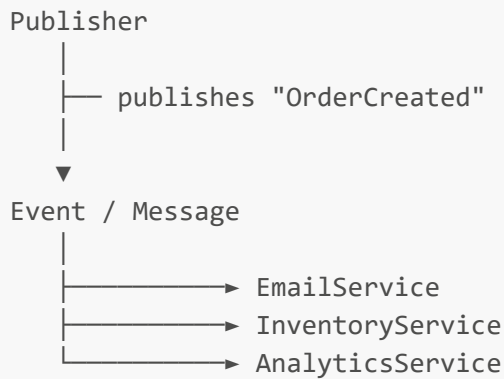
17. Pub/Sub

The **Pub/Sub (Publisher–Subscriber)** pattern in C# is a behavioral/event-driven design pattern where:

- **Publisher** produces events/messages.

- **Subscribers** express interest in specific events.
- The publisher **doesn't need to know who the subscribers are**.
- Subscribers react when the event is published.

A simple mental model:



1. Basic C# example using events

C#'s built-in **event** mechanism is a natural implementation of Pub/Sub.

```

public class Order
{
    public int Id { get; set; }
    public decimal Amount { get; set; }
}
  
```

The publisher:

```

public class OrderService
{
    public event EventHandler<Order>? OrderCreated;

    public void CreateOrder(Order order)
    {
        Console.WriteLine($"Creating order {order.Id}");

        // Publish event
        OrderCreated?.Invoke(this, order);
    }
}
  
```

Subscribers:

```

public class EmailService
{
  
```



```
    public void OnOrderCreated(object? sender, Order order)
    {
        Console.WriteLine(
            $"Sending email for order {order.Id}");
    }
}

public class InventoryService
{
    public void OnOrderCreated(object? sender, Order order)
    {
        Console.WriteLine(
            $"Updating inventory for order {order.Id}");
    }
}
```

Wire them together:

```
var orderService = new OrderService();

var emailService = new EmailService();
var inventoryService = new InventoryService();

orderService.OrderCreated += emailService.OnOrderCreated;
orderService.OrderCreated += inventoryService.OnOrderCreated;

orderService.CreateOrder(new Order
{
    Id = 101,
    Amount = 500
});
```

Output:

```
Creating order 101
Sending email for order 101
Updating inventory for order 101
```

2. Why is this Pub/Sub?

The important part is that `OrderService` doesn't know about `EmailService` or `InventoryService`.

It only knows:

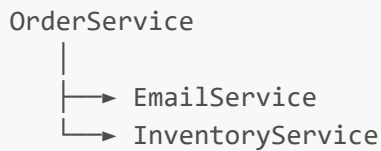
```
public event EventHandler<Order>? OrderCreated;
```

Subscribers decide independently whether they want to listen:

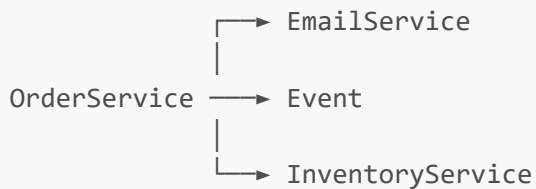
```
orderService.OrderCreated += emailService.OnOrderCreated;
```

This gives you **loose coupling**.

Without Pub/Sub:



With Pub/Sub:



3. A more explicit Event Bus

For larger applications, you might create an **EventBus** instead of having events directly on individual services.

```

public interface IEvent
{
}

public class OrderCreated : IEvent
{
    public int OrderId { get; init; }
    public decimal Amount { get; init; }
}
  
```

Event bus:

```

public class EventBus
{
    private readonly Dictionary<Type, List<Delegate>> _handlers = new();

    public void Subscribe<T>(Action<T> handler)
        where T : IEvent
    {
  
```

```
        var type = typeof(T);

        if (!_handlers.ContainsKey(type))
            _handlers[type] = new List<Delegate>();

        _handlers[type].Add(handler);
    }

    public void Publish<T>(T @event)
        where T : IEvent
    {
        var type = typeof(T);

        if (!_handlers.TryGetValue(type, out var handlers))
            return;

        foreach (var handler in handlers)
        {
            ((Action<T>)handler)(@event);
        }
    }
}
```

Usage:

```
var eventBus = new EventBus();

eventBus.Subscribe<OrderCreated>(e =>
{
    Console.WriteLine($"Email order {e.OrderId}");
});

eventBus.Subscribe<OrderCreated>(e =>
{
    Console.WriteLine($"Update inventory for {e.OrderId}");
});

eventBus.Publish(new OrderCreated
{
    OrderId = 101,
    Amount = 500
});
```

4. In real-world .NET applications

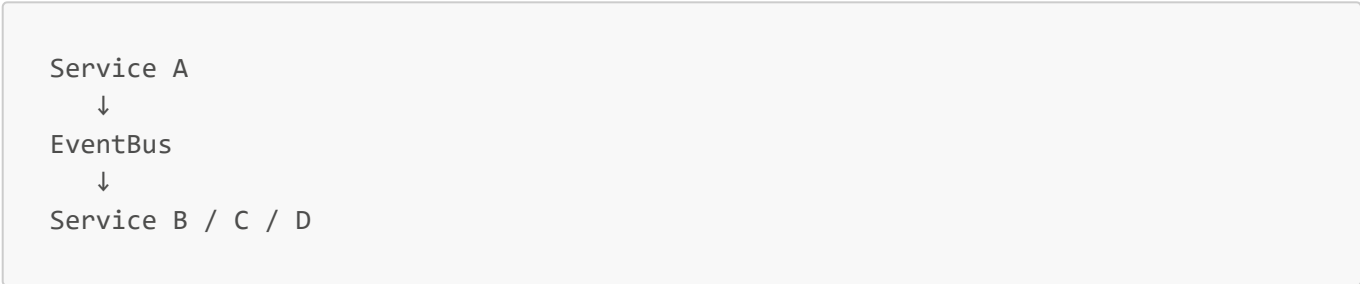
There are several levels of Pub/Sub:

Approach	Communication	Typical use
event	In-process	Simple application events

Approach	Communication	Typical use
Event aggregator / EventBus	In-process	Decoupled application components
MediatR notifications	In-process	CQRS/clean architecture
RabbitMQ	Across processes	Microservices
Azure Service Bus	Across processes	Enterprise/cloud messaging
Kafka	Distributed event streaming	High-volume event streams

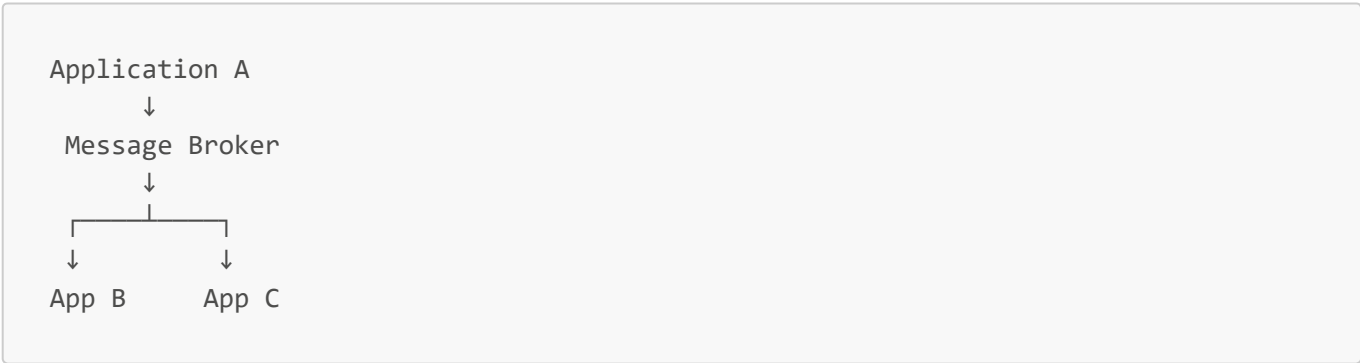
The key distinction is:

In-process Pub/Sub



Everything runs inside the same application/process.

Distributed Pub/Sub



The broker allows publishers and subscribers to be separate applications.

When should you use Pub/Sub?

It's particularly useful when **one action should trigger multiple independent reactions**.

For example:



Instead of putting all of that inside `CreateOrder()`, publish an `OrderCreated` event and let each component subscribe.

Main benefit: low coupling and easy extensibility.

Main trade-off: the flow becomes less obvious, and with asynchronous/distributed Pub/Sub you also have to deal with retries, duplicate messages, ordering, failures, and eventual consistency.

If you're learning this for **C#/.NET interviews**, the important thing to understand next is the difference between **Observer pattern, C# events, EventBus, and RabbitMQ Pub/Sub**—they're related but not interchangeable.

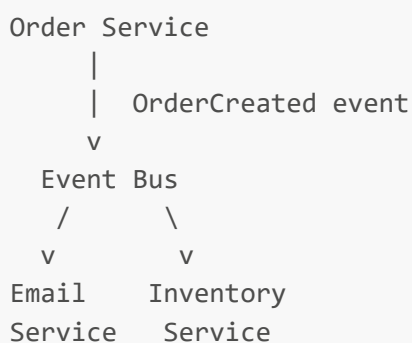
18. Event-Driven Architecture

Event-Driven Architecture (EDA) is a software architecture pattern where components communicate by **publishing and consuming events** rather than calling each other directly.

In C#, it's commonly used when you want systems to be **loosely coupled, scalable, and responsive**.

1. Basic idea

Imagine an e-commerce system:



When an order is created, the Order Service publishes an `OrderCreated` event.

Other services subscribe to that event and perform their own work.

The Order Service **doesn't need to know** that Email or Inventory services exist.

2. Define an event

In C#, an event can simply be a class/record containing the information that happened:

```
public record OrderCreatedEvent(  
    Guid OrderId,  
    Guid CustomerId,  
    decimal TotalAmount,  
);
```

```
        DateTime CreatedAt  
    );
```

The important distinction is that an event describes **something that already happened**.

Good:

```
OrderCreatedEvent  
PaymentCompletedEvent  
CustomerRegisteredEvent
```

Less appropriate for an event:

```
CreateOrderCommand  
ProcessPaymentCommand
```

Those are commands because they ask someone to perform an action.

3. Publisher

Suppose we have an Order Service:

```
public class OrderService  
{  
    private readonly IEventBus _eventBus;  
  
    public OrderService(IEventBus eventBus)  
    {  
        _eventBus = eventBus;  
    }  
  
    public async Task CreateOrderAsync(Order order)  
    {  
        // Save order  
        // await _repository.SaveAsync(order);  
  
        var orderCreated = new OrderCreatedEvent(  
            order.Id,  
            order.CustomerId,  
            order.TotalAmount,  
            DateTime.UtcNow  
        );  
  
        await _eventBus.PublishAsync(orderCreated);  
    }  
}
```

The service publishes the event without knowing who will consume it.

4. Event consumer

The Email Service can subscribe to the event:

```
public class OrderCreatedEmailHandler
{
    public async Task HandleAsync(OrderCreatedEvent @event)
    {
        Console.WriteLine(
            $"Sending confirmation email for order {@event.OrderId}"
        );

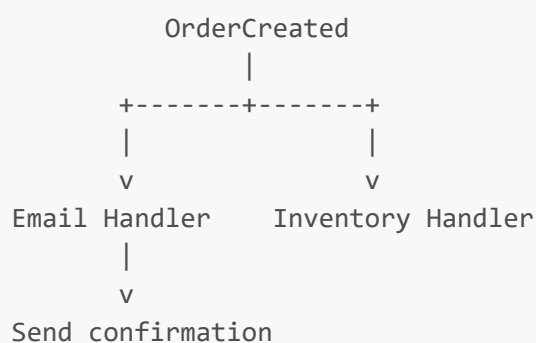
        // Send email
        await Task.CompletedTask;
    }
}
```

The Inventory Service can independently consume the **same event**:

```
public class OrderCreatedInventoryHandler
{
    public async Task HandleAsync(OrderCreatedEvent @event)
    {
        Console.WriteLine(
            $"Reserving inventory for order {@event.OrderId}"
        );

        // Reserve products
        await Task.CompletedTask;
    }
}
```

So one event can have multiple consumers:



5. Simple in-process implementation

If you're building a single ASP.NET Core application, you can implement a lightweight event bus using interfaces:

```
public interface IEvent
{
}

public interface IEventHandler<TEvent>
    where TEvent : IEvent
{
    Task HandleAsync(TEvent @event);
}
```

Event:

```
public record OrderCreatedEvent(
    Guid OrderId,
    decimal Amount
) : IEvent;
```

Handler:

```
public class SendOrderEmailHandler
    : IEventHandler<OrderCreatedEvent>
{
    public Task HandleAsync(OrderCreatedEvent @event)
    {
        Console.WriteLine(
            $"Email sent for order {@event.OrderId}"
        );

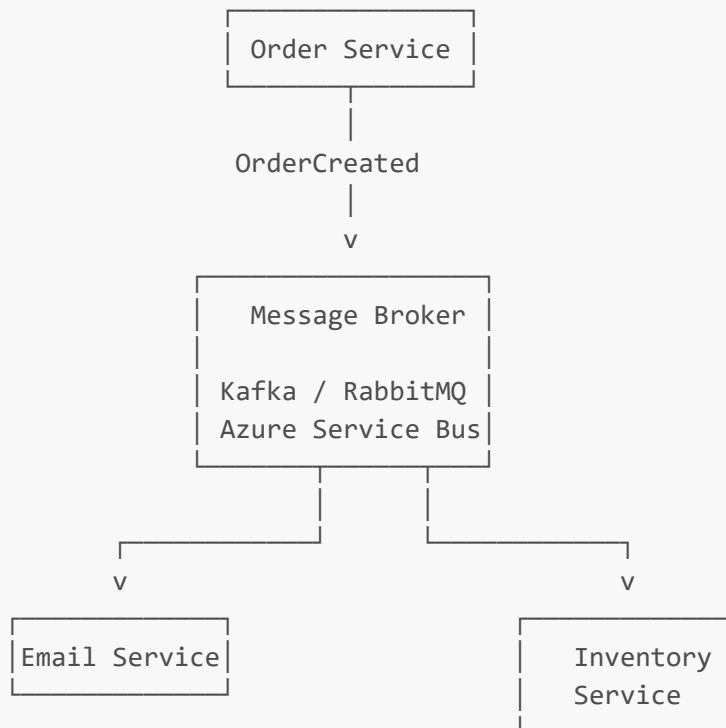
        return Task.CompletedTask;
    }
}
```

You could then have an `EventBus` resolve all handlers for an event and invoke them.

In a real application, however, you will often use a message broker instead of building the infrastructure yourself.

6. Distributed EDA

For microservices, the architecture typically looks like:



Popular choices in the .NET ecosystem include:

- **RabbitMQ** — traditional message broker, queues/routing.
- **Apache Kafka** — high-throughput event streaming.
- **Azure Service Bus** — managed Azure messaging.
- **Amazon SNS/SQS** — managed AWS messaging.
- **MassTransit** — .NET messaging abstraction that works with brokers such as RabbitMQ and Azure Service Bus.

7. EDA vs normal service calls

Traditional synchronous architecture



The Order API depends on the other services being available.

Event-driven architecture

```

Order API
  |
  | OrderCreated
  v
Message Broker
  |
  +----> Inventory
  |
  +----> Email
  |
  +----> Analytics

```

The Order API only needs to successfully publish the event.

This gives you **looser coupling**.

8. Important concepts in production EDA

If you're implementing this in a real C# application, there are several things beyond simply publishing an event.

Idempotency

A message may be delivered more than once.

For example:

```

OrderCreated
OrderCreated    <-- duplicate

```

Your handler should safely process the same event multiple times.

```

if (await _processedEvents.ExistsAsync(eventId))
    return;

await ProcessOrderAsync(orderId);

await _processedEvents.MarkAsProcessedAsync(eventId);

```

Retry

Transient failures should normally be retried:

```

Consumer
  |

```

```

X
|
Retry
|
X
|
Retry
|
v
Success

```

Dead-letter queue

Messages that repeatedly fail can be moved to a dead-letter queue for investigation.

```

Queue
|
v
Consumer
|
X
|
Retry
|
X
|
Dead Letter Queue

```

Outbox Pattern

One of the most important patterns when using EDA with a database.

Suppose you do:

```

await db.SaveOrderAsync(order);
await eventBus.PublishAsync(new OrderCreatedEvent(...));

```

What happens if the database save succeeds but the application crashes before `PublishAsync()`?

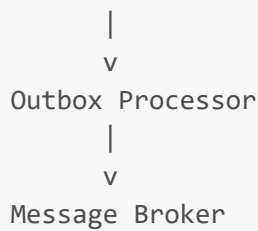
You now have an order that exists but **no event was published**.

The Outbox Pattern solves this by storing the event in the same database transaction:

```

Database Transaction
┌
| Save Order
| Save OrderCreated to Outbox
└

```



This is extremely common in production .NET microservices.

9. When should you use EDA?

EDA is a good fit when:

- Multiple systems need to react to the same business event.
- You want services to be loosely coupled.
- You need asynchronous processing.
- You have microservices.
- You need high scalability.
- Some operations don't need an immediate response.

For example:

```

CustomerRegistered
|
+--> Send Welcome Email
+--> Create Loyalty Account
+--> Create Analytics Record
+--> Notify CRM
  
```

The registration service doesn't need direct dependencies on all four systems.

When not to use it

EDA can add significant complexity.

For a simple CRUD application:

```

Controller
|
v
Service
|
v
Database
  
```

introducing Kafka/RabbitMQ and asynchronous events may be unnecessary.

A useful rule: use events when the business requirement is naturally "**something happened, and interested parties should react to it.**" Use a direct method/API call when you need "**please do this and give me the result.**"

The mental model

Think of EDA as:

COMMAND

"Create this order"

|

v

Order Service

|

v

EVENT

"Order was created"

|

v

v

Email

v

Inventory

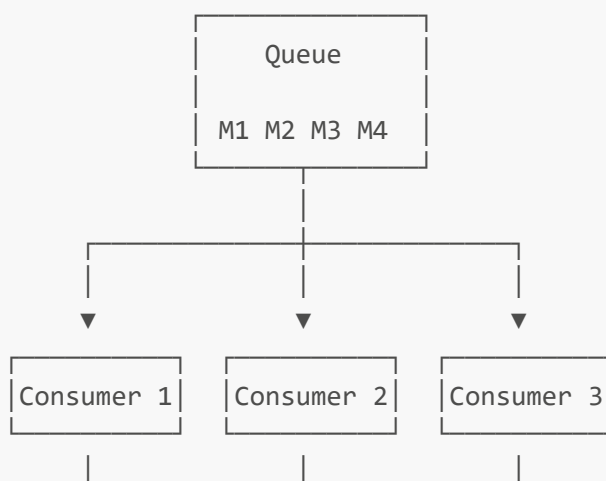
That distinction between **commands and events**, combined with **message brokers, retries, idempotency, and the Outbox Pattern**, is the core of production-grade Event-Driven Architecture in C#.

19. Competing Consumers

The **Competing Consumers** pattern is a messaging/concurrency pattern where **multiple consumers listen to the same queue**, and each message is processed by **exactly one available consumer**.

It is useful when you want to **scale message processing horizontally**.

Basic idea





If Consumer 1 is busy processing M1, another consumer can take M2. This allows work to be processed concurrently.

Simple C# example

You can implement the idea using `System.Threading.Channels`:

```
using System.Threading.Channels;

var queue = Channel.CreateUnbounded<int>();

// Producer
_ = Task.Run(async () =>
{
    for (int i = 1; i <= 10; i++)
    {
        await queue.Writer.WriteAsync(i);
        Console.WriteLine($"Produced: {i}");
    }

    queue.Writer.Complete();
});

// Consumer
async Task ConsumeAsync(string consumerName)
{
    await foreach (var message in queue.Reader.ReadAllAsync())
    {
        Console.WriteLine($"{consumerName} processing {message}");

        // Simulate work
        await Task.Delay(1000);

        Console.WriteLine($"{consumerName} completed {message}");
    }
}

// Multiple competing consumers
var consumers = new[]
{
    ConsumeAsync("Consumer 1"),
    ConsumeAsync("Consumer 2"),
    ConsumeAsync("Consumer 3")
};

await Task.WhenAll(consumers);
```

What happens?

Suppose the producer puts:

```
1 2 3 4 5 6 7 8 9 10
```

The consumers compete for messages:

```
Consumer 1 → 1  
Consumer 2 → 2  
Consumer 3 → 3  
Consumer 1 → 4  
Consumer 2 → 5  
Consumer 3 → 6  
...
```

The exact distribution isn't guaranteed. The important point is that **each message is consumed by one consumer**, rather than every consumer receiving every message.

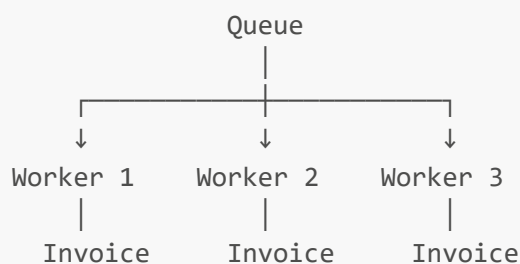
Why use it?

Imagine you have an e-commerce application generating 100,000 orders that need invoice generation.

Instead of:

```
Queue  
↓  
One consumer  
↓  
100,000 invoices
```

you can have:



If processing takes too long, you can add more consumers:

```
3 consumers → 10 consumers → 50 consumers
```

This is **horizontal scaling of message processing**.

Important considerations

1. Messages should generally be independent

If message **B** depends on message **A**, simply distributing them across consumers can cause problems.

2. Make processing idempotent

A message may sometimes be delivered again due to failures/retries.

For example, don't blindly do:

```
account.Balance -= 100;
```

if the same message could be processed twice.

Instead, use an idempotency key/message ID to ensure the operation isn't applied twice.

3. Handle failures

A real implementation needs a strategy for:

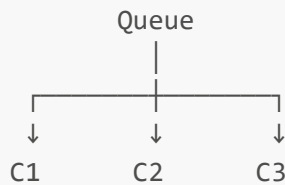
```
Consumer crashes
  ↓
Message wasn't successfully processed
  ↓
Message becomes available again
  ↓
Another consumer retries it
```

This is commonly handled by message brokers using acknowledgements, visibility timeouts, retries, or dead-letter queues.

Competing Consumers vs Pub/Sub

This distinction is important.

Competing Consumers



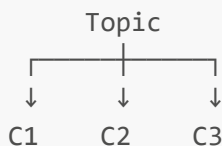
Message 1 → C1

Message 2 → C2

Message 3 → C3

One consumer processes each message.

Publish/Subscribe



Message 1 → C1

Message 1 → C2

Message 1 → C3

Every subscriber receives the message.

So, if you have a **OrderCreated** event and want **Billing, Inventory, and Notifications all to react**, Pub/Sub is appropriate.

If you have 1 million image-processing jobs and want to **divide the workload among workers**, Competing Consumers is appropriate.

In real C# applications

Competing Consumers are commonly implemented with message brokers such as **RabbitMQ, Azure Service Bus, Amazon SQS, or Kafka consumer groups**, rather than an in-memory **Channel<T>**.

A useful mental model is:

Queue + multiple workers = Competing Consumers.

If you're preparing for a **C#/.NET interview**, I can also explain this pattern with a **real RabbitMQ/Azure Service Bus example**, including acknowledgements, retries, and dead-letter queues.

20. Leader Election

The **Leader Election** pattern is used in distributed systems when multiple instances of the same application are running, but **only one instance should act as the leader** for a particular task.

For example, suppose you have 5 instances of a background worker:

```

Worker 1  ┌
Worker 2  │
Worker 3  └─> Leader Election ─> Worker 3 becomes Leader
Worker 4  │
Worker 5  └

```

Only Worker 3 executes the leader-only job. If Worker 3 crashes, another worker can become leader.

Typical use cases

- Running a scheduled job on **only one server**
- Processing a queue where one coordinator is needed
- Managing a singleton background service
- Failover between application instances
- Distributed cache maintenance
- Cluster coordination

A simple C# example

The important part is that **the election mechanism must be shared by all instances**. A local `lock` or `static` variable isn't sufficient because each server/process has its own memory.

For demonstration, imagine a shared store that supports an atomic "acquire lease" operation:

```

public interface ILeaderElection
{
    Task<bool> TryBecomeLeaderAsync(
        string leaderName,
        TimeSpan leaseDuration,
        CancellationToken cancellationToken);

    Task RenewLeadershipAsync(
        string leaderName,
        TimeSpan leaseDuration,
        CancellationToken cancellationToken);

    Task ReleaseLeadershipAsync(
        string leaderName,
        CancellationToken cancellationToken);
}

```

A background service can then use it:

```

public class LeaderWorker : BackgroundService
{
    private readonly ILeaderElection _leaderElection;
}

```

```

public LeaderWorker(ILeaderElection leaderElection)
{
    _leaderElection = leaderElection;
}

protected override async Task ExecuteAsync(
    CancellationToken stoppingToken)
{
    const string leaderName = "my-background-job";

    while (!stoppingToken.IsCancellationRequested)
    {
        bool isLeader =
            await _leaderElection.TryBecomeLeaderAsync(
                leaderName,
                TimeSpan.FromSeconds(30),
                stoppingToken);

        if (isLeader)
        {
            Console.WriteLine("I am the leader!");

            await DoLeaderWorkAsync(stoppingToken);

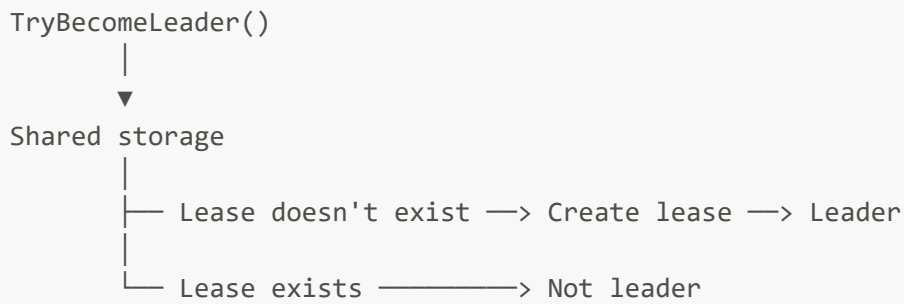
            await _leaderElection.RenewLeadershipAsync(
                leaderName,
                TimeSpan.FromSeconds(30),
                stoppingToken);
        }
        else
        {
            Console.WriteLine("Another instance is the leader.");
        }

        await Task.Delay(
            TimeSpan.FromSeconds(10),
            stoppingToken);
    }
}

private async Task DoLeaderWorkAsync(
    CancellationToken cancellationTokentoken)
{
    // Work that must only be performed by one instance.
    await Task.Delay(1000, cancellationTokentoken);
}
}

```

The crucial operation is something like:



The lease is important

You generally don't want to simply write:

```
if (!leaderExists)
{
    becomeLeader();
}
```

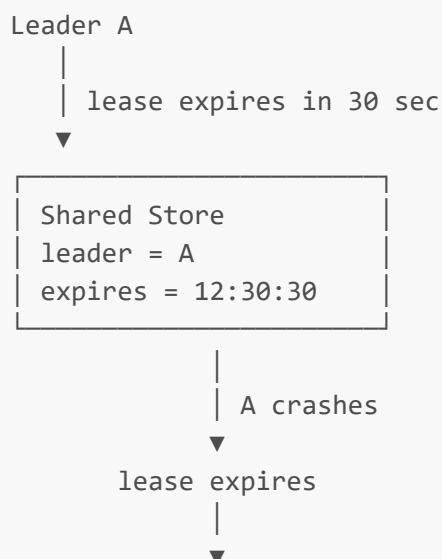
because two servers could execute this simultaneously:

```
Server A: "No leader exists"
Server B: "No leader exists"

Server A: "I am leader"
Server B: "I am leader"    ← Problem
```

The acquisition needs to be **atomic**.

A common approach is a **lease with expiration**:



Worker B
becomes leader

This is why leader election is often implemented using infrastructure such as **Redis, SQL Server, ZooKeeper, etcd, or Kubernetes**, rather than just C# synchronization primitives.

Using Redis

For example, Redis can implement the basic idea using an atomic **SET** with **NX** and an expiration:

```
var acquired = await database.StringSetAsync(  
    "my-app:leader",  
    instanceId,  
    TimeSpan.FromSeconds(30),  
    When.NotExists);
```

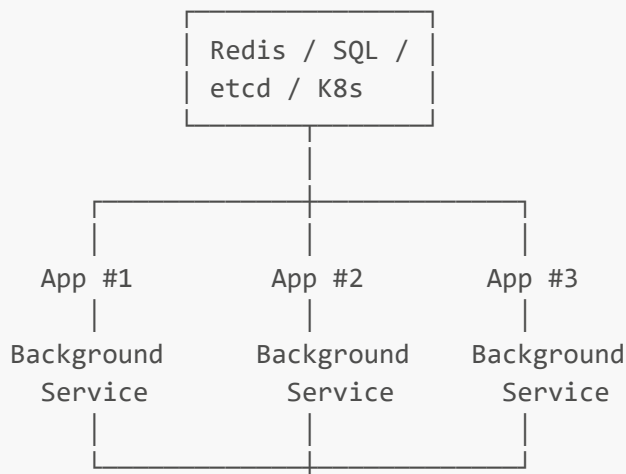
If **acquired** is **true**, this instance acquired the leadership lease.

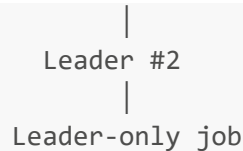
```
if (acquired)  
{  
    // We are leader  
    await RunLeaderTasksAsync();  
}
```

But there's an important subtlety: **renewal and releasing the lease should verify that the stored value still belongs to your instance**. Otherwise, an expired lease could have been acquired by another instance and the old leader might accidentally delete or modify the new leader's lease.

In ASP.NET Core

A common architecture is:





The important distinction is:

Leader election is a distributed coordination pattern, not a C# synchronization pattern.

`lock`, `Monitor`, `Mutex`, and `Semaphore` can coordinate threads/processes under certain conditions, but they don't by themselves solve leader election across multiple machines.

If you're designing this for a **real .NET application**, I can also show the recommended implementation using **Redis + `BackgroundService`**, **SQL Server**, or **Kubernetes**, including failure handling and lease renewal.

8. Concurrency Patterns

Useful when working with **C#, threading and high-performance applications**.

1. Producer–Consumer

The **Producer–Consumer pattern** is a concurrency design pattern used when:

- A **Producer** generates data/work items.
- A **Consumer** processes those items.
- Both operate independently, often at different speeds.
- A **thread-safe queue/buffer** sits between them.

A common C# implementation uses `Channel<T>`.

Basic example

```

using System;
using System.Threading;
using System.Threading.Channels;
using System.Threading.Tasks;

class Program
{
    static async Task Main()
    {
        var channel = Channel.CreateBounded<int>(10);

        // Producer
        var producer = Task.Run(async () =>
        {
            for (int i = 1; i <= 20; i++)
            {
                await channel.Writer.WriteAsync(i);
            }
        });
    }
}
  
```

```

        Console.WriteLine($"Produced: {i}");

        await Task.Delay(100);
    }

    channel.Writer.Complete();
});

// Consumer
var consumer = Task.Run(async () =>
{
    await foreach (var item in channel.Reader.ReadAllAsync())
    {
        Console.WriteLine($"Consumed: {item}");

        await Task.Delay(300);
    }
});

await Task.WhenAll(producer, consumer);
}

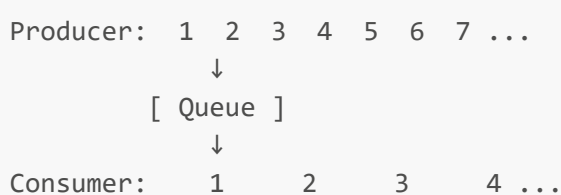
```

How it works



The producer doesn't need to know **when or how** the consumer processes the data.

For example:



Because the consumer is slower, items temporarily wait in the channel.

Why use `CreateBounded`?

```
var channel = Channel.CreateBounded<int>(10);
```

This limits the buffer to **10 items**.

If the producer gets ahead:

```

Producer → [1][2][3][4][5][6][7][8][9][10] → Consumer
              ↑
            Buffer full
  
```

`WriteAsync()` will wait until space becomes available. This is called **backpressure**.

Without a bounded buffer, a very fast producer could potentially keep adding work and consume excessive memory.

Multiple producers and consumers

The pattern becomes particularly useful with worker pools:

```

Producer 1 ┌──┐
Producer 2 ┤──┤ → [ Shared Channel ] → Consumer 1
Producer 3 └──┘
                                   ├───> Consumer 2
                                   └───> Consumer 3
  
```

For example, you might have:

- 1 producer reading messages from a network/API
- 4 consumers processing those messages
- A `Channel<Order>` between them

```

var channel = Channel.CreateBounded<Order>(100);

async Task Produce()
{
    foreach (var order in orders)
    {
        await channel.Writer.WriteAsync(order);
    }

    channel.Writer.Complete();
}

async Task Consume()
{
  
```



```
    await foreach (var order in channel.Reader.ReadAllAsync())
    {
        await ProcessOrder(order);
    }
}
```

Then start several consumers:

```
var consumers = Enumerable.Range(0, 4)
    .Select(_ => Consume());

await Task.WhenAll(consumers);
```

Each order is taken by one consumer, so the work is distributed across the workers.

Channel<T> vs ConcurrentQueue<T>

Both can be used, but they solve slightly different problems:

	Channel<T>	ConcurrentQueue<T>
Thread-safe	☑	☑
Async waiting	☑ Built in	✗
Backpressure	☑	✗ Built in
Producer/consumer semantics	☑	Manual
Great for async pipelines	☑	Sometimes

For modern asynchronous C# applications, **Channel<T>** is usually the cleaner choice for implementing Producer–Consumer.

Real-world examples

The pattern is useful for:

- Background job processing
- Order processing
- Logging pipelines
- Image/video processing
- Message processing
- File processing
- Web crawling
- Sending emails
- CPU-intensive worker pools

Key idea: Producer and Consumer are decoupled by a thread-safe buffer, allowing each side to work at its own pace while controlling concurrency and backpressure.

2. Thread Pool

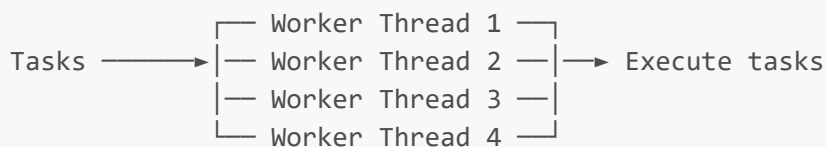
The **Thread Pool** design pattern in C# is a concurrency pattern where a set of reusable worker threads is maintained so that applications don't need to create a new thread for every task.

Core idea

Instead of:

```
Task 1 → Create Thread → Execute → Destroy
Task 2 → Create Thread → Execute → Destroy
Task 3 → Create Thread → Execute → Destroy
```

a thread pool works like:



Threads are **reused**, which reduces the overhead of repeatedly creating and destroying threads.

C# example

Modern C# applications normally use the **Task Parallel Library (TPL)**, which uses the .NET thread pool underneath:

```
using System;
using System.Threading;
using System.Threading.Tasks;

class Program
{
    static void Main()
    {
        for (int i = 1; i <= 5; i++)
        {
            int taskId = i;

            Task.Run(() =>
            {
                Console.WriteLine(
                    $"Task {taskId} running on thread {Thread.CurrentThread.ManagedThreadId}");

                Thread.Sleep(1000);
            });
        }
    }
}
```

```
        Console.ReadLine();  
    }  
}
```

Here:

```
Task.Run(...)
```

queues work to the **.NET ThreadPool**. The runtime chooses an available pool thread to execute it.

Using **ThreadPool** directly

C# also provides the lower-level **ThreadPool** API:

```
using System;  
using System.Threading;  
  
class Program  
{  
    static void Main()  
    {  
        ThreadPool.QueueUserWorkItem(DoWork, "Task 1");  
        ThreadPool.QueueUserWorkItem(DoWork, "Task 2");  
        ThreadPool.QueueUserWorkItem(DoWork, "Task 3");  
  
        Console.ReadLine();  
    }  
  
    static void DoWork(object? state)  
    {  
        Console.WriteLine(  
            $"{state} running on thread " +  
            $"{Thread.CurrentThread.ManagedThreadId}");  
    }  
}
```

Why use a Thread Pool?

Advantages:

- Avoids the cost of creating a thread for every small task.
- Reuses existing threads.
- Limits the number of concurrent worker threads.
- Automatically manages thread creation and retirement.
- Works well for many short-lived background operations.

- Forms the foundation for APIs such as `Task.Run`, async continuations, timers, and parts of ASP.NET Core.

Thread Pool vs creating `Thread`

Thread	Thread Pool
You create/manage the thread	Runtime manages threads
Higher creation overhead	Threads are reused
Good for dedicated long-running work	Good for short/medium work
You manage lifetime	Runtime manages lifetime
Limited control over scheduling	Runtime controls scheduling

Important interview point

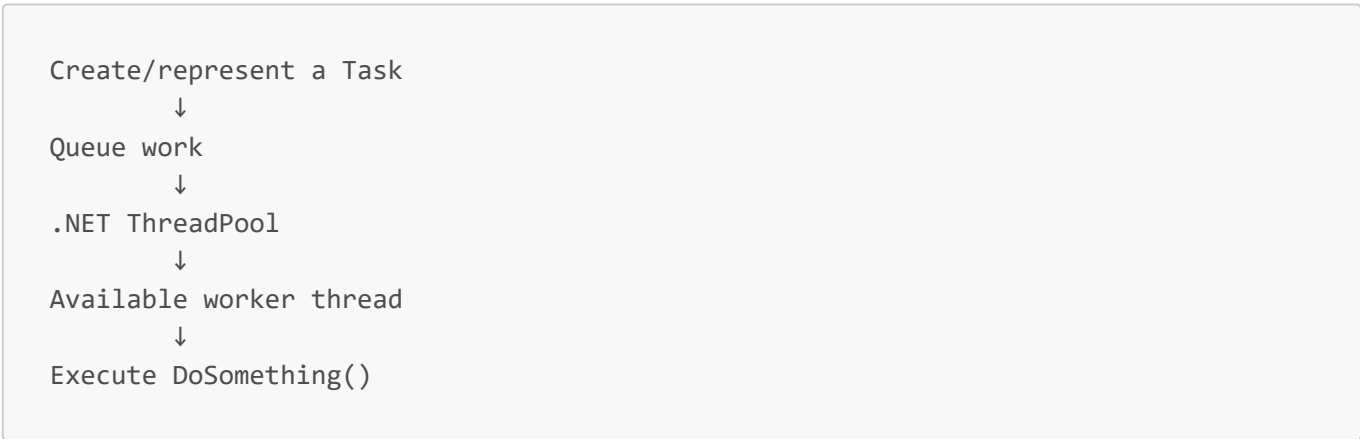
Thread Pool $\neq$ Task.

`Task` represents **work**; the .NET ThreadPool provides threads that can execute that work.

For example:

```
Task.Run(() => DoSomething());
```

conceptually means:



Also, **don't use `Task.Run` merely to make naturally asynchronous I/O asynchronous**. For example, database/network/file APIs that already provide `async` methods should generally be awaited directly:

```
var data = await httpClient.GetStringAsync(url);
```

rather than:

```
var data = await Task.Run(() =>
    httpClient.GetStringAsync(url));
```

The latter unnecessarily involves the thread pool for an operation that doesn't need a worker thread while waiting on I/O.

3. Future / Promise

In C#, **Future / Promise** is a design pattern for representing a result that **will be available later**—typically when an asynchronous operation finishes.

In modern C#, the equivalent is mainly:

- **Task<T>** → a future/promise for a value
- **Task** → a future/promise for completion without a value
- **async / await** → convenient syntax for consuming those futures

1. The basic idea

Imagine you ask someone:

"Get me the user's profile. I don't need it right now, but give it to me when you have it."

The **Future/Promise** represents that pending result.

```
Task<User> future = GetUserAsync();
```

At this point, **future** doesn't contain the **User** yet. It represents a promise that eventually it will contain one.

Later:

```
User user = await future;
```

await waits asynchronously until the result is available.

2. Example

```
public async Task<string> GetDataAsync()
{
    await Task.Delay(2000);

    return "Hello from server";
}
```

Calling it:

```
Task<string> future = GetDataAsync();

Console.WriteLine("Doing other work...");

string result = await future;

Console.WriteLine(result);
```

Conceptually:

```
GetDataAsync()
  |
  v
+-----+
| Task<string> |
| Pending      |
+-----+
  |
  | 2 seconds
  v
+-----+
| "Hello..." |
+-----+
  |
  v
  await
```

3. Promise vs Future

The terminology comes from languages/frameworks such as JavaScript, Java, and C++.

A useful distinction is:

Future

"Here is something that will eventually give you a result."

Promise

"I will eventually provide a result."

In C#, `Task<T>` can effectively play **both roles**, although C# doesn't normally call it a "Promise."

For example:

```
Task<int> future = CalculateAsync();
```

The caller gets the **future**.

If you're implementing an operation yourself, `TaskCompletionSource<T>` lets you control when the **promise** is completed:

```
var tcs = new TaskCompletionSource<int>();

Task<int> future = tcs.Task;

// Somewhere later...
tcs.SetResult(42);

// Consumer:
int result = await future;
```

Here:

```
TaskCompletionSource<int>
    |
    | SetResult(42)
    v
Task<int>
    |
    | await
    v
    42
```

4. Why `TaskCompletionSource` is useful

It is particularly useful when an API is **event/callback-based**, but you want to expose a modern `async` API.

For example:

```
public Task<string> WaitForMessageAsync()
{
    var tcs = new TaskCompletionSource<string>();

    messageReceived += message =>
    {
        tcs.SetResult(message);
    };
}
```

```
        return tcs.Task;  
    }
```

Now callers can write:

```
string message = await WaitForMessageAsync();
```

instead of dealing directly with callbacks.

5. The important mental model

Think of this:

```
Task<int> task = CalculateAsync();
```

as:

"I don't have an **int** yet. I have a handle to an **int** that will eventually exist."

And:

```
int value = await task;
```

means:

"Give me the **int** when it becomes available, without blocking the thread."

This is the core idea behind the **Future/Promise pattern**.

6. In an interview

A concise answer would be:

Future/Promise is an asynchronous programming pattern where an object represents a result that isn't available yet but will become available later. In C#, `Task` and `Task<T>` implement this concept, while `TaskCompletionSource<T>` can be used when we need to manually fulfill a promise. `async/await` provides a convenient way to consume these tasks without blocking the thread.

If you're learning **C# design patterns for interviews**, the next useful comparison is **Future/Promise vs Observer vs Callback**, because they look similar but solve different problems.

4. Active Object

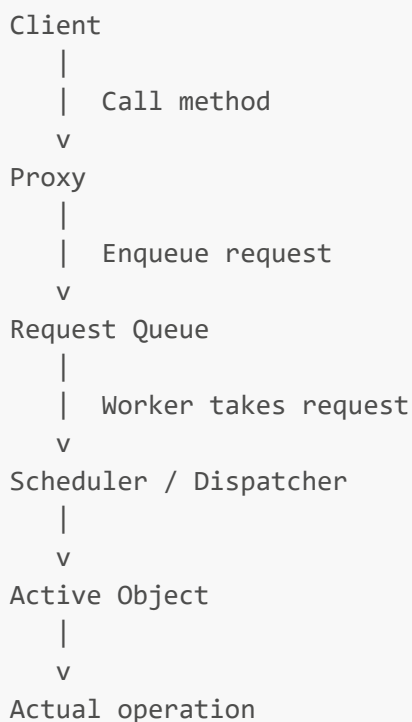
The **Active Object pattern** is a concurrency/design pattern where an object **owns its own thread of execution** and clients interact with it through asynchronous method calls.

The key idea is:

Separate the method invocation from the method execution.

Instead of the caller directly executing a method, the caller puts a request into a queue. The Active Object processes those requests, typically on a dedicated worker thread.

Basic structure



C# example

Here's a simple implementation using `BlockingCollection<T>`:

```
using System;
using System.Collections.Concurrent;
using System.Threading;
using System.Threading.Tasks;

public class ActiveObject
{
    private readonly BlockingCollection<Action> _queue = new();

    public ActiveObject()
    {
        Task.Run(ProcessQueue);
    }
}
```

```

// Called by clients
public Task<int> CalculateAsync(int x, int y)
{
    var tcs = new TaskCompletionSource<int>();

    _queue.Add(() =>
    {
        try
        {
            // Actual work happens on the Active Object's worker
            int result = x + y;
            tcs.SetResult(result);
        }
        catch (Exception ex)
        {
            tcs.SetException(ex);
        }
    });

    return tcs.Task;
}

private void ProcessQueue()
{
    foreach (var action in _queue.GetConsumingEnumerable())
    {
        action();
    }
}

public void Stop()
{
    _queue.CompleteAdding();
}
}

```

Client code:

```

var activeObject = new ActiveObject();

Task<int> result = activeObject.CalculateAsync(10, 20);

Console.WriteLine("Client continues doing other work...");

Console.WriteLine(await result); // 30

activeObject.Stop();

```

What's happening?

When you call:

```
activeObject.CalculateAsync(10, 20);
```

the calculation **doesn't immediately run on the caller's thread**.

Instead:

1. `CalculateAsync()` creates a request.
2. The request is placed in `_queue`.
3. The client gets a `Task<int>` immediately.
4. The worker thread takes the request from the queue.
5. The worker executes the calculation.
6. `TaskCompletionSource` completes the returned task.
7. The client receives the result when it awaits the task.

Why use Active Object?

It's useful when you want to:

- Serialize access to an object's state.
- Avoid multiple threads directly manipulating the same object.
- Perform expensive work asynchronously.
- Queue operations.
- Control concurrency.
- Decouple **requesting work** from **executing work**.

For example, imagine a `FileWriter` that must write files sequentially:

```
fileWriter.WriteAsync("A");
fileWriter.WriteAsync("B");
fileWriter.WriteAsync("C");
```

Rather than having three callers simultaneously access the file, the Active Object can queue the operations:

```
Write A → Write B → Write C
```

Active Object vs `lock`

They're related to concurrency, but solve different problems.

<code>lock</code>	Active Object
Protects shared state	Encapsulates execution
Caller executes the method	Request is queued

lock	Active Object
Caller may block	Caller can continue
Synchronization primitive	Higher-level concurrency pattern
Good for short critical sections	Good for asynchronous/queued work

Important C# point

In modern C#, you often **don't need to implement the pattern manually**. `Task`, `async/await`, `Channel<T>`, `TPL Dataflow`, and actor-style libraries can provide much of the infrastructure.

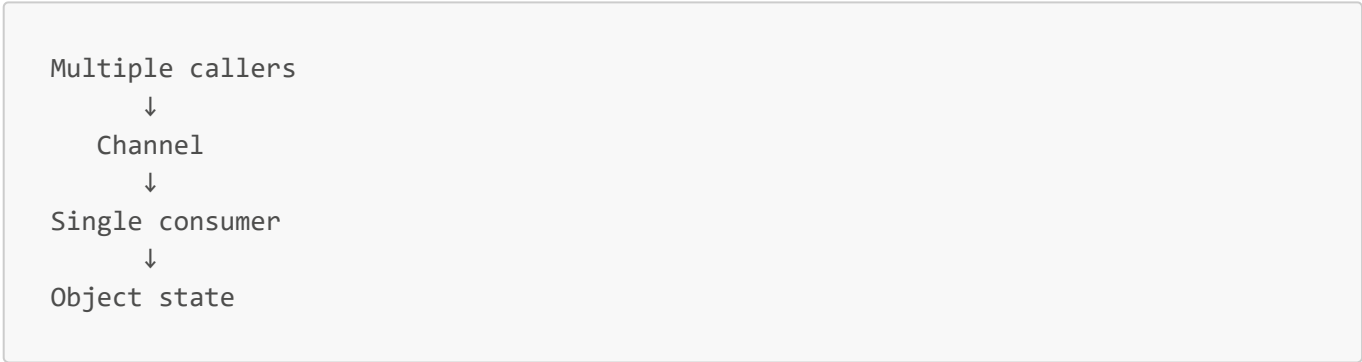
For example, `Channel<T>` is particularly natural for an Active Object:

```
private readonly Channel<Func<Task>> _queue =
    Channel.CreateUnbounded<Func<Task>>();
```

You can then have one consumer process requests sequentially:

```
await foreach (var work in _queue.Reader.ReadAllAsync())
{
    await work();
}
```

This gives you the core Active Object behavior:



In an interview, a concise definition would be:

Active Object is a concurrency pattern that decouples method invocation from method execution by placing requests into a queue and having an independent execution mechanism process them, allowing clients to interact asynchronously while the object's state can be accessed in a controlled manner.

5. Monitor Object

The **Monitor Object design pattern** is a concurrency pattern used to make access to shared state **thread-safe**. It encapsulates the shared resource inside an object and ensures that only one thread at a time can execute operations that modify or inspect that resource.

In C#, the pattern is commonly implemented using the `lock` statement.

Basic example

```
public class BankAccount
{
    private decimal _balance;
    private readonly object _lock = new object();

    public void Deposit(decimal amount)
    {
        lock (_lock)
        {
            _balance += amount;
        }
    }

    public bool Withdraw(decimal amount)
    {
        lock (_lock)
        {
            if (_balance < amount)
                return false;

            _balance -= amount;
            return true;
        }
    }

    public decimal GetBalance()
    {
        lock (_lock)
        {
            return _balance;
        }
    }
}
```

Here, `_balance` is the **shared state**, while `_lock` acts as the **monitor**.

What happens with multiple threads?

Suppose two threads execute:

```
account.Deposit(100);
account.Withdraw(50);
```

The `lock` ensures that the critical section is executed by only **one thread at a time**:



Without synchronization, operations such as:

```
_balance += amount;
```

can suffer from **race conditions**, because the operation is actually a read → modify → write sequence.

lock and Monitor

C#'s:

```
lock (_lock)
{
    // critical section
}
```

is essentially a convenient way of using `Monitor.Enter()` and `Monitor.Exit()`.

Conceptually:

```
Monitor.Enter(_lock);

try
{
    _balance += amount;
}
finally
{
    Monitor.Exit(_lock);
}
```

The `lock` version is preferred because it is less error-prone.

Monitor Object vs normal locking

The important idea is **encapsulation**.

Instead of allowing callers to synchronize access themselves:

```
// Bad design
lock (account)
{
    account.Deposit(100);
}
```

the **BankAccount** object protects its own state:

```
public void Deposit(decimal amount)
{
    lock (_lock)
    {
        _balance += amount;
    }
}
```

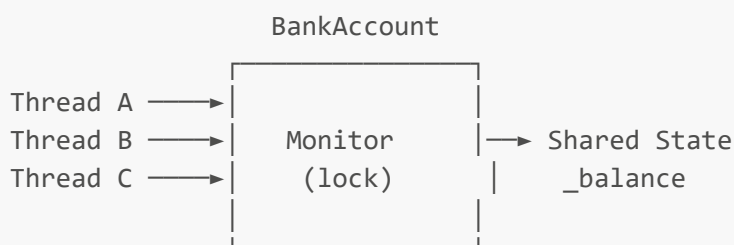
This is what makes it a **Monitor Object**: the object controls access to its own shared state.

When to use it

The pattern is useful when:

- Multiple threads access the same mutable object.
- Operations must be atomic.
- The object maintains invariants that must not be violated concurrently.
- You want synchronization logic hidden inside the object.

For example:



In short: the Monitor Object pattern means “put shared state and the synchronization that protects it together inside an object.” In modern C#, **lock** is the most common mechanism for implementing this pattern.

6. Read-Write Lock

A **Read-Write Lock** is a concurrency pattern used when multiple threads access shared data and you want to allow:

- **Multiple readers simultaneously** — reading doesn't modify data.
- **Only one writer at a time** — writing must be exclusive.
- **No readers while a writer is writing** — prevents inconsistent reads.

In C#, the usual implementation is `ReaderWriterLockSlim`.

Basic example

```
using System;
using System.Threading;

public class Cache
{
    private readonly ReaderWriterLockSlim _lock = new();
    private readonly Dictionary<int, string> _data = new();

    public string? Get(int key)
    {
        _lock.EnterReadLock();

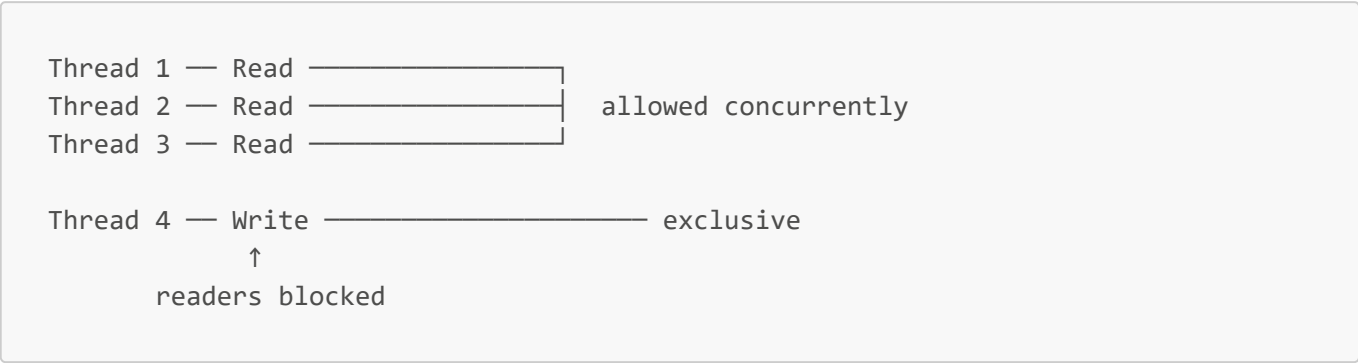
        try
        {
            _data.TryGetValue(key, out var value);
            return value;
        }
        finally
        {
            _lock.ExitReadLock();
        }
    }

    public void Set(int key, string value)
    {
        _lock.EnterWriteLock();

        try
        {
            _data[key] = value;
        }
        finally
        {
            _lock.ExitWriteLock();
        }
    }
}
```

How it behaves

Suppose we have:



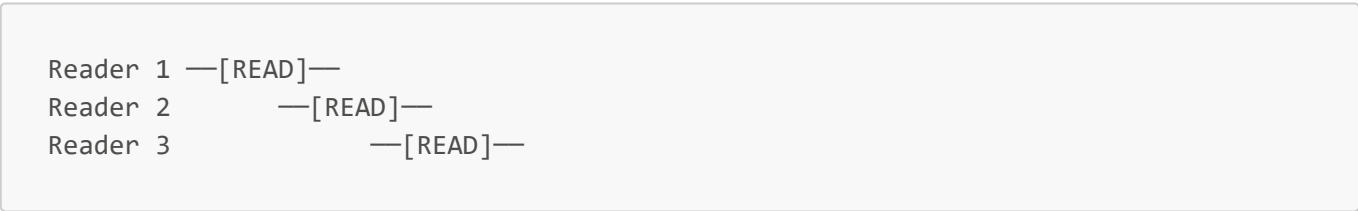
So if three threads are reading at the same time, all three can proceed. But when a writer acquires the lock, other readers and writers have to wait.

Why not just use `lock`?

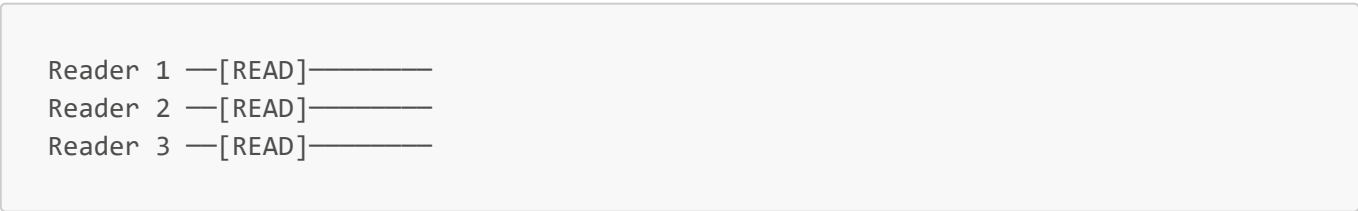
With a normal C# `lock`:

```
lock (_sync)
{
    return _data[key];
}
```

even **readers block other readers**:



With `ReaderWriterLockSlim`:



They can all read concurrently.

This makes a read-write lock particularly useful when the workload is **read-heavy**, such as a cache or configuration store.

Important methods

Method	Purpose
<code>EnterReadLock()</code>	Acquire shared/read access

Method	Purpose
ExitReadLock()	Release read access
EnterWriteLock()	Acquire exclusive/write access
ExitWriteLock()	Release write access
TryEnterReadLock(timeout)	Try to acquire read lock
TryEnterWriteLock(timeout)	Try to acquire write lock

Upgradeable read lock

A particularly useful feature is `EnterUpgradeableReadLock()`.

For example, imagine you want to:

Read an item, and only if it doesn't exist, write it.

You shouldn't simply acquire a read lock and then try to acquire a write lock, because that can lead to deadlock. Instead:

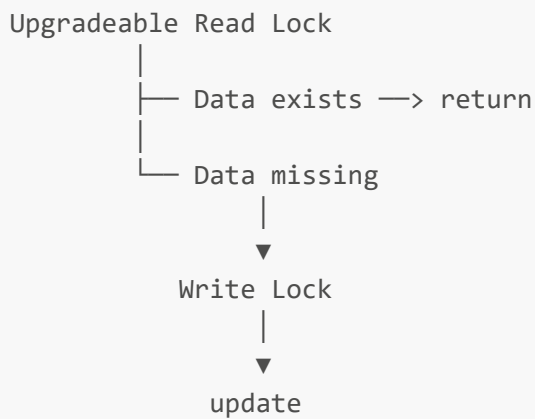
```
public string GetOrCreate(int key)
{
    _lock.EnterUpgradeableReadLock();

    try
    {
        if (_data.TryGetValue(key, out var value))
            return value;

        _lock.EnterWriteLock();

        try
        {
            value = $"Value-{key}";
            _data[key] = value;
            return value;
        }
        finally
        {
            _lock.ExitWriteLock();
        }
    }
    finally
    {
        _lock.ExitUpgradeableReadLock();
    }
}
```

The pattern is:



When should you use it?

A good rule of thumb:

Use **ReaderWriterLockSlim** when:

```

Many reads
+
Few writes
+
Shared mutable state
  
```

For example:

- In-memory caches
- Shared configuration
- Lookup tables
- Frequently-read application state

If reads and writes happen at roughly the same frequency, a simple **lock** may actually be **simpler and faster** because **ReaderWriterLockSlim** has additional synchronization overhead.

Also, **ReaderWriterLockSlim** is generally preferable to the older **ReaderWriterLock** for new C# code.

7. Double-Checked Locking

Double-Checked Locking (DCL) is a design pattern used to implement **lazy initialization of a singleton/shared resource** while avoiding the cost of taking a lock every time the object is accessed.

The problem

A naive singleton might do this:

```

public class Singleton
{
    private static Singleton? _instance;
  
```

```

    private static readonly object _lock = new();

    public static Singleton Instance
    {
        get
        {
            lock (_lock)
            {
                if (_instance == null)
                    _instance = new Singleton();

                return _instance;
            }
        }
    }

    private Singleton() { }
}

```

This is thread-safe, but **every access** to `Instance` acquires the lock.

If the singleton has already been created, locking is unnecessary.

Double-Checked Locking

DCL checks the object **before** acquiring the lock and **again inside** the lock:

```

public sealed class Singleton
{
    private static Singleton? _instance;
    private static readonly object _lock = new();

    public static Singleton Instance
    {
        get
        {
            // First check – avoid locking after initialization
            if (_instance == null)
            {
                lock (_lock)
                {
                    // Second check – another thread may have initialized it
                    if (_instance == null)
                    {
                        _instance = new Singleton();
                    }
                }
            }

            return _instance;
        }
    }
}

```

```

    }

    private Singleton()
    {
    }
}

```

Why two checks?

Imagine two threads, **Thread A** and **Thread B**, accessing the singleton simultaneously.

Initially:

```
_instance = null
```

Both threads perform the first check:

```

Thread A → _instance == null → true
Thread B → _instance == null → true

```

Both try to acquire the lock.

Only one can enter:

```

Thread A → lock acquired
Thread B → waiting

```

Thread A creates the instance:

```
_instance = new Singleton();
```

Then Thread A releases the lock.

Thread B enters the lock, but it **must check again**:

```
if (_instance == null)
```

Now it is false, so Thread B doesn't create another instance.

```

Thread A ┌
         │ ─ lock ─> creates Singleton

```

```
Thread B ─┐
           ↓
         _instance != null
```

The **second check is what prevents multiple instances.**

Why is it called "Double-Checked"?

Because we check the condition twice:

```
if (_instance == null)      // 1st check
{
    lock (_lock)
    {
        if (_instance == null)  // 2nd check
        {
            _instance = new Singleton();
        }
    }
}
```

The first check is for **performance**.

The second check is for **correctness**.

But in modern C#, you usually don't need DCL

For a singleton, .NET provides simpler and safer approaches.

1. `Lazy<T>` — recommended

```
public sealed class Singleton
{
    private static readonly Lazy<Singleton> _instance =
        new Lazy<Singleton>(() => new Singleton());

    public static Singleton Instance => _instance.Value;

    private Singleton()
    {
    }
}
```

Or, with modern C#:

```
public sealed class Singleton
{
    private static readonly Lazy<Singleton> _instance = new();

    public static Singleton Instance => _instance.Value;

    private Singleton()
    {
    }
}
```

`Lazy<T>` handles **thread safety + lazy initialization** for you.

2. Static initialization

Often even simpler:

```
public sealed class Singleton
{
    private static readonly Singleton _instance = new();

    public static Singleton Instance => _instance;

    private Singleton()
    {
    }
}
```

The CLR guarantees thread-safe initialization of static fields.

When should you know DCL?

DCL is important to understand because it demonstrates several core concepts:

- **Race conditions**
- **Thread synchronization**
- **Locks**
- **Lazy initialization**
- **Memory visibility**
- **Performance vs. synchronization overhead**

For a **C# interview**, a good answer is:

"Double-checked locking is a thread-safe lazy initialization technique. We first check whether the instance exists without acquiring a lock. If it doesn't, we acquire the lock and check again because another thread may have initialized it while we were waiting. If it's still null, we create the instance. In

modern C#, `Lazy<T>` is generally preferred because it provides thread-safe lazy initialization without requiring us to implement the locking ourselves."

One important caveat: DCL is easy to get wrong in languages/memory models where ordinary reads/writes don't provide the required visibility guarantees. In modern .NET, the memory model makes correctly implemented patterns possible, but for ordinary C# singleton code, `Lazy<T>` or static initialization is usually the better choice.

8. Immutable Object

The **Immutable Object design pattern** in C# is a way of creating objects whose state **cannot be changed after they are created**.

This is especially useful for objects such as **Money, DateRange, Coordinates, Configuration, User details, or Value Objects**.

1. Basic example

```
public class Person
{
    public string Name { get; }
    public int Age { get; }

    public Person(string name, int age)
    {
        Name = name;
        Age = age;
    }
}
```

Usage:

```
var person = new Person("John", 30);

// person.Name = "Mike"; // ✗ Compile-time error
// person.Age = 40;      // ✗ Compile-time error
```

Once `Person` is created, its `Name` and `Age` cannot be modified.

2. Why use immutability?

An immutable object has several advantages:

- **Thread safety** — multiple threads can safely read the same object.
- **Predictability** — its state doesn't unexpectedly change.
- **Easier debugging** — once created, you know its values remain the same.
- **Safer sharing** — objects can be passed around without worrying that another method will modify them.

- **Good for value objects** — e.g., `Money`, `Address`, `Coordinates`.

3. Using `record` — modern C#

In modern C#, `record` is often the easiest way to create immutable objects:

```
public record Person(string Name, int Age);
```

Then:

```
var person = new Person("John", 30);

Console.WriteLine(person.Name); // John
Console.WriteLine(person.Age);  // 30
```

You can't directly modify it:

```
// person.Age = 40; // ✗
```

But you can create a **new object** with a changed value:

```
var olderPerson = person with { Age = 40 };

Console.WriteLine(person.Age);      // 30
Console.WriteLine(olderPerson.Age); // 40
```

This is an important idea behind immutability:

Don't change the existing object; create a new object with the desired state.

4. A more realistic example

Suppose we have a `Money` object:

```
public sealed class Money
{
    public decimal Amount { get; }
    public string Currency { get; }

    public Money(decimal amount, string currency)
    {
        Amount = amount;
        Currency = currency;
    }
}
```

```
public Money Add(Money other)
{
    if (Currency != other.Currency)
        throw new InvalidOperationException("Currencies must match.");

    return new Money(Amount + other.Amount, Currency);
}
```

Usage:

```
var money1 = new Money(100, "USD");

var money2 = money1.Add(new Money(50, "USD"));

Console.WriteLine(money1.Amount); // 100
Console.WriteLine(money2.Amount); // 150
```

Notice that `Add()` **doesn't modify** `money1`. It creates a new `Money` object.

Important rule

For a truly immutable object, you need to make sure **all mutable state is protected**, not just properties.

For example, this is **not fully immutable**:

```
public class Team
{
    public List<string> Players { get; }

    public Team(List<string> players)
    {
        Players = players;
    }
}
```

Because someone can do:

```
team.Players.Add("John"); // Object changed!
```

A safer version would expose a read-only collection and avoid retaining a mutable collection that callers can modify.

Interview definition

If you're preparing for a C# interview, a good concise answer is:

An immutable object is an object whose state cannot be changed after it has been created. Its properties are typically read-only and initialized through a constructor. Instead of modifying the existing object, operations return a new object containing the updated state. In C#, `record` types are a convenient way to implement immutable data objects.

9. Actor Model

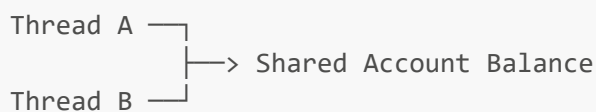
The **Actor Model** is a concurrency design pattern where an application is divided into independent **actors**. Each actor:

- Owns its own state.
- Processes messages sequentially.
- Communicates with other actors by sending messages.
- Doesn't directly access another actor's state.

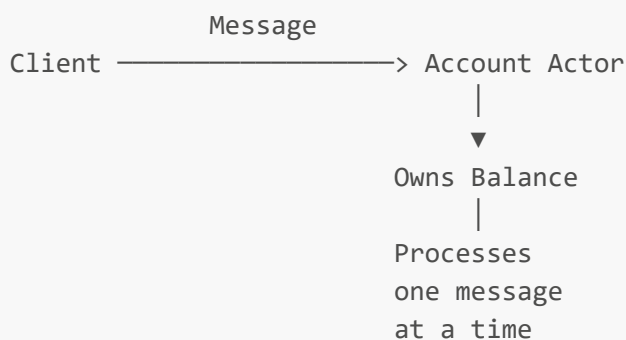
This makes concurrent code easier to reason about and helps avoid problems such as race conditions and shared-state locking.

Basic idea

Instead of:



you have:



Simple C# implementation

You can implement a basic actor using `Channel<T>`:

```

using System.Threading.Channels;

public record Deposit(decimal Amount);
  
```

```
public record Withdraw(decimal Amount);

public class BankAccountActor
{
    private readonly Channel<object> _mailbox =
        Channel.CreateUnbounded<object>();

    private decimal _balance;

    public BankAccountActor(decimal initialBalance)
    {
        _balance = initialBalance;
        _ = ProcessMessagesAsync();
    }

    public ValueTask SendAsync(object message)
    {
        return _mailbox.Writer.WriteAsync(message);
    }

    private async Task ProcessMessagesAsync()
    {
        await foreach (var message in _mailbox.Reader.ReadAllAsync())
        {
            switch (message)
            {
                case Deposit deposit:
                    _balance += deposit.Amount;
                    Console.WriteLine($"Deposited: {deposit.Amount}");
                    break;

                case Withdraw withdraw:
                    if (_balance >= withdraw.Amount)
                    {
                        _balance -= withdraw.Amount;
                        Console.WriteLine($"Withdrawn: {withdraw.Amount}");
                    }
                    else
                    {
                        Console.WriteLine("Insufficient balance");
                    }
                    break;
            }

            Console.WriteLine($"Balance: {_balance}");
        }
    }
}
```

Usage:

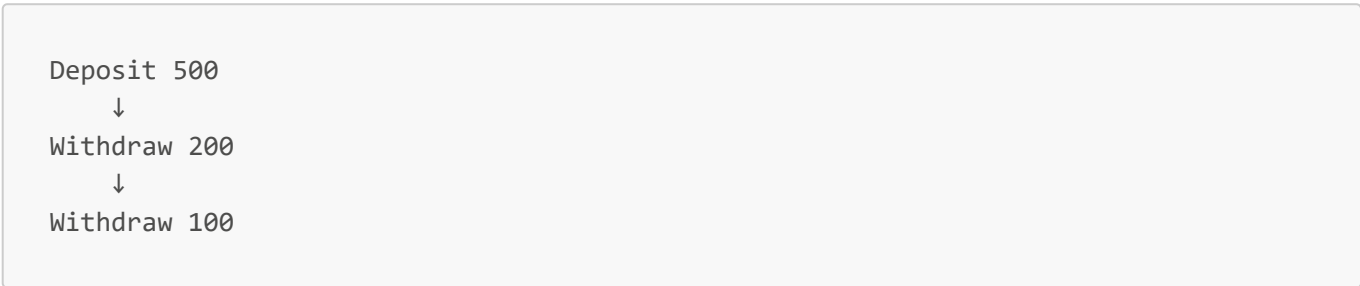
```
var account = new BankAccountActor(1000);

await account.SendAsync(new Deposit(500));
await account.SendAsync(new Withdraw(200));
await account.SendAsync(new Withdraw(100));
```

The important part is that callers **don't modify `_balance` directly**. They send messages to the actor:

```
await account.SendAsync(new Deposit(500));
```

The actor processes those messages sequentially:



So the actor doesn't need a `lock` around `_balance` in this design.

Actor Model vs normal C# code

Traditional approach	Actor Model
Shared mutable state	Private actor state
Threads manipulate state	Actors process messages
<code>lock</code> / <code>Monitor</code> often needed	Sequential mailbox processing
Objects call each other directly	Actors send messages
Concurrency can be difficult	Concurrency is isolated
State is commonly shared	State ownership is explicit

In real-world C#

You generally don't want to build a complete actor framework yourself. A popular option is **Microsoft Orleans**, which provides a virtual-actor model for .NET applications.

Another well-known option is **Akka.NET**, which implements the actor model directly.

The key idea to remember is:

Don't share state; send messages to the component that owns the state.

If you're preparing for a **C#/.NET interview**, I can also explain **Actor Model vs Task vs async/await vs lock vs Channel<T>**, which is where this pattern becomes much clearer.

10. Pipeline